

Desarrollar un CRM omnicanal para comercios que centralice en una sola bandeja de entrada las conversaciones de sus canales de mensajería, las vincule con la gestión de clientes, el calendario, las plantillas de WhatsApp y las estadísticas del negocio, mediante Django, htmx y PostgreSQL, y que pueda operar sobre proveedores reales de WhatsApp sin acoplar la aplicación a ninguno de ellos.

MVP-CRM

Documentación técnica

Autor:

Samuel Pérez Serna

Septiembre 2026

Repositorio: github.com/SamuelPerezCO/MVP-CRM

Stack: Django 6.1 · htmx · Python 3.12 · SQLite en desarrollo, PostgreSQL (Neon) en producción · Vercel.

La segunda parte describe la rama `main` en el commit `6089391`; el Anexo A lista los cambios posteriores.

Sobre este documento

MVP-CRM es un CRM omnicanal para comercios: una bandeja de entrada unificada para los canales de mensajería (WhatsApp, Messenger, Instagram, Facebook, TikTok), gestión de clientes, calendario, plantillas de WhatsApp, embudos de venta, catálogo de productos y estadísticas, todo dentro de un shell de una sola página con barra lateral de iconos. Este documento tiene dos partes. La primera presenta el proyecto: el problema, la pregunta que lo guía, el alcance, los objetivos, los requisitos, el modelo de datos y la interfaz. La segunda documenta el código tal como está en la rama `main`, commit 6089391, y está pensada para quien se incorpora al proyecto o necesita entender cómo funciona una pieza antes de tocarla.

La fuente de verdad es el código. Cada capítulo cita los archivos que documenta y, cuando el código explica el porqué de una decisión en un comentario o docstring, el capítulo lo recoge: esas explicaciones son la parte más valiosa de esta documentación.

Cómo leer la segunda parte

Cada capítulo de la segunda parte se redactó a partir del código fuente y después se verificó afirmación por afirmación (rutas, nombres de URL, campos, variables de entorno, valores por defecto) contrastándola con el código, corrigiendo las discrepancias encontradas. El Anexo B resume ese proceso capítulo por capítulo. Aun así, ante cualquier duda manda el archivo citado.

- Capítulos 1 y 2: la arquitectura del shell y la puerta de entrada (autenticación y agentes). Léelos primero.
- Capítulos 3 a 9: una sección de la aplicación por capítulo, de la más central (Inbox y mensajería) a las más pequeñas.
- Capítulo 10: referencia del modelo de datos y de las migraciones.
- Capítulo 11: configuración, variables de entorno, despliegue en Vercel y operación.
- Capítulo 12: cómo correr las pruebas y las convenciones que el código sigue.
- Anexos: trabajo en curso no integrado en esta versión y el método de verificación.

Nota. Los identificadores de código aparecen en tipografía monoespaciada tal como están en el repositorio (rutas, funciones, variables); la interfaz de usuario está en español y el código en inglés.

Índice

Sobre este documento	2
Parte I. Documento del proyecto	8
Identificación del problema	9
Pregunta de investigación	9
Alcance	9
Objetivo general	10
Objetivos específicos	10
Árbol de problemas	11
Hipótesis de solución	11
Requisitos funcionales	12
Requisitos no funcionales	13
Modelo de datos	14
Mapa conceptual	16
Interfaz	17
Bibliografía	22
Parte II. Documentación técnica del código	24
1. Arquitectura y shell de la aplicación	25
1.1 Propósito y alcance	25
1.2 Cómo funciona: flujo de una petición	25
1.3 Registro de navegación y placeholder automático	26
1.4 Plantilla base, estáticos y vstatic	27
1.5 shell.js: navegación, historial y diálogos	28
1.6 Convenciones que se repiten en todos los módulos	29
1.7 Decisiones de diseño y trampas conocidas	30
1.8 Pruebas que lo cubren	31

2. Autenticación y agentes	33
2.1 Propósito y alcance	33
2.2 Flujo de una petición: el middleware y sus exenciones	33
2.3 APP_AGENTS: quién entra y a quién se asigna	34
2.4 Usuarios espejo de django.contrib.auth	35
2.5 login_view y logout_view	35
2.6 La otra noción de agente: desplegables por usuario activo	36
2.7 Referencia	36
2.8 Decisiones de diseño y trampas conocidas	37
2.9 Pruebas que lo cubren	38
3. Inbox: la bandeja unificada	40
3.1 Propósito y alcance	40
3.2 Filtros y lista de conversaciones	40
3.3 Hilo, compositor y envío	41
3.4 Asignación, etiquetas y respuestas rápidas	42
3.5 Referencia	43
3.6 Decisiones de diseño y trampas conocidas	44
3.7 Pruebas que lo cubren	45
4. Mensajería: modelo, servicios y proveedores	47
4.1 Propósito y alcance	47
4.2 El modelo de datos	47
4.3 Los servicios: recepción, envío e idempotencia	48
4.4 La abstracción de proveedor y el webhook	49
4.5 Medios entrantes y almacenamiento en Vercel Blob	51
4.6 Referencia	51
4.7 Decisiones de diseño y trampas conocidas	53
4.8 Pruebas que lo cubren	53
5. CRM: clientes, listas y etiquetas	55
5.1 Propósito y alcance	55
5.2 Acceso: el portón de sesión	55
5.3 Cómo funciona: nav declarativa y despacho de paneles	56
5.4 La tabla de clientes: búsqueda y paginación	57
5.5 El CRUD de clientes sobre un modal compartido	57
5.6 Reglas de dominio: teléfono, país y validación	58
5.7 Banderas, WhatsApp y listas de clientes	59
5.8 La página de etiquetas	60

5.9 Decisiones de diseño y trampas conocidas	60
5.10 Pruebas que lo cubren	61
6. Mi calendario	63
6.1 Propósito y alcance	63
6.2 Piezas y flujo de arranque	63
6.3 El contrato de zonas horarias	64
6.4 Tipos, paleta y el modal crear/editar	64
6.5 Selección, arrastre, preferencias y mini calendario	65
6.6 Referencia	67
6.7 Decisiones de diseño y trampas conocidas	68
6.8 Pruebas	68
7. Mensajería: plantillas de WhatsApp	70
7.1 Propósito y alcance	70
7.2 La pantalla y su navegación	70
7.3 El modelo MessageTemplate	71
7.4 core/plantillas.py: listas, validación y renderizado	72
7.5 Flujo de creación: tabla, diálogo, editor y galería	73
7.6 Referencia de endpoints y campos del editor	75
7.7 Decisiones de diseño y trampas conocidas	76
7.8 Pruebas que lo cubren	76
8. Estadísticas	78
8.1 Propósito y alcance	78
8.2 Cómo funciona: navegación, paneles y selector de periodos	78
8.3 Las pantallas de detalle: JSON, ECharts y diálogos	79
8.4 Agregación, zona horaria y caché	80
8.5 Referencia	81
8.6 Decisiones de diseño y trampas conocidas	82
8.7 Pruebas que lo cubren	83
9. Embudos, Automatizaciones, Campañas y Mi comercio	85
9.1 Propósito y alcance	85
9.2 El patrón común: módulo de datos, contexto y panel	85
9.3 El componente compartido de secciones de nav	86
9.4 Embudos	86
9.5 Automatizaciones: chatbots de flujo y banner de Academy	87
9.6 Mi comercio: el modelo Product y el catálogo con pestañas	88
9.7 Campañas: segunda entrada a la nav del CRM	88

9.8 Referencia	89
9.9 Decisiones de diseño y trampas conocidas	90
9.10 Pruebas que lo cubren	90
10. Modelo de datos y migraciones	92
10.1 Propósito y alcance	92
10.2 Cómo funciona: un solo contacto para todo el CRM	92
10.3 Referencia de modelos: app core	93
10.4 Referencia de modelos: app messaging	95
10.5 Mapa de relaciones	97
10.6 Historial de migraciones	98
10.7 Comandos que escriben en estas tablas	98
10.8 Decisiones de diseño y trampas conocidas	99
10.9 Registro en el admin de Django	100
10.10 Pruebas que lo cubren	100
11. Configuración, despliegue y operación	102
11.1 Propósito y alcance	102
11.2 Puesta en marcha local	102
11.3 Configuración por entorno: cómo se carga .env	104
11.4 Base de datos y la bandera TESTING	104
11.5 Despliegue en Vercel	105
11.6 Páginas legales públicas y la revisión de Meta	106
11.7 Referencia: variables de entorno, dependencias y archivos	107
11.8 Trampas conocidas y pruebas que lo cubren	108
12. Pruebas y convenciones de desarrollo	110
12.1 Propósito y alcance	110
12.2 Cómo se ejecuta la suite	110
12.3 La bandera TESTING	111
12.4 Las exenciones de la puerta de login	111
12.5 Referencia: módulos de tests	113
12.6 Convenciones de desarrollo observadas	114
12.7 Qué hace shell.js, y qué no	115
12.8 Decisiones de diseño y trampas conocidas	116
12.9 Cómo escribir un módulo de tests nuevo	117
13. Guía práctica: cómo añadir una sección, un panel o una vista nueva	118
13.1 Propósito y alcance	118
13.2 Nivel 1: una sección nueva del sidebar	118

13.3 Nivel 2: columnas 2 y 3 dentro de una sección	119
13.4 Reutilizar una navegación entre dos secciones	120
13.5 Referencia: puntos de extensión y archivos	120
13.6 El ejemplo completo: Mi comercio	121
13.7 Decisiones de diseño y trampas conocidas	122
13.8 Pruebas que lo cubren	123
14. Glosario de términos del dominio	124
14.1 Propósito y alcance	124
14.2 El patrón que se repite: navegación declarativa y paneles	124
14.3 Glosario alfabético (A-E)	126
14.4 Glosario alfabético (I-V)	128
14.5 Claves y valores de referencia	129
14.6 Decisiones de diseño y trampas conocidas	130
14.7 Pruebas que cubren estos términos	131
15. Sistema visual: tokens, convenciones CSS y carga de estilos	133
15.1 Propósito y alcance	133
15.2 Cómo se cargan los estilos	133
15.3 La paleta única: el :root de sidebar.css	134
15.4 El rail de solo iconos y sus tooltips	135
15.5 Convención de nombres y bloques compartidos	136
15.6 La pantalla de bienvenida como estado en reposo	136
15.7 Cache-busting: escribir CSS con {% vstatic %}	137
15.8 Decisiones de diseño y trampas conocidas	138
15.9 Pruebas que lo cubren	139
16. Patrones htmx: fragmentos, swaps e historial	141
16.1 Propósito y alcance	141
16.2 Respuesta dual: página completa o fragmento	141
16.3 Contenedores canónicos, hx-target y hx-push-url	142
16.4 Swaps fuera de banda y los partials *_swap.html	143
16.5 Sondeo con hx-trigger every y las guardas data-hold-poll	144
16.6 Los ganchos de shell.js	145
16.7 Decisiones de diseño y trampas conocidas	146
16.8 Pruebas que lo cubren	147
Anexo A. Estado del repositorio y cambios posteriores a esta versión	149
Anexo B. Método de generación y verificación	150

Parte I

Documento del proyecto

Esta primera parte presenta MVP-CRM como proyecto: el problema que resuelve, la pregunta que lo guía, su alcance y objetivos, los requisitos que cumple, su modelo de datos y su interfaz. La segunda parte documenta el código en detalle, capítulo por capítulo.

Identificación del problema

Los comercios pequeños y medianos de Colombia venden, cada vez más, por mensajería: WhatsApp en primer lugar, y detrás Instagram, Messenger, Facebook y TikTok. Cada uno de esos canales tiene su propia aplicación, su propia bandeja y su propia forma de avisar que alguien escribió. El vendedor atiende desde su teléfono personal, salta de una app a otra, y el historial de cada cliente queda repartido entre chats que nadie consolida.

La consecuencia es doble. Por un lado, el negocio pierde clientes: un mensaje que nadie ve a tiempo es una venta que no ocurre, y cuando la conversación sigue al día siguiente ya no hay quien recuerde qué se había ofrecido. Por otro, el dueño del comercio no tiene datos: no sabe cuántas conversaciones entran por semana, cuánto tarda su equipo en responder, quién atiende a quién, ni cuánto le cuesta cada plantilla de WhatsApp que envía fuera de la ventana de 24 horas que impone Meta.

Existen plataformas comerciales que resuelven esto (Treble, Leadsales, Kommo, entre otras), pero cobran por agente, en dólares y con contratos pensados para equipos de ventas grandes. Para un comercio de dos o tres personas el costo no se justifica, y el resultado es que siguen atendiendo a mano, sin CRM, sin registro y sin métricas.

Pregunta de investigación

A partir de lo anterior se plantea la siguiente pregunta: ¿es posible construir, con Django, htmx y una base de datos PostgreSQL desplegada en infraestructura sin servidor, un CRM omnicanal que centralice en una sola bandeja las conversaciones de un comercio, las vincule con sus clientes y sus ventas, y se conecte a los proveedores reales de WhatsApp (Meta y Twilio) sin que el resto de la aplicación dependa de cuál de ellos se use?

Esa pregunta guía tanto la arquitectura del sistema (una abstracción de proveedor que se elige con una sola variable de entorno) como su alcance (las secciones que se construyeron primero son las que responden a la necesidad más urgente del comercio: recibir, contestar y no perder conversaciones).

Alcance

La aplicación está dirigida al equipo de atención y ventas de un comercio: los agentes que responden mensajes y el usuario maestro que administra al equipo. Es una aplicación web de una sola página con una barra lateral de iconos, en español, que se usa desde el navegador de escritorio.

El producto cubre: una bandeja de entrada unificada (Inbox) con filtros por canal y por asignación, hilo de chat con refresco automático, compositor con la regla de la ventana de 24 horas de WhatsApp, respuestas rápidas y panel del cliente; la gestión de clientes con alta, edición, ficha, baja, búsqueda, exportación a Excel y listas; un calendario con eventos vinculados a clientes; las plantillas de WhatsApp con editor, vista previa, envío a Meta para aprobación y lectura del veredicto; un catálogo de productos; estadísticas de mensajería, ventas, etiquetas y embudos; y la administración del equipo por un usuario maestro.

Queda fuera de esta versión el envío masivo de campañas, la construcción visual de chatbots y las integraciones de pago: esas secciones existen en la navegación como andamiaje con estados vacíos, y así lo documenta la segunda parte. Tampoco se construyó una aplicación móvil: la interfaz responde en pantallas de escritorio y tabletas.

Tecnológicamente, el sistema se desarrolló con Python 3.12 y Django 6.1 en el servidor, htmx para los paneles dinámicos sin build de frontend, CSS y SVG propios, SQLite en desarrollo y PostgreSQL (Neon) en producción, desplegado en Vercel con los archivos de medios en Vercel Blob. La integración con WhatsApp vive detrás de una abstracción de proveedor con tres implementaciones: fake para desarrollo local, Twilio y la Cloud API de Meta.

Objetivo general

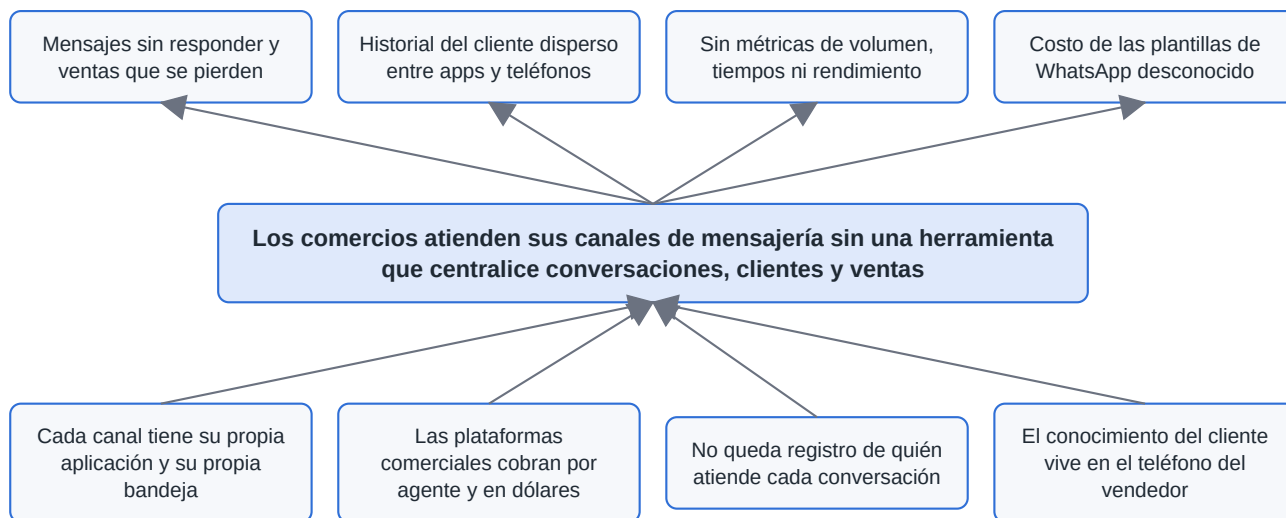
Desarrollar un CRM omnicanal para comercios que centralice en una sola bandeja de entrada las conversaciones de sus canales de mensajería, las vincule con la gestión de clientes, el calendario, las plantillas de WhatsApp y las estadísticas del negocio, mediante Django, htmx y PostgreSQL, y que pueda operar sobre proveedores reales de WhatsApp sin acoplar la aplicación a ninguno de ellos.

Objetivos específicos

- Elicitar las necesidades de atención y venta por mensajería de un comercio pequeño y traducirlas en requisitos funcionales y no funcionales.
- Diseñar un modelo de datos en el que el cliente sea una sola entidad compartida por la bandeja de entrada, el calendario y las listas del CRM.
- Construir el shell de una sola página y la navegación declarativa que permite añadir secciones nuevas con un cambio mínimo.
- Implementar la bandeja de entrada unificada con sus filtros, el hilo de chat, la regla de la ventana de 24 horas y las respuestas rápidas.
- Definir una abstracción de proveedor de mensajería con un webhook firmado e idempotente, y sus implementaciones para desarrollo local, Twilio y Meta.
- Desarrollar la gestión de clientes, el calendario, las plantillas de WhatsApp, el catálogo de productos y las estadísticas.
- Cubrir el sistema con una suite de pruebas automatizadas que proteja las reglas de negocio y los contratos con los proveedores.
- Desplegar la aplicación en producción sobre Vercel y Neon, con las herramientas de operación necesarias para salir a producción con datos limpios.

Árbol de problemas

El árbol resume el problema central, las causas que lo producen (abajo) y los efectos que genera en el comercio (arriba).



Árbol de problemas: efectos (arriba), problema central y causas (abajo).

Hipótesis de solución

Desarrollar una aplicación web con Django, htmx y PostgreSQL que reciba los mensajes de los distintos canales a través de webhooks normalizados, los presente en una sola bandeja con el cliente ya identificado, y permita responder, asignar, etiquetar, agendar y medir desde el mismo lugar, permite que un comercio pequeño atienda sus canales de mensajería con la disciplina de un CRM sin pagar una plataforma por agente, y que cambie de proveedor de WhatsApp modificando una sola variable de configuración.

Requisitos funcionales

ID	Nombre del requisito	Descripción	Usuario
RF-001	Autenticación de agentes	El sistema debe exigir inicio de sesión en toda la aplicación y reconocer como agentes a las personas definidas en la configuración o creadas por el usuario maestro.	Agentes
RF-002	Bandeja de entrada unificada	El sistema debe listar en una sola bandeja las conversaciones de WhatsApp, Messenger, Instagram, Facebook y TikTok, con filtros por canal, por asignación (todos, tu inbox, sin asignar) y búsqueda.	Agentes
RF-003	Recepción de mensajes	El sistema debe recibir los mensajes entrantes por el webhook del proveedor, verificar su firma, evitar duplicados por identificador del proveedor y crear o actualizar el cliente y la conversación.	Sistema
RF-004	Envío de mensajes	El sistema debe permitir responder texto e imágenes desde el hilo, registrar quién envió cada mensaje y reflejar su estado (enviado, entregado, leído, fallido).	Agentes
RF-005	Ventana de 24 horas	Fuera de la ventana de 24 horas desde el último mensaje del cliente, el sistema debe bloquear el texto libre y ofrecer únicamente el envío de una plantilla aprobada.	Sistema
RF-006	Nuevo chat	El sistema debe permitir escribirle primero a un cliente del CRM y abrir la conversación resultante en la bandeja.	Agentes
RF-007	Respuestas rápidas	El sistema debe ofrecer respuestas predefinidas de texto o imagen que se envían con un clic desde el compositor.	Agentes
RF-008	Asignación y etiquetas	El sistema debe permitir asignar cada conversación a un agente, cambiar su estado y aplicar o quitar etiquetas, individualmente o en lote.	Agentes
RF-009	Gestión de clientes	El sistema debe permitir crear, consultar, editar y eliminar clientes (nombre, teléfono con país, correo, canal), buscarlos, exportarlos a Excel y agruparlos en listas.	Agentes
RF-010	Calendario	El sistema debe permitir crear, mover y eliminar eventos con tipo, responsable y cliente vinculado, en la zona horaria de Bogotá.	Agentes
RF-011	Plantillas de WhatsApp	El sistema debe permitir crear plantillas con cabecera, cuerpo con variables, pie y botones, previsualizarlas, enviarlas a Meta para aprobación y leer el veredicto.	Agentes
RF-012	Costo de las plantillas	El sistema debe estimar el costo de cada plantilla según la tarifa vigente de Meta y el mercado del destinatario, y corregirlo con el recibo de entrega.	Sistema
RF-013	Catálogo de productos	El sistema debe permitir crear e importar productos con precio, categoría, marca y estado.	Agentes
RF-014	Estadísticas	El sistema debe mostrar volumen de mensajes, tiempos de respuesta, rendimiento de agentes, perfil de clientes, ventas y etiquetas por periodo.	Usuario maestro
RF-015	Gestión del equipo	El usuario maestro debe poder crear, editar y desactivar usuarios desde la aplicación, sin tocar la configuración ni volver a desplegar.	Usuario maestro
RF-016	Salida a producción	El sistema debe ofrecer comandos que vacíen los datos de prueba conservando al equipo, en modo simulación por defecto.	Administrador

Requisitos funcionales de MVP-CRM.

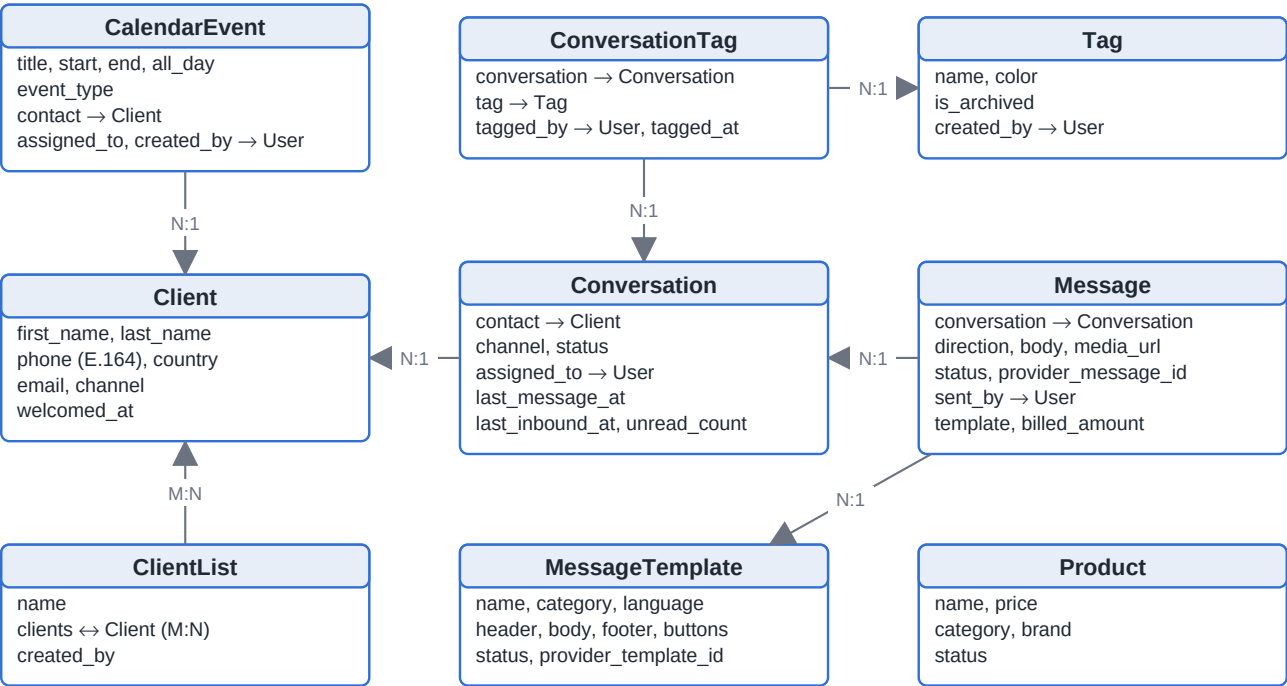
Requisitos no funcionales

ID	Nombre del requisito	Descripción
RNF-001	Seguridad	Toda la aplicación exige sesión; los webhooks verifican la firma del proveedor antes de leer el cuerpo; las contraseñas de los agentes se guardan como hash; las cookies son seguras en producción y el simulador de mensajes no responde fuera de desarrollo.
RNF-002	Independencia del proveedor	Cambiar de proveedor de WhatsApp (simulador, Twilio, Meta) debe ser un cambio de una sola variable de entorno; ninguna vista, modelo o plantilla debe depender del proveedor activo.
RNF-003	Disponibilidad	La aplicación debe correr en infraestructura sin servidor (Vercel) con base de datos administrada (Neon) y almacenamiento de medios externo (Vercel Blob), de modo que no dependa de un servidor propio.
RNF-004	Usabilidad	La interfaz es una sola página con barra lateral de iconos, en español, con paneles que se actualizan sin recargar y una bandeja que se refresca sola cada pocos segundos.
RNF-005	Mantenibilidad	Añadir una sección nueva debe ser un cambio mínimo en el registro de navegación; cada regla de negocio debe estar cubierta por pruebas automatizadas y cada módulo debe documentar sus decisiones en el propio código.
RNF-006	Rendimiento	Las estadísticas se agregan en la base de datos y se cachean por periodo; la bandeja usa sondeo ligero por fragmentos htmx en lugar de recargar la página.
RNF-007	Configurabilidad	Todo lo que cambia entre entornos (base de datos, proveedor, credenciales, dominio, agentes) se define por variables de entorno documentadas en <code>.env.example</code> .
RNF-008	Interoperabilidad	La base de datos puede recibir escrituras de automatizaciones externas (por ejemplo n8n) siempre que respeten el contrato de columnas documentado, y la aplicación debe tolerar valores desconocidos sin caerse.
RNF-009	Trazabilidad	Cada mensaje enviado registra qué agente lo escribió y cada conversación conserva su historial aunque el agente se desactive.

Requisitos no funcionales de MVP-CRM.

Modelo de datos

El modelo gira alrededor de una sola entidad: el cliente (Client). La persona con la que se chatea en el Inbox es la misma fila de la tabla Clientes del CRM, la misma que aparece en los eventos del calendario y en las listas. Cada conversación pertenece a un cliente y a un canal, y contiene los mensajes en ambas direcciones; las etiquetas se aplican a las conversaciones a través de una tabla intermedia que registra quién etiquetó y cuándo.



Modelo de datos: entidades principales y sus relaciones. Las flechas van del lado N al lado 1.

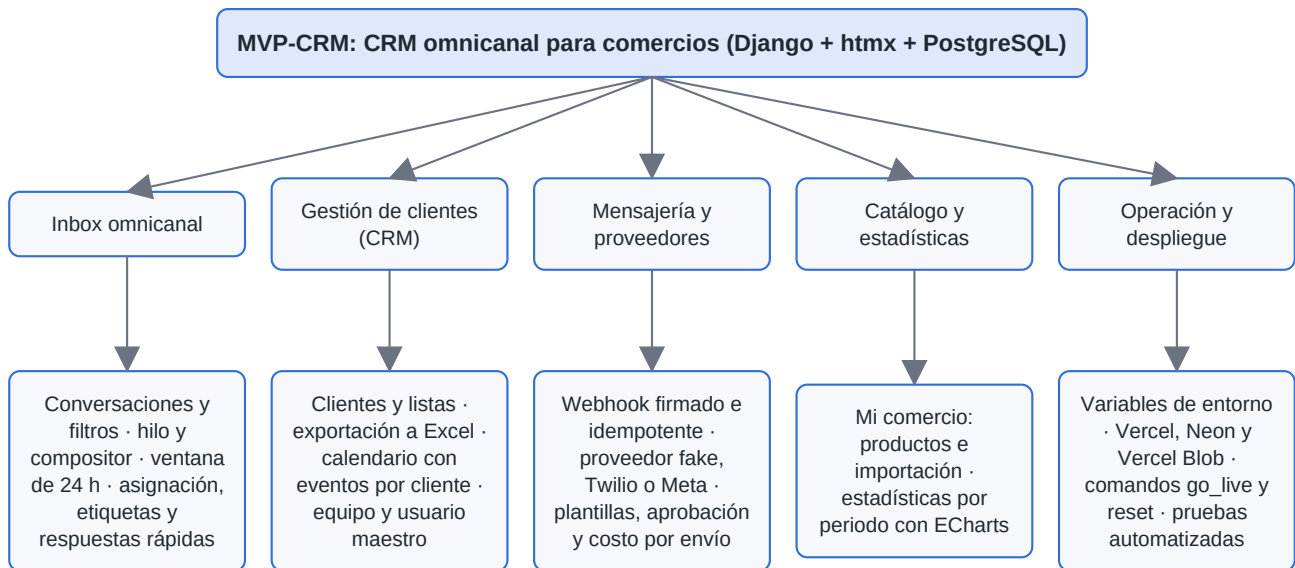
Entidad	App	Qué representa
Client	core	Un cliente del comercio: nombres, teléfono en formato E.164 con país, correo y canal de origen. Es el contacto de las conversaciones, el vinculado a los eventos y el miembro de las listas.
Conversation	messaging	Un hilo con un cliente en un canal. Guarda estado (abierta, pendiente, resuelta), agente asignado y las marcas de tiempo que ordenan la bandeja y deciden la ventana de 24 horas.
Message	messaging	Un mensaje entrante o saliente: cuerpo, medio adjunto, estado de entrega, identificador del proveedor, quién lo envió, plantilla usada y costo facturado.
Tag y ConversationTag	messaging	Etiquetas con color y su aplicación a conversaciones, con quién etiquetó y cuándo.
CalendarEvent	core	Un evento del calendario con tipo, responsable y cliente vinculado.
ClientList	core	Un grupo de clientes (relación muchos a muchos).
MessageTemplate	core	Una plantilla de WhatsApp con su cabecera, cuerpo con variables, pie, botones y el estado de aprobación devuelto por Meta.
Product	core	Un producto del catálogo con precio, categoría, marca y estado.
QuickReply, MessagingSettings, StoredFile	core	Respuestas rápidas del compositor, ajustes de mensajería (bienvenida, asignación automática, widget) y archivos servidos desde la base de datos. Son posteriores al snapshot que documenta la segunda parte.

Entidades del modelo de datos y la app Django a la que pertenecen.

Nota. Las claves foráneas hacia el usuario (`assigned_to`, `sent_by`, `created_by`, `tagged_by`) admiten nulo y se ponen en nulo al borrar el usuario, de modo que desactivar o eliminar a un agente nunca borra historial. Borrar un cliente sí arrastra sus conversaciones y mensajes; los eventos del calendario sobreviven con el cliente en nulo.

Mapa conceptual

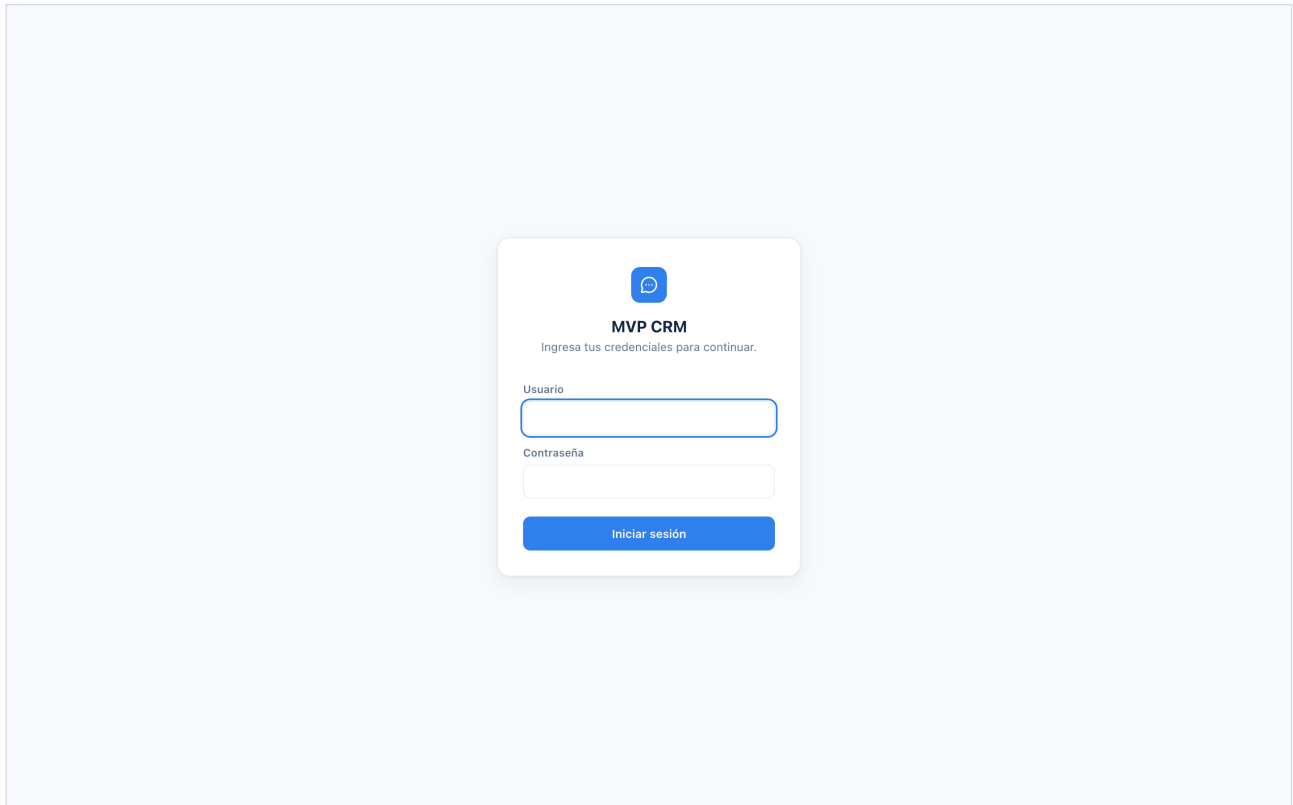
El mapa organiza los conceptos del sistema en tres niveles: la aplicación, sus áreas funcionales y las piezas concretas que cada área aporta.



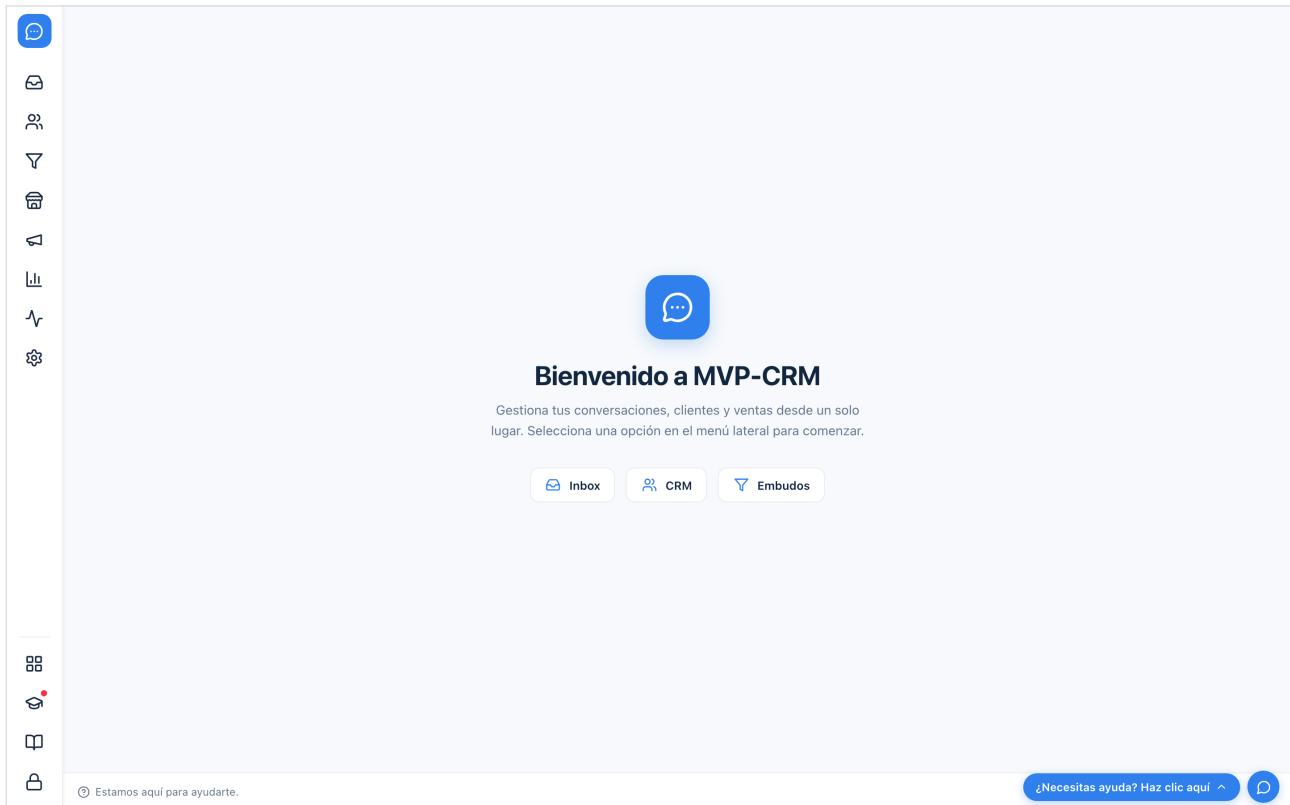
Mapa conceptual de MVP-CRM.

Interfaz

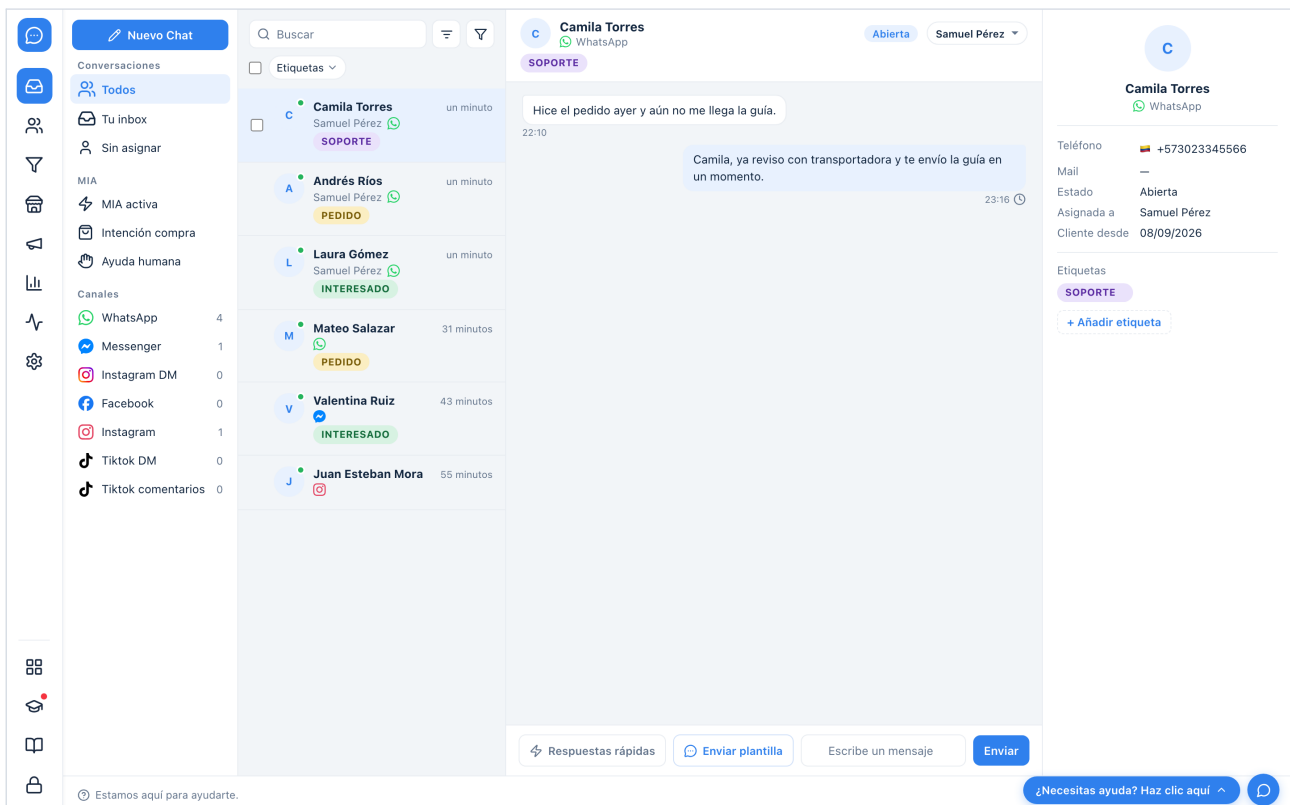
Las siguientes capturas muestran la aplicación en funcionamiento con datos de ejemplo. La barra lateral izquierda es la navegación principal (Inbox, CRM, Embudos, Mi comercio, Campañas, Estadísticas, Integraciones y Configuración de mensajería); el área central cambia sin recargar la página.



Inicio de sesión. Toda la aplicación exige una sesión de agente.



Pantalla de bienvenida al entrar, con accesos directos a Inbox, CRM y Embudos.



Inbox: filtros por conversación y canal, lista de conversaciones con etiquetas y agente, hilo de chat con estados de entrega, compositor con respuestas rápidas y plantillas, y panel del cliente.

CRM, Clientes: tabla con teléfono y bandera del país, correo, canal de origen, acceso directo a iniciar conversación y acciones de ver, editar y eliminar.

CRM, Mi calendario: vista semanal con eventos vinculados a clientes, mini calendario y preferencias de visualización.



Mi cuenta

Gestión de clientes

Calendario

Equipo





















Usuarios

+ Crear usuario

Nombre	Usuario	Rol	Origen	Estado	Acciones
Samuel Pérez · tú	Samuel	MAESTRO	Entorno (APP_AGENTS)	Activo	

CRM, Equipo: usuarios del equipo administrados por el usuario maestro.

Widget de WhatsApp

Plantillas de WhatsApp

Respuestas rápidas

Mensajes de bienvenida

Temas de conversación

Reglas de mensajería

Asignación automática

Buscar por nombre

Sincronizar con WhatsApp

Crear plantilla +

Todas	Pendientes	Aceptadas	Rechazadas	Desactivadas		
Nombre	Tipo	Categoría	Texto	Equipo	Activo	Estado
bienvenida_tienda	Mensaje personalizado	Marketing	¡Hola {{1}}! Gracias por escribirnos a {{2}}. ¿En qué podemos...		Sí	Pendiente sin enviar a WhatsApp
confirmacion_pedido	Mensaje personalizado	Utility	Hola {{1}}, tu pedido #{{2}} fue confirmado y sale mañana.		Sí	Pendiente sin enviar a WhatsApp
recordatorio_pago	Mensaje personalizado	Utility	Hola {{1}}, te recordamos que el pago de tu pedido vence el...		Sí	Pendiente sin enviar a WhatsApp

Configuración de mensajería, Plantillas de WhatsApp: tabla por estado de aprobación, sincronización con WhatsApp y creación de plantillas.



Mi cuenta



Catálogo



Órdenes



Pagos



Envíos



Configuración del ...









Productos

Q

Buscar productos...

Importar

Crear +

Todos los productos

Activos

Inactivos

☐	Nombre	Stock	Precio	Categoría	Marca	Estado	Sincronizado con
☐	Combo 2 unidades crema hidratante	0	119900.00	Cuidado corporal	Glow	Activo	
☐	Kit skincare tono claro	0	89900.00	Cuidado facial	Glow	Activo	
☐	Protector solar SPF 50	0	62000.00	Cuidado facial	Sol	Activo	
☐	Sérum vitamina C 30 ml	0	54900.00	Cuidado facial	Glow	Activo	

Mi comercio, Productos: catálogo con pestañas por estado, importación y creación.

Ventas

Etiquetas

Embudos

Atribuciones Alpha

Temas de conversación

Agentes de IA

Mensajería

Volumen de Mensajes

Mensajes por plataforma, horarios pico y distribución por canal de comunicación.

Tiempos de Respuesta

Tiempo promedio, horario laboral y respuesta humana post-MIA. Identifica cuellos de botella en la atención.

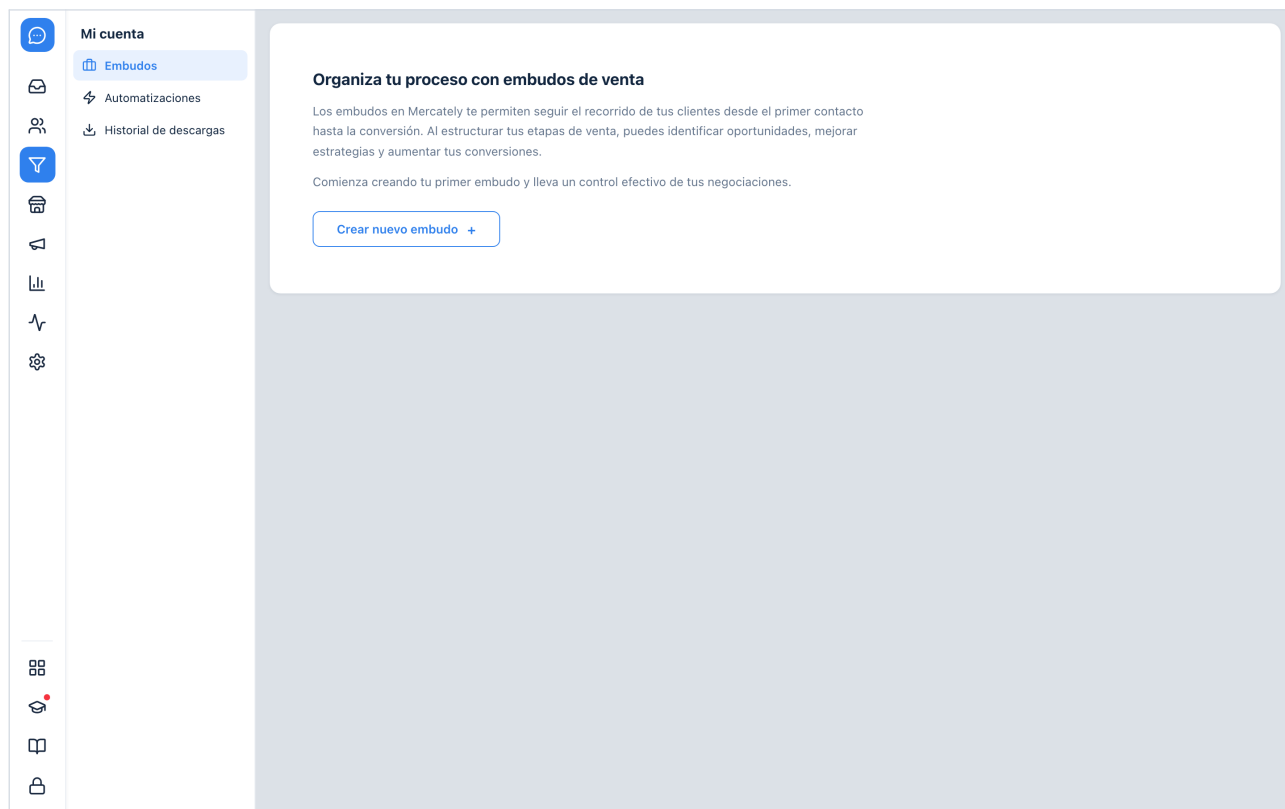
Rendimiento de Agentes

Carga de trabajo, mensajes enviados y estado de conversaciones por agente.

Perfil de Clientes

Métricas de toda la cuenta en el periodo: primer contacto, recurrentes, leads y etiquetas.

Estadísticas de mensajería: tarjetas de volumen, tiempos de respuesta, rendimiento de agentes y perfil de clientes.



Embudos: panel de embudos de venta con su navegación secundaria.

Bibliografía

- Django Software Foundation (2026) *Django documentation, release 6.1*. Disponible en: <https://docs.djangoproject.com/en/6.1/> (consultado en septiembre de 2026).
- Big Sky Software (2026) *htmx documentation*. Disponible en: <https://htmx.org/docs/> (consultado en septiembre de 2026).
- Python Software Foundation (2026) *Python 3.12 documentation*. Disponible en: <https://docs.python.org/3.12/> (consultado en septiembre de 2026).
- Meta Platforms (2026) *WhatsApp Business Platform: Cloud API*. Disponible en: <https://developers.facebook.com/docs/whatsapp/cloud-api/> (consultado en septiembre de 2026).
- Meta Platforms (2026) *WhatsApp Business Platform: pricing*. Disponible en: <https://developers.facebook.com/docs/whatsapp/pricing/> (consultado en septiembre de 2026).
- Twilio (2026) *WhatsApp Business API with Twilio*. Disponible en: <https://www.twilio.com/docs/whatsapp> (consultado en septiembre de 2026).
- Vercel (2026) *Vercel documentation: Python runtime and Vercel Blob*. Disponible en: <https://vercel.com/docs> (consultado en septiembre de 2026).
- Neon (2026) *Neon documentation: serverless Postgres*. Disponible en: <https://neon.com/docs> (consultado en septiembre de 2026).
- PostgreSQL Global Development Group (2026) *PostgreSQL documentation*. Disponible en: <https://www.postgresql.org/docs/> (consultado en septiembre de 2026).
- Shaw, A. (2026) *FullCalendar documentation*. Disponible en: <https://fullcalendar.io/docs> (consultado en septiembre de 2026).
- Apache Software Foundation (2026) *Apache ECharts documentation*. Disponible en: <https://echarts.apache.org/en/index.html> (consultado en septiembre de 2026).

Parte II

Documentación técnica del código

Los capítulos que siguen documentan el código de MVP-CRM tal como está en la rama `main`, commit 6089391. Cada capítulo cita los archivos que describe; el Anexo A lista los commits posteriores que no cubre y el Anexo B el registro de verificación.

1. Arquitectura y shell de la aplicación

MVP-CRM es una aplicación Django 6 con un shell de una sola página: una barra lateral fija de iconos registrada en `core/nav.py` y un área `<main id="content">` que `htmx` rellena con la sección activa. La vista `section()` responde página completa o fragmento según la cabecera `HX-Request`, cualquier sección sin plantilla cae automáticamente en un placeholder, el tag `vstatic` estampa los estáticos con un hash de contenido y `shell.js` sincroniza el estado activo, el título y los `<dialog>` que `htmx` no cubre. Cada pantalla con `nav` secundaria repite la misma forma: un módulo de datos, un constructor de contexto en `SECTION_CONTEXT`, un registro por panel y un endpoint que reemplaza solo la columna 3.

1.1 Propósito y alcance

MVP-CRM es una aplicación Django 6 cuyo contenedor visual es una sola página: una barra lateral fija de iconos a la izquierda y un elemento `<main id="content">` a la derecha en el que se intercambia la pantalla activa. No hay build de frontend ni framework de componentes; `htmx`, vendorizado en `static/js/vendor/htmx.min.js` para que la aplicación no dependa de ningún CDN en tiempo de ejecución, pide fragmentos HTML al servidor y los intercambia, y `static/js/shell.js` añade el pegamento que `htmx` no cubre. Este capítulo describe ese shell: el registro de secciones, la vista que las sirve, la plantilla base, el cache-busting de estáticos, el comportamiento de `shell.js` y las convenciones que cada módulo de pantalla repite.

Quedan fuera los contenidos de cada sección (Inbox, CRM, Embudos, Mi comercio, Estadísticas, Configuración de mensajería), la capa de mensajería y los proveedores de WhatsApp, que tienen capítulos propios. La puerta de entrada (`core.middleware.LoginRequiredMiddleware`) se menciona solo en la medida en que condiciona cada petición del shell.

1.2 Cómo funciona: flujo de una petición

Cada icono de la barra lateral es un `<a href>` real hacia `/s/<key>/` (nombre de URL `section`, declarado al final de `core/urls.py`), decorado en `templates/partials/nav_item.html` con `hx-get`, `hx-target="#content"`, `hx-swap="innerHTML"` y `hx-push-url="true"`. Sin JavaScript el enlace produce una carga completa; con `htmx` la misma URL se pide con la cabecera `HX-Request: true` y el servidor devuelve solo el fragmento. La vista `section()` de `core/views.py` atiende las dos variantes y decide con `_is_htmx(request)`, que comprueba `request.headers.get("HX-Request") == "true"`. Según su docstring, esa doble respuesta es lo que permite prescindir de recargas completas manteniendo cada URL enlazable, guardable como marcador y compatible con el botón de retroceso.

```
Clic en un icono del rail (a.nav-item href="/s/crm/" hx-get="/s/crm/")
|
|-- sin JS ..... GET /s/crm/                -> documento completo (base.html)
'|-- con htmx ..... GET /s/crm/ HX-Request: true -> solo el fragmento
|
| v
core.middleware.LoginRequiredMiddleware (sesion con app_authenticated,
|                                     o redirect a /login/?next=/s/crm/)
| v
core.views.section("crm")
  NAV_BY_KEY["crm"]           -> 404 si la clave no existe
  _section_template("crm")     -> sections/crm.html, o sections/_placeholder.html
  SECTION_CONTEXT["crm"](req) -> datos propios de la pantalla (opcional)
  |
  | v
  render_to_string(section_template) -> hx-swap innerHTML en <main id="content">
  hx-push-url actualiza la barra de direcciones; shell.js mueve is-active y document.title
```

En ambos casos el contexto base es el mismo: `primary_nav`, `secondary_nav`, `active_key`, `item`, `page_title` y `section_template`; a eso se suma lo que devuelva el constructor registrado en `SECTION_CONTEXT` para esa clave, si existe. La respuesta completa se genera con `render_to_string("base.html", ...)`, que incluye `partials/sidebar.html` y después `{% include section_template %}`; la respuesta `htmx` renderiza directamente `section_template`. La raíz / (nombre de URL home) la sirve `welcome()`, que responde de las mismas dos formas y renderiza `sections/welcome.html`, pero con `active_key=None`: como `nav_item.html` compara cada `item.key` contra ese valor y ninguna clave puede ser `None`, el rail queda entero sin seleccionar.

Hay una segunda vía de render de página completa además de `section()`. `_mensajeria_page(request, panel_template, panel_context)` reconstruye a mano el mismo contexto base (`primary_nav`, `secondary_nav`, `active_key="mensajeria"`, `item`, `page_title` y `_section_template("mensajeria")`), le añade `_mensajeria_context(request)` y sustituye `panel_template` por el que reciba. Su docstring explica el porqué: la navegación sin `htmx` llega ahí, por ejemplo al abrir en una pestaña nueva una tarjeta del selector de plantillas o al enviar un formulario sin JavaScript, y esas URLs deben renderizarse dentro del shell en vez de como fragmento suelto. La usan `plantilla_gallery` y `plantilla_editor`, que devuelven el panel desnudo cuando la petición es `htmx`.

El enrutado de proyecto en `config/urls.py` es mínimo: `admin/` para el admin de Django, `webhooks/messaging/` que incluye `messaging.urls`, y la raíz que incluye `core.urls`. Al final añade `static(settings.MEDIA_URL, ...)` para servir uploads en desarrollo, que es un no-op cuando `DEBUG` está apagado.

1.3 Registro de navegación y placeholder automático

`core/nav.py` es la única fuente de verdad de la barra lateral: su docstring lo define así y explica que añadir, reordenar o renombrar una sección es un cambio de una línea en ese archivo, sin tocar plantillas. Cada entrada es un `NavItem` (dataclass congelada) con `key` (slug de URL y nombre de la plantilla de sección), `label` (etiqueta en español que se muestra como tooltip y se usa como `aria-label`, el único texto que nombra el control porque el rail es solo iconos), `icon` (glifo SVG bajo `templates/icons/`, que la propiedad `icon_template` resuelve como `icons/<icon>.svg`) y `badge` (punto rojo de notificación, por defecto `False`). `PRIMARY_NAV` tiene 8 entradas y `SECONDARY_NAV` 3; `ALL_NAV` las concatena y `NAV_BY_KEY` es el índice con el que `section()` valida el slug y responde `Http404` cuando no existe. `DEFAULT_SECTION = "inbox"` es el valor por defecto del parámetro `key` de `section()`, aunque ninguna URL lo usa; `WELCOME_SHORTCUTS = ["inbox", "crm", "embudos"]` define las tarjetas de acceso rápido de la bienvenida, que la vista resuelve a `NavItem` para reutilizar etiqueta y glifo.

Construir una sección nueva no requiere tocar vistas ni URLs. `_section_template(key)` intenta cargar `sections/<key>.html` con `get_template` y, si salta `TemplateDoesNotExist`, devuelve `PLACEHOLDER_TEMPLATE`, que vale `sections/_placeholder.html`. Ese placeholder muestra `item.label` como título en un `page-head__title` y una tarjeta vacía con el texto "próximamente"; el archivo real se recoge en la siguiente petición. El propio docstring de `_section_template` lo llama el punto de extensión: para desarrollar una sección, crea el archivo.

Clave	Etiqueta	Grupo	Icono	Pantalla propia
inbox	Inbox	principal	inbox	Sí, sections/inbox.html
crm	CRM	principal	users	Sí, sections/crm.html
embudos	Embudos	principal	funnel	Sí, sections/embudos.html
mi-comercio	Mi comercio	principal	store	Sí, sections/mi-comercio.html
campanas	Campañas	principal	megaphone	Sí, sections/campanas.html
estadisticas	Estadísticas	principal	bar-chart	Sí, sections/estadisticas.html

Clave	Etiqueta	Grupo	Icono	Pantalla propia
integraciones	Integraciones	principal	activity	No, placeholder
mensajería	Configuración de mensajería	principal	settings	Sí, sections/mensajería.html
apps	Aplicaciones	secundario	grid	No, placeholder
academy	Academy	secundario	graduation-cap	No, placeholder (única con badge)
recursos	Recursos	secundario	book	No, placeholder

Secciones registradas en `core/nav.py`, en el orden en que se renderizan.

Nota. El 2026-08-26 se retiraron del rail automatizaciones, performance-hub y crecimiento; siguen comentadas en `core/nav.py` para restaurarlas con solo volver a añadir el `NavItem`. Automatizaciones conserva plantilla, vistas y entrada en `SECTION_CONTEXT`, pero al no estar en `NAV_BY_KEY`, `/s/automatizaciones/` responde 404 mientras tanto.

1.4 Plantilla base, estáticos y vstatic

`templates/base.html` es el documento HTML completo del shell (`login.html` es un documento aparte que no pasa por él). Carga `{% load assets %}`, enlaza 14 hojas de estilo, una por módulo, todas a través de `{% vstatic %}`, y carga `htmx` y `shell.js` con `defer`. El elemento `<main class="content" id="content" hx-history-elt>` es el destino de todos los swaps de sección y, por `hx-history-elt`, lo que `htmx` guarda en su caché de historial para restaurar en `back/forward`. `partials/sidebar.html` renderiza el rail como `<aside class="sidebar" data-nav-group>` con la marca enlazada a `home`, dos `<nav>` que iteran `primary_nav` y `secondary_nav` e incluyen `nav_item.html`, y un enlace de cierre de sesión.

`{% vstatic %}` vive en `core/templatetags/assets.py` y existe, según su `docstring`, porque el build de Django en Vercel ejecuta `collectstatic` sin nombres hasheados: `{% static %}` emite una ruta estable y el navegador y el CDN siguen sirviendo la copia antigua después de un despliegue. El tag añade `?v=<hash>` con los 10 primeros caracteres hexadecimales del SHA-256 del contenido del archivo, localizado con `finders.find`. Fuera de `DEBUG` el hash se calcula una vez por proceso y se guarda en el diccionario de módulo `_hashes`, incluidos los fallos como cadena vacía, porque la respuesta no puede cambiar sin redespigüe; en desarrollo se recalcula en cada render para que una edición se vea en la siguiente recarga.

Archivo	Función en el shell
<code>config/settings.py</code>	TEMPLATES con <code>DIRS = BASE_DIR / 'templates'</code> ; <code>STATIC_URL 'static/'</code> , <code>STATICFILES_DIRS</code> y <code>STATIC_ROOT = BASE_DIR / 'staticfiles'</code> ; <code>MEDIA_URL 'media/'</code> ; <code>middleware core.middleware.LoginRequiredMiddleware</code>
<code>config/urls.py</code>	<code>admin/</code> , <code>webhooks/messaging/</code> (<code>messaging.urls</code>), raíz (<code>core.urls</code>), <code>media</code> en <code>DEBUG</code>
<code>core/urls.py</code>	<code>home</code> , <code>login</code> , <code>logout</code> , <code>privacy</code> , <code>data_deletion</code> , <code>section</code> y los endpoints de panel y columna de cada pantalla
<code>core/nav.py</code>	<code>NavItem</code> , <code>PRIMARY_NAV</code> , <code>SECONDARY_NAV</code> , <code>ALL_NAV</code> , <code>NAV_BY_KEY</code> , <code>DEFAULT_SECTION</code> , <code>WELCOME_SHORTCUTS</code>
<code>core/views.py</code>	<code>PLACEHOLDER_TEMPLATE</code> , <code>_section_template</code> , <code>_is_htmx</code> , <code>SECTION_CONTEXT</code> , <code>welcome</code> , <code>section</code> , <code>_mensajería_page</code>
<code>core/templatetags/assets.py</code>	tag <code>vstatic</code> y caché <code>_hashes</code>
<code>templates/base.html</code>	documento completo: hojas de estilo, <code>htmx</code> , <code><main id="content"></code> , <code>shell.js</code>

Archivo	Función en el shell
templates/partials/sidebar.html, nav_item.html	rail de iconos y cada enlace con sus atributos hx-* y aria
templates/partials/account_nav_panel.html	nav secundaria Mi cuenta compartida por CRM y Campañas
templates/partials/side_nav_section.html, side_nav_rows.html	componente de grupo colapsable y bucle de filas, usados por account_nav_panel y comercio
templates/partials/footer.html	barra inferior y controles de ayuda compartidos por Inbox y bienvenida
templates/sections/welcome.html, _placeholder.html	pantalla de bienvenida y fallback de sección
static/js/shell.js	estado activo, historial, título, dialogs y otras piezas delegadas
static/css/sidebar.css, welcome.css, modal.css	tokens de diseño y estilos del rail, la bienvenida y el <dialog> compartido

static/css/sidebar.css define los tokens que el resto de hojas reutiliza: --sidebar-w: 64px para el rail solo de iconos, --nav-hit: 40px como área mínima de clic con un glifo de 21px, el acento --accent: #2f80ed y --muted: #65768d, que el comentario justifica por su contraste 4.6:1 sobre blanco. También reafirma [hidden] { display: none !important; } porque cualquier display de autor gana a la regla del navegador y silenciosamente rompería cada el.hidden = true de la aplicación, y atenúa #content.htmx-request a opacidad 0.55 mientras una petición está en vuelo. modal.css es la piel del <dialog> compartido (hoy el selector de plantillas) y remite a shell.js para el comportamiento; .tag-dialog en tags.css es anterior y conserva su propia piel a propósito.

1.5 shell.js: navegación, historial y diálogos

static/js/shell.js es mejora progresiva: sin él cada elemento de nav sigue siendo un enlace normal y el servidor renderiza el estado activo correcto en una carga completa. Su cabecera explica el problema que resuelve: htmx solo reemplaza parte de la página, así que cualquier nav que quede fuera de la región intercambiada (el rail, siempre fuera de #content; el panel de filtros del Inbox, dentro de #content pero cuya columna 3 se intercambia sola) debe mover su propio estado. En lugar de tratar cada caso, todo contenedor [data-nav-group] obtiene el comportamiento: al hacer clic en un [data-nav-item] interior se le pasa la clase is-active y aria-current (page dentro de .sidebar, true en el resto), como retroalimentación inmediata sin esperar al fragmento. Todos los listeners se delegan desde document y los nodos se buscan bajo demanda, porque un nodo cacheado queda obsoleto en cuanto un swap re-renderiza su subárbol.

El historial se sincroniza en popstate y htmx:historyRestore con syncSidebarFromUrl, que busca .nav-item[href=<pathname>] en el rail; si no hay coincidencia (la raíz /) limpia el rail explícitamente, porque setActive(null) es un no-op y el caso vacío no se resolvería solo. Las tarjetas de la bienvenida están fuera del rail, así que llevan data-nav-for=<url> con el mismo href que el icono correspondiente, y shell.js lo usa para encender ese icono en una navegación que el rail no originó. htmx:historyRestore no se limita al rail: reejecuta también syncTitle, hideDismissed(document) y syncTemplateEditor, porque back/forward restaura un snapshot sin disparar htmx:afterSwap.

document.title vive en <head>, fuera de cualquier swap, y se reconstruye tras htmx:afterSwap a partir de #content .page-head__title más el sufijo · MVP CRM, por lo que un fragmento debe viajar con su propio encabezado. La bienvenida es la excepción: su título es un h1 con clase welcome__title y la pantalla no emite ningún page-head__title, así que syncTitle no encuentra encabezado y deja el título anterior al volver a / por htmx. En una carga completa el problema no aparece, porque base.html toma el título de page_title, que welcome() fija en "Bienvenido".

Los <dialog> se manejan con atributos de datos genéricos, y showModal() aporta de forma nativa el foco atrapado, la tecla Escape y la devolución del foco. La detección del clic en el backdrop exige que el

pointerdown haya empezado fuera de la caja del diálogo y que `event.detail === 1`: un arrastre que empieza en el contenido y termina fuera también reporta el diálogo como objetivo, y el segundo clic de un doble clic sobre el botón que abre aterriza en el backdrop del diálogo recién abierto (los clics sintetizados por teclado reportan `detail 0`). Un swap que hace desaparecer el diálogo que lo originó deja el foco en `<body>`, así que `shell.js` lo mueve al primer campo inválido o al primer `h1/h2` del contenido nuevo.

- `[data-dialog-open=<id>]` abre el `<dialog>` con ese id (no llama a `showModal()` si ya está abierto, porque lanzaría).
- `[data-dialog-close]` cierra el diálogo que lo contiene.
- `dialog[data-dialog-backdrop-close]` se cierra además al hacer clic en el backdrop.
- `[data-close-on-success]` cierra su diálogo cuando la petición `htmx` termina con éxito; si es un `<form>`, además lo resetea.
- `[data-dialog-dismiss]` en la respuesta indica que el servidor terminó con el modal (lo usa el CRUD de Clientes junto a un swap out-of-band de la tabla); es dirigido por la respuesta porque solo el servidor sabe si el envío fue aceptado.

El resto del archivo son piezas de otras pantallas que conviven aquí por la misma razón de delegación: banners descartables persistidos en `localStorage` con el prefijo `dismissed:` y reaplicados tras cada swap, la posición de scroll del hilo `#chat-messages`, la multiselección de conversaciones del Inbox (la casilla `[data-select-all]` marca las `.conv-item__check`, y `refreshSelection` recalcula la clase `has-selection` de `#bulk-form` y el chip `[data-selection-count]` tras cada swap de `#conv-list`, porque la lista vuelve del servidor sin marcar), los `details[data-dropdown]` que se cierran con clic fuera o Escape, el editor de plantillas `[data-plantilla-form]`, la vista previa del modal de etiquetas y el selector de respuestas rápidas. Se documentan en los capítulos de sus pantallas.

1.6 Convenciones que se repiten en todos los módulos

Todas las pantallas con nav secundaria siguen la misma forma, que `core/crm.py` describe como la de `core.nav` y `core.inbox`: declarar las filas una vez y dejar que las plantillas iteren. Un módulo de datos por pantalla (`core/crm.py`, `core/embudos.py`, `core/automatizaciones.py`, `core/comercio.py`, `core/estadisticas.py`, `core/mensajería.py`) declara sus vistas como `dataclasses` y expone `VIEW_BY_KEY`, `DEFAULT_VIEW`, `PLACEHOLDER_PANEL` y una función `panel_template(view_key)` que replica `_section_template` a nivel de panel: devuelve `partials/<pantalla>/panels/<view_key>.html` si existe o el placeholder del módulo. En `core/views.py` cada pantalla tiene un constructor `<pantalla>_context(request)` registrado en `SECTION_CONTEXT`, que lee `?view=` y cae a `DEFAULT_VIEW` en lugar de responder 404 para que un marcador antiguo siga abriendo, y un diccionario de contexto por panel (`PANEL_CONTEXT` para el CRM, `EMBUDOS_PANEL_CONTEXT`, `AUTOM_PANEL_CONTEXT`, `COMERCIO_PANEL_CONTEXT`, `STATS_PANEL_CONTEXT`, `MENSAJERIA_PANEL_CONTEXT`) que asocia clave de vista con un constructor de datos; los paneles sin datos no necesitan entrada.

La plantilla de sección es un fragmento de dos columnas: la nav secundaria (columna 2) y un `<main>` con id propio (columna 3) que incluye `panel_template`. Cada fila de la nav apunta con `href` a `/s/<section_key>/?view=<key>` pero con `hx-get` al endpoint de panel, `hx-target` a la columna 3 y `hx-push-url` a la URL recargable; así elegir una página re-renderiza solo el panel y la nav conserva su estado plegado o desplegado. Cada fila lleva además `data-nav-item`, y el contenedor `.side-nav__scroll` lleva `data-nav-group`, que es lo que acota el pintado de la píldora activa de `shell.js` a esa lista.

Ese patrón no está centralizado en una sola plantilla. Solo el CRM y Campañas, a través de `partials/account_nav_panel.html`, y Mi comercio, en `partials/comercio/nav_panel.html`, incluyen el componente compartido `partials/side_nav_section.html`, que agrupa las filas en `<details>/<summary>` nativos y delega el bucle de enlaces en `partials/side_nav_rows.html`.

Embudos, Estadísticas, Configuración de mensajería y Automatizaciones tienen cada uno su propio `partials/<pantalla>/nav_panel.html` con las filas escritas en línea: mismos atributos `hx-*` y mismas clases `.side-nav`, pero sin compartir esas dos plantillas. El comentario de `side_nav_section.html` justifica el `<details>` nativo frente a JavaScript: alterna, es operable con teclado y anuncia su estado expandido a los lectores de pantalla sin coste, cada incluye es su propio `<details>` para que las secciones se plieguen de forma independiente, y todas arrancan cerradas.

Nota. `side_nav_section.html` acepta un parámetro `flat` que renderiza el mismo título y las mismas filas como grupo plano siempre abierto, pero hoy ninguna plantilla del proyecto lo pasa. Campañas monta el mismo `account_nav_panel.html` que el CRM sin `flat`, es decir con los desplegables colapsables; el comentario de `side_nav_section.html` que dice que Campañas usa la variante plana está obsoleto.

Campañas y CRM comparten `core.crm`, sus secciones, sus paneles y el endpoint `crm_panel`; solo difieren el `section_key` que se empuja en las URLs y el id de la columna 3 (`#crm-panel` frente a `#campanas-panel`), de modo que un panel construido una vez se enciende bajo los dos puntos de entrada del rail. `partials/account_nav_panel.html` no nombra ninguna pantalla concreta: una tercera sección que quisiera el mismo panel solo tendría que incluirlo con sus propios parámetros de montaje.

Pantalla	Endpoint de panel (nombre de URL)	Ruta	Destino del swap
CRM y Campañas	<code>crm_panel</code>	<code>crm/panel/<slug:view_key>/</code>	<code>#crm-panel (CRM) y #campanas-panel</code>
Embudos	<code>embudos_panel</code>	<code>embudos/panel/<slug:view_key>/</code>	<code>#embudos-panel</code>
Automatizaciones	<code>automatizaciones_panel</code>	<code>automatizaciones/panel/<slug:view_key>/</code>	<code>#autom-panel</code>
Mi comercio	<code>comercio_panel</code>	<code>comercio/panel/<slug:view_key>/</code>	<code>#comercio-panel</code>
Estadísticas	<code>estadisticas_panel</code>	<code>estadisticas/panel/<slug:view_key>/</code>	<code>#estadisticas-panel</code>
Configuración de mensajería	<code>mensajeria_panel</code>	<code>mensajeria/panel/<slug:view_key>/</code>	<code>#mensajeria-panel</code>

Cada vista de panel valida `view_key` contra `VIEW_BY_KEY` (404 si no existe) y devuelve solo la columna 3.

1.7 Decisiones de diseño y trampas conocidas

Nota. La raíz `/` no es una sección a propósito. `welcome()` no pasa por `section()` y ningún icono apunta a `/`, porque `shell.js` enciende `.nav-item[href="<pathname>"]` y un icono con ese href se seleccionaría solo en la bienvenida; `WelcomeScreenTests.test_root_matches_no_sidebar_href` guarda esa regresión.

El enlace de cerrar sesión en `sidebar.html` es un enlace normal sin `hx-get`: `logout` redirige a `login.html`, un documento independiente que un swap de `#content` no podría renderizar. `LoginRequiredMiddleware` no usa `django.contrib.auth` como puerta; comprueba la marca `app_authenticated (SESSION_KEY)` en la sesión y redirige a `/login/` en caso contrario, exceptuando `/login/`, `/privacidad/`, `/eliminacion-de-datos/` y los prefijos `/webhooks/`, `/static/`, `/media/` y `/admin/`. El parámetro `?next=<path>` solo se añade cuando `request.path` no es `/`: una petición no autenticada a la raíz acaba en `/login/` a secas. `settings.TESTING` (verdadero cuando `test` aparece en `sys.argv`) desactiva la puerta para que el cliente de pruebas, que nunca inicia sesión, no quede bloqueado; `login_view` valida `?next` con `url_has_allowed_host_and_scheme` para evitar el open redirect clásico tras el login.

`login_view` hace algo más que marcar la sesión. Autentica contra `core.agents`, que lee las credenciales del entorno, y con el agente válido abre una sesión real de `django.contrib.auth` sobre el user espejo de ese agente. El docstring explica el porqué: eso convierte al agente en una identidad y no en un booleano, de modo que "Tu inbox" puede filtrar por `assigned_to=request.user`, los mensajes salientes registran

quién los escribió y el desplegable de asignación del Inbox tiene un valor por defecto sensato. La llamada a `auth_login` nombra el backend `django.contrib.auth.backends.ModelBackend` de forma explícita porque el User espejo tiene una contraseña inutilizable y ningún backend podría verificarla; justo después se vuelve a escribir `SESSION_KEY` en la sesión, ya que `auth_login` cicla la clave de sesión. `logout_view` llama a `auth_logout` y luego a `request.session.flush()`.

`config/settings.py` carga `BASE_DIR/.env` con `load_dotenv` antes de leer nada, sin sobrescribir variables ya presentes, de modo que las de Vercel ganen en producción. `DEBUG` vale `True` salvo que la variable diga otra cosa; `ALLOWED_HOSTS` parte de una lista vacía a la que se añaden `VERCEL_URL` y `VERCEL_PROJECT_PRODUCTION_URL` si existen, más dos alias fijos del proyecto solo cuando `VERCEL` está definida, porque en local esas entradas anularían el fallback de localhost de Django. De esa lista se deriva `CSRF_TRUSTED_ORIGINS` como `https://<host>` por cada entrada. `SECURE_PROXY_SSL_HEADER` confía en `X-Forwarded-Proto` porque Vercel termina TLS delante de la aplicación. Cuando `BLOB_READ_WRITE_TOKEN` está presente y no se está en tests, `STORAGES` cambia el backend por defecto a `core.storage.VercelBlobStorage`, porque el filesystem de las funciones es efímero.

Variable	Valor por defecto	Efecto en el shell
<code>SECRET_KEY</code>	clave de desarrollo insegura	firma de sesiones; obligatoria en despliegue
<code>DEBUG</code>	'True'	sirve media en desarrollo; <code>vstatic</code> recalcula hashes por render
<code>ALLOWED_HOSTS</code>	" (lista vacía)	dominios propios; los de Vercel se añaden solos y <code>CSRF_TRUSTED_ORIGINS</code> se deriva de la lista
<code>APP_AGENTS</code>	"	lista de agentes que pueden pasar la puerta de entrada
<code>APP_LOGIN_USERNAME / APP_LOGIN_PASSWORD</code>	"	par antiguo, tratado como lista de un solo agente
<code>DATABASE_URL</code>	no definida (SQLite)	Postgres en producción con <code>conn_max_age=600</code> ; ignorada en tests
<code>BLOB_READ_WRITE_TOKEN</code>	no definida	activa <code>VercelBlobStorage</code> para uploads; ignorada en tests
<code>MESSAGING_PROVIDER</code>	'fake'	proveedor de mensajería; bajo <code>TESTING</code> se fuerza a 'fake' pase lo que pase

Dos trampas documentadas en el propio código. El docstring de `core/views.py` sigue diciendo que `section()` sirve 14 destinos del rail, pero tras la retirada de tres entradas `core/nav.py` registra 11 y `NavDefinitionTests` fija 8 más 3. Y los comentarios `{# ... #}` de Django son de una sola línea: uno de varias líneas no es comentario y se filtra como texto visible, algo que según `TemplateCommentLeakTests` ya se ha desplegado dos veces, por lo que hay una prueba que renderiza cada pantalla y comprueba que no aparecen `{# ni #}`.

1.8 Pruebas que lo cubren

`core/tests.py` es el módulo del shell: su docstring lo resume como pruebas de enrutado, estado activo y fragmentos htmx. La cabecera `HTMX = {"HX-Request": "true"}` simula las peticiones de htmx, y el helper `a_placeholder_section()` busca en tiempo de ejecución una clave que todavía caiga en `PLACEHOLDER_TEMPLATE` en vez de fijarla, porque una clave fija empezaría a probar lo equivocado a medida que se construyen pantallas; si todas tuvieran plantilla, lanza un `AssertionError` pidiendo reescribir la prueba. Las clases de pantalla del Inbox que viven en el mismo archivo (`InboxScreenTests`, `InboxListEndpointTests`) pertenecen al capítulo del Inbox.

Módulo	Clase	Qué comprueba
core/tests.py	NavDefinitionTests	8 primarias y 3 secundarias, claves únicas, solo academy con badge, todo icono SVG existe
core/tests.py	RoutingTests	/ renderiza la bienvenida con active_key None; toda clave de ALL_NAV responde 200 con su active_key; slug desconocido da 404
core/tests.py	SidebarRenderTests	aria-label y tooltip por icono, una sola pill activa con aria-current, un solo badge, cada icono es un href real
core/tests.py	HtmxSwapTests	con HX-Request no hay <html> ni sidebar pero sí page-head__title; sin ella el documento completo; el fragmento está contenido en la página completa
core/tests.py	SectionTemplateTests	inbox usa sections/inbox.html; una sección sin plantilla usa _placeholder.html y muestra próximamente
core/tests.py	WelcomeScreenTests	ningún icono activo, ningún href a /, título Bienvenido · MVP CRM, tarjetas con data-nav-for, fragmento htmx sin sidebar
core/tests.py	TemplateCommentLeakTests	ninguna sección ni la bienvenida filtra {# o #}
core/tests_auth.py	LoginGateTests	la puerta de entrada real, reactivando con override_settings lo que TESTING desactiva; la raíz redirige sin ?next y una sección con él
core/tests_campanas.py	CampanasScreenTests	Campañas monta las mismas secciones, vistas y endpoint crm_panel que el CRM
core/tests_crm.py, core/tests_embudos.py	CrmPanelEndpointTests, EmbudosPanelEndpointTests	los endpoints de columna 3 devuelven solo el panel
core/tests_legal.py	LegalPagesTests	las páginas legales son accesibles sin sesión

```
python manage.py test core.tests
python manage.py test core.tests.HtmxSwapTests
```

Archivos de referencia: README.md, config/settings.py, config/urls.py, core/urls.py, core/nav.py, core/views.py, core/middleware.py, core/agents.py, core/crm.py, core/embudos.py, core/comercio.py, core/mensajeria.py, core/templatetags/assets.py, core/tests.py, core/tests_auth.py, core/tests_campanas.py, templates/base.html, templates/partials/sidebar.html, templates/partials/nav_item.html, templates/partials/account_nav_panel.html, templates/partials/side_nav_section.html, templates/partials/side_nav_rows.html, templates/partials/comercio/nav_panel.html, templates/partials/embudos/nav_panel.html, templates/partials/estadisticas/nav_panel.html, templates/partials/mensajeria/nav_panel.html, templates/partials/automatizaciones/nav_panel.html, templates/partials/footer.html, templates/sections/welcome.html, templates/sections/_placeholder.html, templates/sections/crm.html, templates/sections/campanas.html, templates/sections/embudos.html, static/js/shell.js, static/css/sidebar.css, static/css/modal.css

2. Autenticación y agentes

Cómo se entra en MVP-CRM y quiénes son las personas que la aplicación reconoce: el middleware `LoginRequiredMiddleware` y sus rutas exentas, la variable `APP_AGENTS` como única fuente de verdad de logins y asignatarios (con fallback al par heredado `APP_LOGIN_USERNAME/APP_LOGIN_PASSWORD`), los usuarios espejo de `django.contrib.auth` con contraseña inutilizable, las vistas `login_view` y `logout_view`, la segunda noción de agente que usan los desplegables de Estadísticas y Mi calendario, y los tests de `core/tests_auth.py` y `core/tests_agents.py` que lo cubren.

2.1 Propósito y alcance

Este capítulo describe cómo se entra en MVP-CRM y quiénes son las personas que la aplicación reconoce. No hay registro, recuperación de contraseña ni pantalla de gestión de usuarios: la lista de credenciales vive en la variable de entorno `APP_AGENTS`, un middleware exige que la sesión lleve una marca de autenticación y, al iniciar sesión, la app abre además una sesión real de `django.contrib.auth` contra un usuario espejo de ese agente. Con eso un agente es a la vez un login y un asignatario: la misma identidad que pasa la puerta es la que puede figurar como responsable de una conversación en el Inbox, la que filtra la vista Tu inbox y la que queda registrada en los mensajes salientes.

- `core/middleware.py`: `LoginRequiredMiddleware`, la puerta que redirige a `/login/` y la lista de rutas exentas.
- `core/agents.py`: parseo de `APP_AGENTS`, autenticación en tiempo constante y usuarios espejo.
- `core/views.py`: `login_view` y `logout_view`, y los desplegables que usan otra noción de agente.
- `templates/login.html` y `static/css/login.css`: la pantalla de acceso.
- `config/settings.py`, `config/urls.py`, `core/urls.py` y `.env.example`: lectura de variables, montaje de rutas exentas y documentación de configuración.

2.2 Flujo de una petición: el middleware y sus exenciones

`LoginRequiredMiddleware` se registra en `MIDDLEWARE` justo después de `django.contrib.auth.middleware.AuthenticationMiddleware` y antes de `MessageMiddleware`, de modo que cuando actúa ya existen `request.session` y `request.user`. En cada petición deja pasar si se cumple cualquiera de tres condiciones: `settings.TESTING` es verdadero, la sesión contiene la clave `SESSION_KEY` (cuyo valor es `app_authenticated`) o la ruta está exenta. En caso contrario responde con una redirección a `reverse('login')`; si la ruta pedida no es `/`, añade `?next=` con `request.path` para volver allí tras autenticarse. Solo viaja la ruta, no la query string, y el valor se concatena sin codificar.

Las exenciones se declaran en dos constantes del módulo. `EXEMPT_PATHS` es un conjunto de rutas exactas y `EXEMPT_PREFIXES` una tupla de prefijos comprobados con `startswith` en `_is_exempt`. Cada exención tiene un motivo recogido en el docstring del módulo o en los comentarios de `core/urls.py`.

- `/login/`: la propia pantalla de acceso.
- `/privacidad/` y `/eliminacion-de-datos/`: páginas legales públicas a propósito, porque la revisión de apps de Meta las lee sin cuenta y no publica una app cuya política de privacidad exija login.
- `/webhooks/`: prefijo del que cuelgan los webhooks de proveedor, montados en `config/urls.py` como `path("webhooks/messaging/", include("messaging.urls"))`, es decir bajo `/webhooks/messaging/<provider>/`; se autentican con su propia comprobación de firma, no con sesión de navegador.
- `/static/` y `/media/`: activos estáticos y archivos subidos. `/media/` solo se sirve realmente porque `config/urls.py` añade `static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)`, que es un no-op con `DEBUG` apagado, tal como dice su comentario.

- `/admin/`: el admin de Django, protegido por separado por `django.contrib.auth`.

Nota. `TESTING` se define en `config/settings.py` como `'test' in sys.argv`; con ese valor el middleware es un no-op para que el cliente de pruebas, que nunca inicia sesión, no quede fuera. Los tests que quieren ejercitar la puerta real lo desactivan con `override_settings(TESTING=False)`.

El middleware no distingue peticiones HTMX de navegaciones completas: los `hx-get` de los paneles y los polls del Inbox reciben exactamente la misma redirección a `/login/?next=...`. Como HTMX sigue la redirección y coloca la respuesta en el destino del intercambio, una sesión caducada acaba inyectando la página de login dentro de un panel en lugar de recargar la pantalla entera. No hay ninguna cabecera de respuesta (`HX-Redirect` o similar) que corrija ese caso.

2.3 APP_AGENTS: quién entra y a quién se asigna

`core/agents.py` es la fuente de verdad de quién puede entrar. `APP_AGENTS` es una lista separada por comas de entradas `username:password:Nombre`; el nombre visible es opcional y, si falta, se usa el nombre de usuario. Las contraseñas no pueden contener `:` ni `,`, porque son los separadores. `config/settings.py` la lee con `os.environ.get('APP_AGENTS', '')` y `.env.example` la documenta con un ejemplo comentado; el bloque de comentario de `config/settings.py` usa otro ejemplo, `APP_AGENTS=Admin:sup3rsecret:Admin, Samuel:1234:Samuel`.

```
# App-wide login gate (core/middleware.py) and the Inbox's list of assignable
# agents -- one list for both (core/agents.py). Comma-separated
# `username:password:Nombre` entries; the display name is optional.
#APP_AGENTS=Admin:cambia-esta-clave:Admin, Samuel:1234:Samuel

# Legacy single pair, still honored when APP_AGENTS is unset (it becomes the
# one agent). Prefer APP_AGENTS above.
#APP_LOGIN_USERNAME=
#APP_LOGIN_PASSWORD=
```

Nota. En `.env.example` las tres líneas están comentadas: el archivo es plantilla, no configuración activa. Además, su comentario dice que el par heredado se honra «when `APP_AGENTS` is unset», redacción más estrecha que el código: `configured_agents()` cae al par siempre que el parseo no produzca ningún agente válido, aunque `APP_AGENTS` esté definida con basura.

`configured_agents()` convierte esa cadena en una lista de `Agent`, un `dataclass` congelado con `username`, `password` y `display_name`. Se lee en cada llamada y no al importar el módulo, para que `override_settings` en los tests y un cambio de variable tras un redesplicue surtan efecto. La configuración se toma con `getattr(settings, "APP_AGENTS", "")` or `""` y el mismo patrón para el par heredado, así que un entorno donde falten esos settings como atributos no revienta: simplemente no hay agentes. Las reglas de parseo son deliberadamente tolerantes:

- Cada entrada se recorta de espacios y se divide con `split(":", 2)`, así que solo las dos primeras partes son obligatorias y el nombre visible puede llevar dos puntos.
- Cada parte resultante se vuelve a recortar (`parts = [part.strip() for part in entry.split(":", 2)]`), de modo que los espacios iniciales y finales de usuario, contraseña y nombre visible se pierden: una contraseña puede llevar espacios interiores (una clave larga funciona, y hay test para ello), pero `" 1234 "` se guardaría como `1234`.
- Las entradas sin dos puntos, con usuario vacío o con contraseña vacía se ignoran: según el docstring, un error tipográfico debe costar el acceso a ese agente, no bloquear a todo el equipo.
- Un nombre de usuario repetido conserva la primera aparición.
- Si tras el parseo no queda ningún agente, se usa el par heredado `APP_LOGIN_USERNAME/APP_LOGIN_PASSWORD` como lista de un único agente; si tampoco está definido, la

lista queda vacía.

`authenticate(username, password)` devuelve el Agent cuyas credenciales coinciden o `None`. Compara contra todos los agentes configurados sin salir del bucle en la primera coincidencia y usa `hmac.compare_digest` para usuario y contraseña, combinados con `&` en lugar de `and`, de manera que el tiempo de respuesta no revele qué nombres de usuario existen.

Nota. `hmac.compare_digest` sobre cadenas solo admite caracteres ASCII: un usuario o una contraseña con tildes o ñ, ya venga de `APP_AGENTS` o tecleado en el formulario, hace que `authenticate` lance `TypeError` en lugar de devolver `None`, y la petición termina en error 500 en vez de en la pantalla de credenciales incorrectas.

Nota. Una `APP_AGENTS` en blanco sin par heredado significa que nadie puede entrar, no que la puerta esté desactivada; así lo advierten tanto el comentario de `config/settings.py` como `.env.example`.

2.4 Usuarios espejo de `django.contrib.auth`

`Conversation.assigned_to` es una clave foránea a `AUTH_USER_MODEL`, y los mensajes salientes registran en `sent_by` quién los escribió, así que cada agente del entorno necesita una fila real de `User` a la que apuntar. Esas filas son espejos: `_mirror(username, display_name)` hace `get_or_create` por `username`, rellena `first_name` con el nombre visible truncado a 150 caracteres y, solo cuando la fila es nueva, llama a `set_unusable_password()` y guarda el campo `password`. La propiedad `Agent.user` devuelve ese espejo, creándolo si falta.

`agent_users()` devuelve las filas `User` de todos los agentes en el orden del entorno y es lo que rellena el desplegable de asignación del Inbox, por lo que debe listar a compañeros que aún no han iniciado sesión. En régimen estable es un único `SELECT` con `username__in`; solo escribe la primera vez que un agente aparece en la lista. `assignment_options(conversation)` parte de esa lista y añade al `assigned_to` actual cuando ya no es un agente configurado, porque, como explica su docstring, un `<select>` sin opción coincidente mostraría su primera entrada y afirmarí en silencio que la conversación está Sin asignar.

Nota. Consecuencia práctica: `agent_users()` puede escribir en la base de datos durante una petición de lectura. `core/views.py` llama a `agents.assignment_options(conversation)` al construir el contexto del chat, de modo que un `GET` a `inbox_chat` crea filas `User` como efecto secundario si algún agente del entorno todavía no tenía espejo.

Nota. Los espejos existen para ser referenciados, nunca para autenticar: con contraseña inutilizable ningún backend puede verificarlos y el entorno sigue siendo la única vía de entrada.

2.5 `login_view` y `logout_view`

`login_view` está registrada en `core/urls.py` como `login/` con nombre `login`. Calcula `next_url` a partir de `?next` en la query string, del campo oculto `next` del POST o de `reverse('home')`, y lo valida con `url_has_allowed_host_and_scheme(next_url, allowed_hosts={request.get_host()})`; si no pasa, vuelve a `reverse('home')`. El comentario del código lo justifica: `?next` es alcanzable por un atacante y, sin la comprobación, un `next=https://evil.example` redirigiría un login correcto fuera del sitio, el clásico open redirect post-login.

En un POST la vista llama a `agents.authenticate` con los campos `username` y `password`. Si hay coincidencia, invoca `auth_login(request, agent.user, backend='django.contrib.auth.backends.ModelBackend')`: el backend se nombra explícitamente porque el espejo tiene contraseña inutilizable y `django.contrib.auth.authenticate()` fallaría, correctamente, al verificarla. Acto seguido escribe `request.session[SESSION_KEY] = True`, y el orden

importa: `auth_login` rota la clave de sesión y la marca de la puerta debe sobrevivir en la nueva sesión. Después redirige a `next_url`. Si las credenciales no coinciden, el error es `Usuario o contraseña incorrectos`, y se vuelve a renderizar el formulario con `render_to_string('login.html', ...)` envuelto en `HttpResponse`. No hay atajo para sesiones ya autenticadas: un GET a `/login/` siempre muestra el formulario.

Según su docstring, esa sesión real de `django.contrib.auth` es lo que convierte al agente en una identidad y no en un booleano: Tu inbox puede filtrar por `assigned_to=request.user`, los mensajes salientes registran quién los escribió y el desplegable de asignación tiene un valor por defecto sensato.

`templates/login.html` es una página independiente del shell: carga `css/sidebar.css` y `css/login.css` con la etiqueta `vstatic` de la librería `assets`, incluye `icons/logo.svg` y envía el formulario por POST a `{% url 'login' %}` con `csrf_token`, el campo oculto `next` y los inputs `username` y `password` con `autocomplete` de `username` y `current-password`. El error se muestra en un párrafo con `role="alert"`. `static/css/login.css` reutiliza los tokens de `sidebar.css` (`--panel-bg`, `--sidebar-border`, `--accent`, `--muted`) y define una tarjeta centrada de 340 px de ancho máximo con las clases `login-card` y `login-form`.

`logout_view`, registrada como `logout/` con nombre `logout`, llama a `auth_logout(request)`, hace `request.session.flush()` y redirige a `login`. No restringe el método HTTP; los tests la invocan con GET. Como `/logout/` no está entre las rutas exentas, una petición sin sesión a esa ruta acaba en `/login/?next=/logout/`.

2.6 La otra noción de agente: desplegables por usuario activo

No todos los desplegables de personas se alimentan de `APP_AGENTS`. El panel de filtros de Estadísticas mayor que Tiempos de Respuesta rellena su lista con `get_user_model().objects.filter(is_active=True).order_by("username")` en `_tiempos_context`, y el modal de eventos de Mi calendario hace lo mismo para su clave `advisors` en `_mi_calendario_context`. Ahí aparecen, por tanto, usuarios que no son agentes configurados: el asesor que crea el comando `seed_conversations` de `messaging`, o cualquiera dado de alta desde `/admin/`.

Nota. Conviene tener presente la asimetría al añadir pantallas: el Inbox usa `agents.agent_users()` y `agents.assignment_options()`, que se ciñen al entorno, mientras que Estadísticas y Mi calendario listan todos los usuarios activos de la base de datos. Las dos listas pueden no coincidir.

2.7 Referencia

Nombre	Valor por defecto	Dónde se lee	Descripción
APP_AGENTS	" (cadena vacía)	<code>config/settings.py</code> , <code>core/agents.py</code>	Lista <code>username:password:Nombre</code> separada por comas. En blanco y sin par heredado, nadie entra.
APP_LOGIN_USERNAME	" (cadena vacía)	<code>config/settings.py</code> , <code>core/agents.py</code>	Usuario del par heredado; solo cuenta si <code>APP_AGENTS</code> no aporta ningún agente válido.
APP_LOGIN_PASSWORD	" (cadena vacía)	<code>config/settings.py</code> , <code>core/agents.py</code>	Contraseña del par heredado.
TESTING (setting, no variable)	'test' in <code>sys.argv</code>	<code>config/settings.py</code> , <code>core/middleware.py</code>	Convierte el middleware en un no-op bajo <code>manage.py test</code> .

Variables de entorno y settings derivados de este capítulo

Ruta	Nombre de URL	Vista	Exenta del middleware
/login/	login	login_view	Sí
/logout/	logout	logout_view	No
/privacidad/	privacy	privacy	Sí
/eliminacion-de-datos/	data_deletion	data_deletion	Sí
/webhooks/messaging/<provider>/	messaging_webhook	messaging (config/urls.py)	Sí, por el prefijo /webhooks/

Rutas relacionadas

Función	Devuelve	Notas
configured_agents()	list[Agent]	Relee settings con getattr en cada llamada; fallback al par heredado; entradas malformadas y duplicadas ignoradas.
authenticate(username, password)	Agent o None	hmac.compare_digest en ambos campos, sin salida anticipada del bucle; TypeError si algún valor no es ASCII.
agent_users()	list[User]	Orden del entorno; un SELECT en régimen estable; crea los espejos que falten, también durante un GET.
assignment_options(conversation)	list[User]	agent_users() más el assigned_to actual si ya no es agente.
_mirror(username, display_name)	User	get_or_create por username; set_unusable_password solo al crear.
Agent.user	User	Propiedad; delega en _mirror.

API pública e interna de core/agents.py

Archivo	Papel
core/middleware.py	LoginRequiredMiddleware, SESSION_KEY, EXEMPT_PATHS, EXEMPT_PREFIXES
core/agents.py	Agent, configured_agents, authenticate, agent_users, assignment_options, _mirror
core/views.py	login_view, logout_view, inbox_assign y los contextos _tiempos_context y _mi_calendario_context
core/urls.py	Rutas login, logout, privacy, data_deletion y home
config/urls.py	Montaje de /admin/, /webhooks/messaging/ y el static() de MEDIA_URL
templates/login.html	Formulario de acceso
static/css/login.css	Estilos de la tarjeta de login sobre los tokens de sidebar.css
config/settings.py	TESTING, APP_AGENTS, APP_LOGIN_USERNAME, APP_LOGIN_PASSWORD, orden de MIDDLEWARE
.env.example	Documentación y ejemplo comentado de las variables
messaging/management/commands/seed_conversations.py	Crea el usuario demo asesor / asesor123, que no es un agente
core/tests_auth.py	Tests de la puerta de entrada
core/tests_agents.py	Tests de agentes, espejos, login y asignación

Archivos del capítulo

2.8 Decisiones de diseño y trampas conocidas

Buena parte de las decisiones de este módulo está explicada en comentarios y docstrings, y conviene conocerlas antes de tocar el código. Las siguientes son las que más afectan a quien mantiene la configuración o amplía el modelo de usuarios.

- Credenciales en el entorno, no en la base de datos: añadir un compañero es editar APP_AGENTS y redespargar; no hay registro, recuperación de contraseña ni pantalla de usuarios que construir.
- Espejos con contraseña inutilizable: nadie puede autenticarse a través del ORM, y por eso login_view debe nombrar el backend en auth_login en lugar de pasar por `django.contrib.auth.authenticate()`.
- El docstring de `core/middleware.py` es anterior a `core/agents.py` y aún describe un único par APP_LOGIN_USERNAME/APP_LOGIN_PASSWORD comprobado en login_view; el comportamiento vigente es el de `core/agents.py`, que solo usa ese par como fallback.
- Renombrar un agente en el entorno no actualiza el first_name de un espejo ya creado, porque display_name solo se aplica como defaults de get_or_create.
- Quitar un agente de APP_AGENTS no borra su fila User: deja de listarse en el desplegable, pero las conversaciones que tenía asignadas conservan su nombre gracias a assignment_options.
- /admin/ está exento del middleware pero lo protege `django.contrib.auth`; como los espejos no tienen contraseña utilizable, un agente no puede entrar al admin con su clave de APP_AGENTS.
- Usuarios creados fuera de la lista (el asesor de seed_conversations, o uno creado desde /admin/) pueden figurar como asignados en el Inbox solo mientras tengan esa conversación, pero sí aparecen siempre en los desplegables de Tiempos de Respuesta y Mi calendario, que listan usuarios activos.

Nota. Todos los agentes tienen las mismas capacidades: no hay roles ni permisos por agente, tal como recoge la sección Agentes del README.md.

2.9 Pruebas que lo cubren

Dos módulos cubren este capítulo. `core/tests_auth.py` ejercita la puerta con `LoginGateTests`, decorada con `override_settings(TESTING=False, APP_AGENTS="", APP_LOGIN_USERNAME="tester", APP_LOGIN_PASSWORD="secret-pw")`. El comentario del módulo aclara que vaciar APP_AGENTS es imprescindible: `core.agents` la prefiere siempre que no esté vacía, así que un `.env local` con agentes haría que las credenciales fijadas nunca se alcanzaran.

- Sin sesión, home redirige a login y una sección redirige a login con `?next=` a esa ruta.
- La página de login carga con estado 200 y contiene Iniciar sesión.
- Credenciales erróneas muestran incorrectos y no dejan SESSION_KEY en la sesión; las correctas redirigen a home y la dejan.
- `?next` seguro se respeta; `?next=https://evil.example/phish` cae a home.
- `logout` elimina la marca y la siguiente petición vuelve a redirigir a login.
- Con APP_LOGIN_USERNAME y APP_LOGIN_PASSWORD vacíos, un POST vacío nunca satisface la puerta.
- El webhook `messaging_webhook` del proveedor fake responde 401 sin firma, nunca una redirección a login.

`core/tests_agents.py` cubre la lista de agentes, el login que respalda y la asignación del Inbox. Fija las credenciales con `override_settings` en lugar de leer el `.env local`, lo que funciona precisamente porque `core.agents` relee settings en cada llamada. La constante `TWO_AGENTS`

(Admin:admin-pw:Admin, Samuel:1234:Samuel) es el entorno de referencia de casi todas las clases.

Clase	Qué comprueba
<code>ConfiguredAgentsTests</code>	Parseo en orden del entorno, nombre visible opcional, espacios ignorados, entradas malformadas omitidas sin error, duplicados conservan la primera, contraseña con espacios interiores, <code>authenticate</code> acierta y no cruza credenciales entre agentes.
<code>LegacyCredentialFallbackTests</code>	El par heredado se convierte en el único agente; APP_AGENTS gana cuando ambos están definidos; sin nada configurado no entra nadie.

Clase	Qué comprueba
AgentUserMirrorTests	Espejos creados bajo demanda en orden, reutilizados sin duplicar, sin contraseña utilizable ni check_password; un agente retirado deja de listarse pero su fila sobrevive; sin agentes no hay opciones.
AgentLoginTests	Con TESTING=False, cada agente entra con su clave y _auth_user_id apunta a su espejo; request.user es ese agente; clave errónea no abre sesión auth; logout la cierra.
ConversationAssignmentTests	inbox_assign asigna, desasigna y reemplaza; rechaza con 400 usuarios que no son agentes y valores no numéricos; GET da 405 y conversación inexistente 404; la respuesta trae el control y la línea Asignada a fuera de banda; el desplegable lista a todos y mantiene a un asignado retirado del entorno.
TuInboxFilterTests	El filtro tu-inbox muestra solo las conversaciones del agente logueado; sin-asignar muestra la que no tiene responsable.

Clases de core/tests_agents.py

```
python manage.py test core.tests_auth core.tests_agents
```

Archivos de referencia: core/middleware.py, core/agents.py, core/views.py, core/urls.py, config/urls.py, config/settings.py, templates/login.html, static/css/login.css, .env.example, README.md, messaging/management/commands/seed_conversations.py, core/tests_auth.py, core/tests_agents.py

3. Inbox: la bandeja unificada

Documenta la sección Inbox de MVP-CRM: los filtros declarados en `core/inbox.py`, la lista de conversaciones con su poll `htmx` y su selección múltiple en `shell.js`, el hilo de chat con refresco cada 5 segundos y burbujas pintadas con `display_body`, la regla de la ventana de 24 horas de WhatsApp en el compositor, el envío, la asignación de agente (incluida la opción Sin asignar) con `swaps` fuera de banda, las etiquetas (`picker`, `pills` removibles, archivadas, acciones en bloque) y las respuestas rápidas, con referencia de endpoints, decisiones de diseño y pruebas.

3.1 Propósito y alcance

El Inbox es la bandeja unificada del CRM: una sola pantalla para leer y responder conversaciones de todos los canales. Se sirve como cualquier sección mediante `section` en `core/views.py`, que registra `_inbox_context` en `SECTION_CONTEXT` bajo la clave `inbox`. `templates/sections/inbox.html` es un fragmento sin `<html>` que `htmx` deposita en `#content` y que ensambla cuatro columnas, cada una en su propio parcial bajo `templates/partials/inbox/`: navegación con filtros, lista de conversaciones, hilo de chat y detalles del cliente, con `partials/footer.html` debajo.

Este capítulo documenta la capa de presentación y sus endpoints parciales. La regla de la ventana de 24 horas, el envío y los servicios de etiquetas viven en `messaging/services.py` y `messaging/models.py`; aquí se describe cómo el Inbox los invoca y qué devuelve a `htmx`. Todo lo dinámico es `htmx` más los ganchos de `static/js/shell.js`; no hay build de frontend.

3.2 Filtros y lista de conversaciones

`core/inbox.py` declara los filtros con dos dataclasses congeladas: `Filter` (`key`, `label`, `icon`, `show_count`, más la propiedad `icon_template`, que resuelve a `icons/<icon>.svg`) y `FilterGroup` (`title`, `items`). `FILTER_GROUPS` reúne las listas `CONVERSATIONS`, `MIA` y `CANALES`; de ahí salen `ALL_FILTERS`, `FILTER_BY_KEY` y `CHANNEL_KEYS`, y `DEFAULT_FILTER` vale todos. El docstring del módulo subraya que son filtros sobre conversaciones y no páginas: elegir uno solo re-renderiza la columna 3.

`get_conversations(filter_key, user=None, tag_ids=None)` hace `select_related` de contacto y agente, `prefetch_related("tags")` y ordena por `F("last_message_at").desc(nulls_last=True)` y después `-id`. `tu-inbox` filtra por `assigned_to=user` y devuelve `queryset.none()` a un usuario anónimo, `sin-asignar` usa `assigned_to__isnull=True`, las claves de canal filtran por `channel`, y los tres filtros `MIA` devuelven `queryset.none()` porque todavía no existe un modelo de detección por IA. `get_counts()` devuelve todas las claves a cero y rellena las de canal con una sola consulta agregada sobre conversaciones con `unread_count__gt=0`.

`nav_panel.html` recorre `filter_groups`: cada fila es un enlace real a `/s/inbox/?filter=<key>` que `htmx` convierte en `hx-get` a `inbox_list` con destino `#conv-list`, `hx-push-url` con esa misma URL y `hx-include="#tag-filter input:checked"` para conservar el filtro de etiquetas al saltar de fila. Los contadores se leen con `counts|dict_get:row.key`. `list_panel.html` envuelve la mitad inferior de la columna en el formulario `#bulk-form`, con el checkbox `data-select-all`, el desplegable `#tag-filter`, el desplegable Acciones y la región intercambiable `#conv-list`. El desplegable de etiquetas se alimenta de `all_tags`, que `_inbox_context` calcula como `Tag.objects.filter(is_archived=False)`.

`conversation_list.html` es la respuesta de `inbox_list` e `inbox_tags_bulk`: un input oculto `filter`, una fila por conversación (checkboxbox `selected`, avatar con punto de no leídos, antigüedad con `relative_es`, agente, marca de canal y pills con `id conv-tags-<pk>`) y un div oculto que actúa de poll. Ese poll hace `hx-get` a `inbox_list` cada 5 segundos, se pausa mientras exista un `details[data-hold-poll][open]` o un `.conv-item__check:checked`, se dispara al instante con `change`

from:#tag-filter y envía en hx-vals el id leído de [data-conversation-id] para que ?active= conserve la fila resaltada. inbox_list responde `Http404` a un filtro desconocido y llama antes a `messaging_services.pump_provider_events()` para que el proveedor falso entregue sus acuses pendientes aunque no haya ningún chat abierto.

La selección múltiple de la lista es el único trozo con JavaScript propio, en `static/js/shell.js`. La función `refreshSelection` cuenta los `.conv-item__check:checked`, alterna la clase `has-selection` en `#bulk-form` para mostrar u ocultar el desplegable Acciones y escribe el número de conversaciones marcadas en el [data-selection-count] de su píldora, desmarcando [data-select-all] cuando no queda ninguna. Un `change` sobre [data-select-all] marca o desmarca todas las filas de golpe. Como cada re-render de la lista llega con los checkboxes vacíos, un listener de `htmx:afterSwap` vuelve a derivar el estado siempre que el destino del swap sea `#conv-list`.

3.3 Hilo, compositor y envío

Pulsar una fila hace `hx-get` a `inbox_chat`, que responde con `chat_swap.html: chat_thread.html` para `#chat-panel` más un `<div id="details-panel" hx-swap-oob="innerHTML">` con `details_content.html`, así que un clic actualiza las columnas 4 y 5 en una sola respuesta. `_thread_context(conversation)` es el contexto común a la carga completa con `?chat=`, al swap y al poll: `active_conversation`, `active_conversation_id`, `chat_messages`, `window_open` (tomado de `conversation.is_within_24h_window`) y `assign_options` (de `agents.assignment_options`). `inbox_chat` y `_inbox_context` llaman a `_mark_read`, que pone `unread_count` a cero y guarda con `update_fields=["unread_count"]`.

Dentro del hilo, `#chat-messages` hace `hx-get` a `inbox_thread` con `hx-trigger="every 5s"` y `hx-swap="innerHTML"`; solo se reemplazan las burbujas, de modo que un borrador a medio escribir en el compositor nunca se pierde. `inbox_thread` también ejecuta `pump_provider_events()` y `_mark_read`, porque un entrante que llega con el hilo a la vista se considera leído al pintarse. `chat_messages.html` alinea entrantes a la izquierda y salientes a la derecha, muestra imágenes en línea cuando `is_inline_image` es verdadero, un enlace de adjunto cuando solo hay `media_url`, la hora con `chat_stamp` y, en salientes, el icono de `status_icon_template`.

El texto de la burbuja no sale de `body` sino de `message.display_body`. Su docstring en `messaging/models.py` explica el porqué: cuando la propia imagen se dibuja en línea, el marcador que el proveedor deja como cuerpo (los valores de `MEDIA_PLACEHOLDERS`, «[imagen]» o «[sticker]») repetiría en texto lo que ya se ve, así que `display_body` devuelve cadena vacía en ese caso concreto y conserva íntegro cualquier pie real. El `body` crudo se mantiene sin tocar precisamente para que la vista previa de la fila en la lista tenga algo que enseñar.

El compositor solo se renderiza si `window_open` es verdadero. `is_within_24h_window` en `messaging/models.py` devuelve falso cuando `last_inbound_at` es nulo o supera `SERVICE_WINDOW` (24 horas): la ventana se abre y se reabre con cada mensaje entrante del cliente. Cerrada, `chat_thread.html` sustituye el formulario por el aviso `composer - -closed`, que enlaza a la sección mensajería para reabrir con una plantilla aprobada. Abierta, el formulario hace `hx-post` a `inbox_send` con destino `#chat-messages` y se resetea con `hx-on::after-request` si la petición tuvo éxito.

`inbox_send` solo acepta POST (`HttpResponseNotAllowed(["POST"])`), recorta `body` y, si no está vacío, llama a `messaging_services.send_message(conversation, body, request.user)`. Captura `SendWindowClosed` (la ventana expiró mientras el hilo estaba abierto) y `SendFailed`, y en ambos casos responde igualmente con `chat_messages.html` más `send_error`, que la plantilla pinta como `msg-error` con `role="alert"`. El docstring insiste en que la regla de 24 horas se aplica en el servicio, no en la vista.

3.4 Asignación, etiquetas y respuestas rápidas

`assign_control.html` es un `<select name="agent">` dentro de un formulario `chat-assign-<pk>` que hace `hx-post` a `inbox_assign` en cada `change`, con `hx-target="this"` y `hx-swap="outerHTML"`; no hay botón Guardar. La primera opción es un `value=""` etiquetado Sin asignar, seleccionado cuando `assigned_to_id` es nulo, y detrás van los usuarios de `assign_options`. `inbox_assign` exige POST y trata el valor vacío como desasignar (`assignee = None`); con un id no vacío busca en `agents.assignment_options(conversation)` y responde 400 si no aparece, porque el desplegable es fijo y otra cosa solo puede ser un POST fabricado. Guarda con `update_fields=["assigned_to"]` solo si el valor cambia y responde con `assign_swap.html`: el control re-renderizado más un `<dd id="details-assigned-<pk>" hx-swap-oob="outerHTML">` para la línea Asignada a del panel de detalles. La fila de la lista no se toca: la recoge el poll de 5 segundos, porque puede no estar en el filtro actual.

Las etiquetas se muestran en tres sitios (`conv-tags-<pk>` en la fila, `chat-tags-<pk>` en la cabecera del chat y `details-tags-<pk>` en detalles, estas últimas con `removable_pills.html`) y se editan en dos. El primero es el picker de `templates/partial/tags/picker.html`, un `<details data-dropdown>` cuyo panel carga `conversation_tags` con `hx-trigger="toggle from:closest details"`. `picker_panel.html` trae un buscador con `input changed delay:250ms`, un checkbox por etiqueta que hace `hx-post` con `tag_id` y `action add o remove`, y un botón Crear cuando `offer_create` es verdadero. En POST, `conversation_tags` crea y aplica (`new_name`, vía `create_tag` y `apply_tag`) o alterna (`remove_tag` o `apply_tag`), captura `TagNameTaken` y `ValueError` como `tag_error`, y responde con el panel y `oob_pills=True`, lo que emite los tres swaps fuera de banda. El segundo es el desplegable Acciones: sus botones hacen `hx-post` a `inbox_tags_bulk` con `tag_id`, `action` y `active`; el formulario envolvente aporta los `selected`, el `filter` oculto y los tags del filtro, y la vista devuelve `conversation_list.html` bajo esos mismos filtros.

El aspa de cada pill removable en el panel de detalles no tiene endpoint propio: hace `hx-post` al mismo `conversation_tags` con `tag_id` y `action remove`, lleva su propio `X-CSRFToken` en `hx-headers` y apunta con `hx-target="next .tag-picker__panel"` al panel del picker que va justo después de las pills. Es decir, la respuesta se deposita en un panel que normalmente está cerrado, y quien refresca de verdad las tres zonas de pills son los swaps fuera de banda que ese mismo `picker_panel.html` emite. El comentario de la plantilla lo dice con todas las letras, incluida la razón del `next`.

El picker y el filtro de la lista trabajan solo con etiquetas vivas: `_picker_context` consulta `Tag.objects.filter(is_archived=False)` y `_inbox_context` hace lo mismo para `all_tags`. Las archivadas sí se siguen pintando en las pills, con la clase extra `tag-pill--archived`, porque el historial de una conversación ya etiquetada no debe desaparecer; simplemente no se pueden aplicar de nuevo. En el buscador, `q` filtra la lista con `name__icontains`, mientras que `offer_create` se calcula con `not Tag.objects.filter(name__iexact=q).exists()`: la oferta de crear aparece siempre que no exista una coincidencia exacta, aunque la búsqueda haya devuelto parecidos.

El botón Respuestas rápidas del compositor es un `<details data-quickreplies>` que hace `hx-get` a `inbox_quick_replies` una sola vez (`hx-trigger="toggle once"`) y lleva el token CSRF en `hx-headers`, porque sus entradas son botones ajenos al formulario. `inbox_quick_replies` lista `MessageTemplate.objects.filter(is_active=True).exclude(status="rechazada").order_by("name")` y calcula por plantilla `body` con `plantillas.render_body`, `needs_input` con `plantillas.VARIABLE_RE.search(body)` y `body_json` con `json.dumps(body, ensure_ascii=False)`. En `quick_replies.html` una entrada sin huecos lleva `data-quick-send` y un `hx-post` directo a `inbox_send` con el cuerpo en `hx-vals`, así que un clic la deposita en el hilo; una con `needs_input` lleva `data-quick-body` y la insignia Completar, y `shell.js` la copia al compositor, coloca el cursor al final y deja que el agente rellene el hueco antes de enviar. Tras un clic en `data-quick-send` ese mismo script solo cierra el popover, porque el envío ya lo disparó `htmx`. Las plantillas con estado pendiente se ofrecen con la insignia Pendiente.

3.5 Referencia

Nombre de URL	Ruta y método	Devuelve	Región que reemplaza
section (clave inbox)	/s/inbox/ GET, con ?filter=, ?chat=y ?tags= repetible	sections/inbox.html, página completa o fragmento htmx	#content
inbox_list	/inbox/list/<slug:filter_key>/ GET, con ?active=y tags	partials/inbox/conversation_list.html	#conv-list (innerHTML)
inbox_chat	/inbox/chat/<int:conversation_id>/ GET	partials/inbox/chat_swap.html	#chat-panel (innerHTML) y #details-panel fuera de banda
inbox_thread	/inbox/chat/<int:conversation_id>/mensajes/ GET	partials/inbox/chat_messages.html	#chat-messages (innerHTML)
inbox_send	/inbox/chat/<int:conversation_id>/enviar/ POST, campo body	partials/inbox/chat_messages.html con send_error opcional	#chat-messages (innerHTML)
inbox_assign	/inbox/chat/<int:conversation_id>/asignar/ POST, campo agent (vacío = sin asignar)	partials/inbox/assign_swap.html	formulario chat-assign-<pk> (outerHTML) y details-assigned-<pk> fuera de banda
inbox_quick_replies	/inbox/chat/<int:conversation_id>/respuestas-rapidas/ GET	partials/inbox/quick_replies.html	[data-quickreplies-pop] (innerHTML)
conversation_tags	/inbox/conversacion/<int:conversation_id>/etiquetas/ GET (?q=) y POST (tag_id + action, o new_name)	partials/tags/picker_panel.html	.tag-picker__panel (innerHTML); en POST además conv-tags-<pk>, chat-tags-<pk> y details-tags-<pk> fuera de banda
inbox_tags_bulk	/inbox/etiquetas/bulk/ POST, campos tag_id, action, selected, filter, tags, active	partials/inbox/conversation_list.html	#conv-list (innerHTML)

Endpoints del Inbox (rutas en core/urls.py)

Grupo	Claves	Consulta en get_conversations	Contador
Conversaciones	todos	todas las conversaciones	no
Conversaciones	tu-inbox	assigned_to=user; vacío si el usuario es anónimo	no
Conversaciones	sin-asignar	assigned_to__isnull=True	no
MIA	mia-activa, intencion-compra, ayuda-humana	queryset.none() (sin modelo de IA todavía)	no
Canales	whatsapp, messenger, instagram-dm, facebook, instagram, tiktok-dm, tiktok-coment	channel=<clave>	sí: conversaciones con unread_count mayor que 0

Filtros del panel de navegación (core/inbox.py)

Archivo	Papel
core/inbox.py	Filter, FilterGroup, FILTER_GROUPS, get_conversations, get_counts
core/views.py	_inbox_context, _thread_context, _mark_read, _selected_tag_ids, _picker_context, inbox_list, inbox_chat, inbox_thread, inbox_send, inbox_quick_replies, inbox_assign, conversation_tags, inbox_tags_bulk
core/urls.py	Rutas de los endpoints parciales
core/templatetags/inbox_extras.py	Filtros dict_get, relative_es y chat_stamp
templates/sections/inbox.html	Ensambla las cuatro columnas y el pie
templates/partials/inbox/	16 parciales: uno por columna, uno por estado vacío y uno por cada respuesta de swap
templates/partials/tags/	picker.html, picker_panel.html, pills.html, removable_pills.html
static/js/shell.js	Selección múltiple de la lista (data-select-all, data-selection-count) y popover de respuestas rápidas (data-quick-body, data-quick-send)
static/css/inbox.css	Bloques por columna: shell, nav, lista, chat y detalles, estados vacíos, pie y help-dock
static/css/tags.css	Paleta de diez colores de pill (green, yellow, purple, blue, red, orange, teal, pink, indigo, gray), archivadas con opacidad 0.55, desplegables y picker

Archivos implicados

3.6 Decisiones de diseño y trampas conocidas

Varias decisiones están explicadas en comentarios del código y conviene conocerlas antes de tocar la pantalla. `nulls_last=True` en el orden evita que una conversación recién creada sin mensajes se haga pasar por la más reciente. El filtro por etiquetas es AND: cada `.filter(tags=tag_id)` encadenado añade su propio join, mientras que un único `filter(tags__in=...)` sería OR; el desplegable lo dice explícitamente en su texto de ayuda. El estado de `#tag-filter` vive fuera de `#conv-list` para sobrevivir a polls y cambios de filtro, y el input oculto `filter` vive dentro para no quedar nunca desactualizado.

La lista pausa su poll mientras un `data-hold-poll` esté abierto o haya filas marcadas, porque un re-render borraría el menú o la selección; el hilo solo reemplaza `#chat-messages`, así que el control de asignación de la cabecera no necesita esa guarda, tal como razona el comentario de `assign_control.html`. `data-conversation-id` en `chat_thread.html` tiene carga funcional: el poll de la lista lo lee para conservar el resaltado de la fila abierta. Los swaps fuera de banda apuntan a ids que pueden no estar en pantalla (una fila filtrada, un picker cerrado) y `htmx` los ignora en silencio; `picker_panel.html`, `assign_swap.html` y el aspa de `removable_pills.html` dependen de ese comportamiento.

Nota. En `inbox_assign` las opciones del desplegable se recalculan después de guardar en lugar de reutilizar `choices`: como `assignment_options` incluye al asignado actual aunque ya no figure en `APP_AGENTS`, mover el chat a otra persona debe quitarlo de la lista en la misma respuesta y no en la siguiente recarga.

Nota. Las etiquetas archivadas desaparecen del picker y del filtro de la lista pero siguen pintándose en las pills con `tag-pill--archived`: el historial de las conversaciones ya etiquetadas se conserva y solo se impide aplicarlas de nuevo.

Nota. `inbox_quick_replies` incluye las plantillas con estado pendiente porque el MVP no tiene un flujo real de aprobación de Meta; excluirlas dejaría invisible para siempre cualquier plantilla creada por el usuario. Un envío real fuera de la ventana de 24 horas seguiría necesitando la aprobación.

Hay piezas visibles sin comportamiento: el botón Nuevo Chat del panel de navegación, el buscador de la columna 3 y sus botones de ordenar y filtrar no llevan atributos `htmx` ni ganchos de `shell.js`, y el grupo MIA muestra siempre el estado vacío. `_selected_tag_ids` descarta valores no numéricos para que una URL editada a mano no produzca un 500, `inbox_tags_bulk` hace lo propio con `selected` filtrando por `isdigit()`, y `_inbox_context` degrada un `filter` o un chat desconocidos al valor por defecto o a ninguna selección en vez de responder 404, para que un marcador antiguo siga abriendo la pantalla.

3.7 Pruebas que lo cubren

`core/tests.py` cubre la estructura de la pantalla. `InboxScreenTests` comprueba las cuatro columnas y los tres estados vacíos, que cada fila de `ALL_FILTERS` aparece con su enlace `?filter=`, que todos es el activo por defecto, que `?filter=whatsapp` marca exactamente una fila, que un filtro desconocido responde 200 con todos, que hay exactamente siete `inbox-nav__count` y que las marcas de canal conservan sus colores. `InboxListEndpointTests` verifica que `inbox_list` devuelve solo la lista, que cada filtro tiene endpoint, que uno desconocido es 404, que las filas del nav apuntan a `#conv-list` y empujan `/s/inbox/?filter=...`, y que el fragmento coincide con lo incrustado en la página. `SectionTemplateTests` fija `sections/inbox.html` como plantilla de la sección y `TemplateCommentLeakTests` vigila que ningún comentario `{#` multilínea se filtre al HTML.

`messaging/tests.py` se presenta como pruebas del contrato de webhooks, la regla de 24 horas y el cableado del Inbox. Aquí importan `SendWindowTests` (enviar fuera de la ventana o sin ningún entrante previo lanza `SendWindowClosed` sin crear nada), `InboxUITests` (filas reales en la lista, contadores por no leídos, tu-inbox y sin-asignar, apertura que limpia no leídos, compositor deshabilitado con aviso y mensaje saliente creado desde el compositor), `TagFilterTests` (semántica AND, composición con el filtro de canal y número de consultas plano), `TagPickerTests` (listado con aplicadas marcadas, alternar, swaps fuera de banda, editor en detalles, ausencia de picker en la lista, creación en línea), `BulkTagTests` (añadir y quitar en bloque) e `InlineMediaTests` (el `media_type` del evento llega a `Message` y decide el render en línea). El docstring de `TagPickerTests` aún dice que el picker lo comparten filas y cabecera, pero `test_conversation_list_has_no_picker` afirma lo contrario: hoy vive solo en el panel de detalles.

`core/tests_respuestas_rapidas.py` cubre el picker del compositor en tres clases: `RenderBodyTests` (sustitución de muestras y conservación de huecos en `render_body`), `QuickRepliesEndpointTests` (cuerpos renderizados, entradas con `data-quick-send` hacia `inbox_send`, entradas con hueco que cargan el compositor, exclusión de rechazadas e inactivas, insignia Pendiente, 404 sin conversación, orden por nombre y estado vacío enlazando a mensajería) y `ComposerHookupTests` (carga con `toggle once`, token CSRF presente y envío de extremo a extremo por `inbox_send`). La asignación de agente no se prueba en estos módulos; `inbox_assign` aparece en `core/tests_agents.py`, fuera del alcance de este capítulo.

```
python manage.py test core.tests core.tests_respuestas_rapidas messaging.tests
```

Archivos de referencia: `core/inbox.py`, `core/views.py`, `core/urls.py`, `core/templatetags/inbox_extras.py`, `templates/sections/inbox.html`, `templates/partials/inbox/assign_control.html`, `templates/partials/inbox/assign_swap.html`, `templates/partials/inbox/assigned_name.html`, `templates/partials/inbox/chat_empty.html`, `templates/partials/inbox/chat_messages.html`, `templates/partials/inbox/chat_panel.html`, `templates/partials/inbox/chat_swap.html`, `templates/partials/inbox/chat_thread.html`, `templates/partials/inbox/conversation_list.html`, `templates/partials/inbox/conversation_list_empty.html`, `templates/partials/inbox/details_content.html`, `templates/partials/inbox/details_empty.html`, `templates/partials/inbox/details_panel.html`, `templates/partials/inbox/list_panel.html`, `templates/partials/inbox/nav_panel.html`, `templates/partials/inbox/quick_replies.html`, `templates/partials/tags/picker.html`, `templates/partials/tags/picker_panel.html`, `templates/partials/tags/pills.html`, `templates/partials/tags/removable_pills.html`, `static/css/inbox.css`, `static/css/tags.css`, `static/js/shell.js`, `messaging/models.py`, `core/tests.py`, `messaging/tests.py`,

core/tests_respuestas_rapidas.py

4. Mensajería: modelo, servicios y proveedores

La app messaging concentra todo lo que no es interfaz de la mensajería omnicanal: los modelos Conversation, Message, Tag y ConversationTag; los servicios que aplican eventos entrantes y envían mensajes con la regla de la ventana de 24 horas; y una abstracción de proveedor (fake, twilio, meta) seleccionada por la variable MESSAGING_PROVIDER, con un único endpoint de webhook por proveedor y descarga de medios a Vercel Blob.

4.1 Propósito y alcance

La app messaging es la capa de mensajería del CRM, deliberadamente separada de core, que aloja la interfaz. Su AppConfig (messaging/apps.py, clase MessagingConfig, verbose_name "mensajería") lo dice explícitamente: todo lo que habla con WhatsApp, Meta, o Twilio vive aquí, de modo que cambiar o añadir un proveedor no toca ninguna vista ni ninguna plantilla. Las vistas del Inbox en core/views.py solo consumen tres funciones de servicio: pump_provider_events, send_message y las funciones de etiquetado.

El lado de contacto no tiene modelo propio: la persona con la que se chatea en el Inbox es la misma fila de core.models.Client que aparece en la tabla de Clientes del CRM. El procesamiento del webhook hace upsert de Client por número de teléfono, así que un cliente nuevo entra al CRM por el simple hecho de escribir. Este capítulo cubre modelos, servicios, proveedores, webhook, comandos de gestión y almacenamiento de medios; la pantalla del Inbox en sí queda fuera.

4.2 El modelo de datos

Conversation es un hilo con un cliente en un canal. Un cliente puede tener varias filas a lo largo del tiempo, pero solo un hilo activo por canal: _get_or_create_open_conversation reutiliza el hilo abierto o pendiente más reciente y solo crea uno nuevo si no existe. Las claves de CHANNEL_CHOICES coinciden a propósito con las de core.inbox.CANALES, de forma que un filtro de canal del Inbox es una consulta channel=key directa. Los estados son open, pending y resolved; assigned_to en NULL significa "Sin asignar" en la navegación.

Conversation denormaliza dos marcas de tiempo: last_message_at (el mensaje más reciente en cualquier dirección, y la clave de ordenación de la lista) y last_inbound_at (el último mensaje entrante). La segunda existe para que la comprobación de la ventana de 24 horas y la lista de conversaciones no tengan que reconsultar la tabla de mensajes fila a fila. La propiedad is_within_24h_window compara last_inbound_at con SERVICE_WINDOW, una constante de módulo igual a timedelta(hours=24).

Message guarda cuerpo, media_url, media_type, dirección, estado de entrega y provider_message_id. Ese identificador es único a nivel de base de datos: es la clave de idempotencia frente a los reintentos de los proveedores y lo que usan los acuses de recibo para localizar su mensaje. Es NULL para envíos que fallaron antes de que el proveedor asignara un id. Las propiedades is_inline_image, display_body y status_icon_template sirven al renderizado del hilo; display_body vacía el cuerpo cuando este es exactamente el marcador de MEDIA_PLACEHOLDERS y la imagen ya se muestra en línea, para no repetir "[imagen]" debajo de la propia imagen.

Tag y ConversationTag implementan el etiquetado. Las etiquetas son datos de tiempo de ejecución, no código: el usuario las crea y elige color de una paleta cerrada de diez tokens (green, yellow, purple, blue, red, orange, teal, pink, indigo, gray) que mapean a pares de variables CSS, en lugar de hexadecimales arbitrarios que producirían pastillas ilegibles. El nombre es único sin distinguir mayúsculas mediante una UniqueConstraint sobre Lower("name") llamada tag_name_ci_unique. ConversationTag

es una tabla intermedia explícita en vez de un `ManyToManyField` desnudo precisamente para poder responder quién etiquetó un chat y cuándo (`tagged_by`, `tagged_at`).

Nota. Borrar una etiqueta es archivarla. `set_tag_archived` marca `is_archived` y la etiqueta desaparece de selectores y filtros, pero todas las filas de `ConversationTag` sobreviven: quitar una etiqueta del historial de 500 chats porque alguien ordenó el selector reescribiría el pasado en silencio.

`messaging/admin.py` registra los cuatro modelos en el admin de Django y su docstring explica para qué: es una ventana cruda sobre las conversaciones mientras el Inbox es la superficie principal, útil para cambiar estado o asignación durante el desarrollo. `ConversationAdmin` muestra contacto, canal, estado, asignado, no leídos y `last_message_at`, filtra por canal y estado, y anida un `MessageInline` con los mensajes del hilo. `MessageAdmin` busca por `body` y `provider_message_id`; `TagAdmin` filtra por `is_archived` y `color`; `ConversationTagAdmin` lista conversación, etiqueta, `tagged_by` y `tagged_at`.

4.3 Los servicios: recepción, envío e idempotencia

`messaging/services.py` es todo lo que hay entre la interfaz o el webhook y el proveedor. Tiene dos puntos de entrada importantes. `process_inbound_events` es la mitad pesada del tratamiento del webhook, mantenida fuera de la vista para poder moverla detrás de una cola de tareas más adelante sin tocar el endpoint: la vista encolaría los eventos normalizados en lugar de llamar directamente. Hoy no hay cola porque el proyecto no declara ninguna dependencia de Celery ni de RQ; el docstring del módulo lo resume diciendo que `requirements.txt` es solo Django, lo cual está desactualizado (el fichero incluye además `asgiref`, `sqlparse`, `tzdata`, `requests`, `psycopyg[binary]`, `python-dotenv`, `dj-database-url` y `vercel_blob`). `send_message` es la única vía por la que la aplicación envía un texto libre, y es donde vive la regla de las 24 horas.

`process_inbound_events` recorre los eventos y aísla cada uno en su propia `transaction.atomic`: un evento defectuoso se registra en el log y se salta, y el resto igualmente aterriza. El motivo está en el docstring: el webhook ya respondió 200 por contrato, así que no queda nadie a quien reportar un fallo parcial. Según `event_type`, el evento se despacha a `_apply_status_event`, `_apply_outbound_event` o, por defecto, `_apply_message_event`.

`_apply_message_event` comprueba primero la idempotencia por `provider_message_id` y, si el INSERT pierde la carrera contra un reintento concurrente, captura `IntegrityError` y sale: el mensaje ya está almacenado, que es lo único que importa. Después hace `upsert` del contacto y actualiza el hilo: `last_message_at`, `last_inbound_at`, incremento de `unread_count` y, si la conversación estaba resuelta, reapertura. `_apply_outbound_event` cubre un mensaje que enviamos nosotros por fuera del Inbox (por ejemplo un asesor respondiendo desde el teléfono vinculado): se registra para que el hilo refleje lo que realmente se dijo, pero no toca `last_inbound_at`, `unread_count` ni `status`, porque esos rastrean el lado del cliente.

`_upsert_contact` recibe el lado del evento que es el cliente (`from_number` para un entrante, `to_number` para un saliente registrado fuera del Inbox) y solo crea `Client` si no existe ninguno con ese teléfono. Al crearlo trunca el nombre a 80 caracteres y usa el propio número cuando el proveedor no da nombre; pone `country="co"` únicamente si el teléfono empieza por +57, y deja el campo vacío en vez de adivinar para otros prefijos, porque de ahí sale la bandera en la tabla del CRM. El canal se traduce con `_client_channel`, que colapsa el vocabulario fino de la conversación al más grueso de `Client`: `instagram-dm` pasa a `instagram` y cualquier canal que empiece por `tiktok` pasa a `tiktok`, porque la tabla del CRM no distingue la superficie de mensajería de la marca.

`_apply_status_event` avanza el estado de entrega y es estrictamente hacia adelante. Los proveedores entregan acuses desordenados y reintentan antiguos, así que la comparación se hace con `status_rank` (de `providers/types.py`) y nunca deja que un "read" retroceda a "delivered". `failed` puntúa por encima

de todo porque es terminal, y un valor desconocido puntúa -1 para que jamás sobrescriba nada. Un acuse para un mensaje inexistente se registra y se ignora: puede adelantar al commit del envío o referirse a mensajes enviados fuera de esta aplicación.

`send_message` crea la fila `Message` con estado `queued` antes de llamar al proveedor, para que una caída a mitad del envío deje evidencia en lugar de perder el mensaje; el `id` del proveedor se adjunta cuando la llamada retorna. Si la ventana está cerrada lanza `SendWindowClosed` y ni siquiera crea la fila; si el proveedor falla, la fila queda con `status=failed` y se lanza `SendFailed`. `core/views.py` traduce ambas excepciones en avisos en línea dentro del compositor.

Las funciones de etiquetado (`create_tag`, `update_tag`, `set_tag_archived`, `apply_tag`, `remove_tag`, `next_tag_color`) están agrupadas aquí para que reglas como "una etiqueta archivada no se puede aplicar pero sí quitar" vivan en un solo sitio, sea cual sea la superficie que las invoque. `apply_tag` es idempotente por conversación y devuelve cuántas la ganaron realmente; `next_tag_color` cicla la paleta según el número de etiquetas existentes en vez de amontonar todas las nuevas en un color por defecto.

`pump_provider_events` cierra el hueco de los proveedores basados en extracción. Los proveedores reales empujan webhooks; el proveedor fake no tiene servidor desde el que empujar, así que los endpoints de sondeo del Inbox llaman a esta función, que busca un método `pending_status_events` en el proveedor activo y, si existe, pasa lo que devuelva por `process_inbound_events` exactamente igual que haría un webhook. Es una operación nula para los proveedores que no lo tienen.

4.4 La abstracción de proveedor y el webhook

El contrato está en `messaging/providers/base.py`, clase abstracta `MessagingProvider`, con cinco miembros: `name`, `send_text`, `send_template`, `parse_webhook`, `verify_signature` y un `handshake` con implementación por defecto que devuelve `None`. La forma del contrato está justificada en el docstring del módulo: `parse_webhook` y `verify_signature` reciben el request de Django en crudo, no un diccionario ya parseado, porque Twilio necesita los parámetros de formulario y Meta necesita los bytes exactos del cuerpo para su HMAC; `send_text` y `send_template` reciben números E.164 pelados, y cualquier esquema de direccionamiento como `whatsapp`: es asunto privado del proveedor; y `handshake` existe porque Meta verifica el endpoint con un GET antes de hacer el primer POST.

`messaging/providers/types.py` define los tipos neutros. `InboundEvent` es un dataclass congelado que normaliza cualquier carga útil (el POST con formulario de Twilio, el JSON anidado de Meta, el JSON plano del fake) antes de que el resto de la aplicación la vea; nada fuera de `providers/` debería tocar una carga cruda. En la práctica circulan tres valores de `event_type`: `message`, `outbound_message` y `status`, cada uno con su rama en `process_inbound_events`; el docstring de `InboundEvent` sigue hablando de solo dos formas y está desactualizado. `MessageStatus` es la enumeración `queued`, `sent`, `delivered`, `read`, `failed`, almacenada literalmente en `Message.status`.

`messaging/providers/registry.py` mantiene el diccionario `_PROVIDERS` con las cuatro clases y expone `get_provider(name=None)` e `is_known_provider(name)`. Sin argumento, `get_provider` devuelve el proveedor de `settings.MESSAGING_PROVIDER`; con nombre explícito, el que pide la URL del webhook, porque un callback de estado de Twilio debe parsearse como Twilio aunque la aplicación esté a medio migrar a Meta. Ningún otro módulo importa una clase de proveedor concreta salvo este y los tests.

`messaging/views.py` contiene una sola vista, `webhook(request, provider_name)`, decorada con `csrf_exempt` (la firma la pone el proveedor, no una sesión de navegador) y `require_http_methods(["GET", "POST"])`. Ese decorador corta cualquier otro método con un 405 antes de entrar en el cuerpo de la vista. Un slug desconocido da 404, que es honesto: no es el reintento de un proveedor sino una URL mal configurada. Un GET delega en `handshake` y responde 403 si devuelve `None`. Un POST verifica la firma antes de confiar en un solo byte del cuerpo y responde 401 si falla. A partir de ahí responde 200 pase lo que pase, incluso ante una carga ilegible, porque los proveedores

interpretan cualquier no-200 como "reintenta con agresividad" y se entraría en una tormenta de reintentos sobre la misma basura.

El proveedor fake es el que se usa mientras no hay credenciales reales y mantiene honesta toda la tubería: `send_text` devuelve un id como haría una API real, los webhooks van firmados con X-Fake-Signature contra `MESSAGING_FAKE_SECRET` para que la rama del 401 también se ejercite, y los acuses de entrega ocurren de verdad. `pending_status_events` hace avanzar los mensajes salientes por sent, delivered y read a los 2, 5 y 10 segundos, un paso por sondeo, para que las marcas de verificación progresen visiblemente en lugar de saltar de golpe a leído. Solo considera mensajes salientes cuyo `provider_message_id` empieza por fake- y cuyo estado sigue en queued, sent o delivered, y decide el siguiente paso comparando con `status_rank` en vez de por igualdad, de modo que un mensaje ya entregado avanza a leído en lugar de reemitir sent. El estado vive en la propia tabla Message, no en memoria, así sobrevive a reinicios del servidor de desarrollo y funciona entre procesos.

El proveedor twilio NO está implementado: las cuatro operaciones lanzan `NotImplementedError` y el docstring del módulo documenta la forma que tendrá la implementación cuando lleguen las credenciales (endpoint `Messages.json`, autenticación Basic, esquema whatsapp:+57..., el sid como id de mensaje, y X-Twilio-Signature como HMAC-SHA1 sobre la URL completa más los parámetros ordenados).

El proveedor meta sí es funcional. Envía por Graph API con token Bearer contra `{META_PHONE_NUMBER_ID}/messages`, con la versión de Graph fijada en v25.0 como constante `_GRAPH_API_VERSION` para que subirla sea siempre un acto deliberado. `send_text` desactiva `preview_url` porque las vistas previas de enlaces las obtiene Meta y filtrarían que un mensaje fue procesado antes de que el destinatario lo abra. `send_template` acepta claves numéricas (parámetros posicionales) o nombradas, y una clave reservada `_language` que sobrescribe el código de idioma, por defecto es (constante `_DEFAULT_TEMPLATE_LANGUAGE`, porque el CRM escribe a números colombianos).

El envío pasa por `_post_message`, que falla pronto con un `RuntimeError` explicativo si faltan `META_ACCESS_TOKEN` o `META_PHONE_NUMBER_ID`. Ante un HTTP 400 o superior registra el cuerpo de error de Graph recortado a 500 caracteres (nunca el token) antes de llamar a `raise_for_status`, porque la parte accionable (plantilla inválida, número fuera de la lista de pruebas, token caducado) va en el cuerpo y `raise_for_status` la tira. Si la respuesta no trae `messages[0].id` lanza `ValueError` con la carga recibida.

`parse_webhook` aplanar `entry[].changes[].value`, de donde salen tanto `messages[]` como `statuses[]`, deduce el canal del campo `object` de nivel superior mediante `_OBJECT_CHANNELS` (whatsapp_business_account a whatsapp, page a messenger, instagram a instagram-dm), convierte los números de dígitos pelados a E.164 y los timestamps de segundos unix a datetimes con zona. `_parse_message` cubre bastante más que texto y medios: button (el texto del botón), interactive (el título de button_reply o list_reply), location (el nombre del sitio o las coordenadas) y, para cualquier tipo no soportado (reacciones, pedidos, contactos, mensajes de sistema), un cuerpo de reserva `[<tipo>]` con un log, para que el hilo no pierda un turno en silencio. `_parse_status` además emite un warning con el campo `errors` cuando Meta reporta un envío fallido: es la única fuente del motivo, sin ella un envío fallido es una marca roja sin explicación en ningún sitio.

La firma de Meta es X-Hub-Signature-256, es decir "sha256=" más el HMAC-SHA256 de `META_APP_SECRET` sobre `request.body` tal cual llegó, comparado en tiempo constante. Tanto `verify_signature` como `handshake` fallan cerrados cuando su ajuste está vacío: registran un error y rechazan, en lugar de dejar pasar el tráfico. El comentario lo justifica sin rodeos: un webhook al que cualquiera en internet puede hacer POST es toda la superficie de ataque de esta integración.

4.5 Medios entrantes y almacenamiento en Vercel Blob

Una foto, un audio o un documento entrante de Meta llega como un identificador, no como una URL. Resolverlo cuesta una segunda llamada autorizada a Graph y lo que vuelve es un enlace de CDN de vida corta y protegido por token: inservible desde el navegador y muerto en minutos. Por eso `_fetch_and_store_media` descarga los bytes en ese mismo momento, mientras el enlace está fresco, los guarda en el almacenamiento por defecto de Django bajo el directorio `whatsapp/` y deja en el mensaje una URL propia y duradera.

Esa descarga corre en línea sobre la petición del webhook, con presupuestos ajustados: `_MEDIA_TIMEOUT` de 5 segundos por llamada (más estricto que los 10 de `_REQUEST_TIMEOUT`, el envío que el usuario está esperando) y un tope `_MEDIA_MAX_BYTES` de 16 MiB. La descarga va en streaming para abandonar un fichero que exceda el tope, incluso si mintió sobre su `file_size`. Cualquier fallo se registra en lugar de propagarse: un webhook nunca debe fallar por una imagen, simplemente aterriza el mensaje sin medios. El nombre de almacenamiento es determinista (`_storage_name` sanea el id y le añade una extensión de una tabla `_MIME_EXTENSIONS` explícita, porque `mimetypes.guess_extension` depende de la plataforma), lo que hace la operación idempotente ante reintentos.

En producción no hay sistema de ficheros persistente: las funciones serverless de Vercel son de solo lectura y efímeras, así que todo lo escrito en `MEDIA_ROOT` desaparecía tras la petición y rompía en silencio cada `default_storage.save()`. `core/storage.py` define `VercelBlobStorage`, un backend de Storage que sube los bytes al almacén Blob del proyecto y devuelve las URLs públicas del CDN. `config/settings.py` lo activa solo cuando `BLOB_READ_WRITE_TOKEN` está presente en el entorno y no se está ejecutando la batería de tests; el desarrollo local y las pruebas conservan el almacenamiento en fichero.

Los nombres se mapean a rutas de blob de forma literal, sin sufijo aleatorio, precisamente para que quien escribió `whatsapp/<media-id>.webp` pueda comprobar después ese mismo nombre con `exists()` y encontrarlo; el webhook de Meta depende de eso para su idempotencia. `url()` responde desde una caché por proceso rellena por `_save` y, cuando falla, con una llamada al endpoint de listado del Blob, filtrando por `pathname` exacto porque el listado por prefijo también devuelve nombres más largos.

4.6 Referencia

Proveedor	Estado	Variables de entorno	Firma del webhook
fake	Funcional, por defecto	<code>MESSAGING_FAKE_SECRET</code> (por defecto <code>dev-secret</code>)	X-Fake-Signature, secreto compartido
twilio	No implementado (<code>NotImplementedError</code>)	<code>TWILIO_ACCOUNT_SID</code> , <code>TWILIO_AUTH_TOKEN</code> , <code>TWILIO_WHATSAPP_FROM</code> (vacías por defecto)	X-Twilio-Signature, HMAC-SHA1 (pendiente)
meta	Funcional	<code>META_ACCESS_TOKEN</code> , <code>META_PHONE_NUMBER_ID</code> , <code>META_APP_SECRET</code> , <code>META_VERIFY_TOKEN</code> (vacías por defecto)	X-Hub-Signature-256, HMAC-SHA256 sobre el cuerpo crudo

Proveedores registrados y su configuración

Ajuste	Valor por defecto	Nota
<code>MESSAGING_PROVIDER</code>	fake	Forzado a fake cuando <code>TESTING</code> , aunque el <code>.env</code> apunte a un proveedor real
<code>MEDIA_URL</code>	media/	Servido por Django solo en <code>DEBUG</code> , ver <code>config/urls.py</code>
<code>MEDIA_ROOT</code>	<code>BASE_DIR / 'media'</code>	Almacenamiento local de medios descargados

Ajuste	Valor por defecto	Nota
STORAGES	Por defecto de Django	Se sustituye por core.storage.VercelBlobStorage si BLOB_READ_WRITE_TOKEN está en el entorno y no hay tests

Ajustes generales de la capa de mensajería en config/settings.py

Método	Ruta	Comportamiento
GET	/webhooks/messaging/<provider>/	Handshake de verificación: devuelve el challenge, o 403 si el proveedor lo rechaza
POST	/webhooks/messaging/<provider>/	401 si la firma no valida; 200 en cualquier otro caso, incluida una carga ilegible
GET o POST	/webhooks/messaging/<slug desconocido>/	404 mediante is_known_provider
Otro método	/webhooks/messaging/<cualquiera>/	405 por require_http_methods, antes de comprobar el proveedor

Endpoints del webhook

El patrón está montado en config/urls.py bajo webhooks/messaging/ e incluye messaging/urls.py, cuyo único path es <slug:provider_name>/ con nombre de URL messaging_webhook. El slug de la URL es el atributo name de la clase del proveedor.

Comando	Para qué sirve	Opciones
seed_conversations	Poblar el entorno de demostración: usuario asesor, etiquetas, conversaciones repartidas entre canales y estados, eventos de calendario y volumen de mensajes para Estadísticas	--fresh, --volume-days (por defecto 30), --volume-scale (por defecto 1.0)
simulate_inbound	Empujar un mensaje entrante falso por el camino real del webhook: firma, parseo, idempotencia y upsert de conversación	phone, message, --name, --channel (por defecto whatsapp), --bad-signature
reset_conversations	Vaciar el Inbox: borra mensajes, ConversationTag, conversaciones y contactos	--yes, --keep-clients

Comandos de gestión

seed_conversations hace bastante más que las conversaciones del Inbox. Crea el usuario de demostración asesor con contraseña asesor123 (impresa al crearlo para que el login de /admin funcione de entrada), un juego fijo de cinco etiquetas en TAG_DEFS obtenidas con get_or_create por nombre para que reejecutar no duplique ni pise ediciones del usuario, eventos de calendario de siembra, y un trasfondo determinista de volumen de mensajes para las gráficas de Estadísticas. Ese trasfondo se define con VOLUME_CHANNELS (whatsapp, messenger e instagram-dm con medias diarias distintas para que salgan tres líneas a distinta altura), factores por día de la semana y pesos por hora, VOLUME_INBOUND_SHARE de 0.37 y la semilla fija VOLUME_RNG_SEED igual a 20260828, de modo que resembrar redibuja exactamente la misma gráfica. Todos los contactos sembrados comparten el prefijo FAKE_PHONE_PREFIX, +57300000000, y los del volumen añaden un 9.

Nota. --fresh borra los contactos por prefijo de teléfono y deja que conversaciones y mensajes caigan en cascada, pero borra aparte los eventos de calendario filtrando por sus títulos y descripción de siembra, porque CalendarEvent.contact es SET_NULL y por tanto no cascadea.

```
python manage.py seed_conversations --fresh
python manage.py simulate_inbound "+573000000001" "Hola, ¿sigue disponible?"
python manage.py reset_conversations --yes --keep-clients
```

4.7 Decisiones de diseño y trampas conocidas

Los hilos resueltos son historia cerrada. `_get_or_create_open_conversation` excluye explícitamente el estado `resolved`, así que un mensaje entrante después de resolver arranca una fila de conversación nueva y las métricas por hilo sobreviven a que el cliente vuelva. Nótese la tensión aparente con `_apply_message_event`, que reabre a `open` una conversación resuelta: el orden de operaciones hace que la búsqueda ocurra antes, por lo que la reapertura solo alcanza a un hilo que se resolvió entre medias.

El comando `reset_conversations` es simulación por defecto y solo borra con `--yes`. Antes de tocar nada imprime el motor y el host o el nombre de la base de datos a la que apunta, porque `DATABASE_URL` es fácil de apuntar al proyecto equivocado y "qué base de datos acabo de borrar" es una mala pregunta para hacerse después. El borrado va en una única transacción y en orden explícito en vez de apoyarse en las cascadas, para que los números reportados sean los realmente eliminados. Etiquetas, plantillas, eventos de calendario y agentes sobreviven; `CalendarEvent.contact` es `SET_NULL`, así que los eventos superan a su contacto.

Tag tiene un TODO explícito de multi-inquilino: como todavía no existe modelo de organización o cuenta, los nombres son únicos globalmente, y cuando llegue la tenencia habrá que añadir la clave ajena de organización y acotar la restricción a `(org, Lower(name))`. En cambio `InboundEvent.to_number` sí se usa hoy, pese a que su propio docstring lo dé por no utilizado: `_apply_outbound_event` lo exige (lanza `ValueError` si falta) y se lo pasa a `_upsert_contact` como el número del cliente, y `MetaProvider` lo rellena tanto en los mensajes (con `display_phone_number` de los metadatos) como en los acuses (con `recipient_id`).

El proveedor fake importa `messaging.models.Message` dentro de la función y no en el ámbito del módulo, con un comentario que explica por qué: los proveedores se cargan junto con los ajustes, antes de que el registro de aplicaciones esté necesariamente listo. Y `VercelBlobStorage._save` permite sobrescribir con `allowOverwrite`, razonando que `Storage.save()` solo llega ahí después de que `get_available_name()` haya elegido un nombre libre, de modo que una sobrescritura real solo ocurre cuando dos escritores compiten por la misma ruta determinista, y entonces ambos escribieron los mismos bytes descargados.

4.8 Pruebas que lo cubren

`messaging/tests.py` se describe a sí mismo como pruebas del contrato del webhook, la regla de 24 horas y el cableado del Inbox. `WebhookTests` cubre el camino completo con el proveedor fake: creación de contacto, conversación y mensaje; que el mismo `provider_message_id` dos veces produzca un solo mensaje y no incremente contadores; el 401 con firma forjada o ausente sin crear nada; el 200 ante una carga ilegible; el 404 de un proveedor desconocido; el eco del challenge en el GET; la reutilización del hilo abierto; y que un entrante sobre un hilo resuelto abra uno nuevo.

`OutboundEventTests` cubre los mensajes enviados fuera del Inbox mediante eventos con `event_type` igual a `outbound_message`, incluido el autochat "Yo" de WhatsApp, y el aislamiento de eventos defectuosos. `StatusEventTests` comprueba que los acuses no retrocedan. `SendWindowTests` cubre la ventana, incluido el caso de un hilo al que el cliente nunca escribió. `FakeStatusProgressionTests` verifica que el proveedor fake avance un solo paso por bombeo aunque el mensaje ya sea viejo. `InlineMediaTests` sigue `media_type` desde el evento hasta la fila `Message` y hasta el `<img>` en la burbuja.

`messaging/tests_reset.py` existe para una sola afirmación: el comando no debe borrar sin `--yes`. Verifica que la simulación no borre nada y nombre la base de datos, que `--yes` vacíe el Inbox, que `--keep-clients` conserve los contactos, que las etiquetas sobrevivan por ser configuración y no datos de conversación, que los eventos de calendario sobrevivan a su contacto, y que un Inbox ya vacío sea una operación nula. `core/tests_storage.py` prueba `VercelBlobStorage` con la API de Blob simulada: lo que se verifica es el contrato de `Storage` del que depende el resto de la aplicación, es decir nombres deterministas, idempotencia basada en `exists()` y resolución de `url()`.

Archivos de referencia: messaging/models.py, messaging/services.py, messaging/views.py, messaging/urls.py, messaging/admin.py, messaging/apps.py, messaging/tests.py, messaging/tests_reset.py, messaging/providers/__init__.py, messaging/providers/base.py, messaging/providers/types.py, messaging/providers/registry.py, messaging/providers/fake.py, messaging/providers/twilio.py, messaging/providers/meta.py, messaging/management/commands/seed_conversations.py, messaging/management/commands/simulate_inbound.py, messaging/management/commands/reset_conversations.py, core/storage.py, core/tests_storage.py, config/settings.py, config/urls.py, requirements.txt, README.md

5. CRM: clientes, listas y etiquetas

La sección CRM (y Campañas, que monta la misma nav "Mi cuenta") se apoya en `core/crm.py` como registro declarativo de secciones y vistas, y en `core/views.py` como despachador de paneles.

Clientes es la pantalla que este capítulo desarrolla: tabla paginada con búsqueda, cuatro pantallas que comparten un solo modal `htmx` y las reglas de teléfono, país y validación que viven en `core/clientes.py`. La administración de etiquetas delega en `messaging.services` y archiva en lugar de borrar; Lista de clientes es una tabla de solo lectura con un botón "+ Crear lista" cableado a un placeholder. Todo ello queda detrás del portón de sesión de `core/middleware.py`.

5.1 Propósito y alcance

El capítulo cubre la sección CRM del shell: la columna 2 con la nav "Mi cuenta" y la columna 3 con el panel activo. La nav está declarada una sola vez en `core/crm.py` y la montan dos entradas del sidebar, CRM y Campañas, a través de `templates/sections/crm.html` y `templates/sections/campanas.html`. Los dos ficheros son casi idénticos: cambian el slug que se empuja a la URL y el id del contenedor del panel (`crm-panel` frente a `campanas-panel`), nada más. El comentario de `templates/sections/campanas.html` lo dice explícitamente: un panel construido una vez se enciende bajo ambas entradas.

De las seis vistas declaradas solo cuatro tienen panel propio en este snapshot. `clientes` es la pantalla que este capítulo desarrolla en detalle; `etiquetas` es una administración que escribe contra el modelo `Tag` de la app `messaging`; `lista-clientes` es una tabla de solo lectura sobre `ClientList`.

`campos-personalizados` y `exportaciones` no tienen plantilla y caen en el placeholder compartido.

`mi-calendario` sí tiene panel y también un CRUD completo de eventos (`calendar_event_create`, `calendar_event_update`, `calendar_event_move` y `calendar_event_delete`, todos POST que responden JSON), pero queda fuera del alcance de este capítulo y se documenta con el resto del calendario.

Nota. El modelo `Tag` no vive en `core/models.py` sino en `messaging/models.py`, y las mutaciones de la página Etiquetas pasan por `messaging.services`. La página del CRM es solo la interfaz de administración de un objeto que pertenece al Inbox.

5.2 Acceso: el portón de sesión

Ninguna de estas pantallas es alcanzable sin sesión. `core/middleware.py` define `LoginRequiredMiddleware`, que redirige a la vista `login` cualquier petición cuya sesión no tenga la bandera `app_authenticated` (la constante `SESSION_KEY`), añadiendo `?next=<ruta>` salvo cuando la ruta es `/`. Su docstring aclara que esto no es `django.contrib.auth` en el sentido habitual: no hay registro ni recuperación de contraseña, las credenciales viven en el entorno y la sesión solo recuerda una bandera. Están exentos `EXEMPT_PATHS` (`/login/`, `/privacidad/` y `/eliminacion-de-datos/`, públicas porque la revisión de app de Meta las lee sin cuenta) y `EXEMPT_PREFIXES` (`/webhooks/`, `/static/`, `/media/` y `/admin/`, este último gatado por su propia autenticación).

Las credenciales las parsea `core/agents.py` desde `APP_AGENTS`, una lista separada por comas de entradas `username:password:Nombre` (el nombre visible es opcional). Si `APP_AGENTS` está vacía se usa el par antiguo `APP_LOGIN_USERNAME/APP_LOGIN_PASSWORD` como lista de un solo agente, para que un entorno anterior a ese módulo siga funcionando. `core.views.login_view` autentica contra esa lista y, además de poner la bandera de sesión, abre una sesión real de `django.contrib.auth` contra el user espejo del agente. Como `settings.TESTING` cortocircuita el middleware, las pruebas del CRM piden las URLs directamente sin iniciar sesión.

Variable	Valor por defecto	Uso
APP_AGENTS	cadena vacía	lista de agentes username:password:Nombre
APP_LOGIN_USERNAME	cadena vacía	usuario del par antiguo, solo si APP_AGENTS está vacía
APP_LOGIN_PASSWORD	cadena vacía	contraseña del par antiguo

Variables de entorno del portón (config/settings.py)

5.3 Cómo funciona: nav declarativa y despacho de paneles

core/crm.py declara dos dataclasses congeladas, View (con key y label) y Section (con key, title, icon y views), y una lista SECTIONS con los dos grupos colapsables. De ahí se derivan ALL_VIEWS y VIEW_BY_KEY. La constante DEFAULT_VIEW vale "clientes" y PLACEHOLDER_PANEL apunta a partials/crm/panels/_placeholder.html. La función panel_template(view_key) intenta cargar partials/crm/panels/<view_key>.html con get_template y devuelve el placeholder si lanza TemplateDoesNotExist; su docstring aclara que construir una de estas páginas consiste en crear el archivo, sin tocar vistas ni URLs, igual que hace core.views._section_template a nivel de sección.

En core/views.py, SECTION_CONTEXT registra el mismo builder _crm_context para las claves "crm" y "campanas". _crm_context lee ?view= de la query string, cae a DEFAULT_VIEW si la clave no existe (una URL guardada que ya no es válida abre en vez de dar 404) y expone crm_sections, active_view, crm_view y panel_template. Después consulta PANEL_CONTEXT, un diccionario que mapea clave de vista a un callable que recibe el request y devuelve un dict; los paneles sin entrada no necesitan datos.

La vista crm_panel sirve la columna 3 aislada. Valida view_key contra VIEW_BY_KEY y lanza Http404 si no lo reconoce, arma el mismo contexto reducido (active_view, crm_view más lo que aporte PANEL_CONTEXT) y devuelve render_to_string(crm.panel_template(view_key), ...). Las filas de la nav apuntan aquí con hx-get, de forma que elegir una página vuelve a renderizar solo el panel y la nav conserva qué secciones tenía abiertas. El partial templates/partial/account_nav_panel.html no nombra ninguna pantalla concreta: recibe sections, active_view, section_key, panel_url_name y panel_target por {% include ... with %} y delega cada grupo en partials/side_nav_section.html, que a su vez saca las filas a partials/side_nav_rows.html.

partials/side_nav_section.html tiene dos renderizados. El de por defecto usa <details>/<summary> nativos en vez de JavaScript, porque así el grupo se pliega, es operable con teclado y anuncia su estado a la tecnología asistiva sin código propio, y cada include es su propio <details>, de modo que las secciones se abren y cierran de forma independiente y todas arrancan colapsadas. El segundo se activa con el parámetro flat=True y pinta el mismo título como grupo estático siempre abierto. Su comentario afirma que Campañas monta la nav así, pero es un comentario obsoleto:

templates/sections/campanas.html no pasa flat, y core/tests_campanas.py fija justo lo contrario, que ambos montajes contienen <details> y ninguno side-nav__row--static.

Clave	Etiqueta	Sección	Panel	Contexto
clientes	Clientes	gestion-clientes	panels/clientes.html	_clientes_context
etiquetas	Etiquetas	gestion-clientes	panels/etiquetas.html	_etiquetas_context
lista-clientes	Lista de clientes	gestion-clientes	panels/lista-clientes.html	_lista_clientes_context
campos-personalizados	Campos personalizados	gestion-clientes	_placeholder.html	ninguno
exportaciones	Exportaciones	gestion-clientes	_placeholder.html	ninguno
mi-calendario	Mi calendario	calendario	panels/mi-calendario.html	_mi_calendario_context

Vistas de la nav secundaria (core/crm.py)

5.4 La tabla de clientes: búsqueda y paginación

`_clientes_context` construye el queryset de `Client`, aplica la búsqueda si la hay y lo pagina con `Paginator` a `CLIENTS_PER_PAGE`, que vale 25. Devuelve la página bajo dos nombres, `clients` y `page_obj`, más `client_query`, `countries` (la lista de `core.clientes`) y `channels` (`Client.CHANNEL_CHOICES`), que son las listas de opciones que el diálogo de crear/editar necesita renderizar.

La búsqueda combina `icontains` sobre `first_name`, `last_name` y `email`. El teléfono se trata aparte: se extraen los dígitos del término y, si hay alguno, se busca con `phone__contains` sobre esos dígitos, de modo que "316 768" encuentra +573167687288; si el término no tiene dígitos se cae a `phone__icontains`. Los dos parámetros de vista, `q` y `page`, se leen con `_table_param`, que mira primero la query string y luego el cuerpo del POST: los diálogos los llevan como campos ocultos para que un guardado vuelva a renderizar la tabla que el agente estaba mirando en lugar de devolverlo a la página 1 sin filtrar.

`clientes_table` devuelve solo `partials/crm/client_table.html`, es decir filas más paginador. El comentario de la vista recoge el bug que motivó el endpoint: el paginador antes traía el panel completo de Clientes dentro de `#client-table`, lo que anidaba una segunda barra de herramientas y un `#client-table` duplicado dentro del primero en cada clic. El campo oculto `#client-page` vive dentro de esa región y no en la barra de herramientas, precisamente porque es la parte que se vuelve a renderizar al cambiar de página, así que el valor que cada botón del CRUD arrastra con `hx-include` es siempre la página real en pantalla. La caja de búsqueda dispara con `hx-trigger="input changed delay:300ms, search"` contra `#client-table`.

La tabla tiene tres estados distintos. Con filas, renderiza las columnas Nombres y apellidos, Teléfono, Mail, Canal y Acciones. Con una búsqueda que no devuelve nada, muestra "Sin resultados para «...»", que deliberadamente no es lo mismo que no tener clientes. Sin clientes en absoluto, incluye `partials/crm/client_table_empty.html`, que reutiliza el componente `.empty` construido para las columnas del Inbox. El paginador se pinta solo cuando `page_obj.has_other_pages`, con enlaces reales que `htmx` mejora para intercambiar únicamente esa región.

5.5 El CRUD de clientes sobre un modal compartido

Las cuatro pantallas (ver, crear, editar, eliminar) renderizan dentro de un único `#client-modal-body` declarado en `panels/clientes.html`. El comentario del bloque CRUD en `core/views.py` da la razón: con 25 filas por página, tres diálogos pre-renderizados por fila triplicarían el HTML del panel para un marcado que nadie abre; traer solo el que se pidió mantiene la tabla ligera. Los botones de fila son GET planos, agrupados en un `div.row-actions` que fija `hx-target`, `hx-swap` y `hx-include` una sola vez para los tres.

El token CSRF de estos formularios no viene de la región: cada plantilla que llega al modal trae su propio `{% csrf_token %}`, tanto `partials/crm/client_form.html` como el formulario de borrado de `partials/crm/client_delete.html`. El comentario de `partials/crm/client_table.html` que menciona un `hx-headers` es obsoleto: el marcado real de la región no lleva ese atributo. El único `hx-headers` del CRM está en la página de Etiquetas, sobre `#tag-table`, donde sí hace falta porque los botones de archivar y restaurar son `hx-post` sin formulario propio.

`cliente_form` sirve tanto la creación como la edición: `get_object_or_404` resuelve el cliente cuando llega `client_id` y queda en `None` cuando no. En GET renderiza el formulario con `clientes.form_state`; en POST construye el estado desde `request.POST`, lo pasa por `clientes.validate` y, si no hay errores, aplica los cambios con `clientes.apply`. Su docstring apunta a `core.plantillas.plantilla_editor` como precedente de la misma forma y del mismo contrato de error: entrada inválida vuelve a renderizar el formulario con mensajes por campo en lugar de tirar lo que se escribió. Un guardado correcto responde con `partials/crm/client_saved.html`.

Ese partial de éxito hace tres cosas desde el cuerpo del modal: un `[data-dialog-dismiss]` le dice a `shell.js` que cierre el diálogo, un `swap` fuera de banda sobre `#client-table` refresca las filas de debajo, y otro sobre `#client-notice` anuncia el resultado. El comentario explica por qué el aviso vive en el panel y no dentro del modal: una región viva dentro de un diálogo que se cierra en el mismo tick nunca llega a leerse en voz alta.

La búsqueda y la página viajan de dos maneras según el tipo de petición. Los formularios que hacen POST (el de crear/editar y el de eliminar) los llevan como `<input type="hidden" name="q">` y `name="page"`. La navegación de un fragmento a otro dentro del modal, en cambio, usa query params: el botón "Eliminar" del formulario de edición y el botón "Editar" de la ficha de detalle construyen su `hx-get` con `?q={{ q|urlencode }}&page={{ page|urlencode }}`. En ambos casos el destino es `_table_param`, que lee indistintamente de GET o de POST.

`cliente_detail` es la tarjeta de solo lectura detrás del botón del ojo. Usa `prefetch_related("conversations", "client_lists")` y muestra lo que la fila no tiene columna para mostrar: desde cuándo es cliente, en qué canales tiene hilos y a qué listas pertenece; su botón "Editar" cambia el mismo cuerpo del modal por el formulario, sin abrir un segundo diálogo. `cliente_delete` responde GET con la confirmación y POST con el borrado. La confirmación no es decorativa: `Conversation.contact` es `on_delete=models.CASCADE`, así que borrar un cliente se lleva por delante todo su historial de mensajes, y el fragmento GET dice cuántas conversaciones son antes de que el agente confirme, con singular y plural distintos.

Nombre de URL	Ruta	Métodos	Vista	Respuesta
<code>clientes_table</code>	<code>crm/clientes/tabla/</code>	GET	<code>clientes_table</code>	<code>partials/crm/client_table.html</code>
<code>cliente_create</code>	<code>crm/clientes/nuevo/</code>	GET, POST	<code>cliente_form</code>	<code>client_form.html</code> o <code>client_saved.html</code>
<code>cliente_update</code>	<code>crm/clientes/<int:client_id>/editar/</code>	GET, POST	<code>cliente_form</code>	<code>client_form.html</code> o <code>client_saved.html</code>
<code>cliente_detail</code>	<code>crm/clientes/<int:client_id>/</code>	GET	<code>cliente_detail</code>	<code>client_detail.html</code>
<code>cliente_delete</code>	<code>crm/clientes/<int:client_id>/eliminar/</code>	GET, POST	<code>cliente_delete</code>	<code>client_delete.html</code> o <code>client_saved.html</code>
<code>crm_panel</code>	<code>crm/panel/<slug:view_key>/</code>	GET	<code>crm_panel</code>	el panel de la vista, 404 si es desconocida
<code>lista_create</code>	<code>crm/listas/nueva/</code>	GET	<code>lista_create</code>	<code>panels/_nueva_lista.html</code> (placeholder)
<code>tag_create</code>	<code>crm/etiquetas/nueva/</code>	POST	<code>tag_create</code>	<code>partials/crm/tag_table.html</code>
<code>tag_update</code>	<code>crm/etiquetas/<int:tag_id>/editar/</code>	POST	<code>tag_update</code>	<code>partials/crm/tag_table.html</code>
<code>tag_archive</code>	<code>crm/etiquetas/<int:tag_id>/archivo/</code>	POST	<code>tag_archive</code>	<code>partials/crm/tag_table.html</code>

Endpoints del CRUD y de la tabla (core/urls.py)

5.6 Reglas de dominio: teléfono, país y validación

`core/clientes.py` guarda lo que no es manejo de request. Su docstring explica por qué: `messaging.services._upsert_contact` localiza el `Client` de un mensaje entrante por coincidencia exacta de `phone`, así que un número tecleado con espacios ("`+57 316 768 7288`") crearía un segundo cliente la próxima vez que esa persona escribiera. `normalize_phone` es lo que mantiene a los dos lados hablando la misma cadena: quita todo lo que no sea dígito, antepone `+` y devuelve cadena vacía si no queda ningún dígito.

El rango admitido va de `PHONE_MIN_DIGITS` (7) a `PHONE_MAX_DIGITS` (15). El comentario justifica la asimetría: 15 es el tope de la ITU, mientras que el suelo es deliberadamente laxo porque existen formatos

nacionales cortos y rechazar el número real de un cliente es peor que guardar uno raro. `COUNTRIES` es una lista de 27 países con código ISO alpha-2, nombre en español y prefijo internacional; no pretende ser exhaustiva porque cubre los mercados del producto y `Client.country` es un `CharField` libre, de modo que un país llegado por webhook igualmente se guarda y recibe su bandera.

`country_from_phone` recorre `_BY_DIALING`, la misma lista ordenada por prefijo más largo primero, para que +1809 (República Dominicana) gane a +1. Ante empate de longitud se conserva el orden de `COUNTRIES`, y por eso +1 cae en Canadá y no en Estados Unidos: es una moneda al aire y el agente puede corregirlo en el selector. Solo rellena un país en blanco; una elección explícita nunca se cuestiona, cosa que `apply` respeta al escribir `state["country"]` or `country_from_phone(phone)`.

`form_state` produce un único dict que alimenta a la vez la plantilla y a `validate`, con lo que un envío rechazado vuelve a renderizar mostrando exactamente lo que se escribió. No es una copia literal del POST: recorta espacios de los seis campos y pasa `country` a mayúsculas, así que un `co` escrito en minúsculas valida igual contra `COUNTRY_CODES`. Cuando no hay POST el estado sale del cliente en edición y, si tampoco lo hay, de una plantilla de cadenas vacías. `validate` devuelve un mapa de campo a mensaje en español, vacío cuando el estado es guardable, y acepta el cliente en edición para excluirlo del control de teléfono duplicado.

Campo	Control	Obligatorio	Validación
<code>first_name</code>	texto	sí	no vacío; máximo 80 caracteres
<code>last_name</code>	texto	no	máximo 80 caracteres
<code>phone</code>	tel	sí	normalizado; entre 7 y 15 dígitos; sin otro cliente con el mismo número
<code>country</code>	select	no	vacío o código presente en <code>COUNTRY_CODES</code> ; en blanco se deduce del prefijo
<code>email</code>	email	no	vacío o coincidente con <code>EMAIL_RE</code>
<code>channel</code>	select	no	vacío o clave de <code>Client.CHANNEL_CHOICES</code>

Campos del formulario de cliente y sus validaciones (core/clientes.py)

Nota. El teléfono único no es una restricción de base de datos: el comentario de `validate` explica que los datos de producción son anteriores a esta pantalla y pueden contener duplicados. La comprobación existe porque el Inbox enruta por coincidencia exacta y dos clientes con un mismo número partirían el historial de esa persona en dos.

5.7 Banderas, WhatsApp y listas de clientes

`Client` deriva varias propiedades en lugar de almacenarlas. `full_name` concatena y recorta, `initials` da hasta dos letras para los avatares del Inbox, `has_whatsapp` es simplemente `channel == "whatsapp"` (así el canal y la disponibilidad de WhatsApp no pueden discrepar) y `whatsapp_url` construye un enlace `https://wa.me/` solo con los dígitos del número. La bandera tiene dos caminos: la propiedad `flag` compone el emoji a partir de indicadores regionales, y el tag `flag_template` de `core/templatetags/crm_extras.py` busca un SVG vendorizado en `icons/flags/<cc>.svg`. La plantilla prefiere el SVG y solo cae al emoji cuando no hay uno, porque Windows no trae glifos de bandera y un par de indicadores regionales se ve allí como letras sueltas ("CO"). Añadir un país es dejar caer el archivo SVG, sin cambio de código.

`_lista_clientes_context` anota el número de contactos con `Count("clients")` sobre el M2M en una sola consulta y lo expone como `contact_count`, nunca contando por fila en la plantilla; el docstring señala que derivarlo así impide que el conteo discrepe de los miembros reales de la lista. `created_by` es un `CharField` de texto plano hasta que la aplicación tenga usuarios y autenticación de verdad. `partials/crm/client_list_table.html` renderiza cuatro columnas (Nombre del grupo, Número de

contactos, Fecha, Creado por) dentro de una tarjeta con borde y usa un `{% empty %}` de texto centrado ("Sin lista de clientes") en lugar del estado vacío decorado de Clientes.

El botón "+ Crear lista" apunta a `lista_create`, que devuelve `panels/_nueva_lista.html`, un panel "Crear lista — próximamente": la ruta existe para que el botón vaya a algún sitio real, pero el flujo de creación no está construido. Su `hx-target="closest main"` resuelve a `#crm-panel` o a `#campanas-panel` según el montaje, y el mismo elemento conserva un `href` normal para que funcione sin `htmx`.

5.8 La página de etiquetas

La página Etiquetas usa `_etiquetas_context`, que hace `select_related("created_by")`, anota el uso con `Count("conversation_tags")` en una consulta y ordena por `is_archived` y luego `name`, de modo que las archivadas se hunden bajo las activas en vez de desaparecer: siguen etiquetando conversaciones antiguas, así que permanecen visibles y restaurables. También expone `tag_colors` con `Tag.COLOR_CHOICES` para los selectores de color.

`partials/crm/tag_table.html` pinta seis columnas: Etiqueta, Color, Conversaciones, Creada por, Fecha y Acciones. Cada nombre se renderiza como su píldora real, así que la tabla hace de vista previa de la paleta, y las filas archivadas llevan la clase `tag-row--archived` más un sufijo `"- archivada"`. El diálogo de edición por fila se incluye únicamente para etiquetas no archivadas, que son también las únicas que ofrecen los botones de editar y archivar; una archivada solo ofrece "Restaurar". Sin ninguna etiqueta, la tabla se sustituye por un estado vacío "Aún no hay etiquetas".

Las tres mutaciones (`tag_create`, `tag_update`, `tag_archive`) solo aceptan POST, delegan en `messaging.services` y responden todas con `_tag_table_response`, que vuelve a renderizar la región `#tag-table` e inyecta `tag_error` cuando el servicio lanza `TagNameTaken` o `ValueError`. `tag_archive` interpreta `archived=1` como archivar y cualquier otro valor como restaurar. Los colores tienen dos valores por defecto distintos: `tag_update` reutiliza el color actual con `request.POST.get("color", tag.color)`, y `create_tag` sin color llama a `next_tag_color()`, que cicla la paleta con `Tag.objects.count() % len(palette)` para repartir las etiquetas nuevas en vez de amontonarlas en un solo color.

No hay borrado duro de etiquetas por decisión explícita, repetida en la vista, en el servicio y en el docstring del modelo: archivar oculta la etiqueta de los selectores mientras cada conversación ya etiquetada la conserva. `Tag.COLOR_CHOICES` define diez tokens de paleta (de green a gray) que mapean a pares de variables CSS; el diálogo los renderiza como radios de muestra y no ofrece campo hexadecimal libre, para que ninguna píldora quede ilegible. La unicidad del nombre es insensible a mayúsculas mediante un `UniqueConstraint(Lower("name"), name="tag_name_ci_unique")`, con un TODO que anota que, cuando exista un modelo de organización, la restricción deberá acotarse a (organización, nombre).

Nota. `Tag.created_by` sí es una `ForeignKey` real a `AUTH_USER_MODEL` con `on_delete=models.SET_NULL`, a diferencia del `created_by` de texto plano de `ClientList`. La tabla muestra el nombre completo del usuario, con el nombre de usuario como respaldo, y un guion largo cuando la etiqueta no tiene autor.

5.9 Decisiones de diseño y trampas conocidas

- Construir una página nueva del CRM es crear `templates/partial/crm/panels/<clave>.html`; `panel_template` la recoge sola y, si aporta datos, basta con registrar un builder en `PANEL_CONTEXT`.
- Nada dentro de los paneles nombra su montaje: quien necesite el contenedor usa `closest main`, y por eso el mismo fragmento sirve bajo CRM y bajo Campañas.
- `?view=` desconocido cae al valor por defecto en la página completa, pero `crm_panel` sí devuelve 404 ante una clave desconocida: son contratos distintos a propósito, uno para navegación humana y otro

para fragmentos.

- Los endpoints de tabla y de panel están separados para que el paginador y el buscador no vuelvan a traer la barra de herramientas ni dupliquen el id `client-table`.
- `q` y `page` viajan como campos ocultos en los POST del modal, como query params en los `hx-get` de un fragmento a otro y con `hx-include` en cada botón de fila, porque un guardado debe devolver la misma vista filtrada.
- Hay dos comentarios desactualizados que conviene no crear: el de `client_table.html` habla de un `hx-headers` que la región no tiene, y el de `side_nav_section.html` dice que Campañas monta la nav con `flat=True`, cuando la monta colapsable igual que el CRM.
- Los botones "Acciones" y "Filtrar" de las barras de herramientas son inertes: los comentarios de las plantillas indican que están a la espera de contenido, igual que el texto de los `info-dot`, cuya etiqueta accesible es de momento lo único que aportan.

```
# Añadir un país al selector, sin tocar código:
# 1. una entrada más en core/clientes.py::COUNTRIES
#   Country("NL", "Países Bajos", "+31")
# 2. la bandera, opcional:
#   templates/icons/flags/nl.svg
```

5.10 Pruebas que lo cubren

`core/tests_crm.py` se describe como pruebas de la pantalla CRM: ayudantes de modelo, nav secundaria y tabla de clientes. `ClientModelTests` fija `full_name`, la bandera desde el código de país (y su vacío cuando falta o está malformado), `has_whatsapp` solo para el canal WhatsApp y que `whatsapp_url` quita el formato. `CrmScreenTests` comprueba que se renderizan las dos columnas, que ambas secciones empiezan colapsadas, que `clientes` es la vista por defecto con su `panel_template`, que una vista aún placeholder dice "Exportaciones — próximamente", que una clave desconocida cae al defecto en vez de dar 404 y que hay exactamente una fila activa. `ClientTableTests` cubre el estado vacío, el contenido de la fila, el enlace de WhatsApp condicionado, la caída al emoji cuando no hay SVG vendorizado, las etiquetas accesibles por cliente y el paginador, incluida la regresión de que el paginador ataca el endpoint de región y no el panel completo. `CrmPanelEndpointTests` verifica que el fragmento no reenvía la nav, que todas las vistas responden, que una desconocida es 404 y que el fragmento coincide con lo que la página incrusta. `ClientListTests` cubre la cabecera, las cuatro columnas, la ausencia de columna de casillas, el estado vacío mínimo, el conteo derivado del M2M y que el botón de crear lista apunta a una ruta real que devuelve el placeholder.

`core/tests_clientes.py` se anuncia como pruebas del CRUD de Clientes: las reglas de `core.clientes`, los cuatro endpoints detrás del modal compartido y la caja de búsqueda. Incluye `NormalizePhoneTests` (incluido un caso que confirma que normalizar produce lo mismo que guarda el webhook), `CountryFromPhoneTests` con el prefijo largo ganando al corto, `ValidateTests` con obligatorios, rangos, correo, listas cerradas, teléfono duplicado nombrando al otro cliente y el caso de que un cliente no choca consigo mismo, y `ApplyTests` sobre el país derivado o respetado. Las clases de endpoint verifican que un POST correcto cierra el modal y refresca la tabla, que uno rechazado devuelve el formulario con errores y valores, que el diálogo de edición ofrece Eliminar y el de creación no, que la confirmación cuenta el historial que se lleva por delante, y que la página actual viaja con cada petición del CRUD.

`core/tests_campanas.py` existe para fijar el reparto: mismas secciones, mismas vistas, mismo endpoint `crm_panel`, distinto montaje. Comprueba que la nav de Campañas es colapsable como la del CRM (`<details>` presente, `side-nav__row--static` ausente), que las filas apuntan a `/crm/panel/...` con `hx-target="#campanas-panel"` y `hx-push-url="/s/campanas/?view=..."`, que el panel de Lista de clientes renderiza idénticamente desde ambos montajes y que paginar la tabla de clientes no reescribe la barra de direcciones hacia el CRM. Las pruebas de la administración de etiquetas no están en `core`: viven

en `messaging/tests.py`, en `TagAdminUITests`, documentada como "la página Etiquetas (CRM > Gestión de clientes) y sus endpoints", que cubre las píldoras con su conteo de uso, el error de nombre duplicado sin crear una segunda etiqueta, que renombrar actualiza todas las píldoras a la vez y que archivar la oculta de los selectores pero la conserva en las filas ya etiquetadas.

Archivos de referencia: `core/crm.py`, `core/clientes.py`, `core/models.py`, `core/views.py`, `core/urls.py`, `core/middleware.py`, `core/agents.py`, `config/settings.py`, `core/templatetags/crm_extras.py`, `templates/sections/crm.html`, `templates/sections/campanas.html`, `templates/partials/account_nav_panel.html`, `templates/partials/side_nav_section.html`, `templates/partials/crm/client_delete.html`, `templates/partials/crm/client_detail.html`, `templates/partials/crm/client_form.html`, `templates/partials/crm/client_list_table.html`, `templates/partials/crm/client_saved.html`, `templates/partials/crm/client_table.html`, `templates/partials/crm/client_table_empty.html`, `templates/partials/crm/tag_dialog.html`, `templates/partials/crm/tag_table.html`, `templates/partials/crm/panels/_placeholder.html`, `templates/partials/crm/panels/_nueva_lista.html`, `templates/partials/crm/panels/clientes.html`, `templates/partials/crm/panels/etiquetas.html`, `templates/partials/crm/panels/lista-clientes.html`, `static/css/crm.css`, `core/tests_crm.py`, `core/tests_clientes.py`, `core/tests_campanas.py`, `messaging/models.py`, `messaging/services.py`, `messaging/tests.py`

6. Mi calendario

El panel Mi calendario del CRM: FullCalendar vendorizado cargado solo en esa pantalla, el contrato de zonas horarias (UTC en base de datos, America/Bogota en entrada y pantalla, parse_client_dt como único intérprete), los tipos de evento con la paleta compartida con las etiquetas, el modal crear/editar con sus validaciones de servidor y su plomería de errores, la selección sobre la rejilla, el arrastre/redimensionado, las preferencias en sesión y el mini calendario, más la referencia de endpoints, campos y archivos.

6.1 Propósito y alcance

Mi calendario es la única vista de la sección Calendario del CRM. Se declara en `core/crm.py` como `View("mi-calendario", "Mi calendario")` y, como el resto de páginas del CRM, se resuelve por convención: `crm.panel_template` busca `partials/crm/panels/<view_key>.html` y cae en el placeholder si no existe, de modo que construir la pantalla consistió en crear el archivo, sin tocar vistas ni URLs. `core/views.crm_panel` devuelve solo la columna 3 para las peticiones HTMX de la navegación secundaria.

El modelo `CalendarEvent` explica por qué el calendario vive dentro de un CRM: su docstring dice que `contact` es la razón de ser de la pantalla, porque un evento enlaza con un cliente y deja la ficha a un clic. Las FK de usuario (`assigned_to`, `created_by`) son nulas siguiendo el precedente de `Conversation`: la aplicación todavía no tiene login real, así que los eventos creados desde la interfaz no llevan usuario. `created_by` solo se rellena en `calendar_event_create` cuando `request.user.is_authenticated`.

El campo `reminder_minutes_before` se almacena y se valida contra un menú cerrado, pero no existe ningún envío de recordatorios: es un dato guardado, no una notificación. Nada en el código programa avisos.

6.2 Piezas y flujo de arranque

La plantilla `templates/partials/crm/panels/mi-calendario.html` monta un aside de 240px (botón Crear +, mini calendario, tarjeta de ajustes) y la zona principal con una barra de herramientas propia y el contenedor `[data-calendar-grid]`. El elemento raíz `[data-calendar-root]` transporta todo el contrato con el JavaScript en atributos `data-*`: `data-csrf`, `data-events-url`, `data-create-url`, `data-prefs-url`, `data-weekends` y `data-slot`. No hay ningún dato en línea en un `<script>`.

La barra de herramientas es propia, no la de FullCalendar (`headerToolbar` va a falso): botones Hoy, anterior y siguiente que llaman a `calendar.today()`, `calendar.prev()` y `calendar.next()`, el título del rango en `[data-cal-title]` y un selector de vista `[data-cal-view]` con Día (`timeGridDay`), Semana (`timeGridWeek`) y Mes (`dayGridMonth`) que invoca `calendar.changeView`. `datesSet` es quien resincroniza las tres cosas: escribe el título, devuelve el valor del selector al tipo de vista real y redibuja el mini calendario. En vista de mes, `dayMaxEventRows: true` colapsa el exceso en un enlace «+n más» en vez de desbordar la celda.

El bundle vendorizado `static/js/vendor/fullcalendar.min.js` y `static/js/calendario.js` se cargan al final de esa plantilla, no en `base.html`, para que solo esta pantalla pague el coste del bundle. El CSS sí es global: `templates/base.html` enlaza `css/calendario.css` con el tag `{% vstatic %}`, que añade un sello `?v=<hash>` de contenido. Como la plantilla puede reinserarse por HTMX, `calendario.js` se protege con un expando de JS (`root.__calInit`) en lugar de un atributo `data-`, precisamente porque un atributo se serializaría en la instantánea de historial de HTMX y bloquearía la reinicialización tras una navegación atrás/adelante.

Los scripts externos insertados por HTMX se cargan de forma asíncrona, así que el orden no está garantizado: `calendario.js` arranca con `boot(100)`, que sondea `window.FullCalendar` cada 30 ms y se rinde si el bundle nunca llega o el panel se desmonta. Por el mismo motivo el locale español está copiado literalmente dentro del archivo (`ES_LOCALE`, tomado de `@fullcalendar/core 6.1.19`) en vez de cargarse como un segundo `<script>`. Todos los listeners se acotan a elementos del panel, nunca a `document` o `window`, y en `htmx:beforeCleanupElement` se llama a `calendar.destroy()` para soltar el listener de `resize` y los temporizadores de `FullCalendar`.

Nota. Solo tres URLs viajan en el contrato `data-*`: `feed`, creación y preferencias. Las de editar, mover y eliminar las construye `calendario.js` concatenando `eventsUrl` con el identificador y el sufijo `/editar/`, `/mover/` o `/eliminar/`, así que la forma de esas rutas está codificada en el JavaScript y cambiarlas en `core/urls.py` obliga a tocar también el cliente.

6.3 El contrato de zonas horarias

`config/settings.py` fija `TIME_ZONE = 'UTC'` y `USE_TZ = True`: la base de datos guarda UTC. La entrada y la pantalla ocurren en `core/calendario.CALENDAR_TZ`, que es `ZoneInfo("America/Bogota")`, descrita en el código como zona de reloj de pared fija UTC-5 sin horario de verano. `FullCalendar` se configura con `timeZone: "America/Bogota"` y sin plugin de zonas horarias, por lo que sus objetos `Date` son relojes de pared coercionados a UTC: formatearlos con `toISOString()` produce exactamente la cadena de reloj de pared bogotano que el servidor espera. El helper `wallClock(date)` en `calendario.js` es ese `toISOString().slice(0, 19)`, y el archivo prohíbe explícitamente usar `getHours()` o formateo local sobre fechas de `FullCalendar`.

La hora actual necesita el camino contrario, porque `new Date()` sí está en la zona del navegador. Para eso `bogotaNowStr()` formatea con `toLocaleString("sv-SE", { timeZone: "America/Bogota" })`, que da `YYYY-MM-DD HH:mm:ss`, y sustituye el espacio por una `T`; esa función se pasa como opción `now` junto a `nowIndicator: true`, de modo que la línea roja del ahora cae sobre el reloj de pared bogotano sea cual sea la zona del navegador. `bogotaTodayStr()` es su recorte a diez caracteres y alimenta el «hoy» del mini calendario y la fecha por defecto del modal.

En el servidor, `parse_client_dt` es el único punto donde se interpreta una cadena de fecha; el docstring del módulo dice que nada más en el código del calendario puede parsear un `datetime`. La función recorta una `Z` final antes de parsear, porque la coerción de `FullCalendar` emite relojes de pared marcados con `Z` que no son UTC reales; una cadena sin offset se interpreta como `CALENDAR_TZ`; una cadena con offset real se respeta tal cual. El `overflowError` que produce desplazar un reloj de pared del año 9999 a UTC se reconvierte en `ValueError` para que aflore como la entrada inválida que es.

`serialize_event` hace el camino inverso: emite ISO con el offset de `CALENDAR_TZ`, y los eventos de todo el día como fechas planas con fin exclusivo. `normalize_all_day` es el único sitio donde los rangos de todo el día se sanearn: suelo al medianoche del día de inicio, techo a una medianoche exclusiva al menos un día después.

6.4 Tipos, paleta y el modal crear/editar

Las claves y etiquetas de tipo salen de `CalendarEvent.TYPE_CHOICES` como fuente única; `core/calendario.py` solo añade el color con el diccionario `_TYPE_COLORS`, y un tipo añadido a `TYPE_CHOICES` sin color aquí falla ruidosamente en el import. Cada `EventType` expone `css_class` como `cal-event--<color>`, y `static/css/calendario.css` pinta esas clases con los pares `--tag-<color>-bg` y `--tag-<color>-fg` de `tags.css`: colores de evento y píldoras de etiqueta son un solo sistema. `DEFAULT_EVENT_TYPE` es `"reunion"`.

`_mi_calendario_context` en `core/views.py` es quien alimenta el panel: las preferencias de sesión, `EVENT_TYPES`, `SLOT_CHOICES`, `REMINDER_CHOICES` y las dos listas de opciones del modal. `contacts` son todos los `Client` sin filtrar, mientras que `advisors` es `get_user_model().objects.filter(is_active=True).order_by("username")`, de modo que un usuario desactivado desaparece del menú Asignado a y solo sobrevive en un evento ya existente gracias a `setSelectValue`.

`templates/partials/crm/event_dialog.html` es un `<dialog>` nativo sobre el shell `.modal` compartido, reutilizado para crear y editar: `openDialog` rellena los campos, cambia el título y la copia del botón de envío, y solo revela Eliminar en modo edición. El formulario no envía nativamente; `calendario.js` hace `fetch` a `calendar_event_create` o `calendar_event_update` y luego `calendar.refetchEvents()`, de modo que no hay fallback sin JS (el calendario ya requiere JS). El selector de cliente lleva un filtro de texto que reconstruye la lista de `<option>` en cada tecleo, porque el picker nativo de Safari ignora `hidden` y `display` sobre `<option>`; la opción seleccionada permanece listada aunque no coincida con el filtro, y un `role="status"` invisible anuncia cuántos clientes quedan o «Sin resultados».

Los errores del servidor tienen su propia plomería en el cliente. El diccionario `ERROR_FIELDS` mapea cada clave de error (`title`, `event_type`, `when`, `contact`, `assigned_to`, `reminder`) a los controles del formulario que le corresponden; `showFormErrors` marca esos campos con `.field--error`, les pone `aria-invalid` y `aria-describedby="event-dialog-errors"`, junta todos los mensajes en la única línea de alerta del modal y enfoca el primer campo afectado que no esté deshabilitado ni oculto. `clearFormErrors` deshace todo eso al abrir el modal y antes de cada envío. Al cerrar tras guardar o borrar, `recoverFocus()` comprueba en el siguiente frame si el foco acabó en `<body>` (la celda que se pulsó ya no existe) y, si es así, lo lleva al encabezado del rango.

`_calendar_event_from_post` concentra la validación y devuelve (`event`, `errors`), con errores por campo en español y la regla de que nada se guarda mientras `errors` no esté vacío. Valida título obligatorio y de máximo 120 caracteres, tipo dentro del menú, fechas y horas parseables, fin posterior a inicio, existencia de cliente y de usuario asignado, y recordatorio dentro de `REMINDER_KEYS`. Dos casos merecen mención: un evento que termina a las 00:00 se entiende como que acaba al día siguiente, y en modo todo el día el `end_date` opcional (inclusivo en el formulario) se guarda como fin exclusivo, último día más uno.

6.5 Selección, arrastre, preferencias y mini calendario

El calendario se configura con `editable: true`, `selectable: true` y `selectMirror: true`, así que arrastrar sobre la rejilla es la vía principal para crear un evento: el callback `select` construye un `prefill` y abre el modal ya cargado con lo arrastrado. En el carril horario toma las horas con `wallClock(info.start)` y `wallClock(info.end)` recortadas a `HH:mm`; en el carril de todo el día convierte el fin exclusivo de `FullCalendar` en el «Hasta» inclusivo del formulario con `prefill.endDate = wallClock(new Date(info.end.getTime() - DAY_MS))`. Después llama a `calendar.unselect()` para retirar el sombreado. Un clic sobre un evento existente entra por `eventClick` y abre el mismo modal en modo edición.

`eventDrop` y `eventResize` apuntan ambos a `persistMove`, que envía relojes de pared a `calendar_event_move`. Cuando un arrastre cruza de carril `FullCalendar` pierde el fin (`allDayMaintainDuration` es falso por defecto), así que el cliente lo sintetiza: un día para el carril de todo el día, la duración anterior con mínimo una hora para la rejilla horaria. El servidor vuelve a defenderse: con `all_day=1` normaliza a medianoches, rechaza rangos invertidos con 400 y, si fin es igual a inicio en un evento con hora, le da una hora. Tras guardar, el cliente hace `refetchEvents()` para mostrar la verdad almacenada y no la suposición optimista; si la petición falla, llama a `info.revert()` y muestra un toast persistente.

Las preferencias de la barra lateral son dos: mostrar fines de semana y duración del slot, con opciones 00:15:00, 00:30:00 y 01:00:00. Se guardan en la sesión bajo las claves `calendar_weekends` y `calendar_slot`, y `get_prefs` valida el slot contra `SLOT_KEYS` devolviendo `DEFAULT_SLOT` (00:30:00) si no encaja. El docstring de `get_prefs` deja escrito que esto vive en sesión hasta que la aplicación tenga usuarios y autenticación reales, momento en que pasará a ser una consulta de preferencias de usuario con la misma forma. El cliente aplica el cambio al instante con `calendar.setOption` y lo persiste en paralelo.

El mini calendario se dibuja íntegramente en `renderMini`: rejilla fija de 42 celdas que empieza en domingo, construida con aritmética UTC. Usa `tabindex` móvil (una sola parada de tabulación para toda la rejilla) y las flechas recorren las celdas; si la reconstrucción destruye la celda enfocada, el foco se devuelve a la rejilla en vez de caer al body. Las letras de la cabecera van con `aria-hidden` porque dos M sueltas son ruido para un lector de pantalla, y cada botón de día lleva su fecha completa en `aria-label`. Hacer clic llama a `calendar.gotoDate`, y es `dateset` quien vuelve a sincronizar y redibujar el mini.

Nota. Las dos rejillas no empiezan el mismo día: `ES_LOCALE` trae `week: { dow: 1 }`, así que `FullCalendar` arranca la semana en lunes, mientras que `MINI_DAY_LETTERS` y la aritmética de `renderMini` construyen el mini calendario empezando en domingo.

6.6 Referencia

Nombre de URL	Ruta	Método	Parámetros	Respuesta
calendar_events	crm/calendario/eventos/	GET	start, end (reloj de pared o ISO con offset)	Lista JSON de eventos serializados; 400 con {error} si el rango falta, es inválido, está invertido o supera 100 días
calendar_event_create	crm/calendario/eventos/nuevo/	POST	title, description, date, start_time, end_time, end_date, all_day, event_type, contact, assigned_to, reminder	{ok: true, event: {...}} o 400 con {errors: {campo: mensaje}}
calendar_event_update	crm/calendario/eventos/<int:event_id>/editar/	POST	Mismo payload que create	{ok: true, event: {...}} o 400 con {errors}; 404 si no existe
calendar_event_move	crm/calendario/eventos/<int:event_id>/mover/	POST	start, end, all_day	{ok: true} o 400 con {error}
calendar_event_delete	crm/calendario/eventos/<int:event_id>/eliminar/	POST	ninguno	{ok: true}; 404 si no existe
calendar_prefs	crm/calendario/preferencias/	POST	weekends, slot	{ok: true, weekends, slot} ya validados

Endpoints (todos bajo core/urls.py, consumidos por fetch desde calendario.js, no por HTMX)

Nota. Los cinco endpoints de mutación responden 405 a GET mediante `HttpResponseNotAllowed(["POST"])`. `calendar_events` acota el rango a 100 días porque la petición legítima más ancha de `FullCalendar` es un mes acolchado de unas seis semanas, y sin ese tope un rango hecho a mano vaciaría la tabla entera.

Campo	Tipo	Notas
title	CharField(max_length=120)	Obligatorio; el servidor lo recorta y rechaza el vacío
description	TextField(blank=True)	Libre
start / end	DateTimeField	En UTC; fin exclusivo en eventos de todo el día
all_day	BooleanField(default=False)	Inicio a medianoche, fin exclusivo
contact	FK a Client, null, SET_NULL	related_name calendar_events; el enlace con la ficha del cliente
assigned_to / created_by	FK a AUTH_USER_MODEL, null, SET_NULL	Nulables porque aún no hay login real; related_name calendar_events y created_calendar_events

Campo	Tipo	Notas
event_type	CharField(choices=TYPE_CHOICES, default="reunion")	llamada, reunion, seguimiento, otro
reminder_minutes_before	PositiveIntegerField(null=True)	Solo valores del menú; no dispara ningún aviso
created_at / updated_at	DateTimeField auto	Meta: ordering ["start"], índice calendar_start_advisor_idx sobre (start, assigned_to)

Campos de CalendarEvent (core/models.py)

Archivo	Qué contiene
core/calendario.py	CALENDAR_TZ, EventType, EVENT_TYPES, SLOT_CHOICES, REMINDER_CHOICES, get_prefs/set_prefs, parse_client_dt, normalize_all_day, serialize_event
core/views.py	_mi_calendario_context, _calendario_event_from_post y las seis vistas del bloque Mi calendario
templates/partials/crm/panels/mi-calendario.html	Barra lateral, barra de herramientas con el selector de vista, contenedor de la rejilla, toast y las dos etiquetas script
templates/partials/crm/event_dialog.html	El dialog nativo de crear/editar
static/js/calendario.js	Arranque, FullCalendar, mini calendario, modal, filtro de clientes, persistencia
static/css/calendario.css	Layout, tarjetas de la barra lateral, reskin de FullCalendar sobre sus variables --fc-*, modal y toast
core/tests_calendario.py	Las pruebas del capítulo

Archivos

6.7 Decisiones de diseño y trampas conocidas

- Un valor de recordatorio fuera del menú se rechaza en vez de guardarse: el select no podría mostrarlo y se perdería en silencio en la siguiente edición. En el cliente, un valor heredado fuera de menú sí se añade como opción para que sobreviva al guardado.
- `setSelectValue` añade la opción faltante cuando un evento apunta a un cliente o usuario que ya no está en la lista (archivado, desactivado), para que la asociación no se borre sola al guardar.
- `scrollTime` es 07:00 porque nadie agenda a las 3 de la mañana, y se vuelve a aplicar dentro de un `requestAnimationFrame`: FullCalendar calcula el desplazamiento inicial antes de que el layout flex fije la altura definitiva y el viewport queda varado sobre las 03:00.
- El botón Crear + no lleva `data-dialog-open`; su único abridor es el listener de `calendario.js`, de modo que antes de la inicialización el botón es inerte en lugar de abrir un formulario que aún no puede enviarse.
- `.cal-toast[hidden]` y `.event-actions__delete[hidden]` necesitan regla propia porque el `display: flex` o `inline-flex` de la clase le gana al atributo `hidden`.
- El toast es persistente hasta que se cierra: los errores no se desvanecen solos.
- `.mini-cal__day--outside` usa un gris con 4,7:1 de contraste sobre blanco; hoy va delineado y el día seleccionado relleno, distintos a propósito.

6.8 Pruebas

`core/tests_calendario.py` cubre la pantalla en cuatro clases. `CalendarPanelTests` comprueba que el panel rinde barra lateral, rejilla, mini calendario, el dialog y las dos etiquetas script, que la ruta de sección con `?view=mi-calendario` llega al panel, que las preferencias de sesión se reflejan en `data-weekends` y `data-slot`, y que los clientes aparecen en el selector.

`CalendarEventsFeedTests` fija el contrato de zonas horarias con el caso que da nombre al bug clásico: un evento entrado a las 09:00 de Bogotá se almacena como las 14:00 UTC y vuelve del feed como `2026-08-28T09:00:00-05:00`. También verifica que solo salen los eventos que solapan el rango, que se aceptan rangos con offset explícito, que un rango ausente, basura, invertido, de dos siglos o desbordado devuelve 400, que los eventos de todo el día se serializan como fechas con fin exclusivo, y que las clases CSS van por color de paleta (llamada produce `cal-event--green`).

`CalendarEventMutationTests` recorre las validaciones del modal (título obligatorio, fin posterior a inicio, cliente inexistente, recordatorio fuera de menú incluido un valor astronómico que no debe provocar un 500, y el reloj de pared del año 9999 que debe ser error y no 500), los casos de todo el día simple y multi-día, el evento que termina a medianoche, la edición en sitio, el arrastre y el redimensionado, la entrada y salida del carril de todo el día con su normalización y su hora por defecto, el borrado, y que las cinco mutaciones responden 405 a GET. `CalendarPrefsTests` comprueba que las preferencias persisten en la sesión y se ven en el siguiente render, y que un slot desconocido cae a `00:30:00`.

Archivos de referencia: `core/calendario.py`, `core/models.py`, `core/views.py`, `core/urls.py`, `core/crm.py`, `core/templatetags/assets.py`, `config/settings.py`, `templates/base.html`, `templates/partials/crm/panels/mi-calendario.html`, `templates/partials/crm/event_dialog.html`, `static/js/calendario.js`, `static/css/calendario.css`, `static/js/vendor/fullcalendar.min.js`, `core/tests_calendario.py`

7. Mensajería: plantillas de WhatsApp

La sección Configuración de mensajería y, dentro de ella, la única página construida: Plantillas de WhatsApp. Cubre el modelo `MessageTemplate`, el módulo `core/plantillas.py` como fuente única de listas y validación, la tabla con pestañas, el diálogo de creación, el editor con vista previa en vivo, la galería (placeholder) y el selector de respuestas rápidas del Inbox.

7.1 Propósito y alcance

La sección Configuración de mensajería es el ítem de barra lateral `mensajeria` declarado en `core/nav.py` con la etiqueta "Configuración de mensajería", y se sirve en `/s/mensajeria/`. Su nav secundario ofrece siete páginas, pero solo una está construida: Plantillas de WhatsApp, que es a la vez la vista por defecto. Las otras seis renderizan un panel de relleno con el texto "— próximamente". Este capítulo documenta esa página construida de punta a punta: el modelo que guarda las plantillas, el módulo que define las opciones del formulario y las valida, las vistas HTMX que sirven cada trozo de pantalla y las plantillas HTML que las dibujan.

El alcance real del MVP es crear y listar plantillas, no publicarlas. La vista `plantilla_editor` guarda cada plantilla nueva con `status` igual a `pendiente` y lleva un comentario `TODO(meta)` en el punto exacto donde debería enviarse a la Meta Cloud API. Las credenciales ya existen como ajustes (`META_ACCESS_TOKEN`, `META_PHONE_NUMBER_ID` en `config/settings.py`, todas con cadena vacía por defecto) porque las usa el proveedor de mensajería, pero ningún código de plantillas las lee todavía.

Nota. No hay integración con Meta para plantillas: nada se envía a aprobación y ninguna plantilla cambia de estado sola. Tampoco hay pantalla de edición ni admin: `core/admin.py` no registra ningún modelo, así que `status` e `is_active` solo se pueden cambiar por shell o directamente en la base de datos. Por eso el selector del Inbox ofrece también las plantillas pendientes.

7.2 La pantalla y su navegación

`templates/sections/mensajeria.html` es un fragmento sin `<html>`: lo que HTMX deja caer en `#content` al pulsar el icono de la barra lateral. Contiene dos columnas, el nav secundario (`partials/mensajeria/nav_panel.html`) y un `<main class="msgcfg__panel" id="mensajeria-panel">` que incluye la plantilla resuelta en la variable de contexto `panel_template`. Ese `main` es su propio destino de intercambio, de modo que elegir una página del nav reemplaza la columna 3 sin volver a renderizar el nav.

Las filas del nav salen de la lista `VIEWS` de `core/mensajeria.py`, una lista plana de dataclasses `View` congeladas con `key`, `label` e `icon`; la propiedad `icon_template` devuelve `icons/<icon>.svg`. `DEFAULT_VIEW` vale `"plantillas-whatsapp"`. El docstring del módulo apunta una diferencia deliberada con los navs hermanos: este panel no lleva encabezado sobre la lista, para seguir la referencia de diseño, y por eso `nav_panel.html` usa el modificador `side-nav__scroll--headless` y deja el nombre de la sección en el `aria-label` del `<nav>`.

`core.mensajeria.panel_template()` resuelve `partials/mensajeria/panels/<view_key>.html` llamando a `get_template()` y capturando `TemplateDoesNotExist`; si no existe, devuelve `PLACEHOLDER_PANEL`, es decir `partials/mensajeria/panels/_placeholder.html`. Su docstring lo dice explícitamente: construir una de estas páginas consiste en crear el archivo, sin tocar vistas ni URLs. Los datos extra por página se enchufan igual de barato, con el diccionario `MENSAJERIA_PANEL_CONTEXT` de `core/views.py`, que hoy solo tiene una entrada, `plantillas-whatsapp` apuntando a `_plantillas_context`.

- Widget de WhatsApp (widget-whatsapp)
- Plantillas de WhatsApp (plantillas-whatsapp), la única construida y la vista por defecto
- Respuestas rápidas (respuestas-rapidas)
- Mensajes de bienvenida (mensajes-bienvenida)
- Temas de conversación (temas-conversacion)
- Reglas de mensajería (reglas-mensajeria)
- Asignación automática (asignacion-automatica)

Hay dos maneras distintas de tratar una clave desconocida, y la diferencia es intencionada.

`_mensajeria_context` lee `?view=` y, si el valor no está en `VIEW_BY_KEY`, cae al valor por defecto en lugar de devolver 404, para que un marcador caducado siga abriendo la pantalla; `_plantillas_context` hace lo mismo con `?tab=` y `TAB_BY_KEY`. Las vistas de fragmento `mensajeria_panel` y `plantillas_table`, en cambio, lanzan `Http404` ante una clave que no conocen, porque ahí no hay pantalla que salvar.

7.3 El modelo MessageTemplate

`MessageTemplate` vive en `core/models.py` y es a la vez la fila de la tabla de plantillas y el producto del editor. Ordena por `name` y declara una `UniqueConstraint` sobre el par `name` y `language` llamada `unique_template_name_per_language`, porque Meta acota los nombres de plantilla por idioma: el par es la clave, no el nombre solo. El nombre está sujeto a una restricción de Meta, no a un capricho de estilo (minúsculas, dígitos y guion bajo), pero la expresión regular no se duplica aquí: vive en `core.plantillas.NAME_RE`.

El punto que más confunde al llegar es por qué `is_active` y `status` son campos separados. El docstring del modelo lo explica: `is_active` (columna Activo) es el interruptor de encendido y apagado de la propia cuenta, mientras que `status` (columna Estado) es el veredicto de aprobación de WhatsApp. Son ejes independientes, porque una plantilla aprobada puede estar apagada. De ahí que la pestaña Desactivadas filtre por el interruptor y las demás por el estado.

Las listas de opciones del modelo son uniones planas a propósito, para que cualquier valor almacenado se pueda mostrar siempre con `get_..._display`. Qué subtipos ofrece realmente cada categoría, la lista de idiomas y la validación del editor viven todos en `core/plantillas.py`. `team` y `created_by` siguen siendo texto plano hasta que la aplicación tenga usuarios y autenticación de verdad, la misma postura que `ClientList.created_by`.

Campo	Tipo	Notas
<code>name</code>	<code>CharField(120)</code>	Restricción de Meta: minúsculas, dígitos y <code>_</code>
<code>category</code>	<code>CharField(20), choices</code>	marketing, utility, authentication; por defecto marketing
<code>sub_type</code>	<code>CharField(30), choices</code>	custom, limited_time_offer, carousel, auth_code; por defecto custom
<code>language</code>	<code>CharField(10)</code>	Código de idioma de Meta; por defecto es
<code>team</code>	<code>CharField(80), blank</code>	Texto plano, sin modelo Team todavía
<code>header_type</code>	<code>CharField(10), choices</code>	none, text, image, video, document; por defecto none
<code>header_text</code>	<code>CharField(60), blank</code>	Solo se guarda si <code>header_type</code> es text
<code>header_media</code>	<code>FileField, upload_to plantillas/</code>	Solo si <code>header_type</code> es image, video o document
<code>body</code>	<code>TextField, blank</code>	Admite variables <code>{{n}}</code>
<code>body_sample_values</code>	<code>JSONField, default list</code>	Un ejemplo por variable; el elemento <code>i</code> corresponde a <code>{{i+1}}</code>
<code>footer</code>	<code>CharField(60), blank</code>	Sin variables
<code>buttons</code>	<code>JSONField, default list</code>	Dicts con <code>type quick_reply, url o phone</code>

Campo	Tipo	Notas
is_active	BooleanField, default True	Interruptor de la cuenta (columna Activo); el editor no lo expone
status	CharField(10), choices	pendiente, aceptada, rechazada; veredicto de WhatsApp
rejection_reason	TextField, blank	Motivo de rechazo; no se rellena desde la UI
created_by	CharField(80), blank	Texto plano, sin auth real; el editor lo deja vacío
created_at / updated_at	DateTimeField	auto_now_add / auto_now

Campos de MessageTemplate

7.4 core/plantillas.py: listas, validación y renderizado

Este módulo es la definición de datos y la validación del editor Crear plantilla. Su docstring marca la regla que gobierna toda la pantalla: el cliente refleja estas mismas reglas en vivo desde `static/js/shell.js`, pero el módulo es la autoridad y nada llega a la base de datos sin pasar por `validate()`. Las etiquetas visibles no se reescriben aquí, se leen de las listas de choices del modelo (`_CATEGORY_LABELS`, `_SUB_TYPE_LABELS`, `_HEADER_LABELS`), de forma que la tabla y el editor no pueden separarse.

`CATEGORIES` define las tres tarjetas de "Elige la categoría" (Marketing, Utility, Autenticación) con icono y descripción, y `DEFAULT_CATEGORY` es `marketing`. `SUB_TYPES` mapea cada categoría a sus opciones de radio, siendo la primera el valor por defecto de esa categoría. Marketing ofrece tres (mensaje personalizado, oferta de tiempo limitado, carrusel); Utility y Autenticación llevan una sola opción marcada como `PLACEHOLDER` en el comentario, a la espera de sus pantallas de referencia, con la instrucción de sustituir esas listas en vez de ampliarlas. `LANGUAGES` contiene nueve idiomas con código de Meta y nombre en español, y `DEFAULT_LANGUAGE` es `es`. `HEADER_TYPES` se construye directamente desde `HEADER_CHOICES` del modelo, y `BUTTON_KINDS` ofrece `none`, `quick` y `cta`.

`MEDIA_CONTENT_PREFIXES` acota qué archivo admite cada cabecera de medios y no se limita a decir "un archivo": `image` exige un `content_type` que empiece por `image/`, `video` por `video/` y `document` acepta exclusivamente `application/pdf`. `MEDIA_MAX_BYTES` pone el techo en 16 MB. La comprobación es un `startswith` sobre el `content_type` que declara la subida, de modo que un PDF es el único tipo de documento que hoy pasa la validación.

`team_options()` devuelve los nombres de equipo ya almacenados y distintos, para el desplegable Equipo; el valor vacío se renderiza como "Todos los equipos". `form_state()` produce los valores del formulario: los de por defecto en un GET limpio y los enviados, normalizados, al volver a renderizar tras errores. Es también el único sitio donde los valores de enumeración desconocidos se repliegan a su valor por defecto, en lugar de fallar, con el mismo criterio que los parámetros de consulta; `validate()` ya recibe el estado normalizado. `build_buttons()` arma el JSON de botones y `model_kwargs()` prepara los argumentos de creación, vaciando los campos que pertenecen a una opción de cabecera o de botón no elegida para que un cambio de opción no deje datos huérfanos.

`model_kwargs()` también marca el límite de lo que el editor puede fijar: devuelve nombre, categoría, subtipo, idioma, equipo, cabecera, cuerpo, ejemplos, pie, botones y `status` igual a `pendiente`, y nada más. Ni `is_active` ni `created_by` aparecen, así que toda plantilla creada desde la UI nace activa por el `default=True` del modelo y con `created_by` vacío.

`render_body()` sustituye cada `{{n}}` por su valor de ejemplo y es lo que consume el selector de respuestas rápidas del Inbox: los ejemplos hacen el texto enviable tal cual, y lo que no tiene ejemplo conserva su `{{n}}` para que el agente vea que hay un hueco por rellenar en vez de enviar un agujero en silencio. `body_variables()` devuelve los números distintos en orden de primera aparición, que es lo que el editor usa para construir la lista de campos de ejemplo.

Nombre	Valor	Para qué
NAME_RE	<code>^[a-z0-9_]+\$</code>	Restricción dura de nombre de Meta
VARIABLE_RE	<code>\{\{([1-9]\d*)\}\}</code>	Variable canónica, sin ceros a la izquierda
PADDED_VARIABLE_RE	<code>\{\{0\d*\}\}</code>	Casi-variable <code>\{01\}</code> , rechazada con explicación
NAME_MAX / TEAM_MAX	120 / 80	Longitudes máximas de nombre y equipo
BODY_MAX	1024	Longitud máxima del cuerpo
HEADER_TEXT_MAX / FOOTER_MAX	60 / 60	Cabecera de texto y pie de página
QUICK_REPLY_MAX	3	Botones de respuesta rápida
BUTTON_TEXT_MAX	25	Límite de etiqueta de botón de Meta
MEDIA_CONTENT_PREFIXES	image/, video/, application/pdf	Tipo admitido por cada cabecera de medios
MEDIA_MAX_BYTES	16 * 1024 * 1024	Techo del archivo de ejemplo de cabecera
PHONE_RE	<code>^\+?[0-9]{5,20}\$</code>	Teléfono del botón CTA

Constantes y expresiones regulares

7.5 Flujo de creación: tabla, diálogo, editor y galería

El panel `panels/plantillas-whatsapp.html` no lleva título de página, por la referencia: arriba solo va la barra de herramientas alineada a la derecha, con un campo de búsqueda por nombre, un botón de filtro y el botón primario "Crear plantilla +". Ni el buscador ni el botón de filtro llevan atributos HTMX ni enganches de datos, así que hoy son puramente visuales. Debajo va `<div id="template-table">`, la única región que se reemplaza al elegir una pestaña.

`partials/mensajeria/template_table.html` contiene a la vez la fila de pestañas y la tabla, siguiendo el mismo patrón que la tabla de productos de Mi comercio: las pestañas viven dentro de la región para que al elegir una se rerenderice su propio estado activo junto con las filas. Las pestañas son TABS en `core/mensajeria.py` (Todas, Pendientes, Aceptadas, Rechazadas, Desactivadas) y `get_templates()` traduce la clave a un filtro: `_TAB_STATUS` cubre las tres de estado, desactivadas filtra `is_active=False` y todas no filtra nada. Cada enlace de pestaña combina href a `/s/mensajeria/?view=plantillas-whatsapp&tab=<clave>` con `hx-get` a `plantillas_table` y `hx-push-url`, así que funciona con y sin HTMX.

Las columnas son la lista `TABLE_COLUMNS`, y su comentario avisa de que la plantilla recorre esa lista, así que las celdas deben renderizar los valores en el mismo orden. Nombre sale tal cual; Tipo y Categoría usan `get_sub_type_display` y `get_category_display`; Texto es `body|truncatechars:60`; Equipo es el texto plano del campo; Activo se pinta como Sí o No según `is_active`; y Estado usa `get_status_display`. No hay datos de siembra por diseño: con cero filas en la pestaña activa se renderiza la cabecera de tabla y debajo el estado vacío de dos mitades, copia y botón a la izquierda e ilustración a la derecha.

Los dos botones de crear (el de la barra de herramientas y el del estado vacío) abren el mismo `<dialog>` nativo, `template_chooser_dialog.html`, mediante `data-dialog-open="template-chooser-dialog"`. El diálogo se incluye una sola vez desde el panel y queda fuera de `#template-table`, precisamente para sobrevivir a los intercambios de pestaña. Usa `showModal()`, que aporta trampa de foco, cierre con Escape y devolución del foco de forma nativa; `shell.js` añade el cierre al pulsar el fondo (con `data-dialog-backdrop-close`) y cierra el diálogo cuando la petición HTMX de una tarjeta termina bien (`data-close-on-success`). Cada tarjeta es un único enlace, para que quien navegue con teclado tenga una sola parada de tabulación por tarjeta, y el emparejamiento botón sólido contra botón contorneado es deliberado: la ruta recomendada es "desde plantilla".

La primera tarjeta lleva a `plantilla_gallery`, que hoy solo devuelve `panels/_galeria_plantillas.html`, un placeholder con el texto "Galería de plantillas — próximamente". La galería está sin construir; la vista ya distingue entre HTMX (fragmento desnudo) y navegación normal (la página completa mediante `_mensajeria_page`), así que montarla es rellenar esa plantilla.

La segunda tarjeta abre el editor. `plantilla_editor` responde a GET con la rejilla de dos columnas (formulario a la izquierda, vista previa pegajosa a la derecha) y a POST validando con `core.plantillas`. Si no hay errores construye el `MessageTemplate` y lo guarda: a HTMX le devuelve el panel de Plantillas ya rerenderizado, en la misma región donde se había insertado el editor, y a un POST normal lo redirige a `/s/mensajeria/?view=plantillas-whatsapp`. Con errores, vuelve a renderizar el editor con mensajes en línea conservando lo escrito. El guardado va envuelto en un try por `IntegrityError`, porque el `exists()` del validador puede perder una carrera contra un guardado concurrente del mismo par nombre e idioma; la restricción única lo respalda y se responde con el mismo mensaje. En ese camino de error, si había archivo de cabecera se borra con `template.header_media.delete(save=False)`, ya que `FileField` escribe la subida antes de que falle el INSERT.

El bloque de subtipo del editor renderiza a la vez los grupos de radios de las tres categorías, dentro de `<div data-subtype-groups>`, y oculta los que no corresponden a la categoría activa con el atributo `hidden`, además de marcar `disabled` sus radios para que no se envíen. `shell.js` solo alterna esos atributos al cambiar de categoría, sin pedir nada al servidor. La prueba `test_marketing_offers_three_sub_types_others_one` fija ese contrato: las etiquetas de las cuatro opciones están en el HTML y, en el primer render, solo el grupo `marketing` viene sin `hidden`.

La vista previa es `partials/plantillas/whatsapp_preview.html` y está pensada reutilizable con dos modos. En modo vivo (el editor) se incluye sin contexto: la burbuja arranca oculta detrás de la caja gris de estado vacío y `shell.js` la rerenderiza desde el formulario en cada pulsación, leyendo `[data-plantilla-form]` y escribiendo en los ganchos `[data-wa-*)`. En modo estático (la galería, más adelante) se le pasa un diccionario `preview` con `header_text`, `body`, `footer` y `buttons`, y la burbuja se renderiza en servidor sin JavaScript. El aviso ámbar del encabezado deja claro al usuario que la vista del mensaje puede cambiar al enviarlo por WhatsApp.

Fuera de esta sección, las plantillas reaparecen en el Inbox: `inbox_quick_replies` sirve el popover Respuestas rápidas del compositor, con las plantillas activas que no estén rechazadas, ordenadas por nombre. Las pendientes se incluyen (sin canal real de aprobación, una plantilla recién creada no aparecería nunca) pero llevan una insignia "Pendiente", porque un envío real fuera de la ventana de 24 horas sí necesitaría la aprobación. Cada entrada viaja con su cuerpo ya renderizado y con ese cuerpo también como JSON en `body_json`, porque `hx-vals` se parsea como JSON.

Dentro del popover hay dos clases de entrada. La enviable lleva `data-quick-send` más `hx-post` a `inbox_send`, `hx-vals` con el cuerpo y `hx-target="#chat-messages"`, de forma que un clic deja el mensaje en el hilo; el token CSRF no está en el fragmento, se hereda del `hx-headers` del `<details>` del compositor en `chat_thread.html`. La marcada `needs_input` (su cuerpo aún arrastra un `{{n}}` sin ejemplo, y enviarlo mostraría al cliente un literal `"{{2}}"`) no lleva atributos HTMX: solo `data-quick-body`, que `shell.js` copia al compositor, un `title` que dice "Completa los datos entre llaves antes de enviar" y una segunda insignia "Completar" junto a la de Pendiente. Cuando no hay ninguna plantilla ofrecible, el popover muestra "No hay plantillas activas todavía" con un enlace a `{% url 'section' 'mensajeria' %}`, que cierra el círculo devolviendo al agente a esta misma pantalla.

7.6 Referencia de endpoints y campos del editor

Nombre de URL	Ruta	Vista	Devuelve
section	s/mensajería/	views.section	La pantalla completa; acepta ?view= y ?tab=
mensajería_panel	mensajería/panel/<slug:view_key>/	views.mensajería_panel	Solo la columna 3; 404 si la clave no existe
plantillas_table	mensajería/plantillas/tab/<slug:tab_key>/	views.plantillas_table	Pestañas más tabla; 404 si la pestaña no existe
plantilla_gallery	mensajería/plantillas/galería/	views.plantilla_gallery	Placeholder de la galería, fragmento o página completa
plantilla_editor	mensajería/plantillas/editor/	views.plantilla_editor	GET: editor. POST: valida y guarda como pendiente
inbox_quick_replies	inbox/chat/<int:conversation_id>/respuestas-rapidas/	views.inbox_quick_replies	Popover de plantillas usables en esa conversación

Endpoints (core/urls.py)

Nombre del input	Control	Comportamiento del servidor
name	Texto	validate: obligatorio; NAME_RE; máximo 120; único junto con language
category	Radio de tarjetas	form_state repliega a marketing fuera de CATEGORY_BY_KEY; validate no lo revisa
sub_type	Radio por categoría	form_state repliega a la primera opción de la categoría; validate no lo revisa
language	Select	validate: debe estar en LANGUAGE_BY_CODE
team	Select	validate: máximo 80 caracteres
header_type	Radio segmentado	form_state repliega a none fuera de HEADER_TYPE_KEYS; validate solo ramifica
header_text	Texto	Si header_type es text: obligatorio, máximo 60, sin variables
header_media	Archivo	Si la cabecera es media: obligatorio, content_type según MEDIA_CONTENT_PREFIXES, máximo 16 MB
body	Textarea	Obligatorio; máximo 1024; sin {{01}}; variables desde {{1}} y sin saltos
sample_N	Texto, uno por variable	Cada variable necesita un valor de ejemplo no vacío
footer	Texto	Máximo 60; sin variables
button_kind	Radio segmentado	form_state repliega a none fuera de BUTTON_KIND_KEYS; validate solo ramifica

Nombre del input	Control	Comportamiento del servidor
quick_reply_1 a 3	Texto	Con kind quick: al menos uno; máximo 25 caracteres cada uno
cta_url_text y cta_url	Texto y URL	Con kind cta: el par completo o ninguno; URL validada con URLValidator http/https
cta_phone_text y cta_phone	Texto y teléfono	Con kind cta: el par completo o ninguno; PHONE_RE de 5 a 20 caracteres

Campos del editor: normalización en form_state y validación en validate

Nota. El editor no tiene inputs para is_active, created_by, status ni rejection_reason: model_kwargs() no los incluye, salvo status, que fija siempre a pendiente.

7.7 Decisiones de diseño y trampas conocidas

Varias decisiones están razonadas en comentarios del propio código y conviene no revertirlas por accidente. Los ejemplos se guardan en orden numérico y no en orden de lectura: model_kwargs ordena state["samples"] por su número justo para que el elemento i sea siempre el ejemplo de {{i+1}}, aunque el cuerpo escriba {{2}} antes que {{1}}. Las variables con ceros a la izquierda no se tratan como texto literal en silencio, se rechazan con un mensaje que enseña la forma correcta. La URL de un botón CTA se comprueba con URLValidator y no con un startswith, para que "https://" sin host no cuele. En validate() gana el primer fallo de cada campo, así cada campo muestra un solo mensaje.

En las URLs, el segmento literal tab/ de plantillas_table está ahí para que el <slug:tab_key> no se trague las rutas galeria/ y editor/ declaradas después. En CSS hay una trampa repetida: .subtype-group y .wa-bubble__media declaran display: flex, que de otro modo ganaría al atributo hidden, así que ambos añaden una regla explícita [hidden] { display: none; }. static/css/plantillas.css define además dos tokens propios, --accent-text (#1a66c9) y --error-text (#c73e3e), porque los tokens base cumplen 3:1 para bordes e iconos pero no 4.5:1 para el texto pequeño de esta pantalla; el callout ámbar usa hexadecimales fijos a propósito, ya que no existen tokens ámbar.

Nota. El formulario del editor lleva novalidate y delega todo el comportamiento en vivo a shell.js mediante ganchos data-*. Cualquier regla nueva hay que añadirla en core/plantillas.py primero: el JavaScript es un espejo, no la fuente de verdad.

7.8 Pruebas que lo cubren

core/tests_mensajeria.py cubre la pantalla completa en siete clases: MensajeriaScreenTests, PlantillasPanelTests, PlantillasTabTests, PlantillasTableEndpointTests, TemplateChooserTests, PlantillaEditorTests y MensajeriaPanelEndpointTests.

MensajeriaScreenTests comprueba que se renderizan las dos columnas, que el nav no lleva encabezado, que aparecen las siete filas, que Plantillas es la vista por defecto, que ?view= selecciona el panel, que una vista desconocida cae al valor por defecto en vez de dar 404 y que hay exactamente una fila activa. PlantillasPanelTests verifica la barra de herramientas sin título de página, que se renderizan todas las columnas, que no hay columna de casillas, el estado vacío de dos mitades, que ambos botones abren el diálogo y que el estado vacío desaparece en cuanto hay plantillas. PlantillasTabTests comprueba el filtrado por pestaña, y en particular que Desactivadas filtra el interruptor y no el estado.

PlantillasTableEndpointTests y MensajeriaPanelEndpointTests verifican los contratos de fragmento: que el endpoint devuelve solo su región, que toda pestaña y toda vista responden 200, que una clave

desconocida es 404, que los slugs de pestaña no se tragan las rutas del diálogo y que lo que devuelve el endpoint es exactamente lo que la página incrusta. `TemplateChooserTests` cubre la copia y el cableado de accesibilidad del diálogo, los `href` de respaldo junto a los `hx-get`, la jerarquía de botones sólido y contorneado, las dos rutas destino y que el diálogo queda fuera de la región intercambiada.

`PlantillaEditorTests` es la clase más extensa y va decorada con `override_settings(MEDIA_ROOT=tempfile.mkdtemp(...))`, porque las subidas de cabecera aterrizan en `MEDIA_ROOT` y no deben ensuciar el real. Comprueba el camino feliz por HTMX (guarda como pendiente y devuelve la lista, no el editor otra vez) y por POST normal (redirección a la lista), el GET dentro del armazón de página frente al fragmento desnudo, que solo el grupo de subtipos de la categoría por defecto se renderiza visible, y una a una las reglas de validación: la expresión regular del nombre, la unicidad por idioma, los topes de longitud, las variables sin ceros a la izquierda y numeradas sin saltos, el ejemplo obligatorio por variable, el rechazo de variables en cabecera y pie, la cabecera de medios con archivo obligatorio y tipo coincidente, la forma de URL y teléfono de los botones CTA, que los errores conservan lo escrito y que los ejemplos se almacenan en orden numérico aunque el cuerpo los invierta.

El consumo desde el Inbox tiene sus propias pruebas en `core/tests_respuestas_rapidas.py`, dividido en `RenderBodyTests` (sustitución de ejemplos, variable sin ejemplo o con ejemplo vacío que conserva su marcador, variable repetida, cuerpo sin variables), `QuickRepliesEndpointTests` (qué plantillas se ofrecen, que las rechazadas e inactivas no aparecen, la insignia Pendiente, el orden por nombre, la entrada con hueco que carga el compositor en vez de enviar, el 404 de una conversación inexistente y el estado vacío que enlaza a la sección) y `ComposerHookupTests` (carga perezosa del popover, token CSRF y que elegir una respuesta rápida deja el mensaje en el hilo).

Archivos de referencia: `core/mensajeria.py`, `core/plantillas.py`, `core/models.py`, `core/views.py`, `core/urls.py`, `core/nav.py`, `core/admin.py`, `templates/sections/mensajeria.html`, `templates/partials/mensajeria/nav_panel.html`, `templates/partials/mensajeria/template_table.html`, `templates/partials/mensajeria/templates_empty.html`, `templates/partials/mensajeria/template_chooser_dialog.html`, `templates/partials/mensajeria/panels/plantillas-whatsapp.html`, `templates/partials/mensajeria/panels/plantilla_editor.html`, `templates/partials/mensajeria/panels/_galeria_plantillas.html`, `templates/partials/mensajeria/panels/_placeholder.html`, `templates/partials/plantillas/whatsapp_preview.html`, `templates/partials/inbox/quick_replies.html`, `templates/partials/inbox/chat_thread.html`, `static/css/mensajeria.css`, `static/css/plantillas.css`, `core/tests_mensajeria.py`, `core/tests_respuestas_rapidas.py`, `config/settings.py`

8. Estadísticas

La sección Estadísticas del MVP: su navegación secundaria, los seis paneles con datos reales y el placeholder que queda, las dos pantallas de detalle de tarjeta que alimentan ECharts por JSON, el selector de periodos compartido por cuatro paneles, la agregación en base de datos en zona REPORT_TZ con caché de cinco minutos, y los ocho módulos de prueba que cubren todo ello.

8.1 Propósito y alcance

Estadísticas es la sección de informes del CRM. Ocupa dos columnas dentro del área de contenido: una navegación secundaria plana a la izquierda y un panel activo a la derecha. La navegación se define como datos en `core/estadisticas.py`, con la misma forma que `core.embudos` y `core.automatizaciones`: una lista plana de filas, cada una con su icono. Lo específico de esta sección son dos cosas: una fila puede llevar una pastilla badge (la «Alpha» de Atribuciones) y existe además una lista CARDS que el panel de Mensajería recorre para pintar cuatro tarjetas clicables.

El alcance real conviene decirlo pronto, porque la sección mezcla pantallas terminadas con huecos declarados. Tienen datos reales sobre la base de datos los paneles Ventas, Etiquetas, Embudos, Atribuciones y Temas de conversación, más las dos primeras tarjetas de Mensajería (Volumen de Mensajes y Tiempos de Respuesta). Quedan como placeholder «próximamente» la vista Agentes de IA y las tarjetas Rendimiento de Agentes y Perfil de Clientes. No hay datos de demostración en ninguna parte: lo que no está construido se anuncia como tal en pantalla en lugar de inventar cifras.

Varios paneles miden lo que el modelo de datos permite hoy, y sus docstrings lo dicen sin rodeos. Un «tema» no es una clasificación con IA sino un conteo de palabras sobre los mensajes recibidos. Una «venta» no es un pedido sino una conversación etiquetada con una etiqueta cuyo nombre contiene «venta». Un «embudo» no es un embudo configurable sino la vida de una conversación, porque todavía no existe modelo Funnel y `core.embudos.get_funnels` no devuelve nada hasta que exista el constructor. Cada uno de esos módulos se describe a sí mismo como la pieza que se reemplazará cuando llegue la funcionalidad de verdad.

Nota. Atribuciones lleva la pastilla «Alpha» porque la atribución se detiene en el canal: los webhooks aún no capturan los datos de referencia de Meta (anuncio y campaña de click-to-WhatsApp). El módulo insiste en que nada de lo que calcula hoy es matemática de relleno.

8.2 Cómo funciona: navegación, paneles y selector de periodos

`templates/sections/estadisticas.html` es el fragmento que `htmx` inserta en `#content` al elegir el icono de la sección: solo las dos columnas, sin `<html>` ni barra lateral. Dentro, el panel activo vive en `<main class="stats__panel" id="estadisticas-panel">`, que es su propio destino de swap. El contexto de primera carga lo construye `_estadisticas_context` en `core/views.py`, que lee `?view=`, cae al valor por defecto si la clave es desconocida (así un marcador antiguo abre en vez de dar 404) y añade `stats_nav`, `active_view`, `stats_view` y `panel_template`.

Cada fila de la navegación es un enlace real que `htmx` convierte en una petición a `estadisticas_panel`, apuntando a `#estadisticas-panel` con `hx-swap="innerHTML"` y empujando la URL con `hx-push-url`. La vista `estadisticas_panel(request, view_key)` valida la clave, ejecuta el constructor de contexto correspondiente si existe y renderiza únicamente la plantilla del panel. La resolución de plantilla es por nombre: `estadisticas.panel_template` devuelve `partials/estadisticas/panels/<view_key>.html` cuando ese archivo existe y el placeholder cuando no. Construir una pantalla nueva significa por tanto crear el archivo, sin tocar la vista ni las URLs.

Los paneles Temas de conversación, Ventas, Embudos y Atribuciones comparten el selector de periodos de `core/estadisticas_periodos.py`: un menú fijo de ventanas relativas (7, 30 y 90 días, más todo el historial), con `parse_period` cayendo al valor por defecto ante cualquier cosa rara y `start_of` devolviendo el límite inferior o `None`. El `select` dispara un `hx-get` al mismo endpoint `estadisticas_panel` con `?period=`, de modo que el panel entero se vuelve a renderizar, incluido el propio `select`, que así conserva su selección sin estado en cliente. Cada opción es además marcable como `?period=<clave>`.

Etiquetas es el único panel con datos reales que no lleva selector de periodo: su constructor `_etiquetas_stats_context` no lee `?period=` y cuenta sobre todo el historial. Anota los conteos en una sola consulta con `Count("conversation_tags")` y ordena por `is_archived`, luego por conteo descendente y luego por nombre, de forma que las etiquetas archivadas mantienen su sitio en el ranking pero por debajo de las activas; su barra se dibuja atenuada mediante la regla `.tag-row--archived.rank-bar__fill` de `static/css/estadisticas.css`. La justificación está en el docstring: una etiqueta archivada sigue etiquetando conversaciones antiguas, así que sus números son historia real, solo visualmente retirada.

Nota. El selector de periodos de los paneles es deliberadamente distinto del selector de rango de fechas de las tarjetas de detalle: aquellas tienen su propio JS y su feed JSON, mientras que los paneles se intercambian enteros por `htmx` y un menú fijo los mantiene sin estado.

8.3 Las pantallas de detalle: JSON, ECharts y diálogos

Las tarjetas de Mensajería abren su pantalla de detalle a través de `estadisticas_card(request, card_key)`, que sigue exactamente el mismo patrón de resolución por nombre mediante `estadisticas.card_template`. Un `card_key` desconocido sí es 404; una tarjeta conocida sin plantilla propia renderiza `_card_detail.html`. Los constructores de contexto viven en `STATS_CARD_CONTEXT`, con entradas hoy solo para `volumen-mensajes` y `tiempos-respuesta`.

Las dos pantallas construidas se renderizan completas desde el servidor: cabecera, tiles KPI, tabla accesible y el informe entero serializado con el filtro `json_script` (en `volumen-report` y `tiempos-report`). Solo el lienzo necesita JavaScript. La plantilla carga al final `js/vendor/echarts.min.js`, `js/stats_chart.js` y su propio archivo de pegamento. Cambiar un filtro no re-renderiza el panel: `static/js/stats_volumen.js` y `static/js/stats_tiempos.js` piden el mismo informe a `estadisticas_volumen_data` o `estadisticas_tiempos_data` y aplican tiles, tabla y gráfica en sitio, con `AbortController` para descartar una respuesta lenta que llegue después de una rápida.

`static/js/stats_chart.js` es una librería, no una pantalla: no se auto-inicializa y solo toca el elemento que se le entrega. Expone `window.StatsChart` con `line`, `bar`, `formatNumber` y `shortDate`. Impone cuatro reglas de casa que ningún llamante puede saltarse: un solo eje Y (no hay opción para un segundo), el color nunca es el único canal de identidad (el texto de las series se dibuja en tinta, nunca en el color de la serie), los colores vienen de la propia serie buscados por clave y jamás indexando un array de paleta, y el tooltip de eje con cruz es obligatorio. El montaje compartido añade `ResizeObserver` y redibujo al cambiar el esquema de color, porque una gráfica dentro de un panel intercambiado se monta con anchura cero.

Cada pantalla de detalle lleva una pastilla «¿Cómo funciona?» que abre un `<dialog class="modal modal--prose">` con la explicación en prosa de qué se cuenta, en qué reloj y con qué frescura. Son `templates/partial/estadisticas/volumen_howto_dialog.html` y `tiempos_howto_dialog.html`; no llevan `htmx`, son solo copia explicativa, y sus comentarios piden mantenerlos en paso con los módulos de datos correspondientes. Los paneles usan en su lugar puntos de información con `data-tip`, pero solo en ocho sitios concretos: el subtítulo de Ventas, Atribuciones, Temas de conversación, Embudos y Etiquetas, el tile de conversión de Ventas, el tile de conversaciones etiquetadas de Etiquetas y cada etapa del embudo, que recibe su regla exacta desde `stage.tip`. Los tiles de Temas y de Atribuciones no llevan

punto de información, y el del subtítulo de Mensajería tiene `aria-label` pero no `data-tip`, con un comentario que deja la copia como pendiente.

8.4 Agregación, zona horaria y caché

`core/estadisticas_volumen.py` agrega íntegramente en base de datos con `TruncDate` y `ExtractHour` más `values` y `annotate`. El motivo está escrito en su cabecera: la tabla `Message` crece sin límite y traer las filas de un periodo a Python para recorrerlas deja de funcionar mucho antes de que nadie lo note. Los días y las horas se agrupan en `REPORT_TZ`, no en UTC, porque un mensaje enviado a las 22:00 en Bogotá pertenece al punto de ese día y no al del siguiente. `REPORT_TZ` es literalmente `CALENDAR_TZ` reexportado: una aplicación, un solo «hoy».

La ventana de instantes que cubre un rango de fechas es semiabierta a propósito: el límite superior es la medianoche del día siguiente al final, lo que evita jugar con microsegundos y no puede perder un mensaje sellado a las 23:59:59.999. `parse_range` lee `?start=` y `?end=` en ISO y cae a la ventana por defecto ante cualquier valor ausente, mal formado, invertido o más ancho que `MAX_RANGE_DAYS`, tope que existe para que una URL fabricada no pida agrupar una década. `_peak_hour` usa `ExtractHour` y no `TruncHour` porque el tile suma la misma franja de todos los días del periodo, no una hora concreta de un día concreto.

`core/estadisticas_tiempos.py` es la excepción: sus números son hechos de secuencia («cuánto tardó en llegar la siguiente respuesta»), que un `group-by` no sabe expresar. Por eso transmite los mensajes del periodo en orden de conversación con `values_list(...).iterator()` sobre cuatro columnas y los empareja en una pasada con memoria constante, tres variables por conversación. El flujo queda acotado por el mismo `MAX_RANGE_DAYS`. Una respuesta se mide desde el primer mensaje sin contestar de la racha, es decir desde que el cliente empezó a esperar; una escalación «post-MIA» es un mensaje humano saliente que sigue directamente a uno automatizado sin mensaje de cliente en medio.

Los dos filtros de Tiempos no se aplican en el mismo sitio, y conviene saberlo. El de plataforma entra en la consulta SQL de `_collect` como `conversation__channel__in=PLATFORM_CHANNELS[platform]`, así que reduce el flujo antes de recorrerlo; el de agente se aplica en Python dentro de `_build`, filtrando las listas de respuestas y escalaciones por el identificador del responsable después de la pasada de emparejamiento, porque el emparejamiento necesita ver todos los mensajes de la conversación para saber quién contestó. `parse_agent` solo acepta usuarios activos y devuelve `None` ante cualquier valor desconocido o mal formado, igual de indulgente que `parse_range`. El tile «Conversaciones medidas» cuenta conversaciones distintas con al menos una respuesta medida, ya con ambos filtros aplicados.

En la distribución de Tiempos hay dos detalles deliberados. La serie Post-MIA solo se dibuja si hubo escalaciones en el periodo, exactamente igual que un canal silencioso desaparece de la gráfica de Volumen, y «Tiempo promedio» conserva su morado en ambos casos. Y `_distribution` redondea cada banda por separado sin forzar que la fila sume 100: el docstring lo justifica diciendo que las barras responden a dónde caen la mayoría de las respuestas, y falsear una banda por un punto arreglaría un error que nadie lee en un gráfico de barras.

`core/estadisticas_temas.py` no puede agregar en SQL porque lee el texto, así que se acota por dos vías distintas: solo escanea los `SCAN_CAP` mensajes entrantes más recientes del periodo y cachea el informe terminado. El ranking es por conversaciones distintas que mencionan la palabra, no por número de menciones, y la forma que se muestra es la grafía cruda más frecuente con desempate alfabético, para que el mismo dato pinte siempre la misma palabra. `TOP_N` solo recorta la tabla: `total_topics` sigue contando todos los temas hallados. Los tres módulos con caché usan la misma ventana de 300 segundos y la misma justificación: corta para que una pantalla abierta refleje el tráfico del día en minutos, larga para que saltar entre tarjetas o mover el selector no vuelva a ejecutar el `group-by`. La clave de caché incorpora el rango o el periodo y, en tiempos, también el agente y la plataforma.

8.5 Referencia

Clave	Qué muestra	Fuente de datos	Estado
mensajería	Cabecera y rejilla de cuatro tarjetas clicables	core/estadisticas.py (CARDS, definición estática)	Real (es un índice)
ventas	Tiles de ventas, conversión y total histórico, más tabla por canal	core/estadisticas_ventas.py sobre Tag, ConversationTag y Conversation	Real
etiquetas	Ranking de etiquetas con conteo de chats y barras, sin selector de periodo	_etiquetas_stats_context en core/views.py sobre Tag y Conversation	Real
embudos	Embudo de cuatro etapas sobre las conversaciones del periodo	core/estadisticas_embudos.py	Real
atribuciones	Tabla de conversaciones y ventas por canal, con tiles	core/estadisticas_atribuciones.py	Real, marcado Alpha
temas-conversacion	Ranking de palabras usadas por los clientes	core/estadisticas_temas.py sobre Message entrantes	Real (conteo de texto, no IA)
agentes-ia	Título con «próximamente»	Ninguna: sin plantilla ni constructor de contexto	Placeholder

Paneles de la navegación secundaria (todos servidos por el endpoint estadisticas_panel)

Clave de tarjeta	Qué muestra	Fuente de datos	Endpoint JSON
volumen-mensajes	Cuatro tiles KPI, gráfica de líneas diaria por canal y tabla accesible	core/estadisticas_volumen.py	estadisticas_volumen_data
tiempos-respuesta	Tres tiles KPI y distribución de barras por franja de tiempo	core/estadisticas_tiempos.py	estadisticas_tiempos_data
rendimiento-agentes	Título con «próximamente»	Ninguna	No aplica
perfil-clientes	Título con «próximamente»	Ninguna	No aplica

Tarjetas de Mensajería y sus pantallas de detalle

Nombre de URL	Patrón	Vista
estadisticas_panel	estadisticas/panel/<slug:view_key>/	views.estadisticas_panel
estadisticas_volumen_data	estadisticas/mensajería/volumen/datos/	views.estadisticas_volumen_data
estadisticas_tiempos_data	estadisticas/mensajería/tiempos/datos/	views.estadisticas_tiempos_data
estadisticas_card	estadisticas/mensajería/<slug:card_key>/	views.estadisticas_card

Rutas de core/urls.py

Constante	Módulo	Valor	Para qué sirve
REPORT_TZ	core/estadisticas_volumen.py	CALENDAR_TZ, es decir America/Bogotá	Zona en que se agrupan días y horas
DEFAULT_RANGE_DAYS	core/estadisticas_volumen.py	30	Ventana por defecto del selector de fechas
MAX_RANGE_DAYS	core/estadisticas_volumen.py	366	Tope de un rango escrito a mano
CACHE_SECONDS	core/estadisticas_volumen.py	300	Reutilización de un informe, compartida con tiempos y temas
DEFAULT_PERIOD	core/estadisticas_periodos.py	"30"	Opción marcada al abrir un panel
SCAN_CAP	core/estadisticas_temas.py	5000	Máximo de mensajes leídos por informe de temas

Constante	Módulo	Valor	Para qué sirve
TOP_N	core/estadisticas_temas.py	15	Filas de la tabla de temas (total_topics cuenta todos)
MIN_WORD_LENGTH	core/estadisticas_temas.py	3	Longitud mínima de un token
SALE_NAME_TOKEN	core/estadisticas_ventas.py	"venta"	Subcadena que convierte una etiqueta en marcador de venta

Constantes que gobiernan los informes

Archivo	Contenido
static/css/estadisticas.css	Shell de dos columnas, pastilla de badge, cabecera de página, rejilla de tarjetas, paneles de ranking y bloque de embudo
static/css/stats-detail.css	Pantallas de detalle: cabecera, pastilla «¿Cómo funciona?», puntos de información, filtros y daterange, tiles KPI, tarjeta de gráfica y tabla accesible
static/js/stats_chart.js	Envoltorio compartido de ECharts (window.StatsChart con line, bar, formatNumber y shortDate)
static/js/stats_volumen.js	Selector de rango, refetch y conmutador «Ver tabla» de Volumen de Mensajes
static/js/stats_tiempos.js	Misma pauta con tres filtros (agente, plataforma, periodo) y gráfica de barras

Hojas de estilo y scripts

8.6 Decisiones de diseño y trampas conocidas

La paleta de canales de `estadisticas_volumen.CHANNELS` es a la vez leyenda y mapa de color, y se consulta por clave y nunca por índice, para que un canal sin tráfico que desaparece de un periodo no pueda repintar a los que quedan. Cada canal lleva su propio paso dark validado, no una versión aclarada del claro. El diccionario `SOURCE_CHANNEL` pliega los siete canales de `Conversation.CHANNEL_CHOICES` sobre las cuatro series del gráfico (Facebook se une a Messenger, los DM de Instagram a Instagram) y dos asserts de importación fallan ruidosamente si alguien añade un canal al modelo sin darle destino. El filtro de plataforma de Tiempos se construye invirtiendo ese mismo mapa en `PLATFORM_CHANNELS`, así que «Instagram» significa el mismo conjunto en las dos pantallas.

El formateo de números vive en el servidor, no en la plantilla y otra vez en el JS: los tiles y la tabla se escriben desde las mismas cadenas del mismo informe, así que no pueden divergir. La tabla accesible se construye a partir de exactamente la misma serie que dibuja la gráfica en vez de volver a consultar, porque una tabla que pudiera contradecir a la gráfica de al lado sería peor que no tener tabla. En Volumen, la tabla se reconstruye entera al cambiar de periodo porque el número de columnas depende de cuántos canales tuvieron datos.

Los scripts de pantalla se escriben contando con que se ejecuten más de una vez por vida de página: cada listener se acota a la raíz del panel actual y la inicialización se guarda en un `expando` de JavaScript, no en un atributo `data-`, porque un atributo se serializaría en la instantánea de historial de `htmx` y bloquearía la reinicialización tras ir atrás o adelante. Al intercambiar el panel, `htmx:beforeCleanupElement` destruye la instancia de ECharts. Si el bundle nunca llega, el arranque se rinde en silencio: los tiles y la tabla ya están en pantalla.

Hay trampas de dominio que conviene conocer. La tasa de conversión de Ventas divide las ventas del periodo entre las conversaciones creadas en ese mismo periodo, dos conjuntos que se solapan imperfectamente; el punto de información dice exactamente qué divide, y cuando no hay base devuelve `None`, que se pinta como raya, porque cero entre cero no es «0%». Junto a esa cifra el informe expone `total_sales`, el tile «Total histórico»: se calcula antes de aplicar la ventana, así que cuenta las conversaciones con etiqueta de venta de todo el historial y no del periodo. Y `sale_tags` incluye las etiquetas archivadas, porque una «VENTA 2025» retirada sigue fechando las ventas que registró.

El panel de Ventas distingue además dos vacíos distintos. Si no existe ninguna etiqueta con «venta» en el nombre, la pantalla entra en un estado de configuración que enseña la convención: crear la etiqueta en CRM > Etiquetas y aplicarla desde el Inbox a cada conversación cerrada. Si las etiquetas existen pero el periodo no tiene ventas, el vacío es el normal y sugiere ampliar la ventana. Es la misma línea que sigue toda la sección: cuando falta un dato se explica el porqué en lugar de disculparse o rellenar con ceros.

Las cuatro etapas del embudo se miden de forma independiente contra las conversaciones del periodo y no anidadas: un chat puede marcarse como venta sin haber pasado por resuelta, y el docstring dice que en la práctica los conteos encogen etapa a etapa por los datos, no por construcción. Atribuciones alinea sus reglas con Embudos a propósito (ventana por `created_at` más etiqueta de venta) para que sus totales coincidan con las etapas «Creadas» y «Con venta» del mismo periodo. Solo emite fila para los canales con al menos una conversación en la ventana, porque un canal en el que nadie escribió no atribuye nada, y cuenta las ventas con `Count("id", distinct=True)` para que un chat con dos etiquetas de venta no cuente doble. En Temas, el plegado de acentos deja la `ñ` intacta a propósito, para no fundir «año» con otra cosa.

Nota. `static/css/stats-detail.css` no tiene bloque `prefers-color-scheme` a propósito: el shell de la aplicación es solo claro, así que la gráfica elige su paleta midiendo la luminancia de la superficie sobre la que se dibuja (`onDarkSurface` en `stats_chart.js`) en lugar de preguntar al sistema operativo. Si se añade tema oscuro, sus valores deben mantenerse en paso con `THEME.dark`, porque el canvas no puede leer una propiedad personalizada.

8.7 Pruebas que lo cubren

La sección tiene ocho módulos de prueba, uno por pantalla más el de la carcasa.

`core/tests_estadisticas.py` cubre la carcasa: que se rendericen las dos columnas, que la cabecera de la navegación sea «Menú» y no «Mi cuenta», que la pastilla Alpha aparezca exactamente una vez, que la vista por defecto sea Mensajería, que una clave desconocida caiga en vez de dar 404, que la sección devuelva un fragmento desnudo a `htmx` y que cada vista tenga un endpoint de panel funcionando cuyo contenido coincida con el que embebe la página completa.

`core/tests_estadisticas_volumen.py` verifica el formateo en español, el parseo del rango (por defecto, explícito, mal formado, invertido y absurdamente ancho), que todo canal del modelo tenga destino y paleta, que los pasos oscuros sean valores propios, la agregación (huecos como cero, plegado de canales, hora pico sumando la misma franja de todos los días, cubetas en `REPORT_TZ` y no en UTC, último día incluido hasta su minuto final), que la tabla coincida con la serie, que un rango se agregue una sola vez, y la pantalla y su endpoint JSON, incluido que la ruta de datos no la trague la ruta de tarjeta.

`core/tests_estadisticas_tiempos.py` se centra en la pasada de emparejamiento: que una respuesta se mida desde el primer entrante sin contestar, que solo el primer saliente de una racha cuente, que las conversaciones no se contaminen entre sí, que una escalación sea una respuesta humana justo después de una automatizada y deje de serlo si hay un mensaje de cliente en medio, más los filtros de agente y plataforma, la distribución por bandas y la caché por combinación.

Quedan cinco módulos: `core/tests_estadisticas_atribuciones.py`, `core/tests_estadisticas_embudos.py`, `core/tests_estadisticas_etiquetas.py`, `core/tests_estadisticas_temas.py` y `core/tests_estadisticas_ventas.py`. Todos prueban su informe, sus tiles y su estado vacío, y los cuatro que tienen selector (atribuciones, embudos, temas y ventas) prueban además la ventana de periodo. El de etiquetas no incluye ningún test de periodo, porque el panel no lo tiene: comprueba en cambio que el conteo sea de chats y no de aplicaciones de etiqueta, que las archivadas queden por debajo de las activas y marcadas, que la etiqueta más usada reciba la barra completa y que una con cero no dibuje relleno. El de temas cubre también el tokenizador y el

plegado de acentos.

Archivos de referencia: core/estadisticas.py, core/estadisticas_periodos.py, core/estadisticas_volumen.py, core/estadisticas_tiempos.py, core/estadisticas_temas.py, core/estadisticas_ventas.py, core/estadisticas_embudos.py, core/estadisticas_atribuciones.py, core/views.py, core/urls.py, core/calendario.py, templates/sections/estadisticas.html, templates/partials/estadisticas/nav_panel.html, templates/partials/estadisticas/stat_cards.html, templates/partials/estadisticas/volumen_howto_dialog.html, templates/partials/estadisticas/temas_howto_dialog.html, templates/partials/estadisticas/panels/_placeholder.html, templates/partials/estadisticas/panels/_card_detail.html, templates/partials/estadisticas/panels/mensajeria.html, templates/partials/estadisticas/panels/etiquetas.html, templates/partials/estadisticas/panels/temas-conversacion.html, templates/partials/estadisticas/panels/ventas.html, templates/partials/estadisticas/panels/embudos.html, templates/partials/estadisticas/panels/atribuciones.html, templates/partials/estadisticas/cards/volumen-mensajes.html, templates/partials/estadisticas/cards/temas-respuesta.html, static/js/stats_chart.js, static/js/stats_volumen.js, static/js/stats_tiempos.js, static/css/estadisticas.css, static/css/stats-detail.css, core/tests_estadisticas.py, core/tests_estadisticas_volumen.py, core/tests_estadisticas_tiempos.py, core/tests_estadisticas_temas.py, core/tests_estadisticas_ventas.py, core/tests_estadisticas_embudos.py, core/tests_estadisticas_atribuciones.py, core/tests_estadisticas_etiquetas.py

9. Embudos, Automatizaciones, Campañas y Mi comercio

Las cuatro pantallas más pequeñas del CRM comparten un mismo patrón de nav secundaria más panel intercambiable por HTMX. Embudos y Automatizaciones son andamiaje completo con estados vacíos y botones cableados a placeholders; Mi comercio es la única con datos reales (el modelo Product y su tabla con pestañas de estado); Campañas no tiene módulo propio, sino que monta por segunda vez la nav y los paneles del CRM.

9.1 Propósito y alcance

Este capítulo cubre cuatro entradas del sidebar que comparten una misma anatomía y un grado de madurez desigual: Embudos, Mi comercio, Campañas y Automatizaciones. Las tres primeras son entradas vivas de `core/nav.PRIMARY_NAV`; la cuarta fue retirada del sidebar y hoy responde 404 en su URL de sección, aunque su código sigue íntegro en el repositorio. Son las pantallas más pequeñas del proyecto y sirven bien como puerta de entrada al patrón de dos columnas que el resto de secciones repite a mayor escala.

Conviene separar desde el principio qué es funcionalidad real y qué es andamiaje. Lo único con persistencia detrás es el catálogo de productos de Mi comercio: existe el modelo `Product`, su migración y una tabla que itera un `queryset` real filtrado por pestañas. Todo lo demás de estas cuatro pantallas es navegación, maquetación y estados vacíos: los embudos y los flujos no tienen modelo, y los botones de creación e importación aterrizan en paneles que dicen "próximamente".

- Real: la nav secundaria de las tres pantallas, sus endpoints HTMX de panel, el modelo `Product`, el filtrado por pestañas y el doble montaje CRM/Campañas.
- Andamiaje: `get_funnels()` y `get_flows()` devuelven listas vacías fijas, no hay modelos `Funnel` ni `Flow`.
- Placeholder explícito: `embudo_create`, `flujo_create`, `producto_create` y `producto_import` devuelven una tarjeta con el texto "— próximamente".
- Inerte: el buscador, el botón de filtro y los checkboxes de la tabla de Productos no llevan cableado alguno; el chevron del banner de Academy tampoco hace nada.

Embudos aparece además fuera del sidebar: es una de las tres tarjetas de acceso rápido de la pantalla de bienvenida. La lista vive en `core/nav.WELCOME_SHORTCUTS`, que vale `["inbox", "crm", "embudos"]`, y la vista `core/views.welcome` la resuelve a `NavItem` con `[NAV_BY_KEY[key] for key in WELCOME_SHORTCUTS]`, de modo que las tarjetas reutilizan la etiqueta y el glifo del rail sin duplicarlos.

9.2 El patrón común: módulo de datos, contexto y panel

Cada pantalla tiene un módulo de datos que es la única fuente de verdad de su nav secundaria: `core/embudos.py`, `core/automatizaciones.py` y `core/comercio.py`. Los tres declaran una `dataclass` congelada `View` con al menos `key` y `label`, más las constantes `VIEW_BY_KEY`, `DEFAULT_VIEW` y `PLACEHOLDER_PANEL`. La pieza clave es `panel_template(view_key)`: intenta cargar `partials/<screen>/panels/<view_key>.html` con `get_template` y, si salta `TemplateDoesNotExist`, devuelve el placeholder. El docstring lo dice sin rodeos: construir una de esas páginas consiste en crear el archivo, sin tocar ni la vista ni las URLs.

Dónde vive el icono es lo que distingue a un módulo de otro. En `core/embudos.py` y `core/automatizaciones.py` la nav es plana, así que cada `view` lleva su propio campo `icon` y una propiedad `icon_template` que resuelve a `icons/<icon>.svg`. En `core/comercio.py` la `dataclass View` solo declara `key` y `label`: las filas hijas de una sección no llevan icono, y los iconos que sí existen cuelgan

de `Section.icon_template`, para la cabecera de cada grupo colapsable, y de `SingleView.icon_template`, para la fila suelta, cuya subclase fija `icon = "settings"` por defecto.

Del lado de `core/views.py` hay dos capas. Los constructores `_embudos_context`, `_automatizaciones_context` y `_comercio_context` están registrados en `SECTION_CONTEXT` y arman el contexto de la pantalla completa a partir de `?view=`; dentro de cada uno, un segundo diccionario (`EMBUDOS_PANEL_CONTEXT`, `AUTOM_PANEL_CONTEXT`, `COMERCIO_PANEL_CONTEXT`) añade los datos que solo necesita el panel activo. Las vistas `embudos_panel`, `automatizaciones_panel` y `comercio_panel` reutilizan ese mismo segundo diccionario para devolver la columna 3 aislada, que es lo que pide HTMX cuando se pulsa una fila de la nav.

Nota. Hay una asimetría deliberada entre ambas capas: un `?view=` desconocido en la pantalla completa cae al `DEFAULT_VIEW` en lugar de dar 404, para que un marcador antiguo siga abriendo; el endpoint de panel, en cambio, lanza `Http404` ante una clave que no esté en `VIEW_BY_KEY`. La misma regla se aplica a las pestañas de Productos: `_productos_context` cae a `DEFAULT_TAB` cuando `?tab=` no está en `TAB_BY_KEY`, mientras `productos_table` lanza `Http404` ante un `tab_key` desconocido.

Las plantillas de sección (`templates/sections/embudos.html`, `automatizaciones.html`, `mi-comercio.html`) son fragmentos sin `<html>` ni sidebar: un `div` contenedor, el incluye de la nav de columna 2 y un `<main>` con el id que HTMX usa como destino (`#embudos-panel`, `#autom-panel`, `#comercio-panel`). Dentro de ese `main` se incluye `panel_template`, la variable que el contexto resolvió. Así la nav no se re-renderiza al cambiar de página y conserva su estado de plegado.

9.3 El componente compartido de secciones de nav

Las navs con grupos (Mi comercio y la nav de cuenta de CRM y Campañas) no repiten marcado: incluyen `templates/partials/side_nav_section.html`, que renderiza un grupo titulado con sus filas. El componente tiene dos variantes elegidas por el parámetro `flat`. La de por defecto es colapsable: un `<details>` con un `<summary>` de clase `.side-nav__row--summary` y un chevron. La variante `flat=True` dibuja el mismo título y las mismas filas como un grupo estático siempre abierto, con la clase `.side-nav__row--static`, sin chevron y sin nada que plegar. Las filas hijas de ambas variantes salen de un único archivo, `templates/partials/side_nav_rows.html`, para que las dos compartan fuente.

El parámetro se propaga: `templates/partials/account_nav_panel.html` recibe `flat` y lo pasa tal cual a cada `include` del componente. Hoy, sin embargo, ningún montaje del proyecto pasa `flat=True`, ni siquiera Campañas.

Nota. El comentario del componente afirma que «Campañas mounts the account nav this way» refiriéndose a la variante plana, y eso no es cierto en el código: `templates/sections/campanas.html` incluye `partials/account_nav_panel.html` sin pasar `flat`, y `core/tests_campanas.py` fija justo lo contrario en `test_nav_sections_are_collapsible_like_the_crm`, que exige `<details>` presente y `side-nav__row--static` ausente tanto bajo `campanas` como bajo `crm`. Trata la variante plana como una capacidad disponible pero sin usar, y el comentario como desactualizado.

9.4 Embudos

La nav de Embudos es una lista plana de tres filas, sin agrupaciones colapsables, por lo que cada fila carga su propio icono. El módulo lo documenta como la variante mínima del patrón: "misma forma que `core.crm`, menos la agrupación". La vista por defecto es `embudos`, y las otras dos (`automatizaciones` e `historial-descargas`) no tienen plantilla propia, así que caen en `partials/embudos/panels/_placeholder.html`, que solo imprime el label seguido de "— próximamente".

El panel `embudos.html` bifurca sobre la variable `funnels`, que viene de `embudos.get_funnels()`. Como esa función devuelve siempre una lista vacía, en la práctica se renderiza el `{% else %}`: la tarjeta de bienvenida `_empty.html`, con el titular "Organiza tu proceso con embudos de venta" y un botón "Crear nuevo embudo". La rama `{% if %}` ya existe y recorre `funnels` en un `ul.funnel-list`, pero está declarada como no implementada; el docstring explica que cuando exista un modelo `Funnel` solo hará falta rellenar esa rama, porque vista, URL y nav ya funcionan en ambos casos.

Nota. La ilustración isométrica de la referencia se omite a propósito. En su lugar el texto se limita a una medida legible y la tarjeta conserva un padding generoso, en vez de estirarse a lo ancho del panel.

El botón apunta a la ruta real `embudo_create`, tanto por `href` como por `hx-get` sobre `#embudos-panel`. La vista devuelve `partials/embudos/panels/_nuevo.html`, una tarjeta que anuncia que el flujo de creación se construirá ahí. El patrón se repite en las otras dos pantallas: el botón va a algún sitio de verdad aunque el flujo no exista todavía.

9.5 Automatizaciones: chatbots de flujo y banner de Academy

Automatizaciones repite la lista plana de Embudos con un añadido: la fila "MIA Agentes con IA" lleva `expandable=True` y se renderiza como `<details><summary>` con chevron. Su tupla `children` está vacía, así que el toggle funciona pero no revela nada; el docstring del módulo señala que rellenar `children` es todo lo que hace falta, porque la plantilla ya itera sobre ella. El `<summary>` no es solo un desplegable: lleva el mismo cableado `hx-get/hx-target/hx-push-url` que sus hermanas planas, de modo que al pulsarlo despliega y además carga su panel.

Nota. A diferencia de las filas planas y de las filas hijas, que son elementos `<a>` con `href`, la fila expandible es un `<summary>` sin `href`. Con JavaScript deshabilitado el clic solo abre el `<details>`: no navega a la vista. La navegación sin HTMX se pierde justo en esa fila.

La vista por defecto es `chatbots-flujo` y es la única con panel propio. Ese panel apila, de arriba abajo, el banner de Academy, una cabecera "Flujos" con un `info-dot` cuyo texto de tooltip está pendiente, el botón "+ Añadir flujo" y la lista de flujos. Igual que en Embudos, la lista bifurca sobre `flows` (siempre vacía) y cae en `flows_empty.html`, un estado vacío deliberadamente escueto: solo el texto "Sin flujos" centrado, sin icono ni descripción, a diferencia de los estados vacíos del Inbox y de Embudos.

El banner de Academy es descartable mediante el mecanismo genérico de `static/js/shell.js`: el contenedor lleva `data-dismissible="academy-flujos"` y el botón de cierre `data-dismiss`; la clave se guarda en `localStorage` bajo el prefijo `dismissed:`, con los accesos envueltos en `try/catch` porque pueden fallar en ventanas privadas o con el almacenamiento bloqueado. El servidor lo renderiza siempre y es el cliente quien vuelve a ocultarlo después de cada swap de HTMX. La barra de progreso está fijada a 0% en dos sitios marcados con comentarios (el porcentaje visible y el `width` del relleno), a la espera de datos reales de Academy, y el botón de chevron es inerte porque ese comportamiento no está especificado.

Nota. La entrada automatizaciones fue retirada de `core/nav.PRIMARY_NAV` (comentario fechado el 2026-08-26), junto con `performance-hub` y `crecimiento`. Como `section()` valida el slug contra `NAV_BY_KEY`, `/s/automatizaciones/` devuelve 404; en cambio las rutas `automatizaciones_panel` y `flujo_create` siguen registradas en `core/urls.py` y no consultan la nav, así que continúan respondiendo. Restaurar la pantalla es volver a añadir el `NavItem`.

9.6 Mi comercio: el modelo Product y el catálogo con pestañas

Mi comercio es la más completa de las cuatro. Su nav de columna 2 son cuatro secciones colapsables (catalogo, ordenes, pagos, envios), cada una incluye del componente compartido `partials/side_nav_section.html`, más una fila suelta debajo, `STANDALONE`, con la clave `configuracion-comercio`. Entre secciones y fila suelta suman 12 vistas en `ALL_VIEWS`, de las cuales solo productos tiene plantilla real; el resto cae en el placeholder. El label completo de la fila suelta desborda el ancho del panel a propósito: el recorte lo hace el CSS con puntos suspensivos y el atributo `title` lleva el texto entero, de modo que la cadena truncada nunca aparece en el servidor.

El modelo `Product` mapea directamente sobre las columnas de la tabla. `category` y `brand` son texto plano por ahora y pasarán a claves ajenas cuando las páginas de Categorías y Marcas definan modelos reales. La columna "Sincronizado con" no tiene campo a propósito: será un M2M a un modelo `SalesChannel` cuando existan las integraciones de canal, y hasta entonces la plantilla renderiza una celda vacía.

Sobre la tabla hay tres pestañas de estado. `DEFAULT_TAB` es activos, no todos, siguiendo la referencia. El mapeo de slug de pestaña a valor de `Product.status` vive en `_TAB_STATUS`, junto a `TABS` en `core/comercio.py` y no junto al modelo; su comentario se limita a explicar que todos está ausente del diccionario y que por eso esa pestaña no filtra, que es como `get_products()` sabe que no debe aplicar ningún `filter`. La advertencia de que los slugs de pestaña son plurales mientras las claves de estado son singulares está en el otro extremo, dentro de `core/models.py`, encima de `Product.STATUS_CHOICES`, y es la que remite a `core.comercio._TAB_STATUS`.

Con la base vacía la tabla dibuja la cabecera sobre un cuerpo en blanco, sin mensaje alguno, que es el estado vacío de la referencia y contrasta con los estados decorados de las demás pantallas. La región intercambiable es `#product-table`, y contiene las pestañas además de las filas, igual que el paginador del CRM vive dentro de `#client-table`: al elegir una pestaña se re-renderiza su propio estado activo junto con los datos, y la cabecera y la barra de herramientas de arriba no se tocan. La cabecera de la tabla itera `TABLE_COLUMNS` para que las etiquetas y sus `info-dot` estén en un solo sitio, con la condición explícita de que las celdas rendericen los valores en ese mismo orden.

La barra de herramientas y la tabla tienen varios controles todavía sin conducta. Además del buscador "Buscar productos..." y del botón de filtro, `templates/partials/comercio/product_table.html` dibuja un checkbox de "Seleccionar todos los productos" en la cabecera y otro por fila, etiquetado con el nombre del producto; ninguno de los dos lleva HTMX ni JavaScript detrás, así que la selección no hace nada por ahora.

9.7 Campañas: segunda entrada a la nav del CRM

Campañas no tiene módulo de datos propio, ni endpoint de panel, ni plantillas de panel. Es un segundo montaje del nav "Mi cuenta" y de los paneles del CRM definidos en `core/crm.py`. En `core/views.SECTION_CONTEXT` la clave `campanas` apunta al mismo `_crm_context` que `crm`, y `templates/sections/campanas.html` incluye `partials/account_nav_panel.html` pasando `section_key="campanas"`, `panel_url_name="crm_panel"` y `panel_target="campanas-panel"`.

Lo único que difiere entre ambos montajes es el slug que aparece en las URLs empujadas con `hx-push-url` y el id del contenedor de columna 3. Los fragmentos siguen viniendo de `/crm/panel/...` bajo Campañas. La consecuencia práctica, escrita en el comentario de la plantilla, es que un panel construido una vez se enciende bajo las dos entradas del sidebar sin trabajo adicional.

9.8 Referencia

Nombre de URL	Patrón	Vista	Qué devuelve
embudos_panel	embudos/panel/<slug:view_key>/	embudos_panel	Columna 3 de Embudos
embudo_create	embudos/nuevo/	embudo_create	panels/_nuevo.html (placeholder)
automatizaciones_panel	automatizaciones/panel/<slug:view_key>/	automatizaciones_panel	Columna 3 de Automatizaciones
flujo_create	automatizaciones/flujos/nuevo/	flujo_create	panels/_nuevo_flujo.html (placeholder)
comercio_panel	comercio/panel/<slug:view_key>/	comercio_panel	Columna 3 de Mi comercio
productos_table	comercio/productos/tab/<slug:tab_key>/	productos_table	Pestañas + filas de la tabla
producto_create	comercio/productos/nuevo/	producto_create	panels/_crear.html (placeholder)
producto_import	comercio/productos/importar/	producto_import	panels/_importar.html (placeholder)

Rutas registradas en `core/urls.py` para estas pantallas

Campo	Tipo	Detalle
name	CharField	verbose_name "nombre", max_length=120
stock	PositiveIntegerField	default=0
price	DecimalField	max_digits=10, decimal_places=2
category	CharField	max_length=80, blank=True, texto plano hasta que exista modelo
brand	CharField	max_length=80, blank=True, texto plano hasta que exista modelo
status	CharField	choices=STATUS_CHOICES, default="activo", max_length=10
created_at	DateTimeField	auto_now_add=True

Campos del modelo `Product` (`core/models.py`)

Módulo	Vistas	Por defecto	Dónde vive el icono
<code>core/embudos.py</code>	3 (embudos, automatizaciones, historial-descargas)	embudos	<code>View.icon</code> en cada fila
<code>core/automatizaciones.py</code>	4 (chatbots-flujo, mia-agentes, mensajes-programados, flujos-whatsapp)	chatbots-flujo	<code>View.icon</code> en cada fila
<code>core/comercio.py</code>	12 en <code>ALL_VIEWS</code> (4 secciones + <code>STANDALONE</code>)	productos / pestaña activos	<code>Section.icon</code> y <code>SingleView.icon</code> ; <code>View</code> no tiene icono

Constantes de los tres módulos de datos

Módulo	PLACEHOLDER_PANEL
<code>core/embudos.py</code>	<code>partials/embudos/panels/_placeholder.html</code>
<code>core/automatizaciones.py</code>	<code>partials/automatizaciones/panels/_placeholder.html</code>
<code>core/comercio.py</code>	<code>partials/comercio/panels/_placeholder.html</code>

Plantillas placeholder de cada módulo

Orden	Cabecera	Origen del valor
1	Nombre	product.name
2	Stock	product.stock
3	Precio	product.price
4	Categoría	product.category
5	Marca	product.brand
6	Estado	product.get_status_display
7	Sincronizado con	Celda vacía: aún no hay campo

Columnas de la tabla de Productos (comercio.TABLE_COLUMNS)

9.9 Decisiones de diseño y trampas conocidas

El reparto del CSS no sigue las fronteras de pantalla y conviene saberlo antes de editar.

static/css/embudos.css contiene el bloque .side-nav compartido por Embudos, Automatizaciones, Mi comercio y el nav de cuenta de CRM y Campañas, define --side-nav-w y --page-gray en :root, y también estiliza la variante estática .side-nav__row--static. La extensión colapsable (.side-nav__section, .side-nav__row--summary, .side-nav__chevron) vive en static/css/automatizaciones.css, junto a la pantalla que la introdujo, que además aporta el único token nuevo de la pantalla, --progress-track: #e9e5f4, para la barra de Academy. static/css/comercio.css añade las pestañas, la tarjeta de tabla y la fila suelta, y reutiliza .table y .field de crm.css e inbox.css. Los tres se cargan globalmente desde templates/base.html.

- El segmento literal tab/ en la ruta de pestañas existe para que el slug no se trague nuevo/ e importar/, que cuelgan del mismo prefijo comercio/productos/.
- Las secciones colapsables usan <details>/<summary> nativos en lugar de JavaScript: son operables por teclado, anuncian su estado a la tecnología asistiva y funcionan sin JS. Todas arrancan cerradas.
- Las cabeceras de la tabla llevan info-dot cuyo texto de tooltip está pendiente; lo único que hoy tiene contenido es el aria-label, que es lo que lee la tecnología asistiva.
- El atributo data-nav-group sobre el contenedor de filas acota a esa lista el comportamiento de píldora activa de shell.js.
- La tabla de productos tiene ocho columnas contando la de checkboxes, y desborda en pantallas estrechas: hace scroll dentro de .product-card__scroll en lugar de recortar la tarjeta.

9.10 Pruebas que lo cubren

Los cuatro módulos de test siguen el mismo guion: que la pantalla renderice sus dos columnas, que cada fila de la nav aparezca con su enlace y su label, que el ?view= desconocido caiga al valor por defecto sin 404, que exactamente una fila quede activa, que el endpoint de panel devuelva solo el fragmento (sin <html> ni side-nav) y que ese fragmento coincida literalmente con lo que la página completa embebe. También comprueban que la sección devuelva un fragmento desnudo cuando la petición llega con la cabecera HX-Request.

Módulo	Clases	Qué fija
core/tests_embudos.py	EmbudosScreenTests, EmbudosEmptyStateTests, EmbudosPanelEndpointTests	Nav plana sin <details>, copia de la tarjeta vacía, ausencia de ilustración dentro del panel, botón cableado a embudo_create

Módulo	Clases	Qué fija
core/tests_automatizaciones.py	AutomatizacionesRemovedTests, AutomatizacionesScreenTests, AcademyBannerTests, FlujosEmptyStateTests, AutomatizacionesPanelEndpointTests	Que mia-agentes sea la única fila expandible y su ranura de hijos esté vacía, que el <summary> también haga swap del panel, y el banner descartable con progreso a 0%
core/tests_comercio.py	ComercioScreenTests, ProductosPanelTests, ProductosTabTests, ProductosTableEndpointTests, ComercioPanelEndpointTests	Cuatro <details> cerrados, 12 vistas, truncado por CSS y no por servidor, filtrado por pestaña, checkbox de selección, y que tab/ no se trague las rutas de botón
core/tests_campanas.py	CampanasScreenTests	Que Campañas monte las mismas secciones y vistas del CRM, comparta el endpoint crm_panel, use #campanas-panel como destino, renderice <details> y no side-nav__row- -static, y que paginar no reescriba la URL a /s/crm/

Módulos de test y clases

Nota. Todas las clases de core/tests_automatizaciones.py salvo AutomatizacionesRemovedTests llevan @skip(TAB_REMOVED), porque con la pestaña fuera del sidebar /s/automatizaciones/ devuelve 404. Se saltaron en lugar de borrarse: cuando la entrada vuelva, basta con quitar los decoradores.

En core/tests_comercio.py no hay datos semilla: el helper make_product(name, status) crea cada fila con stock=3 y price=Decimal("9.99"), y las pruebas del estado vacío se apoyan justamente en que la base arranca limpia. La prueba test_row_renders_every_column_value verifica el valor exacto de la celda de stock (<td>3</td>) para que un cambio de orden de columnas no pase inadvertido, y la pareja test_unknown_tab_falls_back_instead_of_404 / test_unknown_tab_is_404 fija la asimetría entre la pantalla completa y el endpoint de tabla.

Archivos de referencia: core/embudos.py, core/automatizaciones.py, core/comercio.py, core/models.py, core/views.py, core/urls.py, core/nav.py, core/crm.py, core/migrations/0002_product.py, core/tests_embudos.py, core/tests_automatizaciones.py, core/tests_comercio.py, core/tests_campanas.py, templates/base.html, templates/sections/embudos.html, templates/sections/automatizaciones.html, templates/sections/mi-comercio.html, templates/sections/campanas.html, templates/sections/crm.html, templates/partials/side_nav_section.html, templates/partials/side_nav_rows.html, templates/partials/account_nav_panel.html, templates/partials/embudos/nav_panel.html, templates/partials/embudos/panels/embudos.html, templates/partials/embudos/panels/_empty.html, templates/partials/embudos/panels/_nuevo.html, templates/partials/embudos/panels/_placeholder.html, templates/partials/automatizaciones/nav_panel.html, templates/partials/automatizaciones/academy_banner.html, templates/partials/automatizaciones/flows_empty.html, templates/partials/automatizaciones/panels/chatbots-flujo.html, templates/partials/automatizaciones/panels/_nuevo_flujo.html, templates/partials/automatizaciones/panels/_placeholder.html, templates/partials/comercio/nav_panel.html, templates/partials/comercio/product_table.html, templates/partials/comercio/panels/productos.html, templates/partials/comercio/panels/_crear.html, templates/partials/comercio/panels/_importar.html, templates/partials/comercio/panels/_placeholder.html, static/css/embudos.css, static/css/automatizaciones.css, static/css/comercio.css, static/js/shell.js

10. Modelo de datos y migraciones

Referencia de la capa de persistencia de MVP-CRM: los cinco modelos de la app core (Client, Product, ClientList, MessageTemplate, CalendarEvent) y los cuatro de la app messaging (Tag, Conversation, ConversationTag, Message), con sus campos, relaciones, Meta, propiedades derivadas, el historial completo de migraciones, los comandos de gestión que escriben en estas tablas y qué queda registrado en el admin de Django.

10.1 Propósito y alcance

El esquema de MVP-CRM vive en dos aplicaciones Django declaradas en `INSTALLED_APPS` de `config/settings.py`: `core`, que guarda las entidades del CRM propiamente dicho, y `messaging`, que guarda las conversaciones y los mensajes que alimentan la pantalla Inbox. Entre las dos suman nueve modelos y once migraciones. No hay modelos en ninguna otra parte del proyecto: `core` tiene más de veinte módulos además de `models.py` (`views.py`, `urls.py`, `clientes.py`, `agents.py`, `middleware.py`, `storage.py`, `nav.py`, `crm.py`, `inbox.py`, `plantillas.py`, `calendario.py`, `comercio.py`, `mensajería.py`, `embudos.py`, `automatizaciones.py` y ocho módulos estadísticas*.py), pero una búsqueda de `models.Model` sólo devuelve `core/models.py` y `messaging/models.py`.

Las dos apps no se configuran igual. `core/apps.py` es un `AppConfig` pelado, con `name = 'core'` y nada más. `messaging/apps.py` declara `default_auto_field = "django.db.models.BigAutoField"`, `verbose_name = "mensajería"` y un docstring que justifica la separación de apps: todo lo que habla con WhatsApp, Meta o Twilio vive en `messaging`, de modo que cambiar o añadir un proveedor nunca toca una vista ni una plantilla.

La base de datos se elige en tiempo de arranque. Si existe la variable de entorno `DATABASE_URL` y no se está ejecutando la suite de tests, `config/settings.py` la parsea con `dj_database_url.parse` pasando `conn_max_age=600` y `conn_health_checks=True`; en caso contrario cae a SQLite en `BASE_DIR / 'db.sqlite3'`. El comentario del propio archivo explica el porqué de usar `dj_database_url` en lugar de descomponer la URL a mano: los parámetros de query de la cadena de conexión viajan a `OPTIONS`, y copiar sólo `NAME/USER/PASSWORD/HOST/PORT` perdería el `sslmode=require&channel_binding=require` de Neon, con lo que libpq caería a `sslmode=prefer` y aceptaría en silencio una conexión sin cifrar. El flag `TESTING`, que es `'test' in sys.argv`, fuerza SQLite aunque el `.env` del desarrollador apunte a una base real.

El proyecto trabaja con `USE_TZ = True` y `TIME_ZONE = 'UTC'`. Todo lo que se guarda en un `DateTimeField` está en UTC; la conversión a la zona en la que el usuario introduce y lee fechas ocurre en la capa de vista, con `CALENDAR_TZ` de `core/calendario.py`, que es `ZoneInfo("America/Bogota")`.

10.2 Cómo funciona: un solo contacto para todo el CRM

La pieza central es `core.models.Client`. El docstring de `messaging/models.py` lo dice explícitamente: la persona con la que se chatea en el Inbox es la misma fila de la tabla Clientes del CRM, y el procesamiento de webhooks hace `upsert` de `Client` por número de teléfono en `services.process_inbound_events`. De ahí cuelgan las conversaciones (`Conversation.contact`), los eventos de calendario (`CalendarEvent.contact`) y las listas de difusión (`ClientList.clients`).

Ningún constraint de base de datos respalda esa identidad. `Client.phone` es un `CharField` normal, sin `unique=True`, y `messaging.services._upsert_contact` localiza al cliente con `Client.objects.filter(phone=phone).first()`, es decir por coincidencia exacta de la cadena. La normalización vive únicamente en el código de formulario: `core.clientes.normalize_phone` reduce lo que se teclea a + seguido de dígitos, y su docstring de módulo advierte de la consecuencia de saltárselo, un número guardado como `" +57 316 768 7288"` crearía un segundo cliente la próxima vez que esa

persona escriba.

Conversation representa un hilo con un cliente en un canal. Un mismo cliente puede tener varias filas a lo largo del tiempo, pero sólo un hilo activo por canal. Esa regla tampoco está en el esquema:

Conversation no declara ninguna UniqueConstraint sobre (contact, channel); la garantiza sólo `_get_or_create_open_conversation`, que busca un hilo del mismo contacto y canal excluyendo los `resolved` antes de crear uno nuevo. Dos campos están desnormalizados a propósito y los mantiene `messaging/services.py`: `last_message_at`, que es la clave de ordenación de la lista, y `last_inbound_at`, que alimenta la comprobación de la ventana de 24 horas de WhatsApp sin volver a consultar la tabla de mensajes fila a fila.

Varias columnas son marcadores de posición declarados como tales en el código y conviene no confundirlas con decisiones finales. `ClientList.created_by`, `MessageTemplate.created_by` y `MessageTemplate.team` son texto plano hasta que la aplicación crezca hacia usuarios modelados en base de datos. En `Product`, `category` y `brand` son texto plano hasta que las páginas Categorías y Marcas definan modelos, y el campo "Sincronizado con" directamente no existe: será un M2M a un futuro modelo `SalesChannel` y, mientras tanto, la tabla pinta una celda vacía.

Nota. Los `ForeignKey` a `settings.AUTH_USER_MODEL` son todos `null=True`, `blank=True` con `on_delete=SET_NULL`. El docstring de `CalendarEvent` atribuye ese patrón a que "la app aún no tiene login real", pero esa frase quedó desfasada: `core.middleware.LoginRequiredMiddleware` protege toda la app y `core.views.login_view` hace `auth_login` contra el User espejo del agente que mantiene `core/agents.py` a partir de `APP_AGENTS`. `calendar_event_create` asigna `event.created_by = request.user` cuando hay sesión, así que los eventos creados desde la UI sí llevan usuario; `assigned_to` se toma del campo del modal.

10.3 Referencia de modelos: app core

Los cinco modelos de `core/models.py`. Todos declaran `verbose_name` y `verbose_name_plural` en español, de modo que el admin y los formularios generados hablan el idioma de la interfaz. Sus claves primarias son `BigAutoField`, pero no porque haya un ajuste vigente que lo imponga:

`config/settings.py` no define `DEFAULT_AUTO_FIELD` y `core/apps.py` no declara `default_auto_field`, así que las tablas de core llevan `BigAutoField` sólo porque así se generaron las migraciones. La única declaración explícita del proyecto está en `MessagingConfig`.

Campo	Tipo	Notas
<code>first_name</code>	<code>CharField(max_length=80)</code>	<code>verbose_name</code> "nombres", obligatorio
<code>last_name</code>	<code>CharField(max_length=80)</code>	<code>verbose_name</code> "apellidos", <code>blank</code>
<code>phone</code>	<code>CharField(max_length=20)</code>	<code>verbose_name</code> "teléfono", <code>help_text</code> "E.164, e.g. +573167687288"; no es <code>unique</code>
<code>country</code>	<code>CharField(max_length=2)</code>	<code>blank</code> , <code>help_text</code> "ISO 3166-1 alpha-2, drives the flag"
<code>email</code>	<code>EmailField</code>	<code>verbose_name</code> "mail", <code>blank</code>
<code>channel</code>	<code>CharField(max_length=20)</code>	<code>blank</code> , choices <code>whatsapp / messenger / instagram / facebook / tiktok</code>
<code>created_at</code>	<code>DateTimeField</code>	<code>auto_now_add</code>

core.Client — Meta: `verbose_name` "cliente" / "clientes", `ordering` ["first_name", "last_name"]

- Propiedades derivadas: `full_name` (une y recorta nombre y apellidos), `initials` (hasta dos letras para los círculos de avatar del Inbox, devuelve "?" si no hay ninguna), `has_whatsapp` (cierto sólo si `channel == "whatsapp"`), `flag` (emoji de bandera construido con indicadores regionales a partir de `country`, cadena vacía si el código falta o está mal formado) y `whatsapp_url` (enlace `https://wa.me/` con sólo los dígitos del teléfono).

- Relaciones inversas: `conversations` (desde `messaging.Conversation`), `calendar_events` (desde `core.CalendarEvent`) y `client_lists` (desde el M2M de `core.ClientList`).

Campo	Tipo	Notas
<code>name</code>	<code>CharField(max_length=120)</code>	<code>verbose_name</code> "nombre"
<code>stock</code>	<code>PositiveIntegerField</code>	<code>default</code> 0
<code>price</code>	<code>DecimalField(max_digits=10, decimal_places=2)</code>	<code>verbose_name</code> "precio"
<code>category</code>	<code>CharField(max_length=80)</code>	blank, texto plano de momento
<code>brand</code>	<code>CharField(max_length=80)</code>	blank, texto plano de momento
<code>status</code>	<code>CharField(max_length=10)</code>	choices activo / inactivo, <code>default</code> "activo"
<code>created_at</code>	<code>DateTimeField</code>	<code>auto_now_add</code>

core.Product — Meta: `verbose_name` "producto" / "productos", `ordering` ["name"]

Nota. Las pestañas de Productos filtran por `status`, pero sus slugs son plurales ("activos") mientras las claves del modelo son singulares; la traducción entre ambos vive en `core.comercio._TAB_STATUS`, junto a la lista de pestañas.

Campo	Tipo	Notas
<code>name</code>	<code>CharField(max_length=120)</code>	<code>verbose_name</code> "nombre del grupo"
<code>clients</code>	<code>ManyToManyField(Client)</code>	<code>related_name</code> "client_lists", blank, <code>verbose_name</code> "clientes"
<code>created_by</code>	<code>CharField(max_length=80)</code>	blank, texto plano hasta que existan usuarios modelados
<code>created_at</code>	<code>DateTimeField</code>	<code>verbose_name</code> "fecha", <code>auto_now_add</code>

core.ClientList — Meta: `verbose_name` "lista de clientes" / "listas de clientes", `ordering` ["name"]

Nota. "Número de contactos" no es una columna: se deriva del M2M `clients` anotando la consulta en la vista, para que el conteo no pueda discrepar de los miembros reales de la lista.

Campo	Tipo	Notas
<code>name</code>	<code>CharField(max_length=120)</code>	sólo minúsculas, dígitos y <code>_</code> ; la regex vive en <code>core.plantillas.NAME_RE</code> , que es <code>^[a-z0-9_]+\$</code>
<code>category</code>	<code>CharField(max_length=20)</code>	choices marketing / utility / authentication, <code>default</code> "marketing"
<code>sub_type</code>	<code>CharField(max_length=30)</code>	choices custom / limited_time_offer / carousel / auth_code, <code>default</code> "custom"
<code>language</code>	<code>CharField(max_length=10)</code>	<code>default</code> "es"
<code>team</code>	<code>CharField(max_length=80)</code>	blank, texto plano
<code>header_type</code>	<code>CharField(max_length=10)</code>	choices none / text / image / video / document, <code>default</code> "none"
<code>header_text</code>	<code>CharField(max_length=60)</code>	blank
<code>header_media</code>	<code>FileField(upload_to="plantillas/")</code>	blank
<code>body</code>	<code>TextField</code>	blank, <code>verbose_name</code> "cuerpo"
<code>body_sample_values</code>	<code>JSONField</code>	<code>default=list</code> , blank; un valor de ejemplo por variable <code>{{n}}</code>
<code>footer</code>	<code>CharField(max_length=60)</code>	blank
<code>buttons</code>	<code>JSONField</code>	<code>default=list</code> , blank; dicts <code>{"type": "quick_reply" "url" "phone", "text": ...}</code>
<code>is_active</code>	<code>BooleanField</code>	<code>default</code> True, <code>verbose_name</code> "activo"
<code>status</code>	<code>CharField(max_length=10)</code>	choices pendiente / aceptada / rechazada, <code>default</code> "pendiente"

Campo	Tipo	Notas
rejection_reason	TextField	blank
created_by	CharField(max_length=80)	blank, texto plano
created_at / updated_at	DateTimeField	auto_now_add / auto_now

core.MessageTemplate — Meta: *verbose_name* "plantilla de WhatsApp" / "plantillas de WhatsApp", *ordering* ["name"], *UniqueConstraint(name, language)* llamada *unique_template_name_per_language*

Nota. `header_media` escribe por el backend de almacenamiento por defecto, que no siempre es el sistema de ficheros: `config/settings.py` cambia `STORAGES['default']` a `core.storage.VercelBlobStorage` en cuanto existe `BLOB_READ_WRITE_TOKEN` y no se está en tests, porque las funciones de Vercel son de sólo lectura y efímeras. En local, y siempre en la suite de tests, los archivos van a `MEDIA_ROOT`.

Campo	Tipo	Notas
title	CharField(max_length=120)	<i>verbose_name</i> "título"
description	TextField	blank
start / end	DateTimeField	obligatorios, almacenados en UTC
all_day	BooleanField	default False; los eventos de día completo empiezan a medianoche y su fin es exclusivo
contact	FK a <i>core.Client</i>	SET_NULL, null, blank, <i>related_name</i> "calendar_events"
assigned_to	FK a <i>AUTH_USER_MODEL</i>	SET_NULL, null, blank, <i>related_name</i> "calendar_events"
created_by	FK a <i>AUTH_USER_MODEL</i>	SET_NULL, null, blank, <i>related_name</i> "created_calendar_events"
event_type	CharField(max_length=20)	choices llamada / reunion / seguimiento / otro, default "reunion"
reminder_minutes_before	PositiveIntegerField	null, blank
created_at / updated_at	DateTimeField	auto_now_add / auto_now

core.CalendarEvent — Meta: *verbose_name* "evento de calendario" / "eventos de calendario", *ordering* ["start"], *Index(start, assigned_to)* llamado *calendar_start_advisor_idx*

Nota. El docstring de *CalendarEvent* justifica su existencia: `contact` es la razón por la que un calendario vive dentro de un CRM, porque un evento enlaza con un cliente y deja su ficha a un clic. `event_type` elige el color reutilizando un par de la paleta de etiquetas: `core.calendario.EVENT_TYPES` se construye recorriendo `CalendarEvent.TYPE_CHOICES` y buscando cada clave en `_TYPE_COLORS`, de forma que un tipo añadido al modelo sin color falla en el import.

10.4 Referencia de modelos: app messaging

`messaging/models.py` define además una constante de módulo, `SERVICE_WINDOW = timedelta(hours=24)`, que es el plazo tras el último mensaje del cliente durante el cual WhatsApp permite respuestas libres. El comentario aclara que se trata de una regla de plataforma y que por eso se aplica en `services.send_message`, no se deja a la interfaz. Los cuatro modelos declaran *verbose_name* en español: etiqueta, conversación, etiqueta de conversación y mensaje.

Campo	Tipo	Notas
name	CharField(max_length=40)	<i>verbose_name</i> "nombre"
color	CharField(max_length=10)	choices green, yellow, purple, blue, red, orange, teal, pink, indigo, gray; default "gray"

Campo	Tipo	Notas
created_by	FK a AUTH_USER_MODEL	SET_NULL, null, blank, related_name "tags_created", verbose_name "creada por"
created_at	DateTimeField	auto_now_add
is_archived	BooleanField	default False, verbose_name "archivada"

messaging.Tag — Meta: verbose_name "etiqueta" / "etiquetas", ordering ["name"], UniqueConstraint(Lower("name")) llamada tag_name_ci_unique

Campo	Tipo	Notas
contact	FK a core.Client	CASCADE, related_name "conversations", obligatorio
channel	CharField(max_length=20)	choices whatsapp, messenger, instagram-dm, facebook, instagram, tiktok-dm, tiktok-coment; default "whatsapp"
status	CharField(max_length=10)	choices open / pending / resolved, default OPEN
assigned_to	FK a AUTH_USER_MODEL	SET_NULL, null, blank, related_name "conversations"; NULL es "Sin asignar"
last_message_at	DateTimeField	null, blank; desnormalizado, clave de orden de la lista
last_inbound_at	DateTimeField	null, blank; desnormalizado, alimenta la ventana de 24 h
unread_count	PositiveIntegerField	default 0
tags	M2M a Tag through ConversationTag	related_name "conversations", blank
created_at	DateTimeField	auto_now_add

messaging.Conversation — Meta: verbose_name "conversación" / "conversaciones", ordering ["-last_message_at"], Index(status, last_message_at)

- `is_within_24h_window`: falso si `last_inbound_at` es None; en otro caso, cierto mientras `timezone.now() - last_inbound_at` sea menor que `SERVICE_WINDOW`. La ventana se abre y reabre con cada mensaje entrante.
- `icon_template`: devuelve `icons/brands/<canal>.svg`, colapsando ambas superficies de TikTok en una sola marca porque comparten logotipo.

Campo	Tipo	Notas
conversation	FK a Conversation	CASCADE, related_name "conversation_tags"
tag	FK a Tag	CASCADE, related_name "conversation_tags"
tagged_by	FK a AUTH_USER_MODEL	SET_NULL, null, blank, related_name "tags_applied"
tagged_at	DateTimeField	auto_now_add

messaging.ConversationTag — tabla intermedia explícita; verbose_name "etiqueta de conversación"; UniqueConstraint(conversation, tag) llamada conversationtag_unique

Campo	Tipo	Notas
conversation	FK a Conversation	CASCADE, related_name "messages"
direction	CharField(max_length=8)	choices inbound / outbound
body	TextField	blank
media_url	URLField	blank
media_type	CharField(max_length=16)	blank; image, video, audio, document o sticker
status	CharField(max_length=10)	choices derivadas de MessageStatus, default MessageStatus.QUEUED.value
provider_message_id	CharField(max_length=255)	unique, null, blank; NULL si el envío falló antes de recibir id

Campo	Tipo	Notas
timestamp	DateTimeField	default timezone.now
sent_by	FK a AUTH_USER_MODEL	SET_NULL, null, blank, related_name "sent_messages"

messaging.Message — Meta: verbose_name "mensaje" / "mensajes", ordering ["timestamp", "id"], Index(conversation, timestamp)

- `is_outbound`: compara `direction` con `Message.OUTBOUND`.
- `is_inline_image`: cierto si hay `media_url` y `media_type` es `image` o `sticker`; los stickers son WebP y el navegador los pinta de forma nativa.
- `display_body`: devuelve cadena vacía cuando la imagen se renderiza en línea y el cuerpo es exactamente el marcador de `MEDIA_PLACEHOLDERS`; los pies de foto reales sí se muestran. El body crudo conserva el marcador para que la vista previa de la lista de conversaciones diga algo.
- `status_icon_template`: mapea cada estado a su icono (`icons/clock.svg`, `icons/check.svg`, `icons/check-check.svg` para `delivered` y `read`, `icons/alert-circle.svg` para `failed`).

10.5 Mapa de relaciones

```

core.Client ■■■1:N■■■> messaging.Conversation (contact, CASCADE, rn=conversations)
core.Client ■■■1:N■■■> core.CalendarEvent (contact, SET_NULL, rn=calendar_events)
core.Client <■■■M:N■■■> core.ClientList (clients, rn=client_lists)

messaging.Conversation ■■■1:N■■■> messaging.Message (conversation, CASCADE, rn=messages)
messaging.Conversation <■■■M:N■■■> messaging.Tag (through messaging.ConversationTag)

AUTH_USER_MODEL ■■■1:N■■■> Conversation.assigned_to, Message.sent_by,
Tag.created_by, ConversationTag.tagged_by,
CalendarEvent.assigned_to, CalendarEvent.created_by
(todas SET_NULL, null=True, blank=True)

core.Product y core.MessageTemplate: sin relaciones, tablas sueltas.
```

Hay exactamente cuatro `ForeignKey` con `on_delete=CASCADE`, y las cuatro están en `messaging`: `Conversation.contact`, `ConversationTag.conversation`, `ConversationTag.tag` y `Message.conversation`. En la práctica eso significa que borrar una conversación borra sus mensajes y sus enlaces de etiqueta, que borrar una etiqueta borra sus enlaces, y que borrar un contacto arrastra sus conversaciones enteras. Todo lo demás es `SET_NULL`, lo que tiene una consecuencia práctica documentada en `messaging/management/commands/reset_conversations.py`: como `CalendarEvent.contact` es `SET_NULL`, los eventos sobreviven al contacto al que apuntaban.

10.6 Historial de migraciones

App	Migración	Qué introduce
core	0001_initial	Crea <code>Client</code> con sus siete campos y su <code>Meta</code>
core	0002_product	Crea <code>Product</code>
core	0003_clientlist	Crea <code>ClientList</code> y el M2M <code>clients</code> hacia <code>core.client</code>
core	0004_messagetemplate	Crea <code>MessageTemplate</code> en su forma de tabla: <code>template_type</code> , <code>category</code> y <code>text</code> como texto libre
core	0005_rename_template_fields	Escrita a mano: <code>RenameField</code> de <code>text</code> a <code>body</code> y de <code>template_type</code> a <code>sub_type</code>
core	0006_messagetemplate_body_sample_values_and_more	Diez <code>AddField</code> del editor (incluidos <code>language</code> , <code>header_type</code> , <code>body_sample_values</code> , <code>buttons</code>), tres <code>AlterField</code> , un <code>RunPython</code> <code>dedupe_names</code> y el <code>AddConstraint</code> <code>unique_template_name_per_language</code>
core	0007_map_legacy_template_values	<code>RunPython</code> <code>map_legacy_values</code> : pliega las categorías y tipos de texto libre a las claves de los nuevos enums
core	0008_calendarevent	Crea <code>CalendarEvent</code> con sus tres FK y el índice <code>calendar_start_advisor_idx</code>
messaging	0001_initial	Crea <code>Conversation</code> y <code>Message</code> más sus dos índices; depende de <code>core.0004_messagetemplate</code>
messaging	0002_tag_conversationtag_conversation_tags_and_more	Crea <code>Tag</code> y <code>ConversationTag</code> , añade el M2M <code>tags</code> y las restricciones <code>tag_name_ci_unique</code> y <code>conversationtag_unique</code>
messaging	0003_message_media_type	Añade <code>Message.media_type</code> como <code>CharField(max_length=16, blank=True)</code>

Once migraciones; todas generadas con Django 6.1, tres de ellas editadas o escritas a mano

```
python manage.py migrate
python manage.py makemigrations core messaging
```

Nota. La migración 0005 se mantuvo separada de la 0006 a propósito, según su propio docstring, para que los renombrados no puedan leerse jamás como un borrar-y-crear que perdería los datos de la columna.

10.7 Comandos que escriben en estas tablas

`messaging/management/commands/` contiene tres comandos que tocan directamente los modelos descritos aquí. `seed_conversations` llena el Inbox de conversaciones de demostración obviamente falsas (apellidos "Pruebas", "Demo", "Ejemplo" y teléfonos bajo el prefijo `FAKE_PHONE_PREFIX`, que es "+5730000000"), repartidas por canales, por estados de asignación y a ambos lados de la frontera de 24 horas; admite `--fresh` para borrar la tanda anterior, reconocida por ese prefijo. `simulate_inbound` construye la carga del webhook del proveedor falso, la firma y llama a la vista `webhook` real, de modo que el mensaje recorre la comprobación de firma, el parseo, la idempotencia y el `upsert` de conversación tal como lo hará un proveedor de verdad.

`reset_conversations` vacía el Inbox borrando mensajes, enlaces de etiqueta, conversaciones y contactos, en ese orden explícito y dentro de una única transacción para que los números reportados sean los realmente borrados. Su docstring recoge las dos salvaguardas: es una simulación por defecto, y

sólo con `--yes` borra de verdad, imprimiendo antes el motor y el host de la base de datos a la que apunta, porque "qué base acabo de borrar" es una mala pregunta para hacerse después. La opción `--keep-clients` borra conversaciones y mensajes pero conserva los contactos, para que el historial de clientes sobreviva a un reinicio del Inbox.

```
python manage.py seed_conversations --fresh
python manage.py simulate_inbound "+573000000001" "Hola, ¿sigue disponible?"
python manage.py reset_conversations
python manage.py reset_conversations --yes --keep-clients
```

10.8 Decisiones de diseño y trampas conocidas

Las listas de choices de canal están duplicadas a conciencia y no significan lo mismo.

`Client.CHANNEL_CHOICES` tiene cinco claves y describe el canal grueso del cliente en la tabla del CRM; `Conversation.CHANNEL_CHOICES` tiene siete y dice dónde vive ese hilo concreto, separando `instagram` de `instagram-dm` y `tiktok-dm` de `tiktok-coment`. Sólo las siete de `Conversation` coinciden exactamente con las claves de `core.inbox.CANALES`, de forma que un filtro de canal del Inbox sea una búsqueda directa `channel=key`. `Client` usa `tiktok`, clave que no aparece en `CANALES`, y por eso `messaging.services._client_channel` traduce cualquier canal de conversación que empiece por `tiktok` a esa clave más gruesa.

En `MessageTemplate`, `is_active` y `status` son campos distintos porque se mueven de forma independiente: `is_active` es el interruptor propio de la cuenta y `status` es el veredicto de aprobación de WhatsApp, así que una plantilla aceptada puede estar apagada. La pestaña Desactivadas filtra por el interruptor y las demás por el estado. Las listas de opciones del modelo son uniones planas para que cualquier valor almacenado se pueda mostrar; qué subtipos ofrece cada categoría, la lista de idiomas y la validación del editor viven en `core/plantillas.py`.

La restricción de unicidad de plantillas es sobre el par `(name, language)` porque Meta acota los nombres por idioma. Como violarla era legal antes de la migración 0006, esa migración ejecuta `dedupe_names` justo antes del `AddConstraint`: recorre las plantillas ordenadas por nombre e id y renombra las repetidas a "welcome (2)", "welcome (3)" y así. La 0007 no tiene inversa porque las cadenas de texto libre originales son irre recuperables y las claves nuevas ya son válidas bajo el esquema antiguo; los valores desconocidos caen a `marketing` y `custom` en lugar de fallar.

En `Tag`, la unicidad es insensible a mayúsculas mediante `UniqueConstraint(Lower("name"))`, de modo que "ventas" y "VENTAS" son la misma etiqueta. El código deja ahí un `TODO(tenancy)` explícito: todavía no existe un modelo de organización o cuenta, así que los nombres son únicos globalmente, y cuando llegue la multi-inquilinación habrá que añadir la FK de organización y acotar la restricción a `(org, Lower(name))`. El borrado de etiquetas es archivado, no borrado: una etiqueta archivada desaparece de los selectores pero sigue en todas las conversaciones donde se aplicó, porque quitar una etiqueta del histórico de 500 chats porque alguien ordenó el selector reescribiría el pasado en silencio.

Nota. `Conversation.tags` usa una tabla intermedia explícita en lugar de un `ManyToManyField` pelado precisamente para poder responder "quién etiquetó este chat y cuándo", información que un M2M simple tira a la basura.

`Message.provider_message_id` es `unique=True` para que los reintentos de webhook no puedan insertar el mismo mensaje dos veces: la base de datos respalda la comprobación que ya hace la aplicación, y es la clave con la que las devoluciones de estado localizan su mensaje. Admite `NULL` para las filas salientes cuyo envío falló antes de que el proveedor asignara un id. `Message.status` sólo tiene sentido en mensajes salientes; los entrantes se guardan como `delivered` porque, por definición, ya llegaron. Los colores de `Tag` son tokens de paleta y no hexadecimales crudos: cada token mapea a un par de variables

CSS --tag-<token>-bg/-fg en `static/css/tags.css`, de forma que ninguna píldora resulte ilegible y un futuro tema oscuro reestilice todas a la vez.

10.9 Registro en el admin de Django

El admin está montado en `admin/` desde `config/urls.py`, pero el registro es asimétrico. `core/admin.py` no registra nada: contiene sólo el `from django.contrib import admin` y el comentario `# Register your models here.` generado por Django, de modo que `Client`, `Product`, `ClientList`, `MessageTemplate` y `CalendarEvent` no aparecen en el admin y se gestionan exclusivamente desde las pantallas del CRM. `messaging/admin.py` sí registra sus cuatro modelos, y su docstring lo describe como una ventana en crudo a las conversaciones mientras el Inbox es la superficie principal, útil para cambiar estado o asignación durante el desarrollo.

Modelo	list_display	Otros ajustes
Conversation	contact, channel, status, assigned_to, unread_count, last_message_at	list_filter por channel y status; inline MessageInline (TabularInline, extra=0, timestamp de sólo lectura)
Message	conversation, direction, body, status, timestamp	list_filter por direction y status; search_fields body y provider_message_id
Tag	name, color, is_archived, created_by, created_at	list_filter por is_archived y color; search_fields name
ConversationTag	conversation, tag, tagged_by, tagged_at	list_filter por tag

Registros de `messaging/admin.py`

Nota. `core.middleware.LoginRequiredMiddleware` deja `/admin/` fuera de su puerta con el resto de prefijos exentos, porque el admin lo gatea `django.contrib.auth` por su cuenta. Los User que existen son los espejos creados por `core/agents.py`, con contraseña inutilizable, así que entrar al admin requiere un superusuario creado aparte.

10.10 Pruebas que lo cubren

No hay fixtures ni seeds en las pruebas: cada módulo crea sus propios datos, algo que los docstrings de los helpers dejan escrito literalmente en `core/tests_calendario.py`, `core/tests_comercio.py` y `core/tests_mensajeria.py` ("tests create their own data, there are no seeds"). Las propiedades derivadas de `Client` se prueban en `core/tests_crm.py`, en la clase `ClientModelTests`: que `full_name` une y recorta, que `flag` funciona con el código en mayúsculas o minúsculas y devuelve cadena vacía para "", "C", "COL" y "12", que `has_whatsapp` sólo es cierto para el canal whatsapp y que `whatsapp_url` elimina el formato del teléfono.

El conteo de contactos de una lista se comprueba en otra clase del mismo archivo, `ClientListTests`, que no es una prueba de modelo sino del panel "Lista de clientes": crea dos clientes, los asocia a una `ClientList` y verifica sobre el HTML renderizado de la fila que aparece `<td>2</td>`, es decir el conteo derivado del M2M tal como lo ve el usuario.

El lado de `messaging` se cubre en `messaging/tests.py`. `WebhookTests` comprueba que un entrante crea contacto, conversación y mensaje, que el mismo `provider_message_id` dos veces produce un solo mensaje sin inflar `unread_count`, que un segundo mensaje reutiliza la conversación abierta y que un entrante sobre un hilo resuelto abre uno nuevo. `SendwindowTests` ejerce la ventana de 24 horas, incluido el caso de un hilo al que el cliente nunca escribió. `TagServiceTests` cubre la unicidad insensible a mayúsculas, la idempotencia y auditoría de `ConversationTag`, y la regla de los 500 chats: archivar una etiqueta no borra su historial. `InlineMediaTests` recorre `media_type` desde el evento hasta la fila y comprueba `is_inline_image` y `display_body`, incluidas las filas anteriores a la existencia del campo, que conservan su enlace de descarga.

`core/tests_calendario.py` cubre `CalendarEvent` a través de la pantalla: el panel, el feed JSON de eventos y su contrato de zona horaria (guardar en UTC, mostrar en Bogotá), las mutaciones desde el modal y desde la rejilla, y las preferencias de sesión. `core/tests_mensajeria.py` cubre `MessageTemplate` desde la pantalla de Configuración de mensajería: la tabla con pestañas, el estado vacío y el editor. `messaging/tests_reset.py` verifica las salvaguardas del comando `reset_conversations`: que la simulación por defecto no borra nada, que nombra la base de datos a la que apunta, que `--yes` vacía el Inbox y que `--keep-clients` conserva los contactos.

Archivos de referencia: `core/models.py`, `messaging/models.py`, `core/admin.py`, `messaging/admin.py`, `core/apps.py`, `messaging/apps.py`, `config/settings.py`, `core/clientes.py`, `core/calendario.py`, `core/plantillas.py`, `core/inbox.py`, `core/middleware.py`, `core/agents.py`, `core/storage.py`, `core/views.py`, `messaging/services.py`, `core/migrations/0001_initial.py`, `core/migrations/0002_product.py`, `core/migrations/0003_clientlist.py`, `core/migrations/0004_messagetemplate.py`, `core/migrations/0005_rename_template_fields.py`, `core/migrations/0006_messagetemplate_body_sample_values_and_more.py`, `core/migrations/0007_map_legacy_template_values.py`, `core/migrations/0008_calendarevent.py`, `messaging/migrations/0001_initial.py`, `messaging/migrations/0002_tag_conversationtag_conversation_tags_and_more.py`, `messaging/migrations/0003_message_media_type.py`, `messaging/management/commands/reset_conversations.py`, `messaging/management/commands/seed_conversations.py`, `messaging/management/commands/simulate_inbound.py`, `core/tests_crm.py`, `core/tests_calendario.py`, `core/tests_mensajeria.py`, `messaging/tests.py`, `messaging/tests_reset.py`

11. Configuración, despliegue y operación

Cómo se configura MVP-CRM por entorno (carga de .env, precedencia de variables reales, catálogo completo de variables), qué hace falta para arrancar en local (venv, migraciones, credenciales de la puerta de entrada, seeds), cómo se elige la base de datos (DATABASE_URL con dj_database_url y Neon, o SQLite), cómo se despliega en Vercel (vercel.json, ALLOWED_HOSTS, estáticos y medios en Vercel Blob), qué hace la bandera TESTING, qué papel juegan las páginas legales públicas en la revisión de la app de Meta y por qué está cada dependencia de requirements.txt.

11.1 Propósito y alcance

Este capítulo describe la capa de configuración y operación del proyecto: todo lo que hay que saber para levantar MVP-CRM en una máquina nueva, entender de dónde sale cada valor de configuración y llevarlo a producción en Vercel. Las piezas implicadas son config/settings.py (el único lugar donde se lee el entorno), manage.py, config/wsgi.py y config/asgi.py como puntos de entrada, vercel.json para el despliegue, requirements.txt y .env.example como contrato de dependencias y de variables, core/storage.py para los medios en producción y las páginas legales públicas de templates/legal/.

Queda fuera el funcionamiento interno de las secciones del producto (Inbox, CRM, Embudos, Estadísticas) y la abstracción de proveedores de mensajería, que se documentan en sus propios capítulos. Aquí solo aparecen las variables que esos módulos consumen, para que el catálogo de configuración esté completo en un único sitio.

11.2 Puesta en marcha local

Los valores por defecto de config/settings.py están elegidos para que un checkout limpio arranque sin configuración de infraestructura: base de datos SQLite y proveedor de mensajería fake. La secuencia que documenta el README es crear el entorno virtual, instalar requirements.txt, migrar y arrancar el servidor de desarrollo.

```
python -m venv venv
venv\Scripts\activate          # en Linux/macOS: source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Hay una excepción importante a lo de «sin configuración»: la puerta de entrada sí necesita al menos una credencial. Con APP_AGENTS en blanco y sin el par antiguo APP_LOGIN_USERNAME/APP_LOGIN_PASSWORD, core.agents.configured_agents() devuelve una lista vacía, authenticate() no puede casar con nadie y core.middleware.LoginRequiredMiddleware redirige toda ruta no exenta a /login/. El comentario de config/settings.py lo dice con todas las letras: en blanco significa que la puerta nunca se puede satisfacer, no que esté desactivada. Antes de ver el shell hay que copiar .env.example a .env y definir un agente.

```
# .env
APP_AGENTS=Admin:cambia-esta-clave:Admin,Samuel:1234:Samuel
```

migrate crea el fichero db.sqlite3 en BASE_DIR, que está en .gitignore junto con /media/, /staticfiles/, venv/ y .env. Con una credencial configurada, la aplicación queda en http://127.0.0.1:8000/ y la pantalla de bienvenida que sirve core.views.welcome enlaza a Inbox, CRM y Embudos. Todos los puntos de entrada fijan el módulo de settings con os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings'): manage.py, config/wsgi.py y config/asgi.py hacen exactamente lo mismo, así que no hay divergencia entre el

arranque local y el de la función serverless.

Para ver el Inbox con datos hay tres comandos de gestión en `messaging/management/commands/`. `seed_conversations` crea contactos y conversaciones de demo repartidos por canal, por estado de asignación y a ambos lados de la ventana de 24 horas, además del usuario asesor con contraseña `asesor123`. Su docstring insiste en que todo lo creado es obviamente falso: los apellidos son «Pruebas», «Demo» o «Ejemplo» y los teléfonos viven bajo el prefijo `FAKE_PHONE_PREFIX`, que vale `+57300000000` y es justamente por lo que `--fresh` reconoce y borra los datos sembrados antes de recrearlos.

```
python manage.py seed_conversations --fresh
python manage.py simulate_inbound "+573000000001" "Hola, ¿sigue disponible?"
```

El borrado de `--fresh` no se apoya solo en el prefijo de teléfono. Borra los `Client` cuyo teléfono empieza por `FAKE_PHONE_PREFIX` (conversaciones y mensajes caen por cascada) y, aparte, los `CalendarEvent` sembrados filtrando por `title__in=SEED_EVENT_TITLES` y `description=SEED_EVENT_DESCRIPTION`, porque `CalendarEvent.contact` es `SET_NULL` y por tanto los eventos sobreviven al contacto que referenciaban; la constante de descripción existe precisamente para no borrar un evento creado por el usuario que casualmente comparta título. El fondo de volumen para las gráficas de Estadísticas es una población aparte, con su propio prefijo `VOLUME_PHONE_PREFIX` (`FAKE_PHONE_PREFIX` más "9") y una semilla fija `VOLUME_RNG_SEED = 20260828`, para que resembrar redibuje exactamente la misma gráfica y una captura de pantalla siga siendo comparable entre ejecuciones.

- `seed_conversations --fresh`: borra la siembra anterior (contactos por prefijo de teléfono y eventos de calendario por título y descripción) antes de recrearla.
- `seed_conversations --volume-days`: días de fondo de volumen de mensajes para las gráficas de Estadísticas, 30 por defecto; 0 lo omite.
- `seed_conversations --volume-scale`: multiplicador sobre las medias diarias de ese fondo, 1.0 por defecto; bajarlo acelera la siembra sin cambiar la forma de la gráfica.
- `simulate_inbound <phone> <message>`: empuja un entrante por el mismo código del webhook (firma, parseo, idempotencia).
- `reset_conversations`: vacía el Inbox, pero por defecto es una simulación que no borra nada.

`reset_conversations` merece leerse antes de usarlo. Su docstring explica las dos decisiones de seguridad que lo gobiernan, porque puede correr contra una base de producción y no hay deshacer. La primera es que es una simulación por defecto: sin `--yes` imprime la base de datos a la que apunta (motor y `HOST`, o `NAME` si no lo hay) y los recuentos de lo que borraría, y termina sin tocar nada; nombrar el destino importa porque `DATABASE_URL` es fácil de apuntar al proyecto equivocado. La segunda es que el borrado va dentro de una sola `transaction.atomic()`, en orden explícito (`Message`, `ConversationTag`, `Conversation` y `Client`) en lugar de confiar en la cascada, para que los números reportados sean los realmente borrados. Con `--keep-clients` conserva los contactos y borra solo conversaciones y mensajes; etiquetas, plantillas, listas de clientes, eventos de calendario y agentes sobreviven siempre.

Nota. `reset_conversations` sin `--yes` no borra nada: es un informe. Con `--yes` y sin `--keep-clients` borra también los contactos (`Client`), no solo las conversaciones.

11.3 Configuración por entorno: cómo se carga .env

`config/settings.py` llama a `load_dotenv(BASE_DIR / '.env')` antes de leer ninguna variable, de modo que un `python manage.py ...` normal recoge las credenciales locales sin ningún paso de export. La decisión importante está en el comentario que acompaña a esa llamada: `load_dotenv` no sobrescribe lo que ya está en `os.environ`, así que las variables reales del entorno siempre ganan al fichero. Eso es lo que hace que las variables de proyecto de Vercel manden en producción, donde además el `.env` ni siquiera se despliega porque está en `.gitignore`.

El fichero `.env.example` es la plantilla a copiar y documenta cada variable con su porqué. Advierte de dos trampas de formato: hay que escribir `KEY=value` sin espacios alrededor del `=`, porque una shell POSIX interpreta `KEY = value` como un comando llamado `KEY` y eso rompe un `set -a; . ./env`; y hay que entrecomillar cualquier valor que contenga `&`, `?` o espacios, cosa que ocurre siempre con las URLs de Neon.

No todo lo configurable pasa por el entorno. `config/settings.py` fija `MAILERS` con un único alias `default` cuyo `BACKEND` es `django.core.mail.backends.console.EmailBackend`, es decir, el correo se imprime por consola y no hay servidor SMTP configurado ni variable que lo cambie. Es la otra decisión de configuración de servicios del fichero, junto a la de mensajería.

Nota. Ninguna credencial vive en `config/settings.py`: todos los valores sensibles se leen de `os.environ` con un valor por defecto inocuo, y `.env` está en `.gitignore` para que no se commiten nunca.

11.4 Base de datos y la bandera TESTING

`TESTING = 'test' in sys.argv` se calcula al principio de `config/settings.py` y es verdadero bajo `manage.py test`. No es una comodidad menor: cuatro decisiones de configuración dependen de ella. `core.middleware.LoginRequiredMiddleware` la consulta para no dejar fuera al cliente de test, que nunca inicia sesión; la base de datos vuelve a SQLite aunque el `.env` del desarrollador apunte a una real, porque crear y destruir una base de trabajo contra producción es indeseable; el almacenamiento se mantiene en el sistema de ficheros aunque haya `BLOB_READ_WRITE_TOKEN`; y `MESSAGING_PROVIDER` se fuerza a `fake`, para que ninguna prueba dependa de una conexión viva con Twilio o Meta.

Cuando `DATABASE_URL` está definida y no se está en tests, `DATABASES['default']` se construye con `dj_database_url.parse(os.environ['DATABASE_URL'], conn_max_age=600, conn_health_checks=True)`. El comentario del código explica por qué se usa la librería en lugar de un `urlparse` a mano: `dj_database_url` arrastra los parámetros de la query hasta `OPTIONS`, y eso importa con Neon, cuya URL termina en `?sslmode=require&channel_binding=require`. Parsear la URL a mano y copiar solo `NAME`, `USER`, `PASSWORD`, `HOST` y `PORT` los descarta, y entonces `libpq` cae a `sslmode=prefer`, que acepta en silencio una conexión sin cifrar. `conn_max_age=600` mantiene viva la conexión entre peticiones en vez de rehacer el handshake TLS en cada una, y `conn_health_checks=True` la valida antes de reutilizarla.

Sin `DATABASE_URL` se usa SQLite en `BASE_DIR / 'db.sqlite3'`. El comentario deja claro que esto no es una opción en producción: las funciones serverless de Vercel no tienen disco persistente, así que cualquier despliegue real debe apuntar a un Postgres (Vercel Postgres o Neon). El driver `psycopg[binary]` solo se importa cuando hay `DATABASE_URL`, por lo que el desarrollo local nunca lo carga.

11.5 Despliegue en Vercel

El proyecto se apoya en el soporte nativo de Vercel para Django, que detecta `manage.py` y el `WSGI_APPLICATION` de `config/settings.py` sin `build script`. `vercel.json` es mínimo: declara el `$schema` y sube el `maxDuration` de la función `config/wsgi.py` a 30 segundos. El resto de la adaptación al entorno `serverless` vive en `config/settings.py`.

`ALLOWED_HOSTS` se construye en tres pasos. Primero se parte la variable `ALLOWED_HOSTS` por comas descartando vacíos. Luego se añaden automáticamente `VERCEL_URL`, que Vercel fija en cada build con el `hostname` del deploy (incluidos los previews), y `VERCEL_PROJECT_PRODUCTION_URL`, porque `VERCEL_URL` es por-deploy y no cubre el alias estable de producción. Finalmente, y solo cuando la variable `VERCEL` está presente, se añaden dos alias fijos del proyecto, `mvp-crm-unaneaprogramadora.vercel.app` y `mvp-crm-git-main-unaneaprogramadora.vercel.app`, `hardcodeados` para que sigan funcionando sin editar variables en el dashboard. La guarda por `VERCEL` es deliberada: en local esas entradas harían `ALLOWED_HOSTS` no vacío, lo que desactiva el fallback a `localhost` que Django aplica en modo `DEBUG` y dejaría al servidor de desarrollo fuera de su propia máquina.

De esa lista se deriva `CSRF_TRUSTED_ORIGINS` como `https://<host>` para cada host. Además se fija `SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')`, porque Vercel termina el TLS por delante y reenvía por HTTP: sin esa cabecera `request.is_secure()` sería siempre falso tras el proxy, rompiendo las comprobaciones de CSRF y cualquier comportamiento de cookie o redirección segura.

Los estáticos no requieren `WhiteNoise`: `STATIC_URL` es `static/`, `STATICFILES_DIRS` apunta a `BASE_DIR / 'static'` y el build de Vercel ejecuta `collectstatic` sobre `STATIC_ROOT` (`BASE_DIR / 'staticfiles'`) y sirve el resultado desde su CDN. Como ese `collectstatic` es el plano, sin nombres `hasheados`, el `cache-busting` lo resuelve la etiqueta `{% vstatic %}` de `core/templatetags/assets.py`, que añade un `?v=<hash>` con un `sha256` del contenido truncado a diez caracteres; los hashes se memorizan por proceso fuera de `DEBUG` —incluidos los fallos de búsqueda, que se cachean como cadena vacía porque no pueden cambiar sin un `redespliegue`— y se recalculan en cada `render` durante el desarrollo.

Los medios son el caso que sí necesita almacenamiento externo. En local, `MEDIA_ROOT` es `BASE_DIR / 'media'` y `config/urls.py` los sirve con `static()`, que es un `no-op` cuando `DEBUG` está apagado. En producción no hay sistema de ficheros persistente, así que la presencia de `BLOB_READ_WRITE_TOKEN` (inyectada al conectar un Blob store desde Storage en el dashboard) basta para redefinir `STORAGES` entero: `default` pasa a `core.storage.VercelBlobStorage` y `staticfiles` se declara explícitamente como `django.contrib.staticfiles.storage.StaticFilesStorage`, porque asignar el diccionario completo sustituye también la entrada por defecto de estáticos. Ese backend sube los bytes al Blob store del proyecto y devuelve URLs públicas de su CDN; sin él, `default_storage.save()` fallaba en silencio para las descargas de medios de WhatsApp y las cabeceras de plantillas.

`VercelBlobStorage` mapea los nombres a `pathnames` del blob literalmente, sin sufijo aleatorio, para que quien escribió `whatsapp/<media-id>.webp` pueda comprobar después `exists()` con el mismo nombre y encontrarlo; el webhook de Meta depende de eso para la idempotencia de los reintentos. Parte de ese contrato de nombres deterministas es `_normalize()`, que convierte las barras invertidas en `/` y quita las barras iniciales, y se aplica tanto al guardar como al resolver. `url()` responde desde una caché por proceso que llena `_save` y, si falla, con una única llamada al endpoint de listado, comparando el `pathname` exacto porque el listado hace coincidencia por prefijo; si tampoco así aparece, lanza `ValueError` con el mensaje `no blob stored at ...` en lugar de devolver una URL inválida, mientras que `exists()` simplemente responde falso. Las escrituras pasan `allowoverwrite` porque una sobrescritura real solo puede ocurrir cuando dos escritores compiten por el mismo `pathname` determinista, y entonces ambos escriben los mismos bytes descargados. Todas las llamadas al Blob usan `_REQUEST_TIMEOUT`, que vale 10 segundos.

La lectura es el único punto donde el backend no usa el SDK: `_open()` resuelve la URL pública con `self.url(name)` y descarga el contenido con `requests.get(...)` más `raise_for_status()`, devolviendo un `ContentFile`. Esa es la razón por la que `requests` figura en `requirements.txt` desde el lado del almacenamiento.

11.6 Páginas legales públicas y la revisión de Meta

`core/urls.py` publica dos rutas fuera del shell de la aplicación: `privacidad/` con nombre `privacy` y `eliminacion-de-datos/` con nombre `data_deletion`, servidas por `core.views.privacy` y `core.views.data_deletion`. Existen porque Meta no publica una app sin una política de privacidad y una URL de eliminación de datos legibles públicamente, y como dice el comentario del código, una política detrás de la puerta de login no es una política que nadie —ni un revisor ni un cliente— pueda leer de verdad.

La exención la aplica `core/middleware.py`: `/privacidad/` y `/eliminacion-de-datos/` están en `EXEMPT_PATHS` junto a `/login/`, y los prefijos exentos son `/webhooks/`, `/static/`, `/media/` y `/admin/`. Ambas vistas comparten `_legal_context()`, que expone `updated`, `entity` y `contact_email`. `entity` y `contact_email` salen de `LEGAL_ENTITY_NAME` y `LEGAL_CONTACT_EMAIL`, configurables por despliegue para que las páginas nombren al negocio real antes de apuntar Meta a esas URLs. `updated` es la constante `LEGAL_UPDATED`, con valor `'2 de septiembre de 2026'`; el comentario justifica que sea constante y no la fecha de hoy: una política que dice cambiar cada vez que se renderiza no vale nada.

Las plantillas `templates/legal/privacidad.html` y `templates/legal/eliminacion-de-datos.html` son documentos HTML completos, no extienden el shell, y cargan `static/css/legal.css` mediante `{% vstatic %}`. El CSS lo dice en su cabecera: son autónomas, sin barra lateral, precisamente porque deben alcanzarse sin sesión. La política cubre responsable, datos recolectados, finalidad y autorización, con quién se comparten, conservación, seguridad, derechos según la Ley 1581 de 2012, cambios y contacto; la página de eliminación explica cómo pedirla por correo indicando el número de teléfono, detalla qué se elimina y qué puede conservarse, y fija un plazo de quince días hábiles conforme a la misma ley. Cada página enlaza a la otra.

11.7 Referencia: variables de entorno, dependencias y archivos

Variable	Obligatoria	Por defecto	Para qué sirve
SECRET_KEY	En producción	django-insecure-r9ns&... (valor de desarrollo)	Clave criptográfica de Django; el fallback solo sirve en local
DEBUG	No	True (comparación exacta con la cadena 'True')	Modo depuración; en producción debe ser False
ALLOWED_HOSTS	No	cadena vacía	Hosts extra separados por coma; los dominios de Vercel se confían solos
VERCEL_URL	La inyecta Vercel	sin valor	Hostname del deploy actual, incluidos previews; se añade a ALLOWED_HOSTS
VERCEL_PROJECT_PRODUCTION_URL	La inyecta Vercel	sin valor	Alias estable de producción; se añade a ALLOWED_HOSTS
VERCEL	La inyecta Vercel	sin valor	Su presencia activa los dos alias hardcoded del proyecto
DATABASE_URL	En producción	sin valor (se usa SQLite)	Cadena de conexión Postgres parseada por dj_database_url
BLOB_READ_WRITE_TOKEN	En producción	sin valor	Su presencia redefine STORAGES con core.storage.VercelBlobStorage
APP_AGENTS	Sí para poder entrar	cadena vacía	Lista usuario:contraseña:Nombre de la puerta de entrada y de asignación
APP_LOGIN_USERNAME	No	cadena vacía	Par antiguo, usado como lista de un solo agente si APP_AGENTS falta
APP_LOGIN_PASSWORD	No	cadena vacía	Contraseña del par antiguo
LEGAL_ENTITY_NAME	No	MVP CRM	Responsable del tratamiento mostrado en las páginas legales
LEGAL_CONTACT_EMAIL	No	sernasamuelperez@gmail.com	Correo de contacto de las páginas legales
MESSAGING_PROVIDER	No	fake	Proveedor activo: fake, twilio o meta; en tests se fuerza a fake
MESSAGING_FAKE_SECRET	No	dev-secret	Secreto compartido de la cabecera X-Fake-Signature
TWILIO_ACCOUNT_SID	Con twilio	cadena vacía	Credencial de la consola de Twilio
TWILIO_AUTH_TOKEN	Con twilio	cadena vacía	Credencial y clave de firma de X-Twilio-Signature
TWILIO_WHATSAPP_FROM	Con twilio	cadena vacía	Número emisor en E.164
META_ACCESS_TOKEN	Con meta	cadena vacía	Token de la Cloud API
META_PHONE_NUMBER_ID	Con meta	cadena vacía	Identificador del número en la Cloud API
META_APP_SECRET	Con meta	cadena vacía	Firma los webhooks entrantes (X-Hub-Signature-256); falla cerrado si falta
META_VERIFY_TOKEN	Con meta	cadena vacía	Responde el handshake de suscripción; falla cerrado si falta

Variables de entorno leídas por config/settings.py

Nota. DJANGO_SETTINGS_MODULE no aparece arriba porque config/settings.py no la lee: la fijan con.setdefault los tres puntos de entrada (manage.py, config/wsgi.py y config/asgi.py), siempre al valor config.settings.

Paquete	Versión	Por qué está
Django	==6.1	El framework de la aplicación; línea sin comentario en el fichero
asgiref	==3.12.1	Dependencia de Django, fijada por versión exacta; línea sin comentario
sqlparse	==0.6.0	Dependencia de Django, fijada por versión exacta; línea sin comentario
tzdata	==2026.3	Base de datos de zonas horarias; línea sin comentario
requests	>=2.31,<3	Cliente HTTP; core/storage.py lo usa en _open(); línea sin comentario
psycopg[binary]	>=3.1,<4	Driver Postgres, usado cuando hay DATABASE_URL; en local nunca se importa
python-dotenv	>=1.0,<2	Carga .env en os.environ al importar settings, sin sobrescribir lo ya presente
dj-database-url	>=2.1,<4	Parsea DATABASE_URL; sustituye un urlparse a mano que perdía la query string
vercel_blob	==0.4.2	Cliente del Blob store tras core.storage.VercelBlobStorage; solo se usa con BLOB_READ_WRITE_TOKEN

requirements.txt y la razón de cada dependencia

Nota. Solo cuatro entradas de requirements.txt llevan comentario explicativo: psycopg[binary], python-dotenv, dj-database-url y vercel_blob. Las cinco primeras líneas del fichero van sin comentario alguno.

Ruta	Qué contiene
config/settings.py	Toda la lectura del entorno, base de datos, estáticos, medios, correo y mensajería
config/wsgi.py	Callable WSGI; es el fichero que vercel.json declara como función
config/asgi.py	Callable ASGI, no usado por el despliegue actual
config/urls.py	Admin, webhooks, core.urls y los medios servidos solo en DEBUG
manage.py	Punto de entrada de los comandos de gestión
vercel.json	maxDuration de 30 segundos para config/wsgi.py
requirements.txt	Dependencias; solo cuatro llevan comentario con su porqué
.env.example	Plantilla de variables con el formato y las advertencias de shell
core/storage.py	VercelBlobStorage: medios en producción
core/middleware.py	Puerta de login y rutas exentas
core/agents.py	configured_agents() y authenticate(): de dónde salen las credenciales
core/templatetags/assets.py	Etiqueta vstatic para cache-busting de CSS y JS
templates/legal/	privacidad.html y eliminacion-de-datos.html
static/css/legal.css	Estilos autónomos de las páginas legales

Archivos de este capítulo

11.8 Trampas conocidas y pruebas que lo cubren

- APP_AGENTS en blanco no desactiva la puerta de entrada: significa que nunca se puede satisfacer, es decir, que no entra nadie, ni siquiera en local.
- Las contraseñas de APP_AGENTS no pueden contener : ni , , que son los separadores del formato.

- Los dos alias de Vercel hardcoded solo se añaden si la variable VERCEL existe; añadirlos siempre rompería el desarrollo local.
- Los estáticos de Vercel se sirven sin nombres hasheados, así que un CSS nuevo no llega al navegador si la plantilla no usa `{% vstatic %}`.
- En producción no hay disco: sin BLOB_READ_WRITE_TOKEN los uploads se pierden tras la petición.
- Un `.env` con `KEY = value` o con una URL de Neon sin comillas rompe la carga desde la shell por el `&` de la query.
- `reset_conversations` sin `--yes` solo informa; con `--yes` borra también los contactos salvo que se pase `--keep-clients`.

La suite completa se lanza con `python manage.py test`. Dos módulos cubren directamente lo que describe este capítulo. `core/tests_legal.py` documenta en su docstring que el objetivo de las páginas legales es ser alcanzables sin sesión, y su clase `LegalPagesTests` comprueba que ambas renderizan con 200, que la política contiene la mención a la Ley 1581 de 2012 y no solo un título, que el responsable y el correo salen de los settings mediante `override_settings`, y que cada página enlaza a la otra porque Meta pide las dos URLs. Sus dos pruebas más finas atacan el middleware directamente: como `settings.TESTING` desactiva la puerta por completo, una petición del cliente de test devolvería 200 aunque la ruta estuviera cerrada, así que se llama a `LoginRequiredMiddleware._is_exempt` sobre `/privacidad/` y `/eliminacion-de-datos/`, y también sobre `/`, `/inbox/list/todos/` y `/s/crm/` para verificar que la exención no se ha convertido en un agujero.

`core/tests_storage.py` cubre `VercelBlobStorage` con la API de Blob mockeada; lo que se prueba, según su docstring, es el contrato de Storage del que depende el resto de la aplicación: nombres deterministas, idempotencia basada en `exists()` y resolución de `url()`. `VercelBlobStorageTests` verifica que guardar sube y conserva el nombre, que `url()` inmediatamente después de un `_save` no hace ninguna llamada extra gracias a la caché por proceso, que `exists()` y `url()` resuelven a través del endpoint de listado descartando un pathname más largo que casa por prefijo, que un blob inexistente no se da por encontrado y que `url()` sobre él lanza `ValueError (test_missing_blob_is_not_found)`, y que `delete` apunta a la URL resuelta.

Archivos de referencia: `config/settings.py`, `config/wsgi.py`, `config/asgi.py`, `config/urls.py`, `manage.py`, `vercel.json`, `requirements.txt`, `.env.example`, `.gitignore`, `README.md`, `core/storage.py`, `core/middleware.py`, `core/agents.py`, `core/urls.py`, `core/views.py`, `core/templatetags/assets.py`, `core/tests_legal.py`, `core/tests_storage.py`, `templates/legal/privacidad.html`, `templates/legal/eliminacion-de-datos.html`, `static/css/legal.css`, `messaging/management/commands/seed_conversations.py`, `messaging/management/commands/reset_conversations.py`

12. Pruebas y convenciones de desarrollo

Cómo se ejecuta la suite de MVP-CRM (completa y por módulo), qué desactiva y qué fuerza la bandera TESTING de config/settings.py, qué cubre cada uno de los 24 módulos de tests, y el catálogo de convenciones de desarrollo que el propio código documenta en sus docstrings: módulo de datos por pantalla, registro de builders de contexto, respuestas duales htmx, plantillas que aparecen solas, frontend sin build con librerías vendorizadas, UI en español y código en inglés.

12.1 Propósito y alcance

MVP-CRM se prueba con el corredor de tests estándar de Django: no hay pytest, ni tox, ni fixtures compartidas, ni factories. Cada pantalla del CRM tiene su propio módulo de tests dentro de core/, y la capa de mensajería tiene los suyos en messaging/. En total son veintidós módulos en core/ y dos en messaging/, todos con nombre tests*.py para que el descubrimiento automático de manage.py test los encuentre sin configuración.

Los tests no dependen de datos sembrados: cada clase crea los objetos que necesita. El helper make_event de core/tests_calendario.py lo dice explícitamente en su docstring, «One event; tests create their own data, there are no seeds». Los comandos seed_conversations y simulate_inbound que documenta el README.md son para explorar la aplicación a mano, no para preparar la suite.

Este capítulo cubre dos cosas: cómo correr las pruebas y qué convenciones de desarrollo se observan de forma consistente en el código. Casi todas esas convenciones están explicadas por el propio código en un docstring de módulo o en un comentario, y esa es la fuente que se recoge aquí.

12.2 Cómo se ejecuta la suite

El proyecto se instala en un entorno virtual llamado venv en la raíz, según la sección «Puesta en marcha» del README.md. Una vez activado y con requirements.txt instalado, la suite completa se lanza con un solo comando desde el directorio que contiene manage.py. No hace falta migrar antes: el corredor de Django crea y destruye su propia base de datos de prueba.

```
source venv/bin/activate          # en Windows: venv\Scripts\activate
pip install -r requirements.txt
python manage.py test
```

Para trabajar sobre una sola pantalla conviene correr únicamente su módulo, indicando la ruta de importación en vez del archivo. La misma sintaxis admite bajar a una clase concreta o a un método, que es lo habitual mientras se depura un caso puntual.

```
python manage.py test core.tests_crm
python manage.py test core.tests_clientes.ClienteDeleteTests
python manage.py test messaging.tests.WebhookTests
python manage.py test core messaging
```

Nota. La suite es grande (alrededor de 650 métodos de test repartidos en 24 módulos), así que durante el desarrollo de una pantalla lo normal es correr su módulo y dejar la suite completa para antes de commitear.

12.3 La bandera TESTING

`config/settings.py` define `TESTING = 'test' in sys.argv` en la línea 25, antes de cargar el `.env`. Es una detección deliberadamente simple de «estoy corriendo bajo `manage.py test`», y su propósito es que la suite quede aislada del `.env` local de cada desarrollador. Cuatro comportamientos distintos dependen de ella, y cada uno lleva su comentario explicando por qué.

El primero es la puerta de entrada. `core/middleware.py` implementa `LoginRequiredMiddleware`, que redirige a `/login/` cualquier petición sin la marca `app_authenticated` en sesión; su condición de paso empieza por `settings.TESTING`, de modo que el cliente de test —que nunca inicia sesión— no queda bloqueado en todos los demás módulos. El segundo es la base de datos: aunque el `.env` del desarrollador apunte `DATABASE_URL` a un Postgres real, con `TESTING` activo se ignora y se usa SQLite, porque crear y destruir una base temporal contra la de verdad no es deseable. El tercero es el almacenamiento: `BLOB_READ_WRITE_TOKEN` cambia el backend por defecto a `core.storage.VercelBlobStorage`, pero los tests conservan siempre el sistema de archivos. El cuarto es el proveedor de mensajería: `MESSAGING_PROVIDER = 'fake' if TESTING else os.environ.get('MESSAGING_PROVIDER', 'fake')`, con el comentario que lo justifica: los tests no deben depender de una conexión viva a Twilio o a Meta para pasar.

Dos módulos necesitan probar precisamente lo que `TESTING` desactiva, y lo hacen desactivando la bandera con `override_settings`. `core/tests_auth.py` fija `TESTING=False` en `LoginGateTests` para ejercitar la puerta de login de verdad, y su docstring lo explica sin rodeos. `core/tests_agents.py` hace lo mismo en `AgentLoginTests` para probar el login de cada agente. `core/tests_legal.py` toma otro camino: como no puede confiar en el cliente de test, instancia `LoginRequiredMiddleware` directamente y comprueba `_is_exempt('/privacidad/')`, con un docstring que avisa de que fetchear esas URLs devolvería 200 aunque estuvieran protegidas.

Las credenciales que fija cada uno son distintas y conviene no confundirlas. `core/tests_auth.py` fija `APP_AGENTS=""` junto a `APP_LOGIN_USERNAME` y `APP_LOGIN_PASSWORD`, y un comentario explica por qué ese vaciado importa tanto como el par: `core.agents` prefiere la lista de agentes siempre que no esté vacía, así que un desarrollador con `APP_AGENTS` en su `.env` nunca llegaría a las credenciales heredadas que ese test quiere probar. `AgentLoginTests`, en `core/tests_agents.py`, hace lo contrario: fija `APP_AGENTS=TWO_AGENTS` y vacía las heredadas con `APP_LOGIN_USERNAME=""` y `APP_LOGIN_PASSWORD=""`. El `APP_AGENTS=""` de ese módulo aparece en otra clase, `LegacyCredentialFallbackTests`, que no toca `TESTING`.

Nota. La bandera se lee desde `django.conf.settings` en tiempo de petición, no se captura al importar, y por eso `override_settings` basta para invertirla en una clase concreta sin recargar nada.

12.4 Las exenciones de la puerta de login

El docstring de `core/middleware.py` deja claro que esto no es `django.contrib.auth` en el sentido clásico: no hay registro ni recuperación de contraseña, solo credenciales del entorno comprobadas en `core.views.login_view` y recordadas como una marca de sesión. La exención no se limita a las páginas legales: además de `EXEMPT_PATHS`, que contiene `/login/`, `/privacidad/` y `/eliminacion-de-datos/`, hay `EXEMPT_PREFIXES = ('/webhooks/', '/static/', '/media/', '/admin/')`, y el helper `_is_exempt` acepta un path si está en el conjunto o empieza por alguno de esos prefijos.

Cada exención lleva su motivo escrito. Los webhooks de proveedor los golpean Twilio o Meta sin sesión de navegador, y ya se autentican con su propia verificación de firma. Las páginas legales son públicas a propósito, porque la revisión de aplicación de Meta las lee sin cuenta y los clientes también deben poder hacerlo. Los assets estáticos y de medios tienen que servirse sin sesión, y `/admin/` queda fuera porque va gateado por separado con `django.contrib.auth`.

Entrada	Tipo	Motivo documentado
/login/	Path exacto	La propia pantalla de login
/privacidad/	Path exacto	Pública para la revisión de Meta y para los clientes
/eliminacion-de-datos/	Path exacto	Igual que la anterior
/webhooks/	Prefijo	Los proveedores se autentican con verificación de firma
/static/ y /media/	Prefijo	Assets servidos sin sesión
/admin/	Prefijo	Gateado aparte por django.contrib.auth

Rutas exentas de la puerta de login en core/middleware.py.

`core.views.login_view` hace más que marcar la sesión. Su docstring explica que un login correcto abre además una sesión real de `django.contrib.auth` contra el user espejo del agente, y que eso es lo que convierte al agente en una identidad en vez de un booleano: «Tu inbox» puede filtrar `assigned_to=request.user`, los mensajes salientes registran quién los escribió y el desplegable de asignación del Inbox tiene un valor por defecto sensato. Como el user espejo tiene contraseña inutilizable, la vista nombra el backend explícitamente en `auth_login` en vez de pasar por `django.contrib.auth.authenticate()`, y vuelve a fijar `SESSION_KEY` después, porque `auth_login` cicla la clave de sesión y la marca de la puerta tiene que sobrevivir a esa rotación.

Nota. El parámetro `?next` se valida con `url_has_allowed_host_and_scheme` contra `request.get_host()` antes de usarse: el comentario advierte que sin esa comprobación un `next=https://evil.example` redirigiría al usuario fuera del sitio justo tras autenticarse, el clásico open-redirect de phishing post-login.

12.5 Referencia: módulos de tests

Módulo	Qué cubre
core/tests.py	Shell de barra lateral: rutas, estado activo y fragmentos htmx. Incluye NavDefinitionTests, RoutingTests, SidebarRenderTests, HtmxSwapTests, SectionTemplateTests, InboxScreenTests, InboxListEndpointTests, WelcomeScreenTests y TemplateCommentLeakTests
core/tests_auth.py	Puerta de login de toda la app (core/middleware.py más core.views.login_view), con TESTING desactivado en LoginGateTests
core/tests_agents.py	Agentes: lista configurada por entorno, fallback a credenciales heredadas, User espejo, el login que la respalda (AgentLoginTests, con TESTING desactivado) y la asignación por conversación en el Inbox
core/tests_legal.py	Páginas legales públicas (privacidad y eliminación de datos) y su exención de la puerta de login
core/tests_crm.py	Pantalla CRM: helpers del modelo Client, nav secundaria y tabla de clientes
core/tests_clientes.py	CRUD de Clientes: reglas de core.clientes, los cuatro endpoints del modal compartido y el buscador de la tabla
core/tests_campanas.py	Campañas: monta la misma nav y los mismos paneles que el CRM; fija ese reparto (mismas vistas, mismo endpoint crm_panel, distinto montaje)
core/tests_calendario.py	Mi calendario: panel y barra lateral, feed JSON de eventos y su contrato de zona horaria (guardar en UTC, mostrar en Bogotá), mutaciones desde el modal y la rejilla, y preferencias de sesión
core/tests_embudos.py	Embudos: nav secundaria plana y tarjeta de estado vacío
core/tests_comercio.py	Mi comercio: nav colapsable multi-sección, tabla de Productos con sus pestañas de estado y el cableado del placeholder
core/tests_automatizaciones.py	Automatizaciones: nav secundaria con la fila expandible MIA, banner de Academy y estado vacío de Flujos. Casi todo está con skip porque la pestaña se quitó de la barra lateral
core/tests_mensajería.py	Configuración de mensajería: nav plana sin cabecera, tabla de Plantillas de WhatsApp por pestañas, estado vacío a dos mitades y el modal selector de plantilla
core/tests_respuestas_rapidas.py	Selector de Respuestas rápidas del compositor: core.plantillas.render_body, endpoint del popover y enganche con el compositor
core/tests_estadisticas.py	Estadísticas: nav secundaria plana con el badge Alpha, rejilla de tarjetas de Mensajería y cableado del placeholder
core/tests_estadisticas_volumen.py	Volumen de Mensajes: agregación por ORM en core.estadisticas_volumen, filtro de periodo, caché, pantalla y endpoint JSON
core/tests_estadisticas_tiempos.py	Tiempos de Respuesta: el recorrido de emparejamiento de core.estadisticas_tiempos, los tres filtros, caché, pantalla y endpoint JSON
core/tests_estadisticas_temas.py	Temas de conversación: informe de frecuencia de palabras (tokenizador, plegado de acentos, ventana de periodo) y el panel con su filtro y estado vacío
core/tests_estadisticas_etiquetas.py	Etiquetas: cada etiqueta con cuántas conversaciones la llevan, tiles de resumen y estado vacío
core/tests_estadisticas_ventas.py	Ventas: qué cuenta como venta (etiquetas cuyo nombre contiene «venta»), ventana de periodo sobre tagged_at, tile de conversión, tabla por canal y estado vacío de configuración
core/tests_estadisticas_embudos.py	Embudos de estadísticas: embudo de conversaciones en cuatro etapas (creadas, respondidas, resueltas, con venta), ventana de periodo y estado vacío
core/tests_estadisticas_atribuciones.py	Atribuciones: atribución por canal de conversaciones y ventas, tiles de canal principal y mejor conversión, ventana de periodo y estado vacío

Módulo	Qué cubre
core/tests_storage.py	core.storage.VercelBlobStorage con la API de Blob mockeada: nombres deterministas, idempotencia basada en exists() y resolución de url()
messaging/tests.py	Capa de mensajería: contrato del webhook, regla de 24 horas y cableado del Inbox. Incluye WebhookTests, OutboundEventTests, StatusEventTests, SendWindowTests, FakeStatusProgressionTests, InboxUITests, TagServiceTests, TagFilterTests, TagAdminUITests, TagPickerTests, BulkTagTests, MetaProviderTests e InlineMediaTests
messaging/tests_reset.py	El comando reset_conversations: no debe borrar nada sin --yes

Los 24 módulos de tests y lo que cubre cada uno, según su docstring y sus clases.

12.6 Convenciones de desarrollo observadas

Un módulo de datos por pantalla. Cada pantalla declara sus filas de navegación y sus vistas una sola vez en un módulo Python, y las plantillas se limitan a iterar. `core/nav.py` es el original y se describe como «single source of truth for the sidebar navigation»: agregar, reordenar o renombrar una sección es un cambio de una línea en ese archivo, sin tocar plantillas. `core/inbox.py` se remite explícitamente a él («Same idea as core.nav»), y `core/crm.py` también («Same shape as core.nav and core.inbox: declare the rows once, let the templates loop»). Existen equivalentes para el resto de pantallas: `core/embudos.py`, `core/comercio.py`, `core/estadisticas.py`, `core/mensajería.py`, `core/automatizaciones.py` y `core/ calendario.py`.

Registro de contexto por panel. Una pantalla que necesita datos no añade una rama al `if` de la vista: registra un builder en un diccionario. `core/views.py` documenta esto en su docstring de módulo y lo implementa con `SECTION_CONTEXT`, un mapa de clave de sección a callable que recibe el request y devuelve contexto. Dentro de cada pantalla se repite el patrón un nivel más abajo: `PANEL_CONTEXT` para los paneles del CRM, y `EMBUDOS_PANEL_CONTEXT`, `AUTOM_PANEL_CONTEXT`, `COMERCIO_PANEL_CONTEXT`, `STATS_PANEL_CONTEXT` y `MENSAJERIA_PANEL_CONTEXT` para las demás. La consulta es siempre la misma forma: `builder = PANEL_CONTEXT.get(view_key)` y, si no es `None`, se actualiza el contexto. Los paneles sin entrada en el diccionario simplemente no necesitan datos.

Respuestas duales para `htmx`. La vista `section` responde de dos maneras a la misma URL: petición normal de navegador devuelve la página completa a partir de `base.html`, y petición `htmx` devuelve solo el fragmento de la sección para que `hx-swap="innerHTML"` lo deje dentro de `<main id="content">`. El docstring de `core/views.py` explica el porqué: es lo que hace funcionar el shell sin recargas completas manteniendo cada URL enlazable, marcable y compatible con el botón atrás. La detección vive en un único helper, `_is_htmx(request)`, que comprueba que la cabecera `HX-Request` valga `"true"`.

Crear la plantilla y aparece. Una sección sin pantalla propia no rompe nada: `_section_template(key)` intenta cargar `sections/<key>.html` y, si `get_template` lanza `TemplateDoesNotExist`, devuelve `PLACEHOLDER_TEMPLATE`, que vale `sections/_placeholder.html`. El docstring lo llama «the extension point»: para construir una sección basta con crear el archivo, sin tocar vista, URL ni navegación, y se recoge en la siguiente petición. Hoy existen plantillas propias para `inbox`, `crm`, `campanas`, `embudos`, `automatizaciones`, `mi-comercio`, `estadisticas`, `mensajería` y `welcome`; el resto cae en el placeholder, que muestra «próximamente».

Quitar una entrada de la barra lateral es borrar una línea, y el código lo documenta. `core/nav.py` conserva un bloque comentado con fecha, «Removed for now (2026-08-26), to restore just re-add the NavItem», que lista las tres entradas retiradas: `automatizaciones` («Automatizaciones», icono bot), `performance-hub` («Performance HUB», icono gauge) y `crecimiento` («Herramientas de Crecimiento», icono trending-up). El comentario aclara que en el caso de `Automatizaciones` la pantalla, las vistas y las plantillas siguen todas en su sitio, de modo que restaurarla es únicamente re-añadir el `NavItem` a `PRIMARY_NAV`.

Sin build de frontend y librerías vendorizadas. No hay npm, bundler ni paso de compilación en el proyecto Django: las tres librerías de terceros se guardan minificadas en `static/js/vendor/` (`echarts.min.js`, `fullcalendar.min.js` y `htmx.min.js`) y se cargan con etiquetas `<script>` normales. El JavaScript propio son cinco archivos planos en `static/js/`: `calendario.js`, `shell.js`, `stats_chart.js`, `stats_tiempos.js` y `stats_volumen.js`, más la subcarpeta `vendor/`. Para que un cambio llegue al navegador existe la etiqueta `{% vstatic %}` en `core/templatetags/assets.py`, que añade un hash corto del contenido como query string `?v=<hash>`; su docstring explica el problema que resuelve, que `{% static %}` emite una ruta estable y el build de Django en Vercel corre `collectstatic` sin nombres hasheados, así que una corrección de CSS podía desplegarse sin llegar a quien ya había cargado la página.

La política de caché de `{% vstatic %}` también está razonada. `_content_hash` guarda los digests en el diccionario de módulo `_hashes` y solo lo consulta y lo rellena cuando `settings.DEBUG` es falso, de modo que en producción cada archivo se lee y se hashea una vez por proceso, mientras que en desarrollo el hash se recalcula en cada render para que una edición aparezca en la siguiente recarga. Los fallos de búsqueda se cachean a propósito como cadena vacía, con un comentario que lo justifica: si `finders.find` no encuentra el archivo, la respuesta no puede cambiar sin un redesplicue y reintentar la búsqueda en cada render no haría aparecer el archivo. Cuando el digest sale vacío, `vstatic` devuelve la URL de `{% static %}` sin sufijo.

Iconos SVG como plantillas incluidas. Los glifos no son archivos estáticos referenciados con `<img>` sino plantillas Django bajo `templates/icons/`, incluidas con `{% include %}` para que hereden el color y el tamaño del contexto. `NavItem.icon_template` devuelve `f"icons/{self.icon}.svg"` y las plantillas hacen `{% include item.icon_template %}`. Hay unos 55 iconos de línea, más subcarpetas `templates/icons/brands/` para las marcas de canal (`whatsapp.svg`, `instagram.svg`, `messenger.svg`, `facebook.svg`, `tiktok.svg`, `instagram-dm.svg`) y `templates/icons/flags/` para las banderas de país.

UI en español, código en inglés. Los identificadores, docstrings y comentarios están en inglés; todo el texto visible está en español. La convención está anotada en los propios dataclasses: el campo `label` de `NavItem` se documenta como la etiqueta española del tooltip, y el docstring añade que la barra lateral es solo de iconos, así que ese texto es el único que nombra el control y se renderiza a la vez como tooltip visible y como `aria-label` del enlace. Las claves de URL sí son español sin acentos ni eñes (embudos, mi-comercio, campanas), porque son a la vez slug de URL y nombre de plantilla de sección.

Comentarios y docstrings que explican decisiones. La convención más útil del repositorio es que cada elección no obvia lleva escrito su porqué junto al código, no en un documento aparte.

`config/settings.py` explica por qué se usa `dj_database_url` en vez de un `urlparse` a mano: al copiar solo NAME, USER, PASSWORD, HOST y PORT se pierden los parámetros de la cadena de conexión, y con Neon, cuya URL termina en `?sslmode=require&channel_binding=require`, `libpq` cae a `sslmode=prefer` y acepta en silencio una conexión sin cifrar. En el mismo archivo, `load_dotenv(BASE_DIR / '.env')` viene acompañado de la razón de su orden: las variables de entorno reales tienen precedencia porque `load_dotenv` no sobrescribe lo ya definido, así que las variables de proyecto de Vercel ganan en producción.

12.7 Qué hace `shell.js`, y qué no

`static/js/shell.js` abre con un comentario de bloque que enumera sus responsabilidades. La primera es el estado activo: `htmx` solo intercambia parte de la página, así que cualquier navegación que quede fuera de la región intercambiada tiene que mover su propio estado. Eso afecta hoy a dos navegaciones, la barra lateral (fuera de `#content`, nunca re-renderizada) y el panel de nav del Inbox (dentro de `#content`, pero donde solo se intercambia la columna 3). En vez de tratar cada caso por separado, cualquier contenedor marcado con `data-nav-group` recibe el comportamiento: al hacer clic en un `data-nav-item` de dentro, la clase `is-active` se mueve a ese elemento.

- Actualizar `document.title`, que vive en `<head>` y por tanto queda fuera de todos los intercambios.
- Limpiar el rail en la pantalla de bienvenida: `</>` no selecciona ningún icono, así que hay que vaciarlo en vez de apuntarlo a uno por defecto.
- Delegar los listeners desde `document` y buscar los elementos bajo demanda, sin cachear nodos.
- Nada más: no monta widgets ni pide datos, todo el HTML lo produce el servidor.

Nota. El comentario insiste en que todo esto es mejora progresiva pura: con JavaScript desactivado cada elemento de nav sigue siendo un enlace normal y el servidor ya renderiza el estado activo correcto en una carga completa. Y no se cachean nodos porque un nodo cacheado queda obsoleto en cuanto algo re-renderiza su subárbol, dejando los clics actualizando un elemento desprendido del documento.

12.8 Decisiones de diseño y trampas conocidas

Un parámetro de vista desconocido no es un 404. En todas las pantallas con nav secundaria, un `?view=` que no existe cae a la vista por defecto en lugar de dar error; el comentario de `_crm_context` lo justifica: «so a stale bookmark opens». Lo mismo vale para el `?filter=` del Inbox. En cambio, los endpoints de panel que `htmx` llama directamente (`crm_panel`, `embudos_panel`, `comercio_panel`, etc.) sí devuelven 404 ante una clave desconocida, y hay tests que fijan las dos mitades de esa regla.

Los comentarios de plantilla de Django son de una sola línea. `TemplateCommentLeakTests`, en `core/tests.py`, renderiza todas las pantallas y comprueba que ninguna emite `{# ni #}`, con un docstring que explica la trampa: un comentario `{# ... #}` multilínea no es un comentario en absoluto y su texto sale visible en la página. El propio docstring anota que esto «has shipped twice before», que es la razón de que exista el test.

Una sola puerta de entrada a las fechas del calendario. `core/calendario.py` deja escrito que los tiempos se guardan en UTC y se introducen y muestran en `CALENDAR_TZ`, que es `ZoneInfo("America/Bogota")`. `FullCalendar` corre con `timeZone` nombrado y sin plugin de zonas horarias, de modo que sus objetos `Date` son relojes de pared forzados a UTC; `parse_client_dt` es el único sitio donde se hace esa interpretación, y el módulo prohíbe explícitamente que cualquier otro punto del código del calendario parsee una cadena de fecha.

Los tests no dan por buena una clave fija cuando la realidad cambia. El helper `a_placeholder_section()` de `core/tests.py` no devuelve una sección quemada a fuego: recorre `ALL_NAV` buscando la primera cuya plantilla resuelta sea `PLACEHOLDER_TEMPLATE`, y lanza `AssertionError` con el mensaje «every section has a template now; this test needs rewriting» si ya no queda ninguna. Su docstring explica el motivo: a medida que se construyen pantallas, una clave hardcodeda empieza a probar otra cosa sin que nadie se entere. `core/tests_automatizaciones.py` aplica la misma honestidad al revés: como la pestaña de Automatizaciones se quitó de la barra lateral y `/s/automatizaciones/` da 404, sus tests están marcados con `skip` en vez de borrados, y el docstring indica que basta con quitar los decoradores cuando la pestaña vuelva.

El mismo panel montado dos veces se prueba dos veces. Campañas y el CRM comparten `core/crm.py`, sus secciones, sus paneles y el endpoint `crm_panel`; en `SECTION_CONTEXT` las claves `crm` y `campanas` apuntan a la misma función `_crm_context`, y solo difiere la plantilla de sección. `core/tests_campanas.py` existe precisamente para fijar ese reparto, y uno de sus tests comprueba que la lista de clientes se renderiza idénticamente desde los dos montajes.

12.9 Cómo escribir un módulo de tests nuevo

Los módulos existentes siguen todos la misma forma, así que copiar uno es la vía rápida. El archivo se llama `core/tests_<pantalla>.py` y abre con un docstring de una a tres frases que enumera qué cubre; los de `core/tests_calendario.py` y `core/tests_estadisticas_ventas.py` son buenos modelos. Se importa el módulo de datos de la pantalla (`from core.crm import ALL_VIEWS` en `core/tests_crm.py`, `from core.comercio import ALL_VIEWS, SECTIONS, STANDALONE, TABLE_COLUMNS, TABS` en `core/tests_comercio.py`) y se itera sobre él en vez de repetir las claves, de modo que agregar una vista rompe el test si algo no está cableado.

- Una clase `TestCase` por área, con nombre descriptivo terminado en `Tests`: `CrmScreenTests`, `ClientTableTests`, `CrmPanelEndpointTests`.
- Nombres de método que se leen como una frase:
`test_unknown_filter_falls_back_instead_of_404`,
`test_nine_bogota_comes_back_as_nine_not_fourteen`,
`test_mirror_cannot_authenticate_through_the_orm`.
- La constante `HTMX = {"HX-Request": "true"}` al principio del módulo para probar la respuesta de fragmento.
- `self.subTest(...)` al iterar sobre las filas de nav o las vistas, para que el fallo señale cuál.
- Helpers locales de construcción (`make_product`, `make_event`, `message_event`) en vez de fixtures compartidas.

El repertorio de comprobaciones también se repite entre pantallas: que se rendericen ambas columnas, que exista una fila de nav por cada vista declarada, que exactamente una fila esté activa, que la vista por defecto sea la esperada, que un `?view=` desconocido caiga a la por defecto en vez de dar 404, que el endpoint de panel devuelva solo el fragmento y que su salida coincida con la que la página incrusta, y que la sección devuelva un fragmento desnudo ante una petición `htmx`. Cubrir esa lista para una pantalla nueva es el mínimo que el resto del proyecto ya cumple.

Archivos de referencia: `README.md`, `config/settings.py`, `core/middleware.py`, `core/views.py`, `core/nav.py`, `core/crm.py`, `core/inbox.py`, `core/calendario.py`, `core/templatetags/assets.py`, `static/js/shell.js`, `requirements.txt`, `core/tests.py`, `core/tests_agents.py`, `core/tests_auth.py`, `core/tests_automatizaciones.py`, `core/tests_calendario.py`, `core/tests_campanas.py`, `core/tests_clientes.py`, `core/tests_comercio.py`, `core/tests_crm.py`, `core/tests_embudos.py`, `core/tests_estadisticas.py`, `core/tests_estadisticas_atribuciones.py`, `core/tests_estadisticas_embudos.py`, `core/tests_estadisticas_etiquetas.py`, `core/tests_estadisticas_temas.py`, `core/tests_estadisticas_tiempos.py`, `core/tests_estadisticas_ventas.py`, `core/tests_estadisticas_volumen.py`, `core/tests_legal.py`, `core/tests_mensajeria.py`, `core/tests_respuestas_rapidas.py`, `core/tests_storage.py`, `messaging/tests.py`, `messaging/tests_reset.py`

13. Guía práctica: cómo añadir una sección, un panel o una vista nueva

*Receta paso a paso para el trabajo más frecuente en MVP-CRM: dar de alta una sección del sidebar o un panel dentro de una sección existente, apoyándose en el descubrimiento por convenio de `core/views.py` y `core/comercio.py`, con *Mi comercio* como ejemplo completo de principio a fin.*

13.1 Propósito y alcance

Casi todo el trabajo de producto en este repositorio consiste en una de dos cosas: añadir un icono nuevo al rail de la izquierda, o añadir una página nueva dentro de una sección que ya existe. El shell está construido para que ambas operaciones sean casi siempre declarativas: se declara el elemento en un módulo de datos y se crea un archivo de plantilla con el nombre correcto. El descubrimiento por convenio hace el resto, sin tocar `core/views.py` ni `core/urls.py`.

Este capítulo recorre la receta completa en los dos niveles. Primero la columna 1, el rail de secciones definido en `core/nav.py`. Después las columnas 2 y 3, el patrón de navegación secundaria más panel que repiten CRM, Campañas, Embudos, Automatizaciones, Mi comercio, Estadísticas y Configuración de mensajería. Se cierra con la hoja CSS, el gancho de JavaScript y la prueba mínima, tomando Mi comercio como ejemplo trabajado.

13.2 Nivel 1: una sección nueva del sidebar

El rail es una única fuente de verdad. `core/nav.py` declara el dataclass congelado `NavItem` con cuatro campos: `key`, que es a la vez el slug de la URL y el nombre de la plantilla de sección; `label`, la etiqueta en español que se usa como tooltip visible y como nombre accesible porque el rail es solo de iconos; `icon`, el nombre del SVG bajo `templates/icons/`; y `badge`, que dibuja el punto rojo de notificación y hoy solo lo lleva Academy. La propiedad `icon_template` compone `icons/<icon>.svg`. Añadir, reordenar o renombrar una sección es una línea en `PRIMARY_NAV` o en `SECONDARY_NAV`.

```
# core/nav.py
PRIMARY_NAV = [
    NavItem("inbox", "Inbox", "inbox"),
    ...
    NavItem("mi-seccion", "Mi sección", "grid"),
]
```

Con eso la sección ya resuelve. La ruta `path("s/<slug:key>/", views.section, name="section")` sirve todas las claves; `core.views.section` valida el slug contra `NAV_BY_KEY` y lanza `Http404` si no existe. Mientras no haya una plantilla propia, `_section_template` devuelve `sections/_placeholder.html`, que pinta el título de la sección y el mensaje "próximamente" con el mismo glifo del rail. Para dejar de caer en el placeholder basta con crear `templates/sections/<key>.html`: no hay que tocar la vista, la URL ni la navegación, y se recoge en la petición siguiente.

El tercer paso es opcional y solo aparece cuando la pantalla necesita datos. En vez de añadir ramas a `section()`, se registra un constructor de contexto en el diccionario `SECTION_CONTEXT` de `core/views.py`, que mapea la clave de sección a un invocable que recibe `request` y devuelve un diccionario. La vista lo busca con `SECTION_CONTEXT.get(key)` y hace `context.update(...)` si existe. Campañas es el caso interesante: apunta al mismo `_crm_context` que CRM, de modo que ambas entradas del rail montan la misma navegación y los mismos paneles.

Nota. `core.views.section` responde de dos formas a la misma URL: fragmento cuando `_is_htmx` detecta la cabecera `HX-Request` con valor `"true"`, y documento completo con base `.html` en cualquier otro caso. Por eso toda plantilla bajo `templates/sections/` debe ser un fragmento, sin `<html>` y sin sidebar.

Toda ruta nueva queda además detrás de `LoginRequiredMiddleware`, en `core/middleware.py`. No es `django.contrib.auth`: es un único usuario y contraseña compartidos que vienen de `APP_LOGIN_USERNAME` y `APP_LOGIN_PASSWORD`, se comprueban en `core.views.login_view` y se recuerdan como el flag de sesión `app_authenticated`. Cualquier petición sin ese flag se redirige a `/login/?next=<ruta>`, salvo las rutas exentas de `EXEMPT_PATHS` (`/login/`, `/privacidad/`, `/eliminacion-de-datos/`) y los prefijos de `EXEMPT_PREFIXES` (`/webhooks/`, `/static/`, `/media/`, `/admin/`). El docstring explica el porqué de cada exención: los webhooks de proveedor se autentican con su propia firma, las páginas legales las lee la revisión de aplicaciones de Meta sin cuenta, y el admin de Django lo protege `django.contrib.auth`.

Nota. El middleware se salta por completo cuando `settings.TESTING` es cierto, y `config/settings.py` lo define como `'test' in sys.argv`. Eso es lo que permite que todas las pruebas del capítulo hagan `self.client.get(...)` sin autenticarse antes.

13.3 Nivel 2: columnas 2 y 3 dentro de una sección

Las secciones ya construidas comparten una anatomía de dos columnas: navegación secundaria a la izquierda y panel activo a la derecha. La navegación se declara en un módulo de datos por sección, con la misma forma en todos ellos. `core/crm.py` y `core/comercio.py` agrupan sus vistas en `Section` colapsables; `core/embudos.py`, `core/automatizaciones.py`, `core/estadisticas.py` y `core/mensajeria.py` usan una lista plana `VIEWS` cuyas filas llevan icono propio. En todos los casos hay un `VIEW_BY_KEY` para validar, un `DEFAULT_VIEW` para el primer render y un `PLACEHOLDER_PANEL`.

- `View` es una fila seleccionable: `key` (slug de `?view=`) y `label` (encabezado del panel); en los módulos planos añade además icon.
- `Section` agrupa varias `View` bajo un título y un icono, y expone `icon_template`.
- `SingleView`, solo en `core/comercio.py`, es una fila suelta fuera de los grupos: hereda de `View` y añade su propio icon, con `settings` por defecto.
- `panel_template(view_key)` intenta `get_template` sobre `partials/<screen>/panels/<view_key>.html` y devuelve el placeholder si lanza `TemplateDoesNotExist`.

El componente compartido `templates/partials/side_nav_section.html` renderiza un grupo colapsable, pero solo lo usan las pantallas que tienen grupos. Sus dos únicos consumidores son `templates/partials/comercio/nav_panel.html`, para Mi comercio, y `templates/partials/account_nav_panel.html`, que montan CRM y Campañas. Se incluye una vez por grupo, pasando `section`, `active_view`, `section_key`, `panel_url_name` y `panel_target`; el parámetro opcional `flat` elige entre el `<details>` colapsable y una versión plana siempre abierta. Las filas hijas viven en `templates/partials/side_nav_rows.html` para que ambas variantes compartan una sola fuente.

Las navegaciones planas no pasan por ese componente.

`templates/partials/embudos/nav_panel.html`, `templates/partials/estadisticas/nav_panel.html` y `templates/partials/mensajeria/nav_panel.html` recorren su lista `VIEWS` y escriben la etiqueta `<a class="side-nav__row">` a mano, sin incluir ni `side_nav_section.html` ni `side_nav_rows.html`. Comparten con el resto las clases `.side-nav` y los mismos atributos `HTMX`, pero cada una tiene su propio detalle: Estadísticas usa el encabezado "Menú" en vez de "Mi cuenta" y pinta el pill de `view.badge` (el

"Alpha" de Atribuciones), y Configuración de mensajería no lleva encabezado, con el nombre de la sección solo en el `aria-label` del `<nav>`.

En todas las variantes cada fila es un `<a href>` real que HTMX convierte en un `hx-get` contra el endpoint del panel, con `hx-target` al id de la columna 3, `hx-swap="innerHTML"` y `hx-push-url` a la URL canónica de la sección. La columna 3 tiene su propio endpoint por sección, del estilo `comercio/panel/<slug:view_key>/`. Esa vista valida la clave contra `VIEW_BY_KEY` y responde 404 si no la conoce, monta un contexto mínimo y renderiza `panel_template(view_key)`. Así, elegir una página redibuja solo el panel y la navegación conserva su estado de plegado.

Si el panel necesita datos, se registra un constructor en el diccionario de contexto de paneles de esa pantalla, por ejemplo `COMERCIO_PANEL_CONTEXT`, `STATS_PANEL_CONTEXT`, `AUTOM_PANEL_CONTEXT`, `EMBUDOS_PANEL_CONTEXT`, `MENSAJERIA_PANEL_CONTEXT` o, para el CRM, el que se llama simplemente `PANEL_CONTEXT`. El mismo diccionario lo consultan dos veces: el constructor de contexto de la sección, para el primer render de página completa, y el endpoint del panel, para el swap. Los paneles sin entrada no necesitan datos.

Nota. Hoy ningún llamador pasa `flat=True`. El comentario de `templates/partials/side_nav_section.html` dice que Campañas monta así la navegación, pero `templates/sections/campanas.html` no pasa el parámetro, de modo que la rama plana del componente está sin usar. Conviene saberlo antes de apoyarse en ella para una navegación nueva.

13.4 Reutilizar una navegación entre dos secciones

CRM y Campañas son la misma pantalla montada dos veces, y el mecanismo merece detallarse porque es el patrón a copiar. El partial `templates/partials/account_nav_panel.html` no nombra ninguna sección concreta: recibe `sections`, `active_view`, `section_key`, `panel_url_name`, `panel_target` y el opcional `flat`, y su comentario dice explícitamente que una tercera sección que quiera este panel solo tiene que incluirlo con sus propios parámetros de montaje.

`templates/sections/crm.html` lo incluye con `section_key="crm"` y `panel_target="crm-panel"`, y declara `<main class="crm__panel" id="crm-panel">`. `templates/sections/campanas.html` hace lo mismo con `section_key="campanas"` y `panel_target="campanas-panel"`, sobre `<main class="crm__panel" id="campanas-panel">`. Los dos pasan `panel_url_name="crm_panel"`, es decir, el mismo endpoint de columna 3, y los dos reciben `crm_sections` desde `_crm_context`. Lo único que cambia entre ambos montajes es el slug que se empuja a la URL y el id del contenedor que HTMX reemplaza, así que un panel construido una vez se enciende bajo las dos entradas del rail.

13.5 Referencia: puntos de extensión y archivos

Objetivo	Archivo a editar o crear	Cambio necesario
Icono nuevo en el rail	<code>core/nav.py</code>	Un <code>NavItem</code> en <code>PRIMARY_NAV</code> o <code>SECONDARY_NAV</code>
Pantalla real en vez del placeholder	<code>templates/sections/<key>.html</code>	Crear el archivo (fragmento, sin <code><html></code>)
Datos para la sección	<code>core/views.py</code>	Entrada en <code>SECTION_CONTEXT</code>
Fila nueva en la navegación secundaria	<code>core/crm.py</code> , <code>core/comercio.py</code> , <code>core/embudos.py</code> , <code>core/estadisticas.py</code> , <code>core/automatizaciones.py</code> , <code>core/mensajeria.py</code>	Un <code>View</code> en la <code>Section</code> o en <code>VIEWS</code>
Panel real en vez del placeholder	<code>templates/partials/<screen>/panels/<view>.html</code>	Crear el archivo

Qué tocar según lo que se quiera añadir

Módulo	Estructura	DEFAULT_VIEW	Endpoint del panel
core/crm.py	2 Section colapsables	clientes	crm_panel
core/embudos.py	IEWS plana, 3 filas	embudos	embudos_panel
core/automatizaciones.py	IEWS plana, 4 filas	chatbots-flujo	automatizaciones_panel
core/comercio.py	4 Section + 1 SingleView	productos	comercio_panel
core/estadisticas.py	IEWS plana, 7 filas con badge	mensajería	estadisticas_panel
core/mensajería.py	IEWS plana, 7 filas	plantillas-what sapp	mensajería_panel

Módulos de navegación secundaria y sus constantes

Clave	Contenido
comercio_sections	comercio.SECTIONS
comercio_single	comercio.STANDALONE
active_view	Clave validada de ?view=
comercio_view	El View activo
panel_template	Ruta devuelta por comercio.panel_template

Contexto que publica _comercio_context

13.6 El ejemplo completo: Mi comercio

Mi comercio ejerce todas las piezas a la vez y sirve de plantilla mental. En `core/nav.py` hay un `NavItem("mi-comercio", "Mi comercio", "store")`. En `core/comercio.py` hay cuatro secciones colapsables (Catálogo, Órdenes, Pagos, Envíos), una fila suelta `STANDALONE` con clave `configuracion-comercio`, y `ALL_VIEWS` suma doce vistas. `templates/sections/mi-comercio.html` incluye `partials/comercio/nav_panel.html` y declara `<main class="comercio__panel" id="comercio-panel">`, que es el destino de los `hx-target`.

`templates/partial/comercio/nav_panel.html` incluye cuatro veces `side_nav_section.html` con `panel_url_name="comercio_panel"` y `panel_target="comercio-panel"`, y pinta la fila suelta a mano con los mismos atributos HTMX.

De las doce vistas solo productos tiene panel propio; las otras once caen en `partials/comercio/panels/_placeholder.html`, que muestra el label seguido de "próximamente". El panel de Productos añade un segundo nivel de swap: sus pestañas apuntan a `productos_table` y reemplazan solo `#product-table`, dejando quietos el título y la barra de herramientas. Los botones "Crear +" e "Importar" apuntan a `producto_create` y `producto_import`, que renderizan `partials/comercio/panels/_crear.html` y `partials/comercio/panels/_importar.html`: están cableados para que el botón lleve a algún sitio real, pero los flujos de creación e importación no están contruidos.

El filtrado de la tabla también vive en `core/comercio.py`. `TABS` declara tres pestañas (todos, activos, inactivos) y `DEFAULT_TAB` es activos, no todos, porque la referencia abre en Activos según dice el comentario del código. El mapa privado `_TAB_STATUS` traduce la clave de pestaña al valor de `Product.status` por el que filtra, y omite a propósito todos, que es la ausencia de filtro. `get_products(tab_key)` parte de `Product.objects.all()` y aplica ese filtro si lo hay; su docstring aclara que con la tabla vacía devuelve cero filas y la plantilla pinta la cabecera desnuda, el estado vacío de la referencia, y que no hay datos de ejemplo por decisión de diseño.

Quedan dos enganches fuera de Python. El CSS de la pantalla se enlaza en `templates/base.html` con el tag `{% vstatic %}` de `core/templatetags/assets.py`; para Mi comercio la línea es la de `css/comercio.css`. El JavaScript va en `static/js/shell.js`, cargado con `defer` al final del `<body>`: no hay build de frontend, así que el comportamiento nuevo se añade como listener delegado desde `document` dentro de ese IIFE. Para una navegación basta con marcar el contenedor con `data-nav-group` y cada fila con `data-nav-item`; `shell.js` mueve la clase `is-active` y el `aria-current` por su cuenta.

Conviene saber qué hace exactamente `{% vstatic %}`, porque su docstring explica el problema que resuelve: `{% static %}` emite una ruta estable y la build de Django en Vercel ejecuta `collectstatic` sin nombres con hash, así que un arreglo de CSS puede desplegarse y no llegar a quien ya tenía la página cacheada. `vstatic` calcula un sha256 del contenido del archivo y lo trunca a diez caracteres, y devuelve la URL con `?v=<hash>` detrás. Fuera de `DEBUG` el hash se cachea una vez por proceso; en desarrollo se recalcula en cada render para que una edición se vea al recargar. Si `finders.find` no encuentra el archivo el hash es la cadena vacía y la URL sale sin `?v=`, y ese fallo también se cachea fuera de `DEBUG` porque la respuesta no puede cambiar sin un redesplicue.

13.7 Decisiones de diseño y trampas conocidas

El patrón central es que el placeholder no es un estado de error sino el punto de extensión. Tanto `_section_template` como cada `panel_template` resuelven por existencia de archivo, y sus docstrings lo dicen explícitamente: construir una pantalla significa crear el archivo, sin cambios de vista ni de URL. La contrapartida es que un nombre mal escrito no falla ruidosamente, simplemente sigue mostrando "próximamente".

Los parámetros de consulta son tolerantes y los segmentos de URL no. Un `?view=` o un `?tab=` desconocido cae al valor por defecto en vez de devolver 404, para que un marcador antiguo siga abriendo; en cambio los endpoints de panel y de tabla sí lanzan `Http404` ante una clave desconocida. Otra decisión visible en las rutas: `comercio/productos/tab/<slug:tab_key>/` lleva el prefijo `tab/` precisamente para que el slug no se trague las rutas hermanas `nuevo/` e `importar/`, y `mensajeria/plantillas/tab/<slug:tab_key>/` hace lo mismo frente a `galeria/` y `editor/`.

La navegación colapsable usa `<details>` y `<summary>` nativos en lugar de JavaScript: se pliega, es operable por teclado, anuncia su estado a los lectores de pantalla y sigue funcionando si el JS no carga. En la misma línea, cada fila es un enlace real y HTMX solo la mejora. Conviene recordar además que los comentarios `{# ... #}` de Django son de una sola línea: un comentario multilínea con esa sintaxis no es un comentario y se filtra a la página como texto visible, algo que ya ha llegado a producción dos veces según el docstring de la prueba que lo vigila.

Nota. El docstring de `core/views.py` dice que `section` sirve 14 destinos del sidebar, pero `ALL_NAV` suma 11: el número está desactualizado. En `core/nav.py` lo único comentado, con fecha 2026-08-26, son los tres `NavItem` de Automatizaciones, Performance HUB y Herramientas de Crecimiento.

Ese comentario aclara que para restaurar una entrada basta con volver a añadir el `NavItem`, porque la pantalla, las vistas y las plantillas de Automatizaciones siguen en su sitio y sin comentar: `core/views.py` importa el módulo `automatizaciones`, `SECTION_CONTEXT` conserva la clave `automatizaciones`, `core/urls.py` mantiene la ruta `automatizaciones_panel` y existe `templates/sections/automatizaciones.html`. Performance HUB y Crecimiento, en cambio, nunca llegaron a tener vistas ni plantillas propias.

13.8 Pruebas que lo cubren

La suite completa se ejecuta con `python manage.py test`, y cada pantalla tiene su archivo propio bajo `core/`. Para el nivel 1, `core/tests.py` cubre el shell: `NavDefinitionTests` fija los recuentos de `PRIMARY_NAV` y `SECONDARY_NAV`, comprueba que las claves son únicas, que solo `Academy` lleva badge y que todos los `icon_template` existen; `RoutingTests` resuelve todas las claves de `ALL_NAV` y verifica el 404 de una desconocida; `SidebarRenderTests` comprueba el pill activo y que las filas son enlaces reales; `HtmxSwapTests` contrasta fragmento contra página completa; `SectionTemplateTests` valida las dos ramas de `_section_template`; y `TemplateCommentLeakTests` renderiza todas las secciones para asegurar que no se filtra `{# ni #}`.

Merece la pena copiar un detalle de ese archivo: el helper `a_placeholder_section()` no codifica una clave a mano, sino que recorre `ALL_NAV` buscando la primera cuya plantilla siga siendo el placeholder. Su docstring explica el porqué: según se van construyendo pantallas, una clave fija empieza a probar silenciosamente otra cosa. Si algún día todas tienen plantilla, el helper lanza `AssertionError` pidiendo que se reescriba la prueba.

Para el nivel 2, `core/tests_comercio.py` es el modelo a imitar. `ComercioScreenTests` comprueba que se pintan las dos columnas y el id `comercio-panel`, que las cuatro secciones arrancan plegadas, que todas las filas de `ALL_VIEWS` aparecen, que `?view=` selecciona el panel y que un valor desconocido cae al defecto sin 404. `ComercioPanelEndpointTests` recorre todas las vistas contra `comercio_panel`, comprueba que la respuesta no reenvía la navegación y que una clave inventada da 404.

`ProductosPanelTests`, `ProductosTabTests` y `ProductosTableEndpointTests` cubren el segundo nivel de swap, incluida una prueba de que los slugs de pestaña no se tragan las rutas de los botones. Una prueba mínima para una pantalla nueva son cuatro asserts: la sección responde 200, el panel por defecto es el esperado, el endpoint del panel devuelve solo el fragmento, y una clave desconocida devuelve 404.

Nota. Ninguna de estas pruebas inicia sesión, y no es un descuido: `LoginRequiredMiddleware` se desactiva cuando `settings.TESTING` es cierto. Una prueba que quiera ejercitar el propio gate tiene que hacerlo explícitamente, como hace `core/tests_auth.py`.

Archivos de referencia: `core/nav.py`, `core/views.py`, `core/urls.py`, `core/middleware.py`, `core/comercio.py`, `core/crm.py`, `core/embudos.py`, `core/estadisticas.py`, `core/automatizaciones.py`, `core/mensajeria.py`, `core/templatetags/assets.py`, `core/tests.py`, `core/tests_comercio.py`, `config/settings.py`, `templates/base.html`, `templates/sections/_placeholder.html`, `templates/sections/mi-comercio.html`, `templates/sections/crm.html`, `templates/sections/campanas.html`, `templates/partials/side_nav_section.html`, `templates/partials/side_nav_rows.html`, `templates/partials/account_nav_panel.html`, `templates/partials/comercio/nav_panel.html`, `templates/partials/comercio/panels/_placeholder.html`, `templates/partials/comercio/panels/productos.html`, `templates/partials/embudos/nav_panel.html`, `templates/partials/estadisticas/nav_panel.html`, `templates/partials/mensajeria/nav_panel.html`, `static/js/shell.js`

14. Glosario de términos del dominio

Diccionario de los términos de negocio en español que atraviesan la UI, las URL y los nombres de módulo de MVP-CRM, con su contraparte técnica: qué modelo o dataclass representa cada concepto, en qué módulo vive y qué decisiones de diseño explican los comentarios del código.

14.1 Propósito y alcance

MVP-CRM está escrito en dos idiomas a la vez. La interfaz, las etiquetas de navegación y buena parte de los slugs de URL están en español («Embudos», «Atribuciones», «Respuestas rápidas»), mientras que los modelos, campos y funciones están en inglés (Conversation, MessageTemplate, assigned_to). Este capítulo es el puente entre ambos: para cada término de negocio dice qué significa para el usuario, qué modelo o dataclass lo representa y en qué módulo vive. Sirve para leer un nombre de módulo como `core/embudos.py` o una URL como `/s/estadisticas/?view=atribuciones` sin abrir el código.

El glosario documenta únicamente lo que existe hoy. Varios términos del producto de referencia todavía no tienen modelo propio y el código lo dice abiertamente: no hay modelo Funnel, no hay modelo de pedido, no hay clasificador de IA, no hay modelo Flow y no hay modelo de organización ni de equipo. Donde eso ocurre, la entrada correspondiente lo señala en lugar de describir la funcionalidad prevista. Tampoco es un inventario de la UI visible: `core/nav.py` documenta que el 2026-08-26 se retiraron tres entradas del menú (Automatizaciones, Performance HUB y Herramientas de Crecimiento) dejando en su sitio sus vistas, módulos y plantillas, de modo que hay términos con código vivo y sin icono en la barra lateral.

- Términos cubiertos: inbox, conversación, mensaje, canal, MIA, agente, asignación, cliente, lista de clientes, etiqueta, embudo, etapa, campaña, plantilla de WhatsApp, respuesta rápida, producto, evento de calendario, atribución, tema de conversación, periodo y venta.
- Fuentes principales: `core/nav.py`, `core/models.py`, `messaging/models.py` y los módulos de datos por pantalla (`core/crm.py`, `core/embudos.py`, `core/comercio.py`, `core/mensajería.py`, `core/automatizaciones.py`, `core/plantillas.py`, `core/clientes.py`, `core/agents.py`) y por informe (`core/estadisticas_atribuciones.py`, `core/estadisticas_temas.py`, `core/estadisticas_periodos.py`).
- Fuera de alcance: la capa de proveedores de mensajería, el webhook y las plantillas HTML, que tienen sus propios capítulos.

14.2 El patrón que se repite: navegación declarativa y paneles

Casi todos los términos del glosario aparecen primero como una fila de navegación. El patrón está descrito en la docstring de `core/nav.py`: todo lo que define un elemento del menú (slug de URL, etiqueta en español, icono SVG y si lleva punto de notificación) vive en una dataclass `NavItem`, y la plantilla solo itera sobre la lista. Añadir, reordenar o renombrar una sección es un cambio de una línea en ese archivo, sin tocar plantillas. `PRIMARY_NAV` reúne las ocho secciones que hoy se pintan y `SECONDARY_NAV` las tres fijadas al pie; `NAV_BY_KEY` valida el slug de la URL y `DEFAULT_SECTION` vale "inbox".

`PRIMARY_NAV` no es el censo de lo que existe. Justo debajo, un bloque comentado conserva los tres `NavItem` retirados el 2026-08-26 para poder reinstalarlos con una línea, y el propio comentario aclara que la pantalla de Automatizaciones, sus vistas y sus plantillas siguen en su sitio. La otra aclaración de ese archivo es sobre la raíz: «/» ya no es una sección, sino la pantalla de bienvenida que renderiza `core.views.welcome` sin icono seleccionado; `WELCOME_SHORTCUTS` lista las tres secciones que esa pantalla ofrece como tarjetas de acceso rápido, resueltas contra `NAV_BY_KEY` para reutilizar la misma etiqueta y el mismo glifo de la barra.

Cada sección con navegación secundaria repite la misma forma con su propia dataclass View: `core/crm.py` agrupa sus vistas en `Section` colapsables, `core/embudos.py`, `core/estadisticas.py`, `core/mensajería.py` y `core/automatizaciones.py` usan listas planas donde cada fila lleva su icono, y `core/comercio.py` añade además una fila suelta (STANDALONE) y las pestañas `Tab` de la tabla de Productos. Al menos seis módulos exponen la misma función `panel_template(view_key)`, que devuelve `partials/<sección>/panels/<view_key>.html` si la plantilla existe y, si no, el placeholder. Construir una de esas páginas significa crear el archivo: no hay que tocar ni la vista ni la URL. `core/estadisticas.py` añade una variante del mismo gesto, `card_template(card_key)`, para las cuatro tarjetas del panel de Mensajería.

Los términos de datos, en cambio, viven en modelos. El CRM aporta `Client`, `Product`, `ClientList`, `MessageTemplate` y `CalendarEvent` en `core/models.py`; la mensajería aporta `Tag`, `Conversation`, `ConversationTag` y `Message` en `messaging/models.py`. La costura entre ambos mundos es el cliente: la docstring de `messaging/models.py` lo dice explícitamente, la persona con la que se chatea en el Inbox es la misma fila de la tabla Clientes. El procesamiento de webhooks, sin embargo, solo crea esa fila en el primer contacto: `_upsert_contact` busca por phone exacto y devuelve la fila existente sin tocar nada, así que ni el nombre, ni el canal, ni el país se refrescan en contactos posteriores.

Nota. Las claves de canal de `Conversation.CHANNEL_CHOICES` sí coinciden con las de `core.inbox.CANALES`, para que filtrar el Inbox por canal sea una consulta directa `channel=key`. Las de `Client.CHANNEL_CHOICES` no: son un juego más grueso donde «instagram» y «tiktok» no distinguen `instagram-dm`, `tiktok-dm` ni `tiktok-coment`, y «tiktok» ni siquiera es una clave de `CANALES`, de modo que un `Client` con `channel="tiktok"` no casa con ningún filtro del Inbox. Por eso `messaging.services._client_channel` traduce explícitamente la clave de conversación a la de `Client`.

14.3 Glosario alfabético (A-E)

Término en la UI	Qué significa	Representación en el código	Dónde vive
Agente	La persona que responde conversaciones; es a la vez un login y un destinatario de asignaciones.	Dataclass Agent (username, password, display_name) más filas User espejo creadas bajo demanda por <code>_mirror</code> .	<code>core/agents.py</code>
Asignación	A qué agente pertenece una conversación; sin asignar aparece como «Sin asignar» y asignada aparece en «Tu inbox».	<code>Conversation.assigned_to</code> , FK nutable a <code>AUTH_USER_MODEL</code> ; las opciones del desplegable las da <code>assignment_options(conversation)</code> .	<code>messaging/models.py</code> , <code>core/agents.py</code>
Atribución	De qué canal vienen las conversaciones del periodo y las ventas entre ellas.	Dataclass Row (key, label, conversations, sales, conversion_pct, share_pct) y la función <code>report(period)</code> .	<code>core/estadisticas_atribuciones.py</code>
Campaña	Sección del menú lateral que monta exactamente la misma navegación «Mi cuenta» y los mismos paneles que el CRM.	No tiene modelo propio: <code>core.crm.SECTIONS</code> se reutiliza y <code>core/views.py</code> mapea la clave <code>campanas</code> al mismo constructor de contexto que <code>crm</code> .	<code>core/crm.py</code> , <code>core/views.py</code>
Canal	La superficie por la que llega el mensaje: WhatsApp, Messenger, Instagram DM, Facebook, Instagram, Tiktok DM o Tiktok comentarios.	<code>Conversation.CHANNEL_CHOICES</code> y <code>Conversation.channel</code> ; los filtros equivalentes son <code>core.inbox.CANALES</code> . <code>Client.channel</code> usa un juego más grueso y distinto.	<code>messaging/models.py</code> , <code>core/inbox.py</code>
Cliente	Una persona en la tabla Clientes del CRM y el interlocutor de cualquier conversación.	Modelo <code>Client</code> , con propiedades derivadas <code>full_name</code> , <code>initials</code> , <code>has_whatstapp</code> y el resto de ayudantes de fila.	<code>core/models.py</code>
Conversación (hilo)	Un hilo con un cliente en un canal; una persona puede tener varios a lo largo del tiempo, pero solo uno activo por canal.	Modelo <code>Conversation</code> , con estados <code>open</code> , <code>pending</code> y <code>resolved</code> y los mensajes en <code>Message</code> .	<code>messaging/models.py</code>
Embudo	Pantalla de embudos de venta. No hay embudos personalizados todavía.	<code>core.embudos.get_funnels()</code> devuelve una lista vacía, que es lo que hace al panel renderizar su tarjeta de estado vacío.	<code>core/embudos.py</code>
Etapas	Cada barra del embudo integrado que sí se calcula, en Estadísticas > Embudos.	Dataclass <code>Stage</code> (key, label, count, pct, tone, tip); cuatro etapas fijas: creadas, respondidas, resueltas y venta.	<code>core/estadisticas_embudos.py</code>
Etiqueta	Rótulo que los usuarios inventan y aplican a conversaciones («CLIENTE NUEVO», «VENTA EFECTIVA»).	Modelo <code>Tag</code> con color de paleta y <code>is_archived</code> ; la aplicación concreta es <code>ConversationTag</code> , con <code>tagged_by</code> y <code>tagged_at</code> .	<code>messaging/models.py</code>

Término en la UI	Qué significa	Representación en el código	Dónde vive
Evento de calendario	Una entrada de «Mi calendario» dentro del CRM, opcionalmente ligada a un cliente.	Modelo CalendarEvent (title, start, end, all_day, contact, assigned_to, event_type, reminder_minutes_before).	core/models.py, core/calendario.py

Términos de negocio y su contraparte en el código.

14.4 Glosario alfabético (I-V)

Término en la UI	Qué significa	Representación en el código	Dónde vive
Inbox	La bandeja de conversaciones: primera sección del menú y sección por defecto.	<code>NavItem("inbox", "Inbox", "inbox")</code> y las dataclasses <code>Filter</code> y <code>FilterGroup</code> de la columna de filtros, agrupadas en <code>FILTER_GROUPS</code> .	<code>core/nav.py</code> , <code>core/inbox.py</code>
Lista de clientes	Un grupo con nombre de clientes, una fila de la tabla «Lista de clientes».	Modelo <code>ClientList</code> , con <code>M2M clients</code> ; el «número de contactos» se anota en la vista, no se almacena.	<code>core/models.py</code>
Mensaje	Cada burbuja del hilo, entrante o saliente, con su estado de entrega.	Modelo <code>Message</code> (<code>direction</code> , <code>body</code> , <code>media_url</code> , <code>media_type</code> , <code>status</code> , <code>provider_message_id</code> , <code>timestamp</code> , <code>sent_by</code>).	<code>messaging/models.py</code>
MIA	El asistente de IA. Da nombre al segundo grupo de filtros del Inbox: MIA activa, Intención compra y Ayuda humana.	Tres <code>Filter</code> declarados en la lista MIA; <code>get_conversations</code> devuelve <code>queryset.none()</code> para ellos porque no hay modelo de detección de IA detrás.	<code>core/inbox.py</code>
País	El desplegable que acompaña al teléfono del cliente y determina la bandera.	Dataclass <code>Country</code> (<code>code</code> , <code>name</code> , <code>dialing</code>) y la lista <code>COUNTRIES</code> ; <code>country_from_phone</code> la deduce del prefijo.	<code>core/clientes.py</code>
Periodo	El selector de ventana temporal compartido por los paneles de Estadísticas.	Dataclass <code>Period</code> (<code>key</code> , <code>label</code> , <code>days</code>) y las funciones <code>parse_period(data)</code> y <code>start_of(period)</code> .	<code>core/estadisticas_periodos.py</code>
Plantilla de WhatsApp	Un mensaje preaprobado por Meta: una fila de la tabla Plantillas y el resultado del editor «Crear plantilla».	Modelo <code>MessageTemplate</code> ; las categorías, subtipos, idiomas, cabeceras y la validación viven aparte.	<code>core/models.py</code> , <code>core/plantillas.py</code>
Producto	Una ficha del catálogo de «Mi comercio».	Modelo <code>Product</code> (<code>name</code> , <code>stock</code> , <code>price</code> , <code>category</code> , <code>brand</code> , <code>status</code>); las pestañas filtran por <code>status</code> .	<code>core/models.py</code> , <code>core/comercio.py</code>
Respuesta rápida	Dos cosas distintas con el mismo nombre: el botón <code>quick</code> de una plantilla, y el popover del compositor que inserta una plantilla en el chat.	<code>ButtonKind("quick", "Respuesta rápida")</code> para lo primero; la vista <code>inbox_quick_replies</code> y <code>plantillas.render_body</code> para lo segundo.	<code>core/plantillas.py</code> , <code>core/views.py</code>
Tema de conversación	Una palabra que los clientes usan de verdad, no un tema detectado por IA.	Dataclass <code>Topic</code> (<code>word</code> , <code>conversations</code> , <code>mentions</code>), construida por <code>report(period)</code> sobre mensajes entrantes tokenizados.	<code>core/estadisticas_temas.py</code>
Venta	Una conversación marcada con una etiqueta de venta; no existe modelo de pedido.	<code>SALE_NAME_TOKEN</code> y <code>sale_tags()</code> , que devuelve toda etiqueta cuyo nombre contenga «venta».	<code>core/estadisticas_ventas.py</code>
Vista / panel	Cada página seleccionable dentro de una sección, elegida con <code>?view=<key></code> .	Dataclass <code>View</code> repetida en cada módulo de sección, más <code>panel_template(view_key)</code> con su placeholder.	<code>core/crm.py</code> , <code>core/embudos.py</code> , <code>core/comercio.py</code> , <code>core/estadisticas.py</code> , <code>core/mensajeria.py</code> , <code>core/automatizaciones.py</code>

Continuación del diccionario.

14.5 Claves y valores de referencia

Concepto	Claves	Valor por defecto	Definido en
Secciones del menú principal	inbox, crm, embudos, mi-comercio, campanas, estadísticas, integraciones, mensajería	DEFAULT_SECTION = inbox	core/nav.py
Atajos de la pantalla de bienvenida	inbox, crm, embudos	No aplica	core/nav.py
Filtros del Inbox	todos, tu-inbox, sin-asignar; mia-activa, intencion-compra, ayuda-humana; las siete claves de canal	DEFAULT_FILTER = todos	core/inbox.py
Vistas del CRM	clientes, etiquetas, lista-clientes, campos-personalizados, exportaciones, mi-calendario	DEFAULT_VIEW = clientes	core/crm.py
Vistas de Embudos	embudos, automatizaciones, historial-descargas	DEFAULT_VIEW = embudos	core/embudos.py
Vistas de Estadísticas	mensajería, ventas, etiquetas, embudos, atribuciones (con badge «Alpha»), temas-conversacion, agentes-ia	DEFAULT_VIEW = mensajería	core/estadisticas.py
Vistas de Configuración de mensajería	widget-whatsapp, plantillas-whatsapp, respuestas-rapidas, mensajes-bienvenida, temas-conversacion, reglas-mensajería, asignacion-automatica	DEFAULT_VIEW = plantillas-whatsapp	core/mensajería.py
Vistas de Automatizaciones (sin icono en la barra)	chatbots-flujo, mia-agentes, mensajes-programados, flujos-whatsapp	DEFAULT_VIEW = chatbots-flujo	core/automatizaciones.py
Pestañas de Productos	todos, activos, inactivos	DEFAULT_TAB = activos	core/comercio.py
Pestañas de Plantillas	todas, pendientes, aceptadas, rechazadas, desactivadas	DEFAULT_TAB = todas	core/mensajería.py
Periodos estadísticos	7, 30, 90, todo	DEFAULT_PERIOD = 30	core/estadisticas_periodos.py
Tipos de evento de calendario	llamada, reunion, seguimiento, otro	DEFAULT_EVENT_TYPE = reunion	core/models.py, core/calendario.py
Estados de conversación	open, pending, resolved	OPEN	messaging/models.py
Categorías de plantilla	marketing, utility, authentication	DEFAULT_CATEGORY = marketing	core/models.py, core/plantillas.py

Enumeraciones que un desarrollador nuevo encontrará en URLs y plantillas.

Variable	Para qué sirve	Valor por defecto
APP_AGENTS	Lista de agentes separada por comas, cada entrada username:password:Nombre (el nombre visible es opcional).	Cadena vacía
APP_LOGIN_USERNAME	Usuario del par heredado, usado como agente único cuando APP_AGENTS no está definida.	Cadena vacía
APP_LOGIN_PASSWORD	Contraseña de ese par heredado.	Cadena vacía

Variables de entorno relacionadas con el término «agente».

14.6 Decisiones de diseño y trampas conocidas

El teléfono es la clave real del cliente aunque no sea una restricción de base de datos. La docstring de `core/clientes.py` lo explica: el Inbox localiza al cliente de un mensaje entrante por coincidencia exacta de `phone`, así que un número tecleado con espacios crearía un segundo cliente la próxima vez que esa persona escriba. `normalize_phone` reduce cualquier entrada a + seguido de dígitos (y devuelve cadena vacía si no hay ninguno) y `validate` rechaza duplicados, pero sin constraint en la base. El rango aceptado va de `PHONE_MIN_DIGITS` a `PHONE_MAX_DIGITS`, con un suelo deliberadamente laxo: rechazar el número real de un cliente se considera peor que guardar uno raro.

La lista `COUNTRIES` no pretende ser exhaustiva: cubre los mercados a los que apunta el CRM, y como `Client.country` es un `CharField` libre, un país que llegue por webhook se guarda igual y conserva su bandera. El emparejamiento por prefijo ordena de más largo a más corto (+1809 antes que +1) y los empates conservan el orden de `COUNTRIES`, razón por la que +1 cae en Canadá y no en Estados Unidos; el comentario admite que es una moneda al aire y que el agente puede corregirlo en el selector. `country_from_phone` solo rellena un país en blanco: una elección explícita nunca se sobrescribe.

Borrar una etiqueta es archivarla. Una etiqueta archivada desaparece de los selectores pero sigue en todas las conversaciones a las que se aplicó alguna vez, porque quitarla de cientos de chats por haber ordenado el selector reescribiría el pasado en silencio. Por el mismo motivo la M2M usa un through-model explícito, `ConversationTag`, que responde a «quién etiquetó este chat y cuándo»; una `ManyToManyField` normal tiraría ese dato. Los colores de etiqueta son tokens de paleta, no hex arbitrario, y cada token mapea a un par de variables CSS.

El filtrado del Inbox tiene dos detalles que sorprenden. Los filtros por etiqueta son AND, no OR: `get_conversations` encadena un `.filter(tags=tag_id)` por etiqueta, de modo que cada uno añade su propio join y la conversación debe llevarlas todas; un único `.filter(tags__in=...)` habría sido OR. Y el orden usa `nulls_last` sobre `last_message_at` para que una conversación recién creada sin mensajes se hunda al fondo en lugar de hacerse pasar por la más reciente. Los filtros del grupo MIA devuelven un queryset vacío a propósito: sin modelo de detección de IA, muestran el estado vacío honestamente en vez de fingir.

El término «agente» arrastra una consecuencia importante. Las credenciales viven en el entorno, no en la base de datos, así que dar de alta a un compañero es editar una variable y volver a desplegar: no hay registro, ni recuperación de contraseña, ni pantalla de gestión de usuarios. Como `assigned_to` es una FK a `AUTH_USER_MODEL`, cada agente configurado necesita una fila `User` real; esas filas son espejos creados bajo demanda con contraseña inutilizable, de modo que nadie pueda autenticarse a través del ORM. `authenticate` compara contra todos los agentes sin salir antes de tiempo y con `compare_digest`, para que el tiempo de respuesta no revele qué usuarios existen.

Nota. `assignment_options` añade al desplegable al asignado actual cuando ya no figura en `APP_AGENTS`. Sin esa opción el select caería en su primera entrada y afirmarían en falso que el chat está «Sin asignar».

Tres términos de Estadísticas son honestos sobre lo que miden. «Embudo» mide el único embudo que los datos ya contienen, la vida de una conversación, y sus cuatro etapas se miden de forma independiente contra las conversaciones del periodo, no anidadas: un chat puede marcarse como venta sin haber sido resuelto nunca. «Atribución» se detiene en el canal porque los webhooks todavía no capturan los datos de referencia de los anuncios click-to-WhatsApp; cuando lleguen, las filas de anuncio encajarán bajo su canal, y el módulo insiste en que nada de lo que hay ahí es matemática de relleno. «Tema» no es un tema detectado por IA sino una palabra realmente escrita por los clientes: solo se cuentan mensajes entrantes, se pliegan los acentos y se descartan saludos y palabras vacías.

La definición de «venta» esconde dos decisiones fáciles de pasar por alto. Una venta se fecha por `ConversationTag.tagged_at`, el rastro de auditoría del `through-model`, y no por la edad de la conversación: la venta ocurrió cuando alguien la marcó. Y `sale_tags()` incluye a propósito las etiquetas archivadas, porque una «VENTA 2025» archivada sigue fechando las ventas que registró. Además una conversación con dos etiquetas de venta cuenta como una sola: los recuentos van sobre conversaciones distintas.

Nota. El plegado de acentos de `core/estadisticas_temas.py` traduce solo «áéíóúü» y deja la ñ intacta a propósito: una descomposición unicode completa fundiría «año» con otra palabra. El análisis lee texto en Python, así que está acotado por `SCAN_CAP` mensajes y cacheado `CACHE_SECONDS` segundos por periodo.

En la conversación hay dos campos desnormalizados que sostienen media pantalla. `last_message_at` es la clave de orden de la lista y `last_inbound_at` es lo que hace barata la comprobación de la ventana de servicio, ambos mantenidos por `messaging/services.py` para no reconsultar la tabla de mensajes por fila. Sobre `last_inbound_at` se calcula `is_within_24h_window` contra `SERVICE_WINDOW`. En el mensaje, `status` no tiene lista propia: `STATUS_CHOICES` deriva del enum `MessageStatus` de `messaging/providers/types.py`, el valor por defecto es `queued` y los entrantes se guardan siempre como `delivered`, porque por definición llegaron.

En las plantillas de WhatsApp conviven dos campos que parecen uno: `is_active` («Activo») es el interruptor de la cuenta y `status` («Estado») es el veredicto de aprobación de Meta. Están separados porque se mueven de forma independiente: una plantilla aprobada puede apagarse. La pestaña Desactivadas filtra por el interruptor y las demás por el estado. El nombre está restringido por Meta a minúsculas, dígitos y guion bajo mediante `NAME_RE`, y la unicidad se define sobre el par nombre e idioma, porque Meta acota los nombres por idioma.

Nota. Los eventos de calendario se guardan en UTC y se introducen y muestran en `CALENDAR_TZ`. `parse_client_dt` es el único sitio donde se interpreta una cadena de fecha en todo el código del calendario; ningún otro punto puede parsear `datetimes`.

Por último, varios campos son texto plano a la espera de modelos que aún no existen, y el código lo declara en cada sitio: `ClientList.created_by`, `MessageTemplate.team` y `MessageTemplate.created_by` seguirán siendo `CharField` hasta que haya usuarios y equipos reales; `Product.category` y `Product.brand` se volverán claves ajenas cuando las páginas de Categorías y Marcas definan sus modelos; la columna «Sincronizado con» de Productos no tiene campo todavía y la tabla pinta una celda vacía. Tampoco hay modelo de organización, así que los nombres de etiqueta son únicos globalmente y hay un TODO marcado para acotarlos por organización cuando llegue la multitenencia.

14.7 Pruebas que cubren estos términos

Cada término del glosario tiene un módulo de pruebas que fija su significado. Son útiles como documentación ejecutable: cuando la definición de «venta» o de «tema» cambie, estos archivos son los primeros en romperse. Las pruebas de agentes fijan las credenciales con `override_settings` en lugar de leer el `.env local`, algo que funciona precisamente porque `core.agents` relee la configuración en cada llamada.

Módulo	Qué comprueba
<code>core/tests_crm.py</code>	Ayudantes del modelo <code>Client</code> , la navegación secundaria y la tabla de clientes; incluye <code>ClientListTests</code> .
<code>core/tests_clientes.py</code>	Las reglas de <code>core.clientes</code> (normalización de teléfono, país, validación), los endpoints del modal y el buscador de la tabla.

Módulo	Qué comprueba
core/tests_agents.py	La lista de agentes leída del entorno, el fallback al par heredado, las filas User espejo, el login y la asignación por conversación.
messaging/tests.py	Contrato del webhook, la regla de las 24 horas y el cableado del Inbox, con clases dedicadas a etiquetas: TagServiceTests, TagFilterTests, TagAdminUITests, TagPickerTests y BulkTagTests.
core/tests_embudos.py	Las dos columnas de la pantalla Embudos, la tarjeta de estado vacío y el endpoint de panel.
core/tests_automatizaciones.py	La navegación de Automatizaciones y sus paneles, todavía presentes pese a que la sección salió de la barra lateral.
core/tests_comercio.py	La navegación colapsable de Mi comercio, la tabla de Productos con sus pestañas de estado y el cableado de los placeholders.
core/tests_mensajería.py	La tabla de plantillas de WhatsApp con pestañas, el estado vacío en dos mitades, el modal selector y el editor.
core/tests_respuestas_rapidas.py	plantillas.render_body, el endpoint del popover y su enganche con el compositor.
core/tests_calendario.py	El panel de Mi calendario, el feed JSON y su contrato de zona horaria, las mutaciones de eventos y las preferencias de sesión.
core/tests_estadisticas_atribuciones.py	La atribución por canal de conversaciones y ventas, las tarjetas de canal principal y mejor conversión, la ventana temporal y el estado vacío.
core/tests_estadisticas_ventas.py	Qué cuenta como venta, el fechado por tagged_at y las tarjetas del panel.
core/tests_estadisticas_embudos.py	Las cuatro etapas del embudo integrado y sus porcentajes sobre la base del periodo.
core/tests_estadisticas_temas.py	El tokenizador, el plegado de acentos, la ventana del periodo y el renderizado del panel.
core/tests_estadisticas_etiquetas.py	Cada etiqueta con cuántas conversaciones la llevan, las tarjetas resumen y el estado vacío.

Módulos de test y qué término fijan.

Archivos de referencia: core/nav.py, core/models.py, messaging/models.py, messaging/services.py, messaging/providers/types.py, core/crm.py, core/clientes.py, core/embudos.py, core/plantillas.py, core/agents.py, core/estadisticas_atribuciones.py, core/estadisticas_temas.py, core/estadisticas_periodos.py, core/estadisticas_embudos.py, core/estadisticas_ventas.py, core/comercio.py, core/inbox.py, core/calendario.py, core/estadisticas.py, core/mensajería.py, core/automatizaciones.py, core/urls.py, core/views.py, config/settings.py

15. Sistema visual: tokens, convenciones CSS y carga de estilos

MVP-CRM no tiene build de frontend: la capa visual son dieciséis hojas CSS escritas a mano en `static/css/`, de las cuales catorce se enlazan una a una desde `templates/base.html` mediante la etiqueta `{% vstatic %}`; las dos restantes, `static/css/login.css` y `static/css/legal.css`, las cargan sus propias plantillas. Este capítulo describe el sistema que las mantiene coherentes: la paleta única declarada en el `:root` de `static/css/sidebar.css`, el reset compartido (incluida la regla `[hidden] { display: none !important; }`), el sistema de tooltips que hace usable un rail de solo iconos, la convención de nombres tipo BEM con una hoja por pantalla, los bloques reutilizables que viven en la hoja de la pantalla que los introdujo, el cache-busting por hash de contenido y la pantalla de bienvenida como estado en reposo del shell.

15.1 Propósito y alcance

El proyecto no usa Sass, PostCSS ni ningún empaquetador: lo que se escribe en `static/css/` es exactamente lo que llega al navegador. Eso obliga a que la coherencia visual la sostengan convenciones explícitas y no herramientas. Hay tres: una única paleta de variables CSS declarada en `static/css/sidebar.css`, una hoja por pantalla con nombres de clase tipo BEM prefijados por el bloque de esa pantalla, y un reset compartido que también vive en `sidebar.css` porque es la primera hoja que carga cualquier página del shell.

El directorio `static/css/` contiene dieciséis hojas. Catorce se enlazan desde `templates/base.html` y por tanto están presentes en todas las pantallas del shell autenticado. Las otras dos son autónomas: `static/css/login.css`, que carga `templates/login.html`, y `static/css/legal.css`, que cargan `templates/legal/privacidad.html` y `templates/legal/eliminacion-de-datos.html`. La cabecera de `legal.css` explica que esas páginas renderizan fuera del shell, sin sidebar, porque los revisores de Meta y cualquier lector de una política deben alcanzarlas sin sesión, y remite a `core.middleware.EXEMPT_PATHS`. `templates/login.html` sí carga `static/css/sidebar.css` antes de `static/css/login.css`, precisamente para heredar colores, tipografía y el reset de `[hidden]`.

15.2 Cómo se cargan los estilos

`templates/base.html` es un documento corto: cabecera con catorce elementos `<link rel="stylesheet">`, el script de htmx vendorizado con el comentario de que está así para que la aplicación no tenga dependencia de CDN en tiempo de ejecución, y un cuerpo con `.app`, el sidebar incluido desde `partials/sidebar.html` y un `<main class="content" id="content" hx-history-elt>` que incluye la plantilla de sección. Todas esas hojas se cargan en todas las páginas del shell, independientemente de la sección que se esté viendo. No hay carga condicional ni bloque `{% block extra_css %}`: añadir una pantalla nueva significa crear su hoja y añadir su `<link>` a mano al final de la lista.

El orden de esa lista no es arbitrario. `static/css/sidebar.css` va primero porque aporta los tokens y el reset del que dependen todas las demás; a partir de ahí las hojas están más o menos en el orden en que se fueron construyendo las pantallas. Como las propiedades personalizadas de CSS se resuelven en el momento de usarse, la posición relativa de los bloques `:root` es indiferente:

`static/css/estadisticas.css` puede usar `--page-gray` aunque lo declare `static/css/embudos.css`. Lo que sí depende del orden son las reglas de igual especificidad, y ahí hay una trampa real documentada más abajo.

Cada hoja abre con un comentario de cabecera que declara de qué otras hojas depende.

`static/css/welcome.css` dice que construye sobre los tokens de `sidebar.css` y reutiliza el footer y el help dock de `inbox.css`; `static/css/modal.css` dice que construye sobre los tokens de `sidebar.css` y `inbox.css` y sobre los botones compartidos `.btn-primary` y `.btn-outline`; `static/css/crm.css` avisa de que su columna 2 es el `.side-nav` compartido, montado por CRM y Campañas desde `partials/account_nav_panel.html`, y de que lo único nuevo que aporta son las superficies de tabla. Ese encabezado es el mapa de dependencias del sistema y conviene leerlo antes de tocar nada.

La cabecera más rica es la de `static/css/calendario.css`, y sirve de ejemplo de hasta dónde llega ese mapa: declara que construye sobre los tokens de `sidebar.css` y `inbox.css`, sobre los botones compartidos, sobre el shell `.modal` de `modal.css` y sobre el sistema de formularios `.ffield` junto con los tokens seguros para texto `--error-text` y `--accent-text` de `plantillas.css`, y añade que los colores de los eventos reutilizan los pares de la paleta de etiquetas de `tags.css`. Cierra explicando que FullCalendar v6 inyecta su propio CSS base y que las sobrescrituras están confinadas bajo `.calendario` y se apoyan en sus variables `--fc-*`.

Orden	Hoja	Qué aporta al sistema
1	<code>css/sidebar.css</code>	Tokens <code>:root</code> , reset compartido, rail, tooltips, <code>.content</code> , <code>.page</code> , <code>.panel</code>
2	<code>css/inbox.css</code>	Anchos de columna, <code>--accent-soft</code> , <code>--footer-h</code> , <code>.btn-primary</code> , <code>.inbox-footer</code> , <code>.help-dock</code> , <code>.icon</code>
3	<code>css/tags.css</code>	Paleta de etiquetas <code>--tag-*-bg</code> / <code>--tag-*-fg</code> , <code>.tag-pill</code> , <code>.tag-dialog</code>
4	<code>css/crm.css</code>	<code>--table-head-bg</code> , <code>--wa-green</code> , <code>.table</code> , <code>.field</code> , <code>.info-dot</code>
5	<code>css/embudos.css</code>	<code>--side-nav-w</code> , <code>--page-gray</code> , el bloque compartido <code>.side-nav</code> , <code>.btn-outline</code>
6	<code>css/automatizaciones.css</code>	Extensión colapsable de <code>.side-nav</code> , <code>--progress-track</code> , banner Academy
7	<code>css/comercio.css</code>	Bloque compartido <code>.product-card</code> / <code>.product-tabs</code> , fila de nav autónoma
8	<code>css/estadisticas.css</code>	<code>.nav-badge</code> , <code>.stat-cards</code> , paneles de ranking, embudos de estadísticas
9	<code>css/mensajería.css</code>	Nav sin cabecera y estado vacío de dos mitades
10	<code>css/welcome.css</code>	Pantalla de bienvenida: <code>.welcome</code> , <code>.welcome__body</code> , <code>.welcome-card</code>
11	<code>css/modal.css</code>	Shell compartido de <code><dialog></code> : <code>.modal</code> , <code>.modal__body</code> , <code>.modal-choice</code>
12	<code>css/plantillas.css</code>	<code>--accent-text</code> , <code>--error-text</code> , sistema de formularios <code>.ffield</code> , <code>.wa-preview</code>
13	<code>css/calendario.css</code>	Sobrescrituras de FullCalendar v6 dentro de <code>.calendario</code>
14	<code>css/stats-detail.css</code>	<code>.kpi</code> , <code>.kpi__value</code> , <code>.data-table</code> , cabecera y filtros de detalle

Hojas enlazadas desde `templates/base.html`, en orden

Nota. Las dos hojas que no aparecen en esa lista son `static/css/login.css` y `static/css/legal.css`, cargadas por `templates/login.html` y por las plantillas de `templates/legal/` respectivamente. Añadir un `<link>` a `base.html` es lo que hace visible una hoja en el shell; no hay descubrimiento automático.

15.3 La paleta única: el `:root` de `sidebar.css`

El bloque `:root` de `static/css/sidebar.css` es la paleta del producto. Su comentario de cabecera dice que los colores están muestreados de las capturas de referencia de Mercatelly, y las variables están agrupadas por rol: layout, superficies, tinta y acento. Ninguna hoja de pantalla debería introducir un hex nuevo para un color que ya exista aquí; cuando una pantalla necesita un valor propio, lo declara en su propio `:root` y lo documenta, como hace `static/css/crm.css` con `--table-head-bg` y `--wa-green`.

Dos tokens llevan comentarios que explican decisiones de accesibilidad. `--muted: #65768d` va anotado como 4,6:1 sobre blanco, es decir AA incluso a tamaños pequeños. Y `static/css/plantillas.css` declara `--accent-text: #1a66c9` y `--error-text: #c73e3e` precisamente porque los tokens base pasan 3:1 para bordes e iconos pero no 4,5:1 para el texto pequeño que fija esa pantalla: son las variantes seguras para texto, no reemplazos del acento.

Token	Valor	Rol
<code>--sidebar-w</code>	64px	Ancho del rail de solo iconos; <code>.content</code> se desliza con este valor
<code>--nav-hit</code>	40px	Área de pulsación mínima, con el glifo de 21px dentro
<code>--sidebar-bg</code>	<code>#ffffff</code>	Fondo del rail
<code>--sidebar-border</code>	<code>#e9edf2</code>	Hairline de bordes y divisores en todo el producto
<code>--content-bg</code>	<code>#f7f9fc</code>	Fondo del área de contenido, aplicado en <code>body</code>
<code>--panel-bg</code>	<code>#ffffff</code>	Superficie de panel
<code>--icon-idle</code>	<code>#1b2a41</code>	Azul marino de los iconos no seleccionados
<code>--text</code>	<code>#12263f</code>	Tinta principal
<code>--muted</code>	<code>#65768d</code>	Tinta secundaria; 4,6:1 sobre blanco
<code>--accent</code>	<code>#2f80ed</code>	Azul de acción; píldora activa, botones, focos
<code>--accent-ink</code>	<code>#ffffff</code>	Tinta sobre el acento
<code>--hover-bg</code>	<code>#f1f4f8</code>	Fondo de hover neutro
<code>--badge</code>	<code>#f5333f</code>	Punto de notificación de Academy
<code>--tip-bg</code>	<code>#12263f</code>	Fondo del tooltip del rail
<code>--radius-pill</code>	10px	Radio de la píldora de nav
<code>--speed</code>	140ms	Duración única de todas las transiciones

Tokens declarados en el `:root` de `static/css/sidebar.css`

La segunda familia de tokens de layout la aporta `static/css/inbox.css`, que además de `--accent-soft` y `--footer-h` declara los anchos fijos de tres de las cuatro columnas del Inbox. Están ahí, y no en `sidebar.css`, porque describen esa pantalla y no el shell; pero como `inbox.css` se carga en todas las páginas, cualquier hoja posterior puede referirse a ellos.

Token	Valor	Rol
<code>--nav-panel-w</code>	196px	Ancho de la columna de navegación del Inbox
<code>--list-panel-w</code>	300px	Ancho de la columna de lista de conversaciones
<code>--details-panel-w</code>	280px	Ancho de la columna de detalles
<code>--panel-alt-bg</code>	<code>#f2f5f8</code>	Fondo de la columna de lista de conversaciones
<code>--accent-soft</code>	<code>#e8f1fe</code>	Píldora azul clara detrás de una fila activa
<code>--footer-h</code>	42px	Altura de la barra <code>.inbox-footer</code>

Tokens declarados en el `:root` de `static/css/inbox.css`

15.4 El rail de solo iconos y sus tooltips

`.sidebar` es un elemento `position: fixed` anclado al viewport, de 64px de ancho, para que no se desplace con el área de contenido. Al ser un rail de solo iconos no hay sitio para etiquetas, así que la etiqueta en español de cada sección se revela al pasar el ratón o al recibir foco de teclado. El bloque de tooltips de `static/css/sidebar.css` lo dice de forma explícita en su comentario: el rail es solo de iconos, y por eso hover y focus revelan la etiqueta.

El mecanismo es `.nav-item__tip`, un elemento absolutamente posicionado a la derecha del icono, con fondo `--tip-bg`, `z-index: 200` y una flecha dibujada con `::before` mediante un borde transparente cuyo lado interior toma el color del tooltip. En reposo combina `opacity: 0` con `visibility: hidden` y `pointer-events: none`, y las reglas `.nav-item:hover .nav-item__tip` y `.nav-item:focus-visible .nav-item__tip` lo hacen visible desplazándolo unos píxeles. El contexto de posicionamiento lo aporta `.nav-list__row { position: relative; }`, una única regla de una línea de la que depende todo el bloque.

15.5 Convención de nombres y bloques compartidos

Los nombres de clase siguen una variante de BEM: bloque, doble guion bajo para el elemento y doble guion para el modificador. El bloque suele ser la pantalla o el componente, de modo que `.welcome__body` y `.welcome__cards` pertenecen inequívocamente a la pantalla de bienvenida, `.conv-item__check` a una fila de conversación del Inbox, `.stats-page__kpis` a un panel de Estadísticas y `.modal-choice__action` a las tarjetas del diálogo compartido. Los estados no son elementos: se expresan con clases independientes `.is-active` y `.has-selection`, que en algunos casos se aplican a un ancestro para que descendientes enteros reaccionen a la vez, como hace `.has-selection .conv-item__check { opacity: 1; }`.

Un bloque reutilizable vive en la hoja de la pantalla que lo introdujo, no en una hoja común, y su comentario avisa de que es compartido. El caso más claro lo declara `static/css/embudos.css`: una nota dice que el bloque `.side-nav` es el lenguaje de navegación secundaria de toda la app, que Embudos, Automatizaciones, Mi comercio y la nav de cuenta de CRM y Campañas renderizan sobre esas clases, y que la extensión de secciones colapsables, con `.side-nav__row--summary` y `.side-nav__children`, vive en `static/css/automatizaciones.css` junto a la pantalla que lo introdujo.

El segundo bloque compartido con nota propia está en `static/css/comercio.css`: `.product-card` y `.product-tabs` son el lenguaje de tarjeta con pestañas de la aplicación, y el comentario avisa de que la pantalla de Plantillas de WhatsApp renderiza también sobre esas clases; la cabecera de `static/css/mensajeria.css` lo confirma desde el otro lado, declarando que reutiliza las clases de pestañas y tarjeta de `comercio.css`. La misma lógica de vivir donde nacieron coloca `.btn-primary` en `static/css/inbox.css`, `.btn-outline` en `static/css/embudos.css`, `.table` y `.field` en `static/css/crm.css`, y `.kpi` con `.data-table` en `static/css/stats-detail.css`.

Nota. `.tag-dialog`, en `static/css/tags.css`, es deliberadamente un skin de diálogo compacto propio y no se monta sobre el shell `.modal`: la cabecera de `static/css/modal.css` deja constancia de que es anterior a ese shell y de que lo único compartido entre ambos es el comportamiento de `static/js/shell.js`. Los diálogos nuevos deben empezar por `.modal`.

15.6 La pantalla de bienvenida como estado en reposo

`templates/sections/welcome.html` es la pantalla servida en la raíz, antes de que se elija ninguna sección. Su comentario la describe como un fragmento igual que cualquier otra sección, sin `<html>` ni sidebar, para que `htmx` pueda insertarla directamente en `#content`, pero con una diferencia: ocupa toda el área de contenido, sin panel de navegación secundaria, con un único bloque centrado. La vista que la sirve es `core.views.welcome`, que fija `active_key=None` y `page_title` a Bienvenido, resuelve shortcuts desde `core.nav.WELCOME_SHORTCUTS` y usa `section_template` igual a `sections/welcome.html`.

El docstring de `core.views.welcome` explica el porqué de esos valores. `active_key=None` funciona porque `partials/nav_item.html` compara cada `item.key` contra él, así que un valor que ninguna clave puede igualar deja el rail entero sin seleccionar. Y añade que la vista deliberadamente no pasa por `core.views.section`: la raíz no es un destino de navegación, y mantenerla separada es lo que impide

que la barra lateral se encienda en la pantalla de bienvenida.

La contraparte de estilo es `static/css/welcome.css`, ciento diez líneas, la hoja más pequeña de las catorce que enlaza `base.html`; entre todas las escritas a mano solo `static/css/login.css` es menor, con ciento cuatro. Su cabecera la define como el estado de aterrizaje sereno de la raíz y explica que `.welcome` imita a `.inbox`: una columna a altura completa con el footer anclado abajo y el contexto de posicionamiento al que se ancla el `.help-dock` flotante. `.welcome__body` toma el espacio sobre el footer y centra en ambos ejes; `.welcome__logo` es una baldosa de 72px con el mismo glifo que la marca del sidebar, y `.welcome__logo .brand__icon` sobrescribe a 36px los atributos `width` y `height` que trae el SVG inlineado desde `templates/icons/logo.svg`.

Los atajos son `.welcome-card`, y el comentario de la plantilla explica por qué existe el atributo `data-nav-for`: las tarjetas intercambian `#content` exactamente igual que un icono del rail, pero viven fuera de `[data-nav-group]`, así que `data-nav-for` le nombra a `shell.js` el icono que debe encender al salir, y mantener el `href` idéntico al del rail es lo que hace que esa búsqueda tenga éxito. El bloque `.welcome__text` limita la medida a 46ch con el comentario de que así la longitud de línea sigue siendo legible en pantallas anchas, y `.welcome__cards` lleva la nota de que la fila es deliberadamente ligera: una fila de atajos, no un dashboard.

`templates/partials/footer.html` se incluye al final de `.welcome` y aporta la barra `.inbox-footer` con el aviso "Estamos aquí para ayudarte." y el `.help-dock` con la píldora "¿Necesitas ayuda? Haz clic aquí" y el botón de avatar. Su comentario advierte de dos cosas: que el elemento anfitrión debe ser `position: relative`, porque `.help-dock` se posiciona de forma absoluta contra él, y que sus estilos viven en `static/css/inbox.css`, que se carga en todas las páginas desde `base.html`. Los botones del dock no tienen comportamiento asociado: son controles sin acción todavía.

15.7 Cache-busting: escribir CSS con `{% vstatic %}`

El docstring de `core/templatetags/assets.py` explica el problema de raíz. La etiqueta `{% static %}` emite una ruta estable como `/static/css/plantillas.css`, de modo que un navegador, y la CDN de Vercel, siguen sirviendo la copia que cachearon antes de que se desplegara un arreglo: un cambio de CSS puede aterrizar, desplegarse y aun así no llegar a nadie que ya hubiese cargado la página. El build de Django en Vercel ejecuta `collectstatic` sin nombres de archivo hashados, así que nada aguas arriba resuelve esto por el proyecto.

`{% vstatic %}` añade un hash corto del contenido como cadena de consulta, de modo que la URL cambia exactamente cuando cambian los bytes del archivo y la entrada de caché antigua deja de usarse. El hash es un SHA-256 truncado a diez caracteres. Para quien edita CSS esto significa que no hay ningún paso manual: guardar el archivo cambia el hash y con él la URL. En desarrollo, con `DEBUG` activo, el hash se recalcula en cada render, así que una edición aparece en la siguiente recarga; fuera de `DEBUG` se calcula una sola vez por proceso y se memoiza en el diccionario `_hashes`.

El archivo se localiza con `finders.find(path)` de `django.contrib.staticfiles`, no leyendo directamente de `STATIC_ROOT`. Ese detalle es el que hace que la etiqueta funcione igual en desarrollo, donde los archivos están en los directorios de origen de cada aplicación, y en producción una vez ejecutado `collectstatic`. También es el que permite que una ruta inexistente devuelva la cadena vacía en lugar de lanzar una excepción: `vstatic` comprueba el hash y, si está vacío, emite la URL de `{% static %}` sin sufijo.

```
@register.simple_tag
def vstatic(path: str) -> str:
    """The `{% static %}` URL for ``path`` plus a ``?v=<hash>`` stamp."""
    url = static(path)
    digest = _content_hash(path)
    return f"{url}?v={digest}" if digest else url

# En la plantilla:
# <link rel="stylesheet" href="{% vstatic 'css/sidebar.css' %}">
# Emite:
# /static/css/sidebar.css?v=1a2b3c4d5e
```

core/templatetags/assets.py: la etiqueta y su resultado

Nota. El comentario en `_content_hash` justifica que también se memoicen los fallos, es decir el hash vacío de una ruta sin archivo detrás: la respuesta no puede cambiar sin un redespigüe, y reintentar la búsqueda en cada render no haría aparecer el archivo. La consecuencia práctica es que un `<link>` con la ruta mal escrita seguirá emitiendo una URL sin `?v=` durante toda la vida del proceso.

15.8 Decisiones de diseño y trampas conocidas

El reset compartido de `static/css/sidebar.css` son cuatro reglas: `box-sizing: border-box` universal incluidos pseudo-elementos, `height: 100%` en `html` y `body`, el `body` con margen cero, `--content-bg` de fondo, `--text` de color, la pila tipográfica que empieza por `Inter` y `-webkit-font-smoothing: antialiased`, y la regla de `[hidden]`. Esta última lleva el comentario más importante del archivo: el agente de usuario oculta `[hidden]` con `display: none`, que cualquier declaración de `display` de autor supera después, de modo que un bloque con `display: flex` deja silenciosamente de honrar el atributo y todos los `el.hidden = true` de la aplicación dejan de funcionar sobre él. La regla lo reafirma como ganador una sola vez, aquí. `static/js/shell.js` asigna `el.hidden` en quince sitios, así que esa línea sostiene buena parte del comportamiento del shell.

Nota. Como `static/css/legal.css` es autónoma y las páginas legales no cargan `sidebar.css`, esas páginas no tienen ni los tokens ni el reset de `[hidden]`. Cualquier estilo nuevo para ellas debe declararse dentro de esa hoja.

Otras dos decisiones del shell viven también en `sidebar.css`. La primera es que las secciones que quieren padding de página normal se apuntan con `.page`, mientras que las pantallas a sangre completa como el `Inbox` simplemente no lo hacen y reciben el área entera; el comentario junto a la regla lo formula así, como una adhesión voluntaria. La segunda es la retroalimentación de `htmx: #content.htmx-request` baja la opacidad a 0,55 mientras la petición está en vuelo, con una transición de `--speed`. Cierra el archivo un bloque `@media (prefers-reduced-motion: reduce)` que reduce todas las duraciones de transición y animación a 0,01ms de forma universal, así que ninguna hoja de pantalla necesita repetir esa consideración.

Hay una trampa activa causada por el orden de los `<link>`. `static/css/estadisticas.css` define el modificador `.kpi__value--text` con `font-size: 18px`, con el comentario de que es una baldosa de KPI cuyo valor es un nombre y no un número, el Canal principal de Atribuciones, más pequeño para que las etiquetas de longitud de WhatsApp quepan en una línea. Pero el bloque base `.kpi__value` con `font-size: 26px` vive en `static/css/stats-detail.css`, que `base.html` enlaza en la posición 14, después de `estadisticas.css` en la posición 8. Ambos selectores tienen la misma especificidad de una clase, así que sobre los elementos de `templates/partials/estadisticas/panels/atribuciones.html` que llevan `class="kpi__value kpi__value--text"` gana la regla posterior y el modificador no llega a aplicarse.

Conviene tener presente la regla general que ese caso ilustra: la extensión de un bloque compartido debe estar en una hoja que se enlace después de la que define el bloque, o bien elevar su especificidad. Otras hojas ya siguen la primera opción por construcción; `static/css/automatizaciones.css`, que extiende `.side-nav` de `embudos.css`, se enlaza justo detrás. Y `static/css/comercio.css` toma la segunda: sus reglas `.product-card .table thead th:first-child` y `.product-card .table thead th:last-child` anulan los radios de esquina que `crm.css` da a la cabecera de `.table`, con un comentario que explica el porqué, que dentro de la tarjeta las pestañas quedan encima y una banda redondeada a media altura se ve rota, y que remite explícitamente a `crm.css`.

Nota. `static/css/stats-detail.css` incluye un bloque oscuro deliberadamente limitado a esa pantalla. Su cabecera lo dice sin rodeos: la aplicación todavía no tiene tema oscuro global, aunque la paleta de gráficos sí lo contemple mediante el paso oscuro de `core.estadisticas_volumen.Channel`, y un gráfico oscuro dentro de una tarjeta blanca sería peor que cualquiera de las dos opciones. La cabecera de `static/css/tags.css` apunta en la misma dirección: un futuro tema oscuro repintaría todas las píldoras redefiniendo solo las variables `--tag-*`, y los colores de etiqueta se guardan como tokens de `Tag.COLOR_CHOICES` que nombran esos pares, nunca como hex crudo.

15.9 Pruebas que lo cubren

No existen pruebas de CSS: nada renderiza las hojas ni comprueba valores calculados. Lo que sí está cubierto es el marcado que esas hojas estilizan, principalmente en `core/tests.py`, cuyo docstring de módulo lo describe como las pruebas del shell del sidebar, es decir enrutado, estado activo y fragmentos de `htmx`. Tampoco hay ningún módulo de pruebas que ejecute `vstatic` ni el hash de contenido de `core/templatetags/assets.py`; esa etiqueta hoy no tiene cobertura propia.

La clase `WelcomeScreenTests` es la que cubre el material de este capítulo, con ocho pruebas. Su docstring la resume como el estado de aterrizaje de la raíz: shell completo, texto centrado y rail en reposo. Dentro, `test_root_matches_no_sidebar_href` lleva su propio docstring explicando que guarda la regresión que la pantalla de bienvenida existe para evitar: `shell.js` enciende `.nav-item[href=<pathname>]`, así que si algún icono apuntase alguna vez a la raíz el rail se seleccionaría a sí mismo en la pantalla de bienvenida. Su contraparte es `test_leaving_the_welcome_screen_activates_that_icon`, que comprueba que al navegar a una sección vuelve el estado activo normal.

Clase	Prueba	Qué fija
<code>WelcomeScreenTests</code>	<code>test_no_icon_is_marked_active</code>	El rail se renderiza pero no aparecen <code>is-active</code> ni <code>aria-current</code>
<code>WelcomeScreenTests</code>	<code>test_root_matches_no_sidebar_href</code>	Ninguna URL de sección resuelve a la raíz, para que <code>shell.js</code> no encienda nada
<code>WelcomeScreenTests</code>	<code>test_hero_copy_renders</code>	El título, el texto y la clase <code>welcome__logo</code> llegan al HTML
<code>WelcomeScreenTests</code>	<code>test_title_names_the_screen</code>	El <code><title></code> es Bienvenido · MVP CRM
<code>WelcomeScreenTests</code>	<code>test_shortcuts_link_to_their_sections</code>	Cada atajo lleva <code>href</code> y <code>data-nav-for</code> con la URL de su sección y la etiqueta del rail
<code>WelcomeScreenTests</code>	<code>test_footer_and_help_dock_render</code>	El aviso del footer y la píldora de ayuda aparecen en la bienvenida

Clase	Prueba	Qué fija
WelcomeScreenTests	test_htmx_request_returns_a_bare_fragment	Bajo HX-Request no hay <html> ni sidebar, pero sí welcome__title
WelcomeScreenTests	test_leaving_the_welcome_screen_activates_that_icon	En la sección crm, active_key es crm y hay exactamente un nav-item is-active
TemplateCommentLeakTests	test_welcome_screen_does_not_leak_either	Ni {# ni #} aparecen en la página renderizada
SidebarRenderTests	test_every_icon_renders_with_a_tooltip	Cada icono del rail llega con su nav-item__tip
SidebarRenderTests	test_active_item_gets_pill_and_aria_current	Exactamente un nav-item is-active y un aria-current="page", en el enlace correcto
SidebarRenderTests	test_badge_renders_once	nav-item__badge aparece una sola vez

Pruebas relevantes en core/tests.py

Fuera de core/tests.py hay una prueba que documenta explícitamente el reparto entre servidor y CSS: test_standalone_row_truncates_via_css_not_hardcoding, en core/tests_comercio.py. Comprueba que la etiqueta completa Configuración del comercio aparece dos veces en el marcado, como atributo title y como texto visible, y que la cadena truncada nunca aparece del lado del servidor, porque el recorte es solo de CSS. TemplateCommentLeakTests merece mención aparte: su docstring explica que los comentarios {# de Django son de una sola línea, que uno multilínea no es un comentario en absoluto y se filtra a la página como texto visible, y que esto ya se ha desplegado dos veces.

Archivos de referencia: static/css/sidebar.css, static/css/welcome.css, static/css/modal.css, static/css/inbox.css, static/css/crm.css, static/css/estadisticas.css, static/css/embudos.css, static/css/tags.css, static/css/plantillas.css, static/css/legal.css, static/css/login.css, static/css/stats-detail.css, static/css/comercio.css, static/css/automatizaciones.css, static/css/calendario.css, static/css/mensajería.css, static/js/shell.js, templates/base.html, templates/login.html, templates/legal/privacidad.html, templates/legal/eliminacion-de-datos.html, templates/sections/welcome.html, templates/partials/footer.html, templates/icons/logo.svg, templates/partials/estadisticas/panels/atribuciones.html, core/templatetags/assets.py, core/views.py, core/nav.py, core/tests.py, core/tests_comercio.py

16. Patrones htmx: fragmentos, swaps e historial

*Catálogo transversal de los mecanismos htmx que se repiten en toda la aplicación: la respuesta dual de las vistas según la cabecera HX-Request, el reparto de hx-target/hx-swap/hx-push-url sobre contenedores canónicos y destinos relativos, los swaps fuera de banda y los parciales *_swap.html, el sondeo con hx-trigger every en el Inbox junto con las guardas data-hold-poll, y los ganchos de shell.js sobre htmx:beforeSwap, htmx:afterSwap, htmx:afterRequest y htmx:historyRestore.*

16.1 Propósito y alcance

MVP-CRM es una única shell de aplicación sin build de frontend: htmx se sirve vendorizado desde `static/js/vendor/htmx.min.js` y se carga con `defer` en `templates/base.html`, con el comentario que explica el porqué («Vendored so the app has no CDN dependency at runtime»). No hay framework de cliente ni empaquetador; toda la navegación interna consiste en pedir un fragmento HTML al servidor e intercambiarlo dentro de un contenedor conocido. Este capítulo recoge los mecanismos que se repiten en todas las secciones y que, hasta ahora, sólo se veían a trozos dentro de cada capítulo funcional.

El ámbito es transversal: la respuesta dual de las vistas, los contenedores destino, los intercambios fuera de banda, el sondeo periódico y los ganchos de JavaScript que rehidratan estado tras cada intercambio. Quedan fuera los dos consumidores de JSON que no usan htmx: el calendario (`calendar_events` y compañía, consumido por `FullCalendar`) y los informes de Estadísticas (`estadisticas_volumen_data` y `estadisticas_tiempos_data`, consumidos por `ECharts`), que devuelven `JsonResponse`. `messaging/urls.py` tampoco entra: sólo expone el webhook de proveedores.

Nota. Todo lo descrito aquí es mejora progresiva. Cada disparador htmx se monta sobre un `<a href>` o un `<form>` real, de modo que sin JavaScript la aplicación sigue navegando con recargas completas.

16.2 Respuesta dual: página completa o fragmento

La pieza central es `_is_htmx` en `core/views.py`, que devuelve `True` cuando `request.headers.get("HX-Request") == "true"`. Las vistas que sirven una pantalla completa consultan ese predicado y responden de dos formas con el mismo contexto: a una petición normal del navegador le devuelven `base.html` (documento, barra lateral y sección); a una petición htmx le devuelven sólo la plantilla de la sección. El docstring del módulo lo enuncia como la razón de ser de la shell: es lo que permite navegar sin recargas manteniendo cada URL directamente enlazable, marcable y compatible con el botón atrás.

`section` aplica ese patrón a los destinos de la barra lateral, resolviendo la plantilla con `_section_template`, que devuelve `sections/<key>.html` si existe y `PLACEHOLDER_TEMPLATE` (`sections/_placeholder.html`) si no. `welcome` hace lo mismo para la raíz, pero deliberadamente no pasa por `section`: su `active_key` es `None` para que ningún icono del raíl quede seleccionado. En Configuración de mensajería hay una tercera variante, `_mensajeria_page`, que reconstruye la página completa con el panel sustituido; su docstring explica el caso que cubre: abrir en pestaña nueva una tarjeta del selector o un POST de formulario sin JavaScript, para que esas URLs rindan dentro de la shell en lugar de como fragmento suelto. `plantilla_gallery` y `plantilla_editor` son las dos vistas que la usan.

El resto de endpoints son fragmentarios por naturaleza y no comprueban `HX-Request`: `inbox_list`, `inbox_chat`, `inbox_thread`, `crm_panel`, `clientes_table`, `productos_table`, `plantillas_table` o `estadisticas_card` devuelven siempre su trozo. Eso es intencionado y los tests lo aprovechan: el

fragmento que emite el endpoint debe aparecer literalmente dentro de la página completa.

16.3 Contenedores canónicos, hx-target y hx-push-url

El destino de nivel superior es `<main class="content" id="content" hx-history-elt>` en `templates/base.html`. El atributo `hx-history-elt` marca ese elemento como el que `htmx` guarda y restaura en su caché de historial, coherentemente con que sea el único trozo que la navegación de sección reemplaza. Los iconos del raíl (`templates/partials/nav_item.html`) y las tarjetas de acceso rápido de la pantalla de bienvenida (`templates/sections/welcome.html`) son los dos únicos sitios que apuntan a `#content`, ambos con `hx-swap="innerHTML"` y `hx-push-url="true"`.

Un nivel más abajo, la navegación secundaria de cada sección intercambia sólo su columna 3, pero el marcado no está unificado. `templates/partials/side_nav_rows.html` es la plantilla compartida por CRM, Campañas y Mi comercio: recibe `panel_target` por contexto y genera `hx-target="#{{ panel_target }}"`. Sólo llegan a ella tres montajes, a través de `templates/partials/account_nav_panel.html` y `templates/partials/side_nav_section.html`, que son quienes pasan `panel_target` (`crm-panel`, `campanas-panel` y `comercio-panel`). Embudos, Automatizaciones, Estadísticas y Configuración de mensajería tienen cada uno su propio `nav_panel.html` con el `hx-target` escrito a mano sobre `#embudos-panel`, `#autom-panel`, `#estadisticas-panel` y `#mensajeria-panel`.

Lo que sí es constante en todas esas filas es el reparto de atributos: `hx-get` va al endpoint parcial y `hx-push-url` va a la URL de sección recargable con `?view=`, nunca al endpoint del fragmento. Los constructores de contexto cierran el círculo leyendo esos parámetros al recargar (`?view=` en `_crm_context`, `?filter=`, `?chat=` y `?tags=` en `_inbox_context`), y ambos hacen lo mismo con un valor desconocido: caer al valor por defecto en lugar de devolver 404, para que un marcador antiguo siga abriendo.

Prácticamente todos los intercambios son `hx-swap="innerHTML"`; la excepción es el desplegable de asignación (`templates/partials/inbox/assign_control.html`), que usa `hx-target="this"` con `hx-swap="outerHTML"` porque la vista responde con exactamente el mismo marcado del formulario. Junto a los destinos por `id` conviven varios destinos relativos, que se usan allí donde el marcado se monta en más de un sitio o se repite por fila. El botón «+ Crear lista» de `templates/partials/crm/panels/lista-clientes.html` usa `hx-target="closest main"` porque la pantalla se alcanza desde CRM y desde Campañas; el test correspondiente anota que así se resuelve bajo cualquiera de los dos montajes, `#crm-panel` o `#campanas-panel`.

Destino	Declarado en	Para qué
<code>hx-target="find [data-quickreplies-pop]"</code>	<code>partials/inbox/chat_thread.html</code>	carga diferida del popover de respuestas rápidas
<code>hx-target="this"</code>	<code>partials/inbox/assign_control.html</code>	el formulario de asignación se sustituye a sí mismo
<code>hx-target="this"</code>	<code>partials/tags/picker.html</code>	el panel del picker se rellena a sí mismo al abrir
<code>hx-target="closest main"</code>	<code>partials/crm/panels/lista-clientes.html</code>	sirve bajo <code>#crm-panel</code> y bajo <code>#campanas-panel</code>
<code>hx-target="next .tag-picker__panel"</code>	<code>partials/tags/removable_pills.html</code>	cada aspa de píldora apunta al panel contiguo
<code>hx-target="closest .tag-picker__panel"</code>	<code>partials/tags/picker_panel.html</code>	buscador y filas del picker se re-renderizan en su panel

Destinos relativos en uso

Contenedor	Definido en	Quién lo rellena
<code>#content</code>	<code>templates/base.html</code> (con <code>hx-history-elt</code>)	<code>section</code> , <code>welcome</code>

Contenedor	Definido en	Quién lo rellena
#crm-panel / #campanas-panel	templates/sections/crm.html, campanas.html	crm_panel
#embudos-panel	templates/sections/embudos.html	embudos_panel
#autom-panel	templates/sections/automatizaciones.html	automatizaciones_panel
#comercio-panel	templates/sections/mi-comercio.html	comercio_panel
#estadisticas-panel	templates/sections/estadisticas.html	estadisticas_panel, estadisticas_card
#mensajería-panel	templates/sections/mensajería.html	mensajería_panel, plantilla_gallery, plantilla_editor
#conv-list	templates/partials/inbox/list_panel.html	inbox_list, inbox_tags_bulk
#chat-panel	templates/partials/inbox/chat_panel.html	inbox_chat
#chat-messages	templates/partials/inbox/chat_thread.html	inbox_thread, inbox_send
#client-table	templates/partials/crm/panels/clientes.html	clientes_table y las mutaciones de cliente
#client-modal-body	templates/partials/crm/panels/clientes.html	cliente_form, cliente_detail, cliente_delete
#tag-table	templates/partials/crm/panels/etiquetas.html	tag_create, tag_update, tag_archive
#product-table	templates/partials/comercio/panels/productos.html	productos_table (contenido en comercio/product_table.html)
#template-table	templates/partials/mensajería/panels/plantillas-whatsapp.html	plantillas_table (contenido en mensajería/template_table.html)

Contenedores destino canónicos

Nota. Los contenedores #product-table y #template-table se declaran en los paneles, no en los partials del mismo nombre: product_table.html y template_table.html son sólo el contenido que se intercambia dentro de ellos. En Plantillas el comentario lo dice explícitamente, porque el diálogo selector vive fuera de #template-table para sobrevivir a los cambios de pestaña.

16.4 Swaps fuera de banda y los partials *_swap.html

Cuando una acción tiene que actualizar más de una zona, la respuesta lleva bloques marcados con hx-swap-oob: htmx los extrae del cuerpo y los aplica por id, además del intercambio dirigido a hx-target. La convención de nombre para esas respuestas compuestas es el sufijo _swap.html.

templates/partials/inbox/chat_swap.html es el caso canónico: inbox_chat responde el hilo de chat (que va a #chat-panel, el destino de la petición) y, fuera de banda, el contenido de #details-panel; un clic actualiza dos columnas con una sola respuesta. templates/partials/inbox/assign_swap.html hace lo propio con el control de asignación más la línea «Asignada a» del panel de detalles.

Fuera del Inbox el mismo patrón resuelve el ciclo de los modales.

templates/partials/crm/client_saved.html es lo que devuelve cualquier alta, edición o borrado de cliente con éxito, y hace tres cosas desde el cuerpo del modal: un [data-dialog-dismiss] que ordena a shell.js cerrar el diálogo, un swap fuera de banda de #client-table que refresca las filas debajo, y otro de #client-notice que anuncia el resultado. templates/partials/tags/picker_panel.html añade una variante condicionada: sólo en las respuestas a POST (oob_pills) emite las tres superficies donde se ven las etiquetas de una conversación, #conv-tags-<pk>, #chat-tags-<pk> y #details-tags-<pk>.

Plantilla	Vista	Destino principal	Bloques OOB
partials/inbox/chat_swap.html	inbox_chat	#chat-panel	#details-panel (innerHTML)

Plantilla	Vista	Destino principal	Bloques OOB
partials/inbox/assign_swap.html	inbox_assign	el propio formulario (outerHTML)	#details-assigned-<pk> (outerHTML)
partials/crm/client_saved.html	cliente_form, cliente_delete	#client-modal-body	#client-table, #client-notice (innerHTML)
partials/tags/picker_panel.html	conversation_tags (POST)	el panel del picker	#conv-tags-<pk>, #chat-tags-<pk>, #details-tags-<pk>

Respuestas con swaps fuera de banda

Nota. Un id fuera de banda que no está en la página se ignora sin error, tal y como anota el comentario de `picker_panel.html` para el caso de una fila filtrada fuera de la lista. Eso es lo que permite emitir siempre las tres superficies de etiquetas sin saber cuáles están montadas.

16.5 Sondeo con `hx-trigger every` y las guardas `data-hold-poll`

Hay exactamente dos sondeos, ambos en el Inbox y ambos a cinco segundos. El primero vive en `templates/partials/inbox/chat_thread.html`: `#chat-messages` se refresca a sí mismo contra `inbox_thread` con `hx-trigger="every 5s"`. El comentario explica por qué el sondeo se limita a los mensajes y no al hilo entero: así un borrador a medio escribir en el compositor de abajo nunca se pisa. El segundo vive al final de `templates/partials/inbox/conversation_list.html`, en un elemento oculto que se re-renderiza con cada intercambio, de modo que el filtro activo viaja siempre incrustado y no puede quedarse obsoleto.

Ese segundo disparador es el más elaborado de la aplicación y merece leerse entero, porque cada parte está justificada en el comentario que lo acompaña. La condición entre corchetes pausa el sondeo mientras haya abierto un desplegable marcado `data-hold-poll` o alguna fila marcada, porque un re-render borraría el desplegable abierto o una selección a medio hacer. El segundo disparador, `change from:#tag-filter`, hace que cambiar el filtro de etiquetas refresque la lista al instante. `hx-include` arrastra las casillas de etiqueta marcadas y `hx-vals` con `js`: recupera la conversación abierta leyendo `[data-conversation-id]`, que `chat_thread.html` describe como «load-bearing» precisamente por eso.

```
hx-trigger="every 5s [!document.querySelector('details[data-hold-poll][open], .conv-item__check:checked')], change from:#tag-filter"
hx-target="#conv-list"
hx-swap="innerHTML"
hx-include="#tag-filter input:checked"
```

Los dos elementos que esa condición busca están en `templates/partials/inbox/list_panel.html`, deliberadamente fuera de `#conv-list` para que su estado sobreviva a filtros y sondeos: el desplegable de etiquetas `#tag-filter` y el menú Acciones, ambos `<details class="dropdown" data-dropdown data-hold-poll>`. Toda la mitad inferior del panel es un solo `<form id="bulk-form">`, así que las casillas de fila, el input oculto `filter` (que viaja dentro de `#conv-list`) y las casillas del filtro de etiquetas acompañan automáticamente a cualquier `hx-post` disparado desde dentro.

Ese es justamente el flujo del menú Acciones: cada botón publica a `inbox_tags_bulk` con `hx-vals` en modo `js`: aportando `tag_id`, `action` («add» o «remove») y `active` leído de `[data-conversation-id]`, con `hx-target="#conv-list"`. La vista lee `selected` del formulario, aplica o quita la etiqueta mediante `messaging_services.apply_tag` o `remove_tag`, y re-renderiza `conversation_list.html` bajo el mismo `filter` y las mismas etiquetas seleccionadas, de forma que el usuario recupera exactamente la lista que estaba mirando. Un `filter` desconocido cae a `inbox.DEFAULT_FILTER` en lugar de fallar.

Del lado del servidor, `inbox_list` e `inbox_thread` llaman a `messaging_services.pump_provider_events()` en cada tick para que el proveedor falso entregue sus recibos pendientes; en proveedores reales, basados en push, la llamada no hace nada. Los demás `hx-trigger` de la aplicación no son sondeos sino carga diferida o antirrebote: `toggle` once en el selector

de respuestas rápidas, `toggle from:closest details` en el picker de etiquetas, `input changed delay:300ms`, `search` en el buscador de clientes, `input changed delay:250ms` en el buscador del picker y `change` en el desplegable de asignación.

16.6 Los ganchos de `shell.js`

`static/js/shell.js` es el pegamento de la shell y se declara a sí mismo como mejora progresiva. Su cabecera fija dos reglas que conviene respetar al añadir código nuevo: los listeners se delegan desde `document` y los elementos se buscan bajo demanda, nunca se cachean, porque un nodo cacheado queda obsoleto en cuanto se re-renderiza su subárbol y los clics acaban actualizando un elemento desconectado.

Su trabajo principal ni siquiera es un gancho de `htmx`, sino un `click` delegado: cualquier contenedor marcado `[data-nav-group]` recibe el comportamiento de píldora activa, de modo que al pulsar un `[data-nav-item]` dentro de él la clase `is-active` se mueve al instante, sin esperar a que vuelva el fragmento. La cabecera explica por qué existe: `htmx` sólo intercambia parte de la página, y hay dos navegaciones que quedan fuera de la región intercambiada, la barra lateral (nunca se re-renderiza) y el panel de navegación del Inbox (dentro de `#content`, pero sólo se cambia la columna 3). El mismo listener añade `aria-current="page"` en el raíl y `aria-current="true"` en el resto.

Las tarjetas de la pantalla de bienvenida son el caso que ese esquema no cubre por sí solo: intercambian `#content` desde fuera del raíl, así que llevan `data-nav-for` con la misma URL que el icono correspondiente y `shell.js` usa ese valor para encender el icono de una navegación que no originó. La contrapartida al volver atrás es `syncSidebarFromUrl`, colgada de `popstate` y de `htmx:historyRestore`: rederiva el icono activo buscando `.nav-item[href="<pathname>"]`, y trata aparte el caso de la raíz, porque `«/»` no corresponde a ningún icono y hay que apagar el raíl explícitamente (`setActive(null)` sería un `no-op`).

El único uso de `htmx:beforeSwap` es medir la posición de `scroll` del hilo de chat antes de que llegue el reemplazo, con una tolerancia de 48 píxeles en `nearBottom`: sólo si el usuario ya estaba abajo se le devuelve al fondo después del intercambio, para que un sondeo no arranque de la lectura a quien ha subido a mirar el histórico. El `htmx:afterSwap` que lo acompaña hace algo más que restaurar esa medida: cuando el destino no es `#chat-messages` sino un contenedor que lo contiene (un hilo recién abierto), baja siempre al último mensaje, porque un chat que se acaba de abrir debe empezar por lo nuevo. Y al margen de `htmx` hay un ajuste de primera carga: si la página completa se abrió con `?chat=` en la URL, el hilo viene renderizado desde el servidor y el script lo posiciona abajo nada más cargar (`initialChat`).

Contando ese, `shell.js` registra siete listeners distintos de `htmx:afterSwap`, más cuatro de `htmx:historyRestore` para lo que también debe rehacerse cuando el navegador restaura una instantánea sin disparar `afterSwap`.

- `htmx:afterSwap`: sincroniza `document.title` desde `#content .page-head__title`, que vive en `<head>` fuera de todo intercambio (`syncTitle`).
- `htmx:afterSwap`: reaplica los banners descartados guardados en `localStorage`, porque cada swap los vuelve a traer del servidor (`hideDismissed`).
- `htmx:afterSwap`: restaura el `scroll` del hilo de chat, o baja al último mensaje si el hilo se acaba de abrir.
- `htmx:afterSwap`: cierra el diálogo cuando la respuesta trae `[data-dialog-dismiss]`.
- `htmx:afterSwap`: reubica el foco (listener aparte del anterior) cuando un swap ha eliminado el diálogo que lo contenía.
- `htmx:afterSwap`: vuelve a sincronizar el editor de plantillas (`syncTemplateEditor`), `no-op` en el resto de pantallas.

- `htmx:afterSwap`: rederiva el estado de selección múltiple tras cada swap de `#conv-list` (`refreshSelection`), ya que las listas llegan siempre sin marcar.
- `htmx:historyRestore`: `syncSidebarFromUrl`, `syncTitle`, `hideDismissed` y `syncTemplateEditor`.

Hay además un gancho sobre `htmx:afterRequest`, que cierra el diálogo de cualquier elemento marcado `[data-close-on-success]` cuando la petición ha ido bien, y reinicia el formulario si el elemento lo es. `templates/partial/mensajeria/template_chooser_dialog.html` es el ejemplo vivo: sus dos tarjetas llevan `data-close-on-success` y `hx-target="#mensajeria-panel"`. El comentario del propio partial reparte responsabilidades: el `<dialog>` nativo aporta la trampa de foco, el Escape y la devolución de foco vía `showModal()`, y `shell.js` sólo añade el cierre por clic en el fondo, opt-in con `data-dialog-backdrop-close`.

Ese cierre por fondo necesita dos guardas que el código explica. La primera es `pressOnBackdrop`, medida en `pointerdown`: un clic por sí solo no distingue el fondo del contenido, porque un arrastre que empieza dentro del diálogo y termina más allá de su borde también reporta el `<dialog>` como destino con coordenadas exteriores, y sólo el punto de presión separa los dos casos. La segunda es `event.detail === 1`, que descarta la cola de un doble clic sobre el botón que abre el diálogo (la segunda presión cae ya sobre el fondo del diálogo recién abierto) y los clics sintetizados por teclado, que llegan con `detail 0` en `(0,0)`.

Nota. No todos los ganchos viven en `shell.js`. El compositor del chat usa uno inline por elemento, `hx-on::after-request="if(event.detail.successful) this.reset()"`, para vaciar el campo tras un envío correcto sin pasar por la delegación global.

16.7 Decisiones de diseño y trampas conocidas

Varias decisiones están explicadas en los comentarios del propio código y son las que más ahorran tiempo al llegar nuevo. La primera es el reparto entre swap dirigido y swap fuera de banda.

`assign_swap.html` documenta por qué la fila de la lista de conversaciones no se actualiza fuera de banda: el sondeo de la lista la recoge en su siguiente tick, y forzar un OOB significaría apuntar a un elemento que puede no estar siquiera en el filtro actual. La misma vista `inbox_assign` recalcula `assign_options` después de guardar, no reutilizando las opciones con las que validó, para que un agente que ya no está en `APP_AGENTS` desaparezca del desplegable en lugar de quedarse hasta la siguiente recarga.

Esa vista es también el ejemplo de validación de un disparador `htmx` que no es un formulario libre. `inbox_assign` acepta únicamente ids que estuvieran entre las opciones renderizadas por `agents.assignment_options`; si el valor recibido no está, responde `HttpResponse(status=400)` en lugar de guardar. El docstring lo razona: el desplegable es una lista fija, así que cualquier otro valor es un POST fabricado a mano, y dejarlo pasar apuntaría la conversación a una fila de `User` arbitraria.

La segunda es dónde vive el marcado que no debe re-renderizarse. En `Clientes`, `#client-modal` y la barra de herramientas se quedan fuera de `#client-table`, que es lo único que se reemplaza; el docstring de `clientes_table` recoge la regresión que motivó separarlos: el paginador solía traerse el panel completo dentro de esa región, anidando en cada clic una segunda barra de herramientas y un `#client-table` duplicado dentro del primero. Por el mismo motivo el input oculto `#client-page` vive en `client_table.html` y no en la barra: así el valor que los botones CRUD arrastran con `hx-include` es siempre la página que hay en pantalla. En el `Inbox`, el input oculto `filter` viaja dentro de `#conv-list` con idéntico razonamiento.

La tercera es de accesibilidad. `client_saved.html` explica por qué la región viva `#client-notice` está en el panel y no dentro del modal: una región viva alojada en un diálogo que se cierra en el mismo tick nunca llega a leerse. Y `picker_panel.html` anota que el buscador conserva el foco entre intercambios

porque tiene un id estable, que es a lo que htmx se agarra al asentar el swap.

La cuarta es el CSRF. Cuando el disparador de un `hx-post` es un formulario, el token viaja en su `{% csrf_token %}`: así funcionan el compositor del chat, el formulario de asignación, el `bulk-form` del Inbox y los formularios de alta, edición y borrado de cliente (`client_form.html` y `client_delete.html`). Cuando el disparador es un botón suelto, el token se inyecta con `hx-headers` sobre un contenedor que lo envuelve. En todo el árbol de plantillas hay sólo cuatro `hx-headers`: el `<details>` de respuestas rápidas en `chat_thread.html`, `#tag-table` en `panels/etiquetas.html`, `.tag-picker__body` en `picker_panel.html` y cada botón de aspa de `removable_pills.html`.

Nota. El comentario de `templates/partials/crm/client_table.html` afirma que un `hx-headers` local suministra el token a los formularios del modal, pero el atributo no existe en ese archivo: el comentario está obsoleto y el CSRF de Clientes lo aportan los propios formularios. Al añadir un `hx-post` nuevo que no sea un formulario, hay que comprobar que queda bajo uno de los cuatro contenedores con `hx-headers` o darle el suyo.

16.8 Pruebas que lo cubren

El contrato de la respuesta dual se prueba con una constante compartida por varios módulos, `HTMX = {"HX-Request": "true"}`, que se pasa como `headers` al cliente de test. En `core/tests.py`, la clase `HtmxSwapTests` comprueba las dos direcciones: una petición `htmx` devuelve un fragmento sin `<html>` y sin la barra lateral pero con el encabezado de página, una petición normal devuelve el documento completo, y `test_fragment_and_full_page_agree_on_content` verifica que el fragmento aparece literalmente contenido en la página completa. Ese último aserto es el que impide que las dos ramas de una vista se separen. `InboxListEndpointTests` repite la comprobación para `inbox_list` y añade la del reparto de atributos: `hx-target="#conv-list"` apuntando al endpoint parcial, `hx-push-url` apuntando a la URL de sección recargable.

Los swaps fuera de banda se verifican por marcado en la respuesta. `core/tests_agents.py` comprueba en `test_response_carries_the_new_name_for_both_panels` que la respuesta de `inbox_assign` trae `id="chat-assign-<pk>"`, `id="details-assigned-<pk>"` y `hx-swap-oob="outerHTML"`.

`core/tests_clientes.py` comprueba en

`test_a_successful_post_closes_the_modal_and_refreshes_the_table` que un alta correcta trae `data-dialog-dismiss` y `id="client-table"` `hx-swap-oob="innerHTML"`, y el test hermano de rechazo comprueba lo contrario: sin `data-dialog-dismiss`, con errores en línea y sin perder lo escrito. En `messaging/tests.py`, `test_toggle_response_updates_pills_out_of_band` cubre las tres superficies de etiquetas.

La variante de página completa de mensajería tiene sus propios tests en `core/tests_mensajeria.py`: `test_htmx_get_returns_a_bare_fragment` frente a `test_plain_get_renders_inside_the_page_shell`, y para el POST, `test_valid_htmx_post_saves_pendiente_and_returns_the_list` frente a `test_valid_plain_post_redirects_to_the_list`, que fija el destino de la redirección en `/s/mensajeria/?view=plantillas-whatsapp`. Los módulos por sección (`tests_crm.py`, `tests_comercio.py`, `tests_embudos.py`, `tests_automatizaciones.py`, `tests_estadisticas.py`, `tests_campanas.py`) repiten el mismo par de asertos sobre sus filas de navegación secundaria: el `hx-target` del panel correspondiente y el `hx-push-url` de la URL con `?view=`. `core/tests_crm.py` añade en `test_create_button_points_at_a_real_route` la comprobación del destino relativo `hx-target="closest main"`.

Por último, `TemplateCommentLeakTests` en `core/tests.py` renderiza todas las secciones y fija que ninguna deja escapar `{# ni #}`. Su docstring explica el motivo y admite el historial: los comentarios `{#` de Django son de una sola línea, uno multilínea no es un comentario y se cuela como texto visible, y esto se

ha colado ya dos veces. Dado que casi todos los partials de este capítulo empiezan por un bloque `{% comment %}` largo, es una salvaguarda directamente relevante aquí.

Archivos de referencia: `core/views.py`, `core/urls.py`, `messaging/urls.py`, `static/js/shell.js`,
`templates/base.html`, `templates/sections/welcome.html`, `templates/sections/crm.html`,
`templates/sections/campanas.html`, `templates/sections/mensajeria.html`, `templates/partials/nav_item.html`,
`templates/partials/side_nav_rows.html`, `templates/partials/side_nav_section.html`,
`templates/partials/account_nav_panel.html`, `templates/partials/comercio/nav_panel.html`,
`templates/partials/embudos/nav_panel.html`, `templates/partials/automatizaciones/nav_panel.html`,
`templates/partials/estadisticas/nav_panel.html`, `templates/partials/mensajeria/nav_panel.html`,
`templates/partials/inbox/chat_swap.html`, `templates/partials/inbox/assign_swap.html`,
`templates/partials/inbox/assign_control.html`, `templates/partials/inbox/conversation_list.html`,
`templates/partials/inbox/chat_thread.html`, `templates/partials/inbox/chat_panel.html`,
`templates/partials/inbox/list_panel.html`, `templates/partials/crm/client_saved.html`,
`templates/partials/crm/client_table.html`, `templates/partials/crm/client_form.html`,
`templates/partials/crm/client_delete.html`, `templates/partials/crm/panels/clientes.html`,
`templates/partials/crm/panels/etiquetas.html`, `templates/partials/crm/panels/lista-clientes.html`,
`templates/partials/comercio/panels/productos.html`,
`templates/partials/mensajeria/panels/plantillas-whatsapp.html`, `templates/partials/tags/picker.html`,
`templates/partials/tags/picker_panel.html`, `templates/partials/tags/removable_pills.html`,
`templates/partials/mensajeria/template_chooser_dialog.html`, `core/tests.py`, `core/tests_agents.py`,
`core/tests_clientes.py`, `core/tests_crm.py`, `core/tests_mensajeria.py`, `messaging/tests.py`

Anexo A. Estado del repositorio y cambios posteriores a esta versión

Este documento describe la rama main en el commit 6089391. Ese es el código que los autores y los verificadores leyeron, y es el que describen los capítulos 1 a 16. main avanza rápido: cuando este PDF se incorporó al repositorio ya había ocho commits nuevos por encima de ese punto. Esta tabla los recoge para que sepas exactamente qué queda fuera y no confundas una ausencia con una omisión.

Commit	Qué añade a main
1be6bc6	El cliente vinculado se muestra en cada evento del calendario.
f53a6dc	Exportación de la base de clientes a Excel desde Gestión de clientes.
37d334e	Envío real de mensajes: primer contacto, plantillas y respuestas rápidas con imágenes.
ca404a0	Un usuario maestro que crea y administra al resto del equipo.
10ce778	Puesta en producción: se vacía el CRM y se impide que datos de prueba lleguen a producción.
b4c68e9	Unifica en una sola las dos limpiezas de "solo datos reales".
c6e200c	Envío de plantillas a Meta para aprobación y lectura del veredicto.

Commits en main posteriores al snapshot documentado, sin contar merges.

Esa lista es la que era cierta al publicar el documento, y envejece. Para ver el desfase real en cualquier momento, pide a git los commits que separan el snapshot de la punta actual:

```
git log --oneline --no-merges 6089391..origin/main
```

Nota. Los capítulos más afectados por ese trabajo posterior son 3 (Inbox), 4 (Mensajería), 5 (CRM), 6 (Mi calendario) y 7 (plantillas de WhatsApp): siguen siendo correctos para 6089391, pero ya no describen main por completo. Regenerar el documento sobre un commit más reciente exige volver a lanzar la lectura y la verificación; recomponerlo con el contenido actual es un solo comando (ver docs/README.md).

Anexo B. Método de generación y verificación

Cada capítulo pasó por tres etapas independientes: un lector redactó el capítulo a partir del código del snapshot; un verificador escéptico, con acceso al mismo código pero sin conocer al lector, contrastó cada afirmación concreta (rutas, nombres de URL, campos, variables de entorno, valores por defecto, comandos) e intentó refutarla; y, cuando encontró discrepancias u omisiones, un corrector reescribió el capítulo aplicándolas. La tabla resume el resultado.

Capítulo	Afirmaciones verificadas	Correcciones	Omisiones cubiertas	Resultado
Arquitectura y shell de la aplicación	27	3	6	Corregido
Autenticación y agentes	26	2	6	Corregido
Inbox: la bandeja unificada	26	0	6	Corregido
Mensajería: modelo, servicios y proveedores	29	4	6	Corregido
CRM: clientes, listas y etiquetas	26	2	6	Corregido
Mi calendario	25	0	6	Corregido
Mensajería: plantillas de WhatsApp	29	3	6	Corregido
Estadísticas	26	3	6	Corregido
Embudos, Automatizaciones, Campañas y Mi comercio	25	2	6	Corregido
Modelo de datos y migraciones	27	6	6	Corregido
Configuración, despliegue y operación	27	4	6	Corregido
Pruebas y convenciones de desarrollo	36	3	6	Corregido
Guía práctica: cómo añadir una sección, un panel o una vista nueva	27	2	6	Corregido
Glosario de términos del dominio	25	3	6	Corregido
Sistema visual: tokens, convenciones CSS y carga de estilos	28	2	6	Corregido
Patrones htmx: fragmentos, swaps e historial	31	5	6	Corregido

Registro de verificación por capítulo. "Afirmaciones verificadas" es el número de datos concretos que el capítulo expone al verificador.

Tras los capítulos previstos, un crítico de completitud comparó los archivos del repositorio con lo cubierto y propuso los huecos que dieron lugar a: Guía práctica: cómo añadir una sección, un panel o una vista nueva; Glosario de términos del dominio; Sistema visual: tokens, convenciones CSS y carga de estilos; Patrones htmx: fragmentos, swaps e historial. Esos capítulos pasaron por las mismas tres etapas.